Paging

Abhilash Jindal

Overview

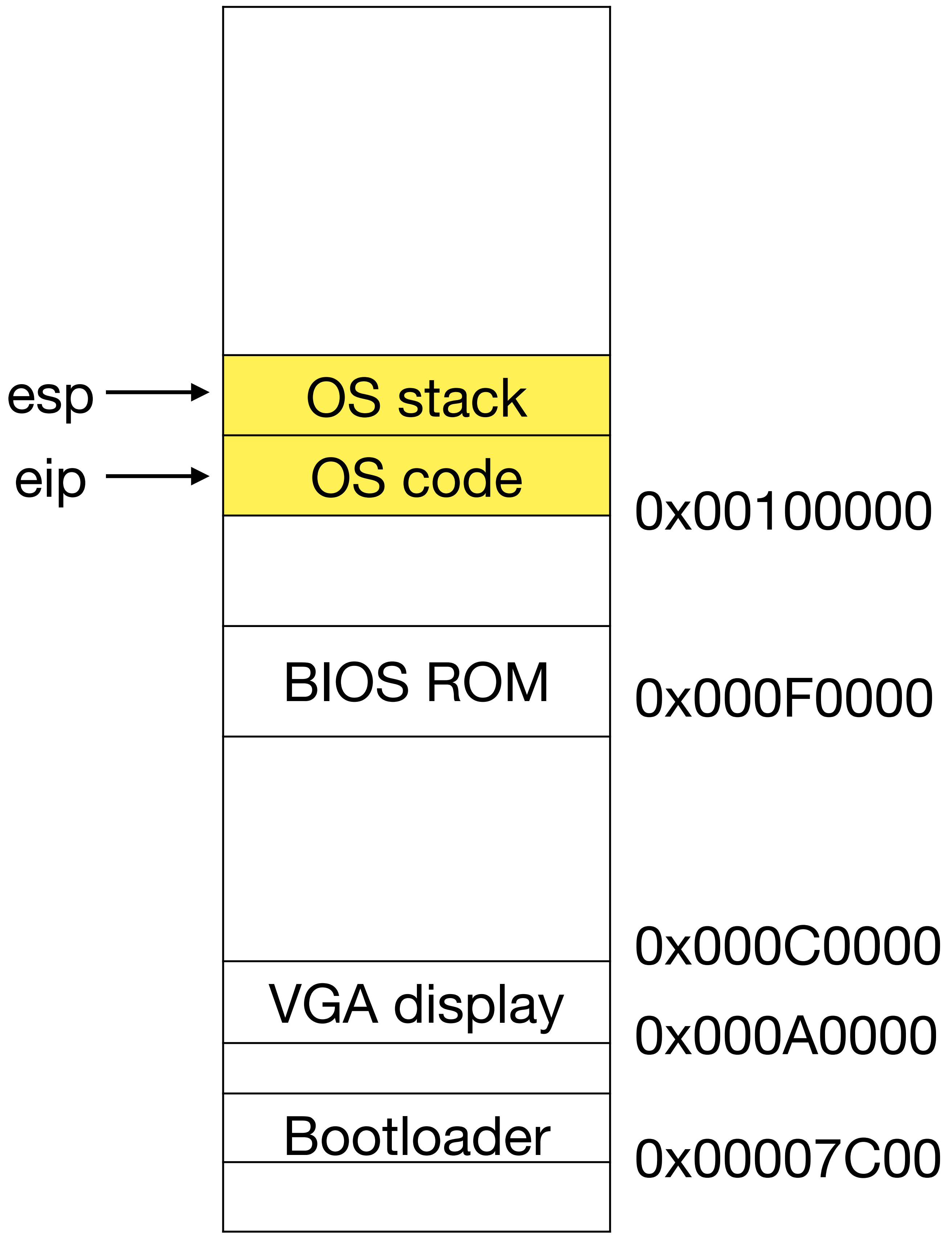
- More flexible address translation with paging (OSTEP Ch 18-20)
  - Paging hardware
- Demand paging: swapping pages to disk when memory becomes full (OSTEP Ch 21-22)
  - Swapping mechanisms
  - Page replacement algorithms
- Paging in action (xv6 book Ch 2, OSTEP Ch 23)
  - Paging on xv6
  - Fork with Copy-on-write, Guard pages

Paging Hardware

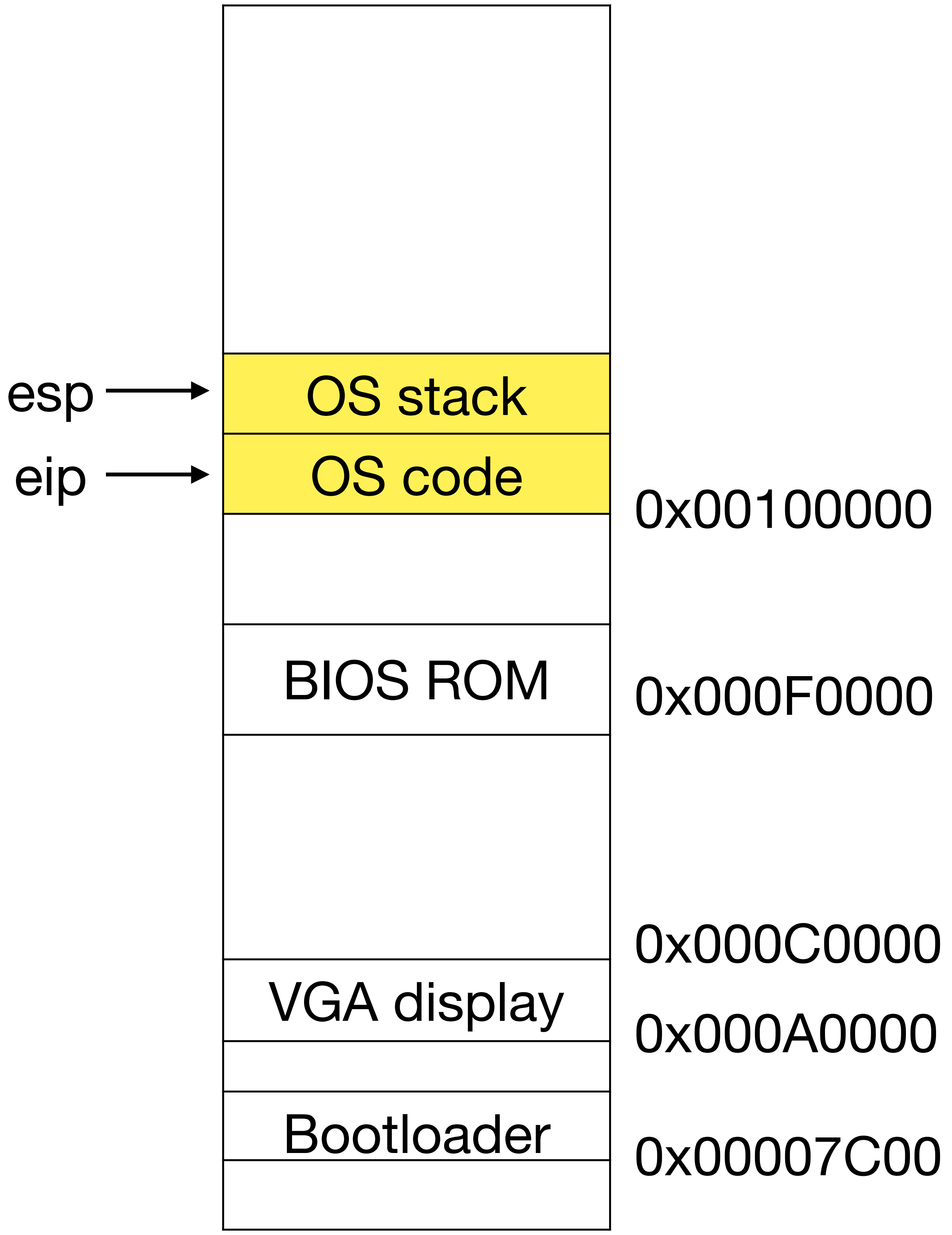
OSTEP Ch 18-20

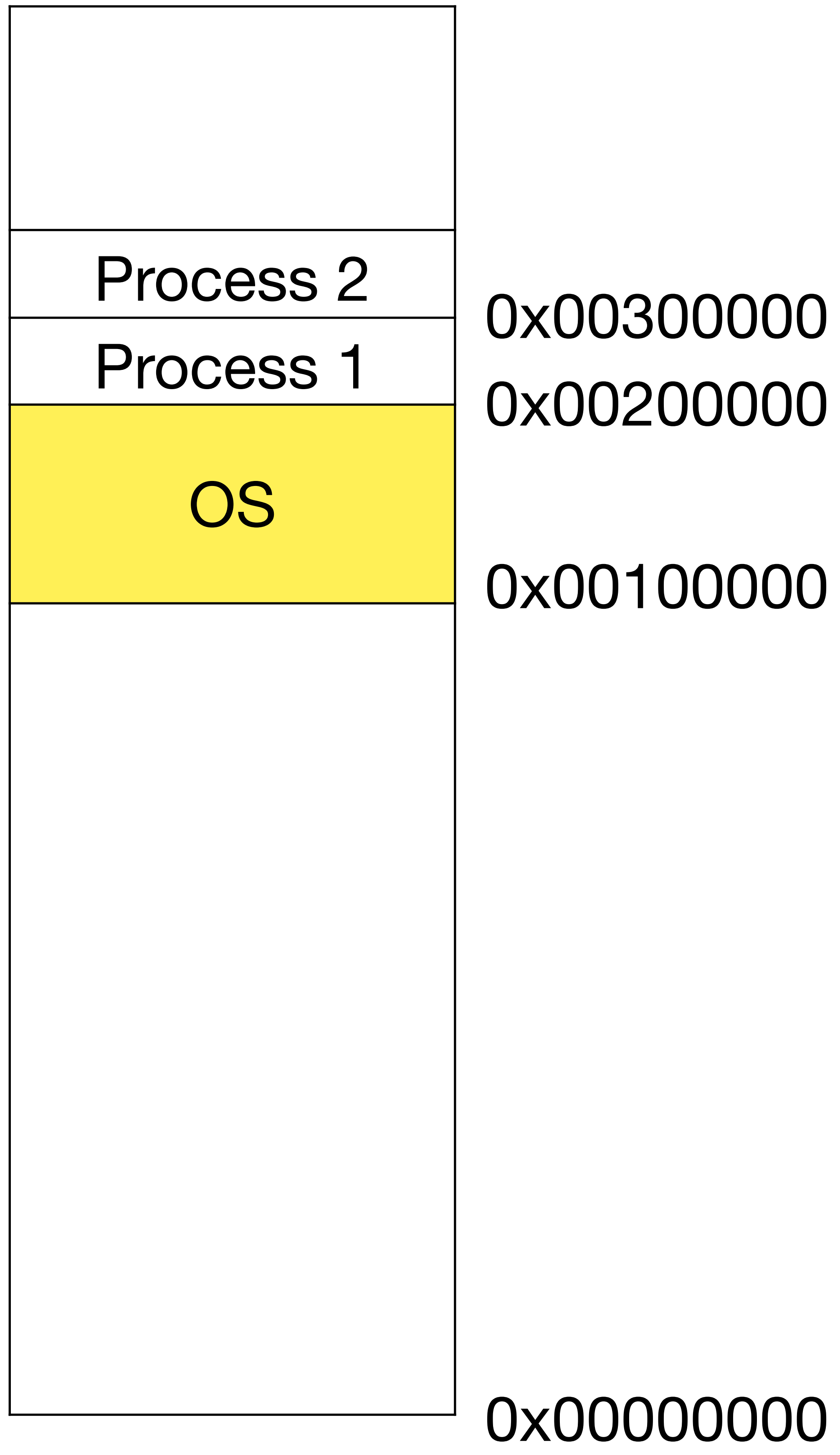
Intel SDM Volume 3A Ch 4

Memory isolation and address space

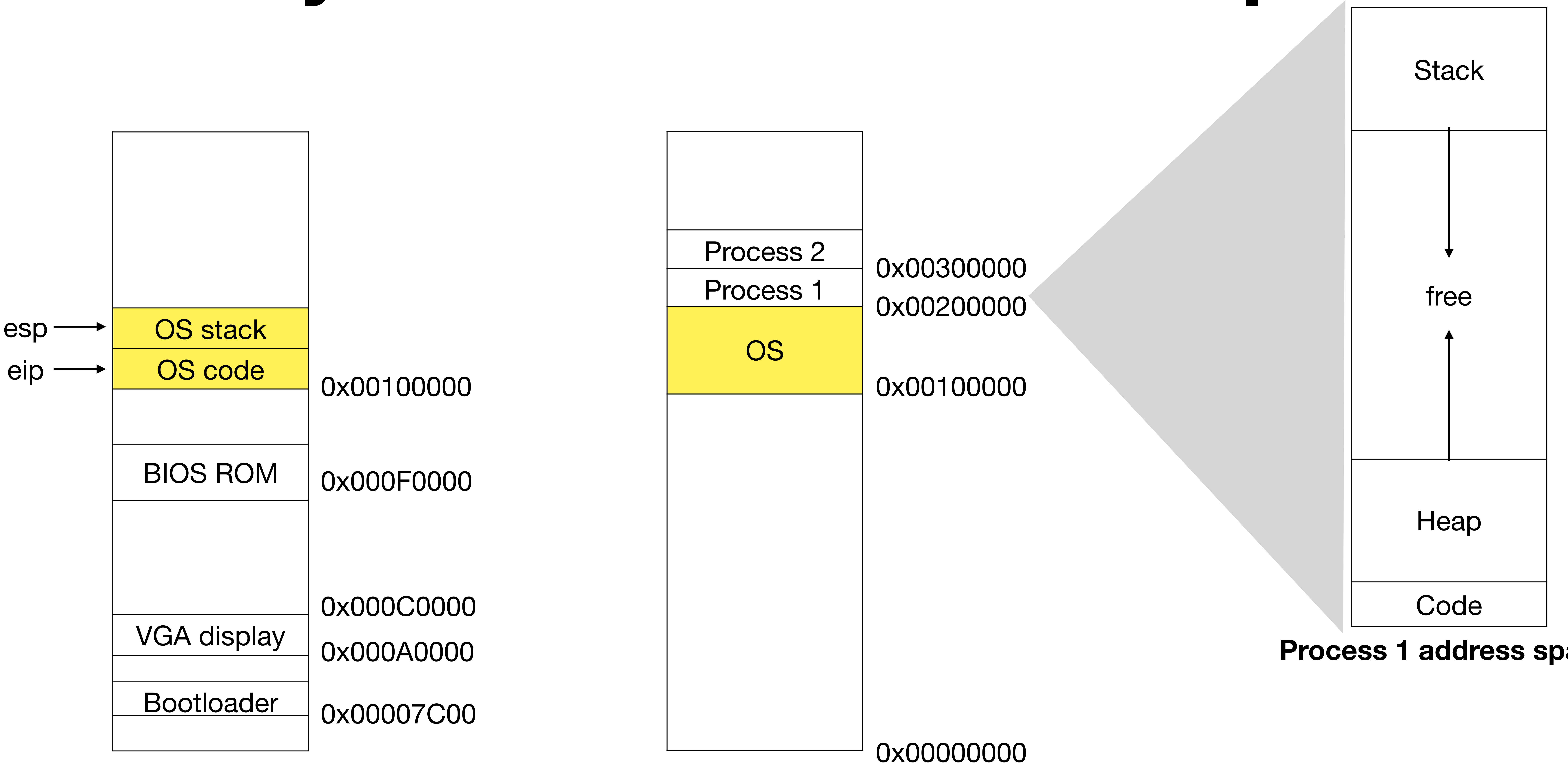


Memory isolation and address space



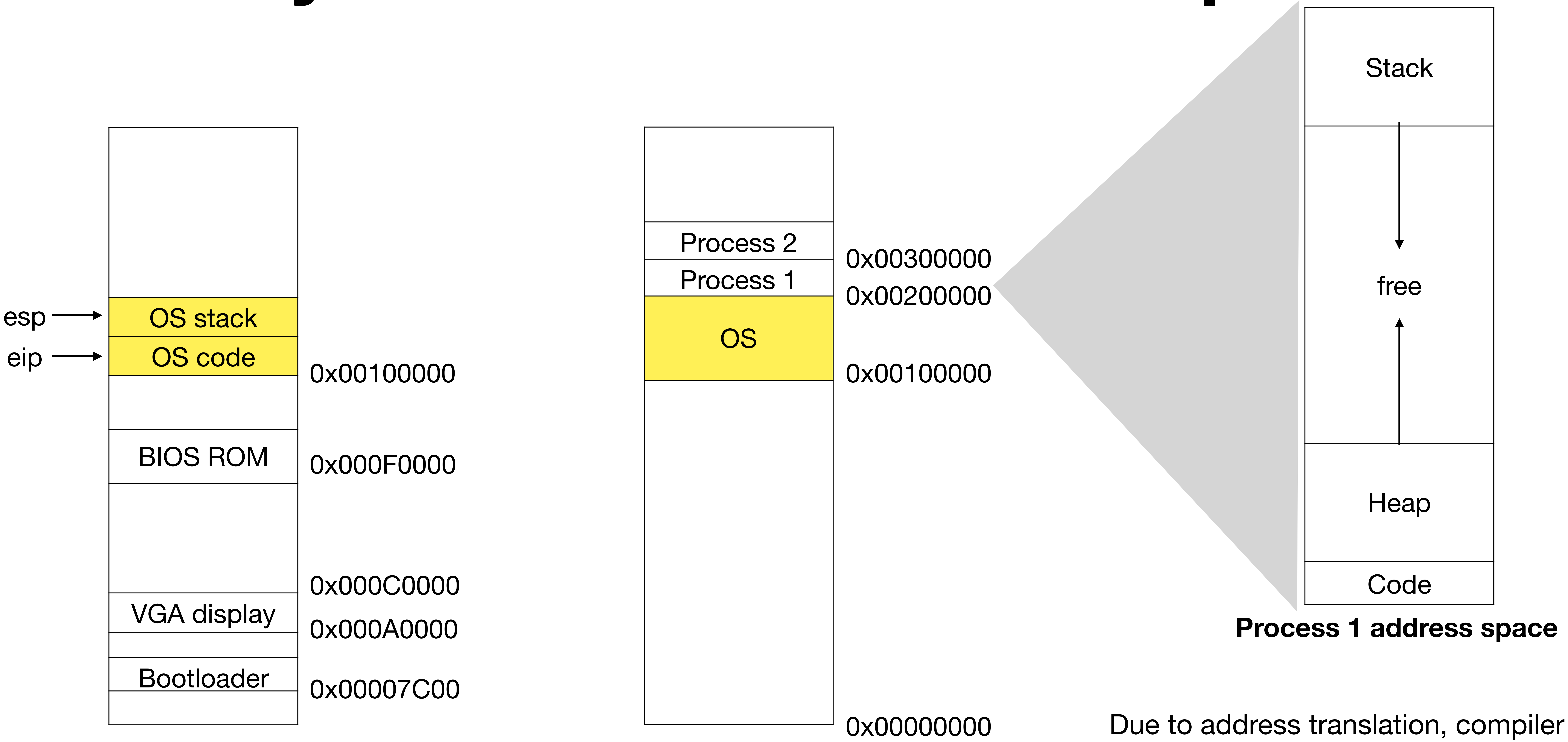


Memory isolation and address space



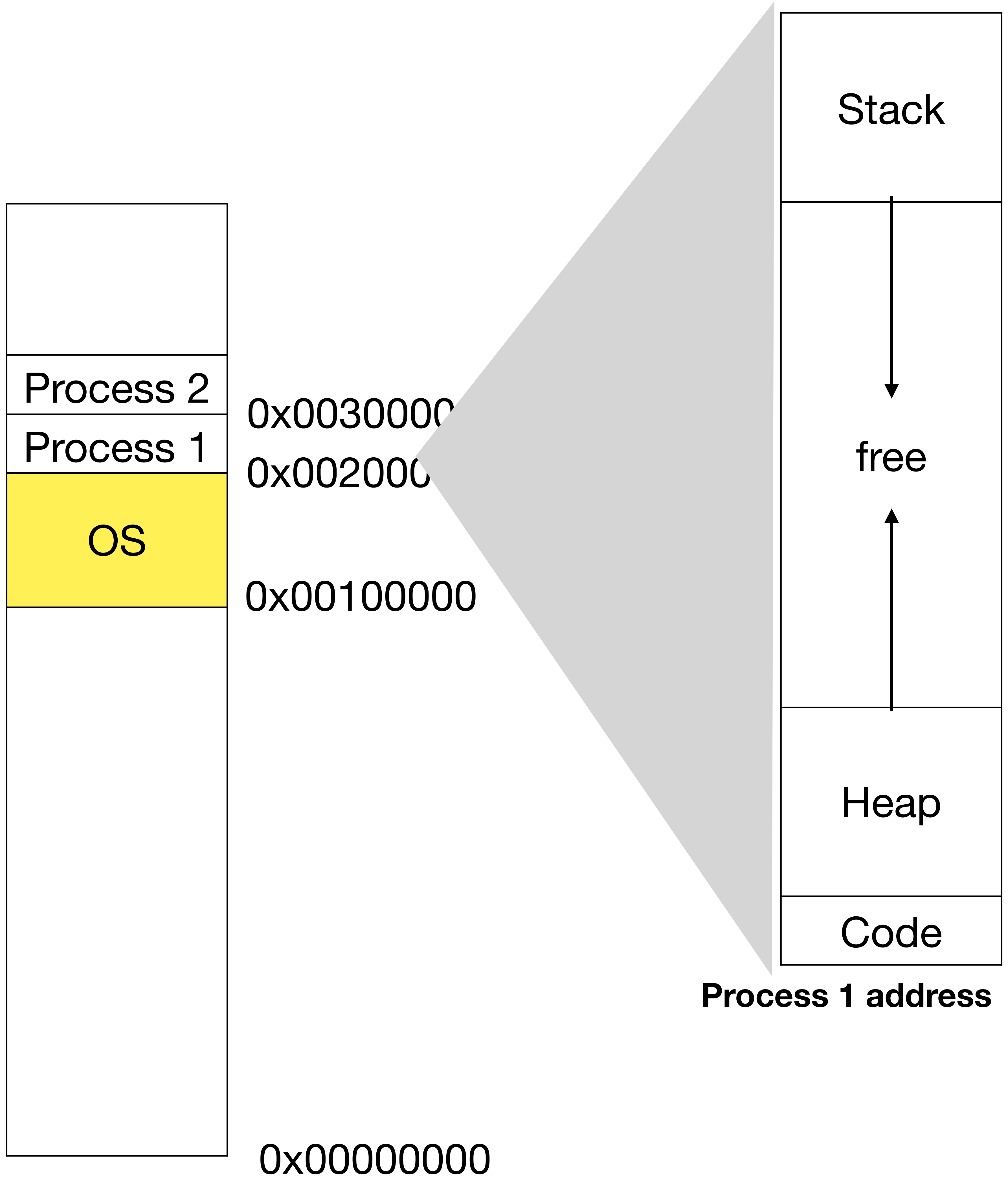
Process 1 address space

Memory isolation and address space

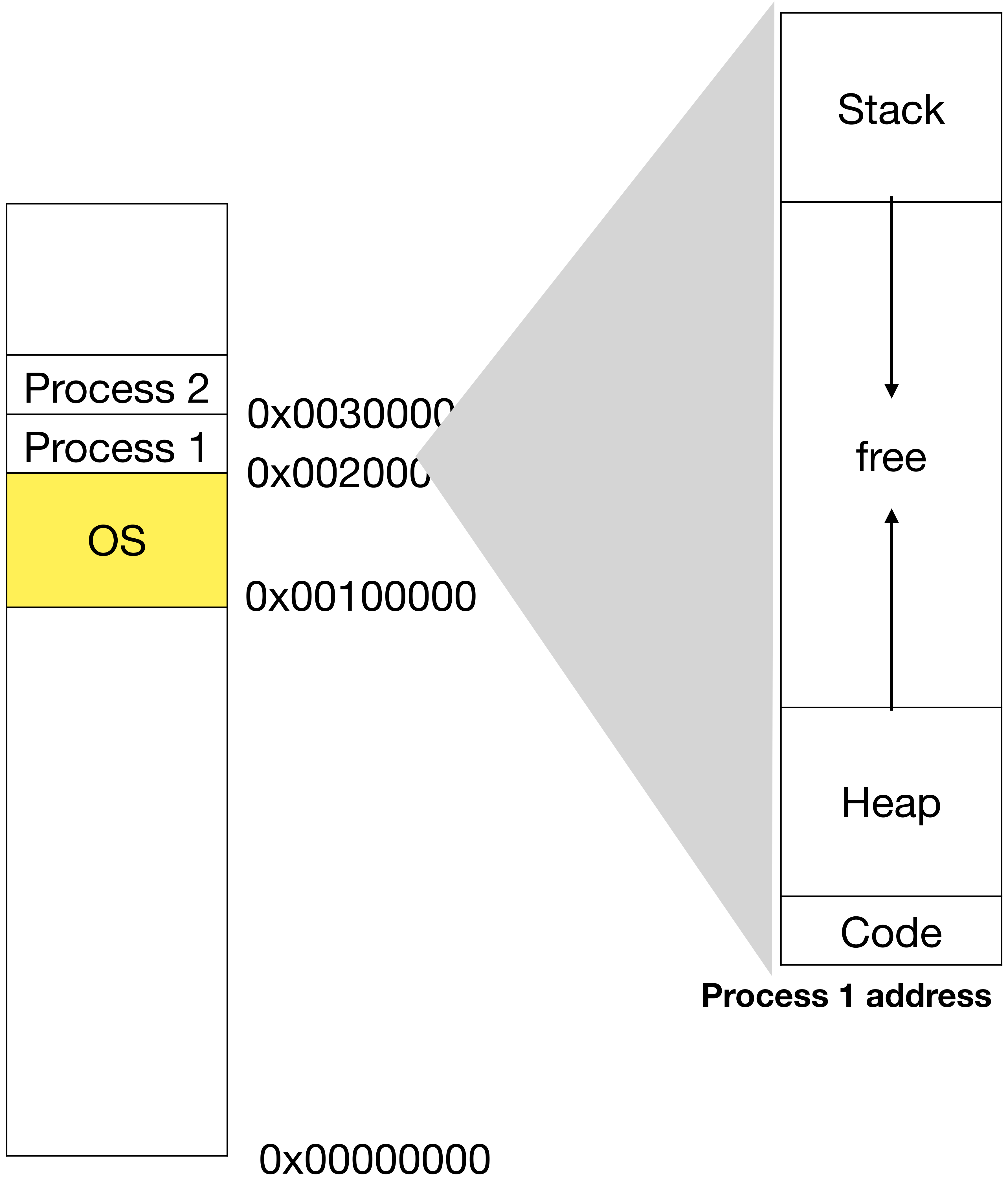


Due to address translation, compiler need not worry where the program will be loaded!

Segmentation

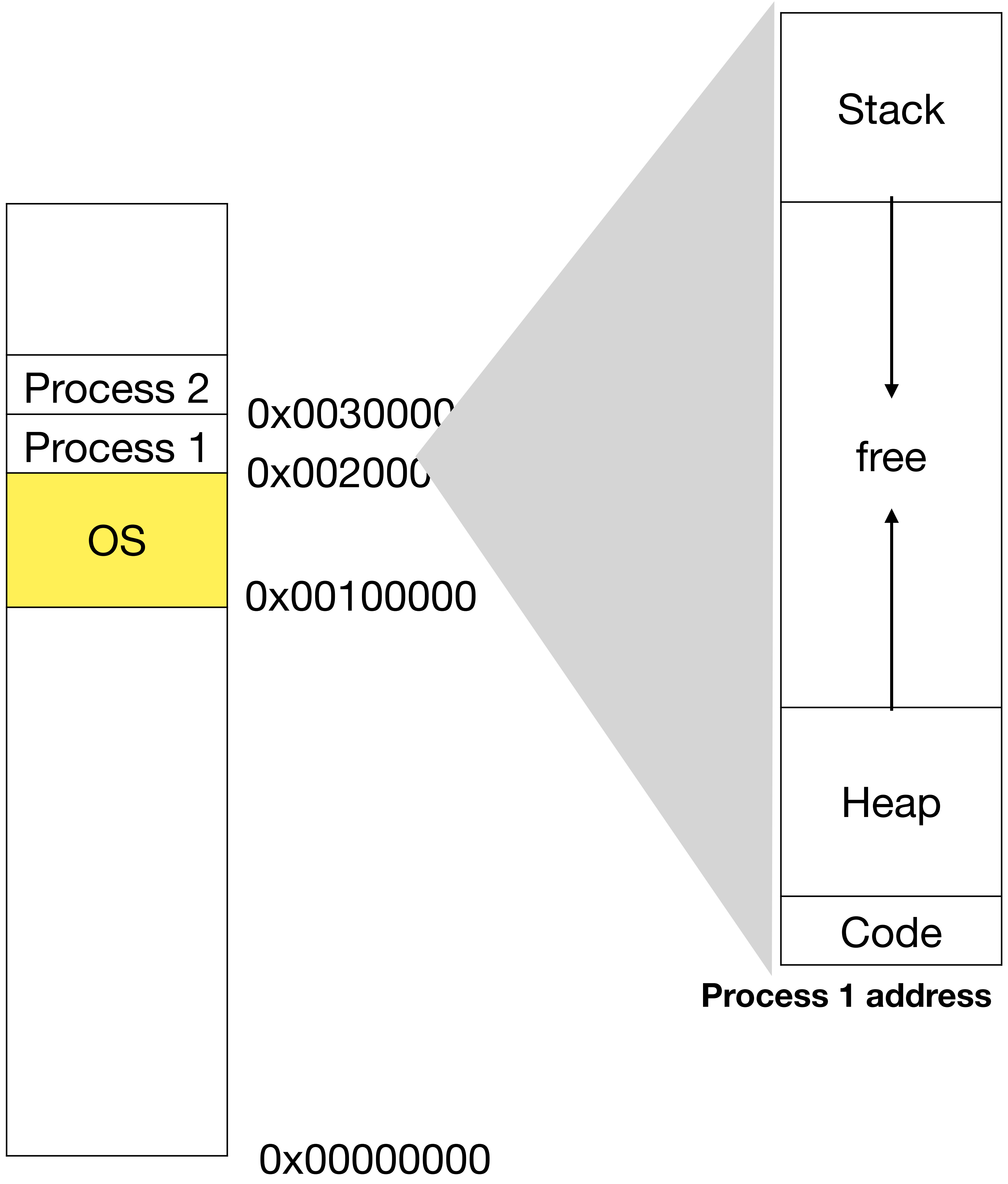


Segmentation



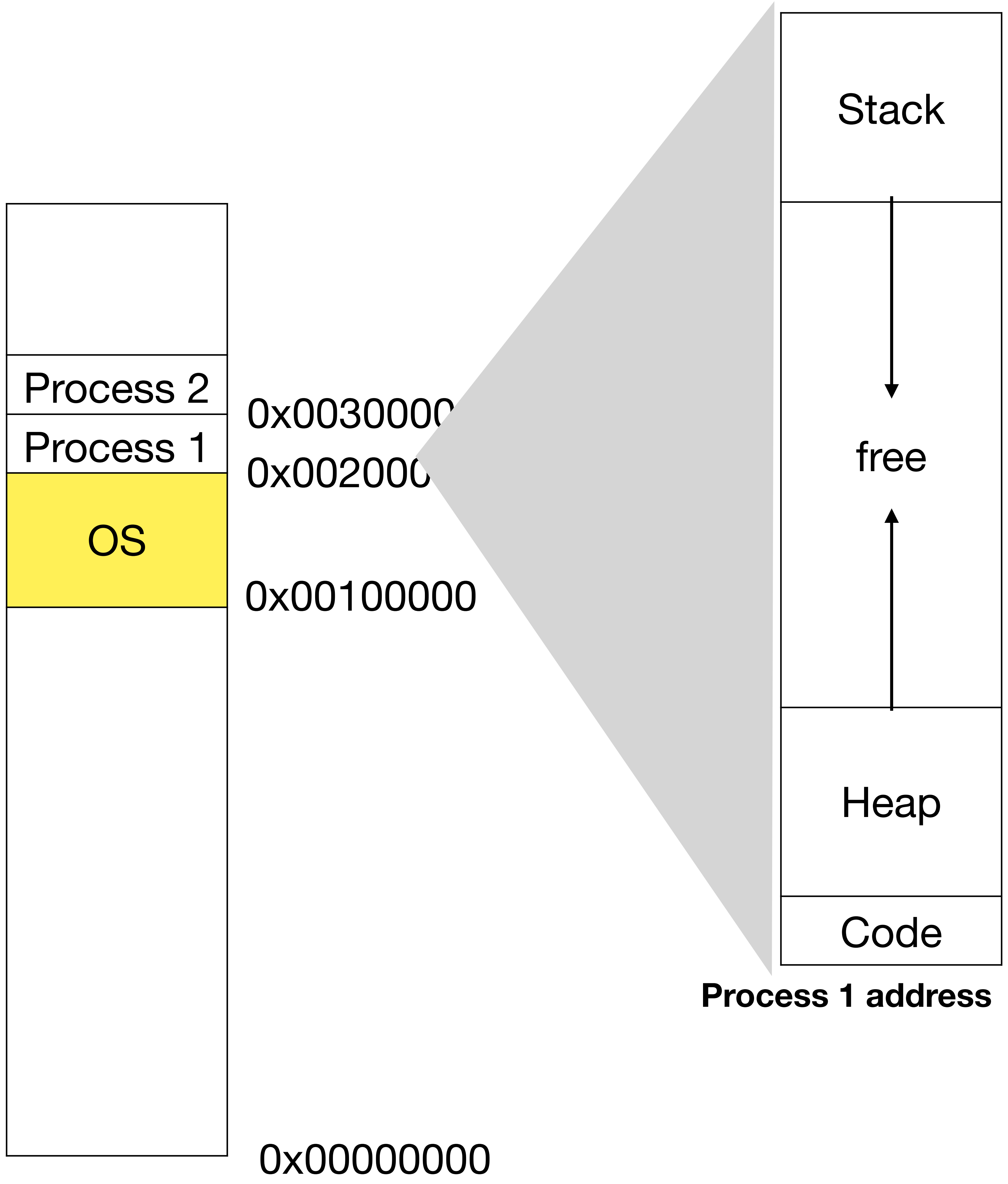
- Mapping large address spaces

Segmentation

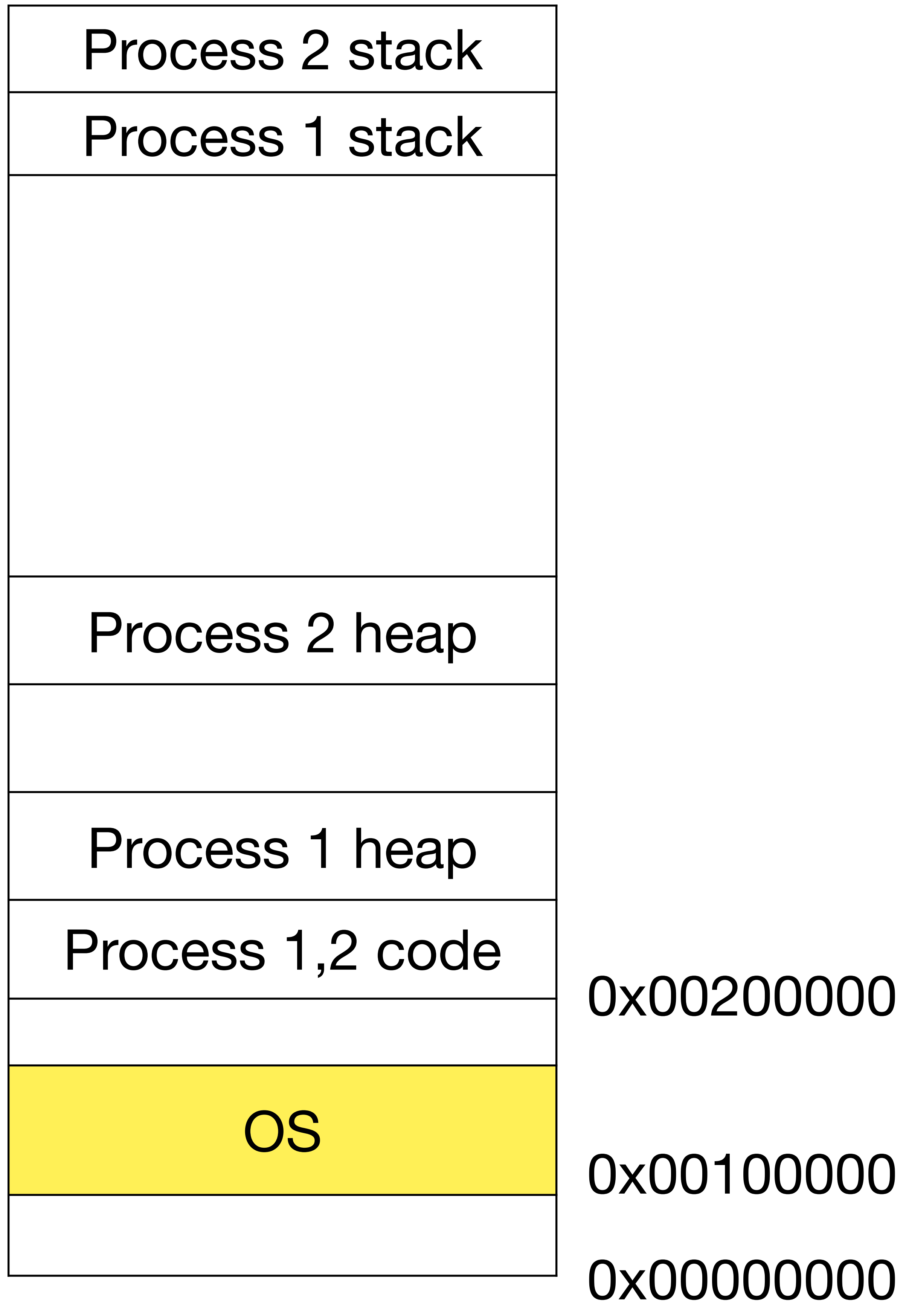


- Mapping large address spaces
    - Place each segment independently to not map free space

Segmentation

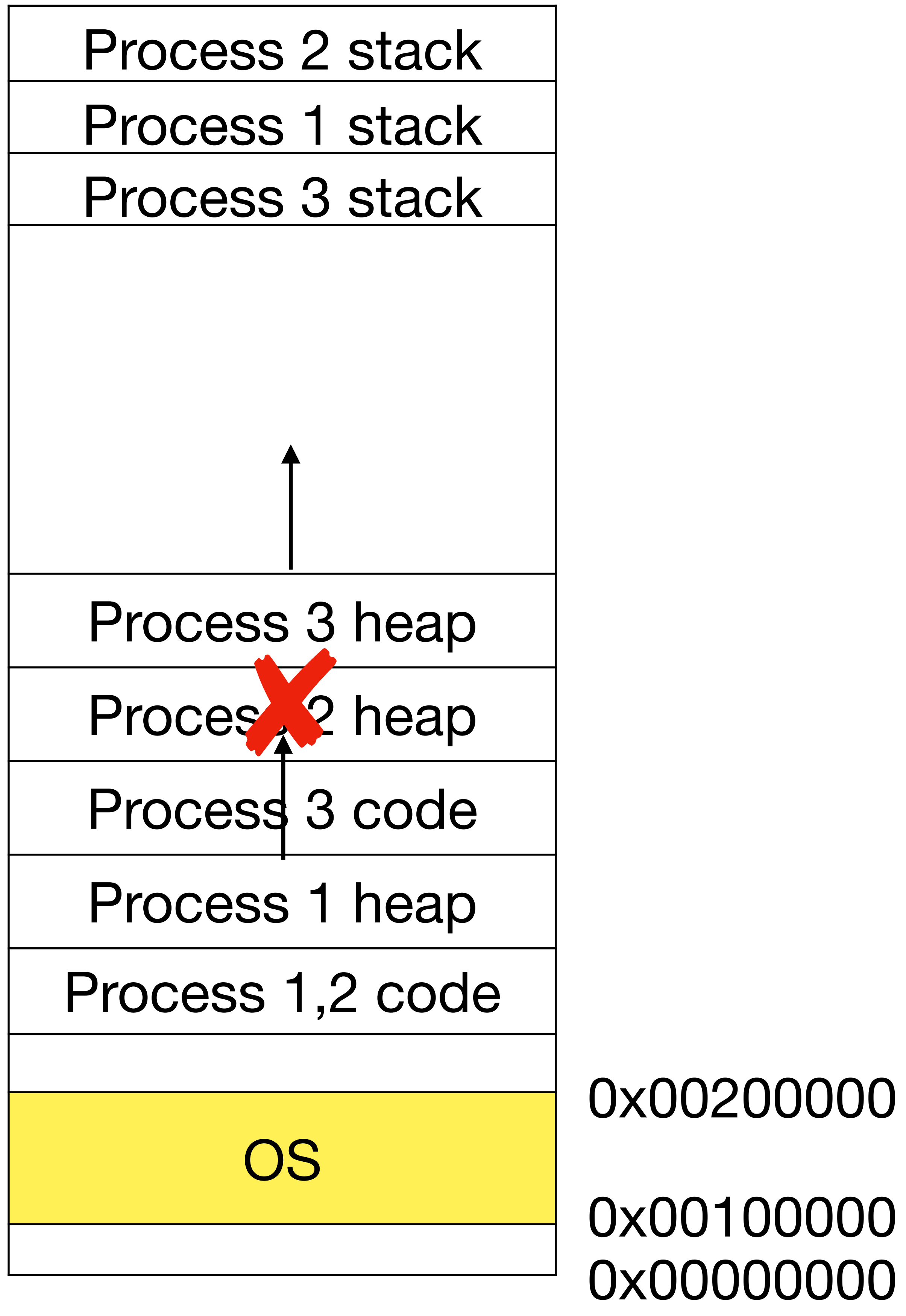


- Mapping large address spaces
  - Place each segment independently to not map free space



Growing and shrinking address space

- Segments need to be contiguous in memory
- Growing might not succeed if there are other segments next to heap segment



Limitations of segmentation

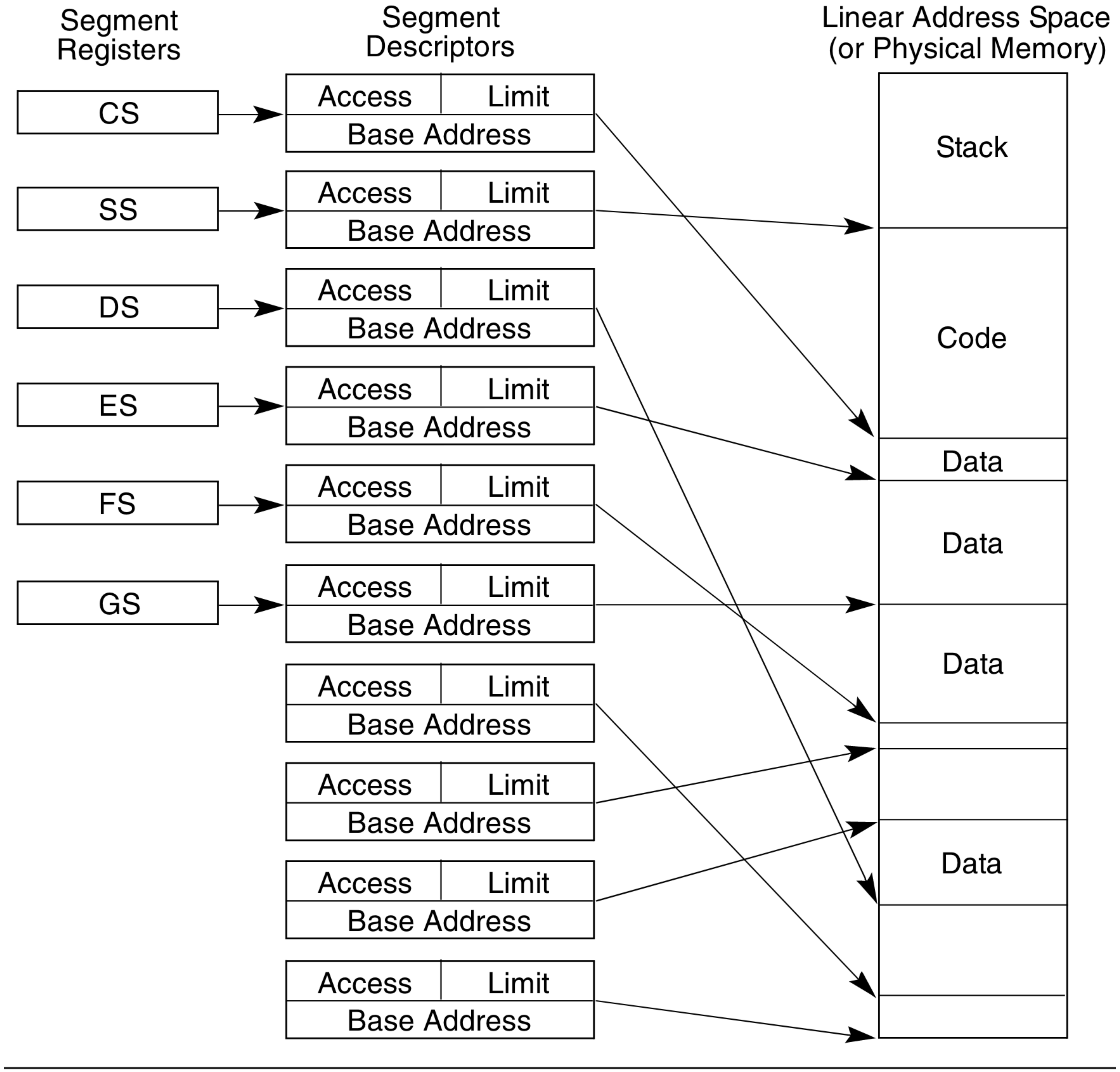


Figure 3-4. Multi-Segment Model

Limitations of segmentation

Limited flexibility:

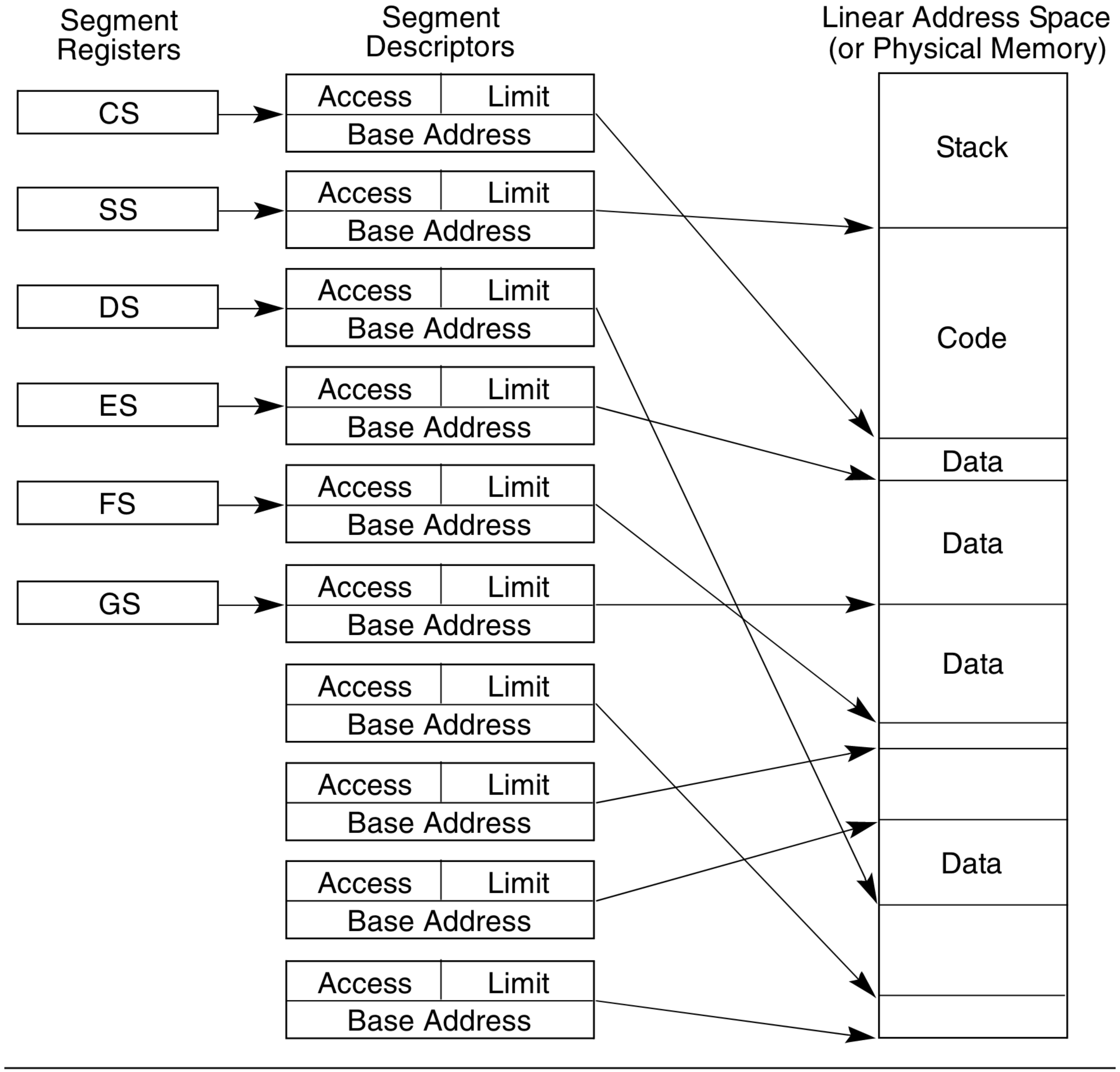


Figure 3-4. Multi-Segment Model

Limitations of segmentation

Limited flexibility:

- To support large address spaces, burden falls on programmer/compiler to manage multiple segments

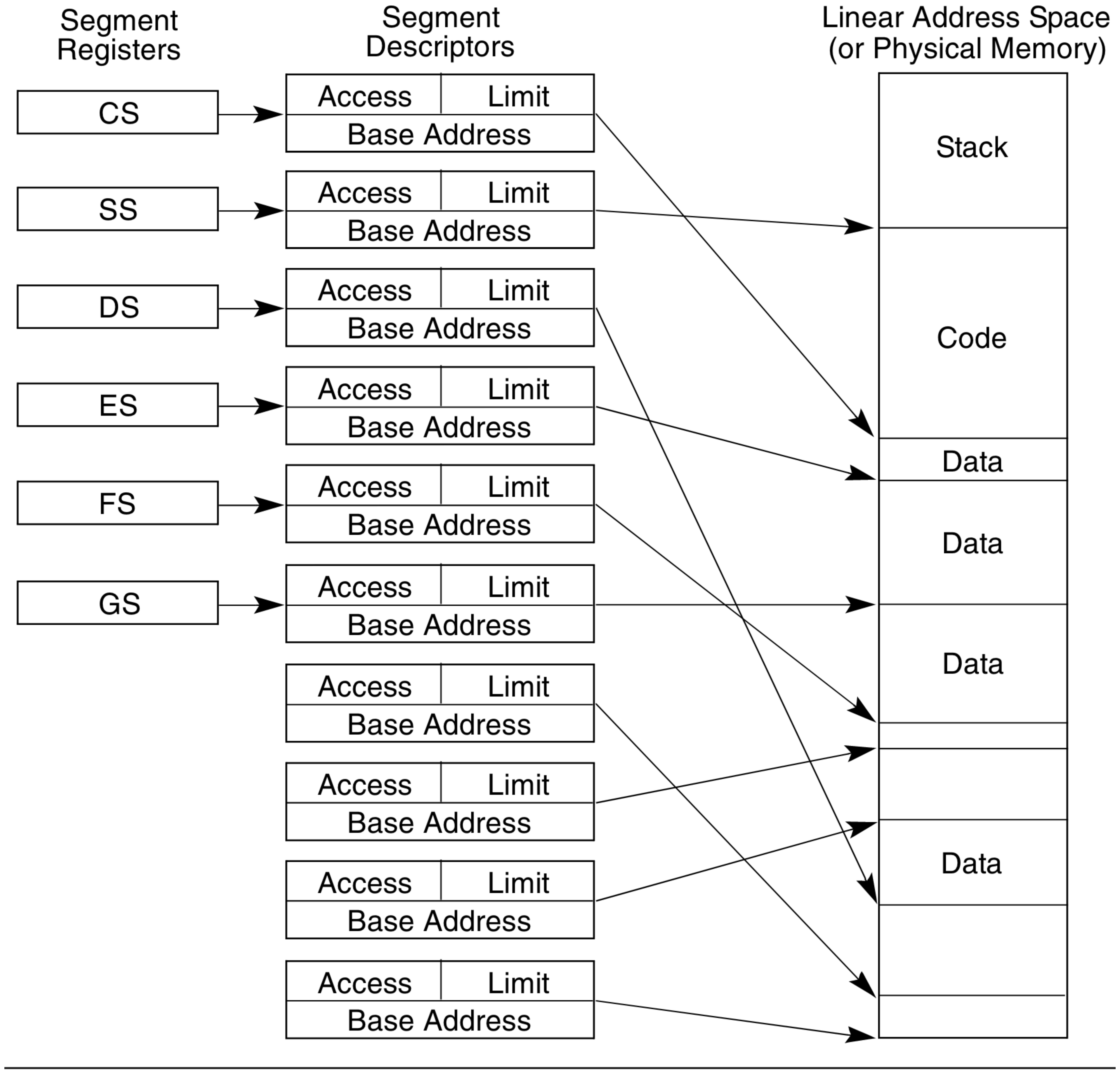


Figure 3-4. Multi-Segment Model

Limitations of segmentation

Limited flexibility:

- To support large address spaces, burden falls on programmer/compiler to manage multiple segments
- Only an entire segment can be shared. Example: cannot share some part of CS (e.g., both processes use same library)

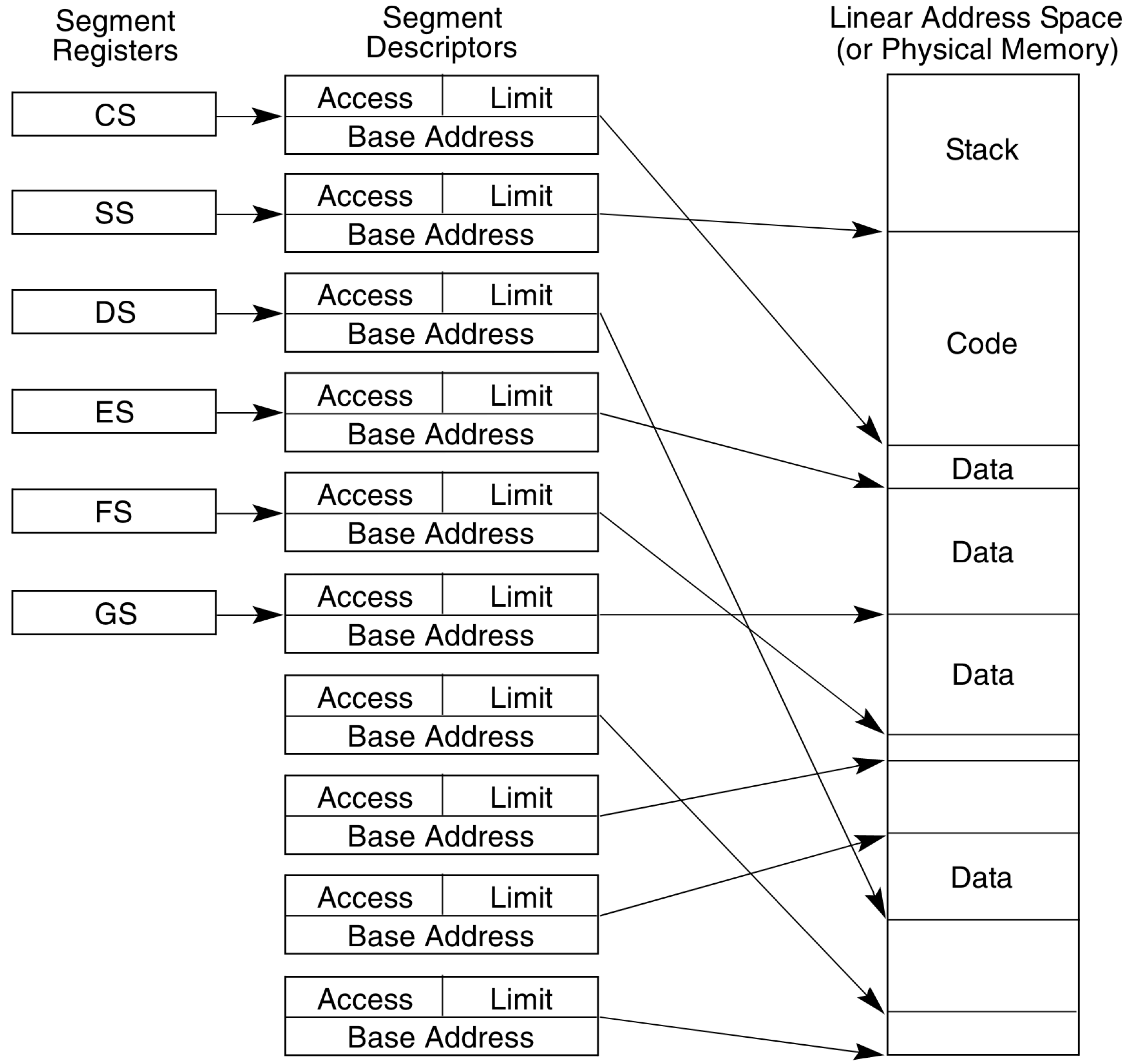


Figure 3-4. Multi-Segment Model

Limitations of segmentation

Limited flexibility:

- To support large address spaces, burden falls on programmer/compiler to manage multiple segments
- Only an entire segment can be shared. Example: cannot share some part of CS (e.g., both processes use same library)

Different sized segments, segments need to be contiguous in physical memory

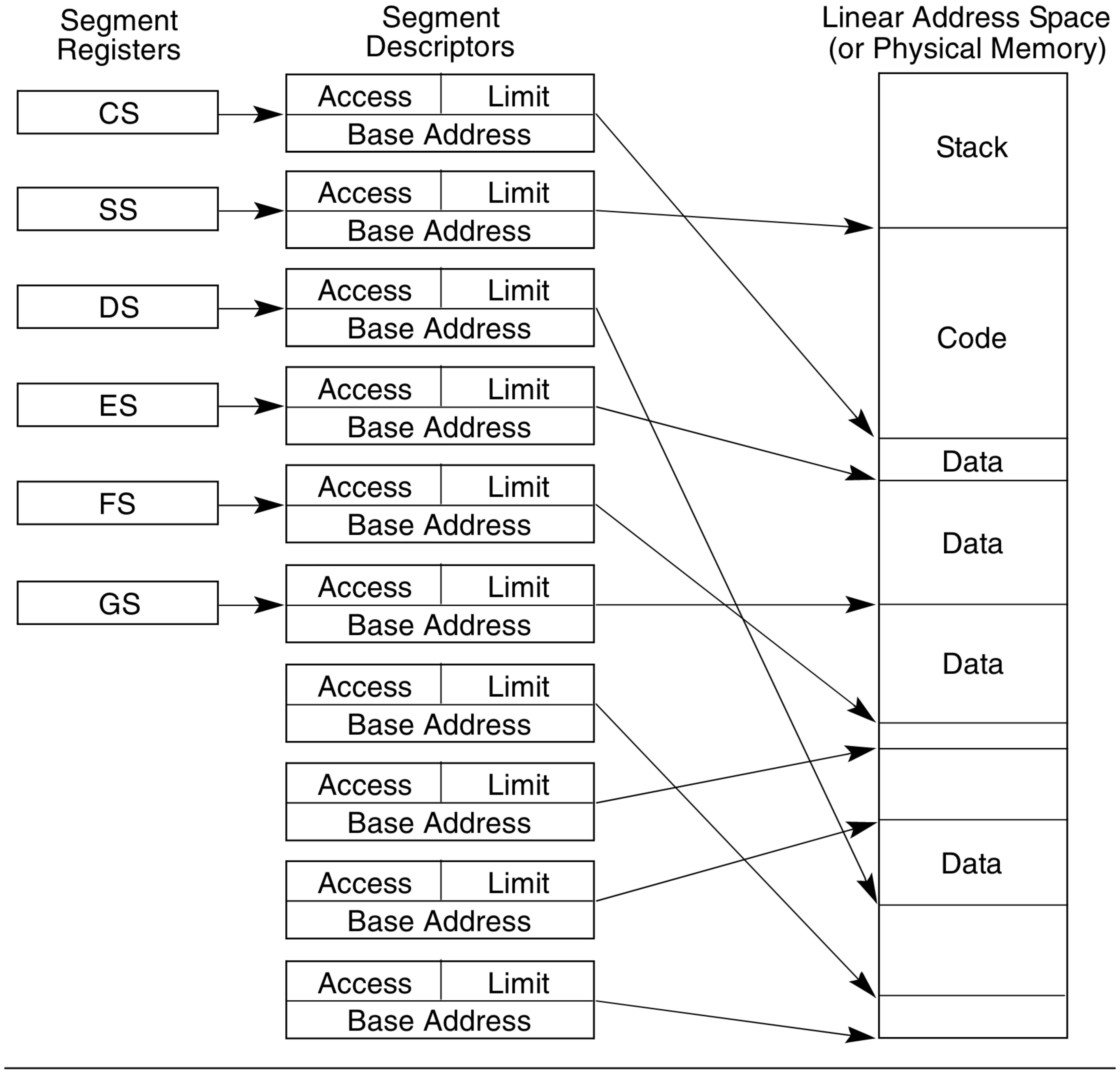


Figure 3-4. Multi-Segment Model

Limitations of segmentation

Limited flexibility:

- To support large address spaces, burden falls on programmer/compiler to manage multiple segments
- Only an entire segment can be shared. Example: cannot share some part of CS (e.g., both processes use same library)

Different sized segments, segments need to be contiguous in physical memory

- complicates physical memory allocator

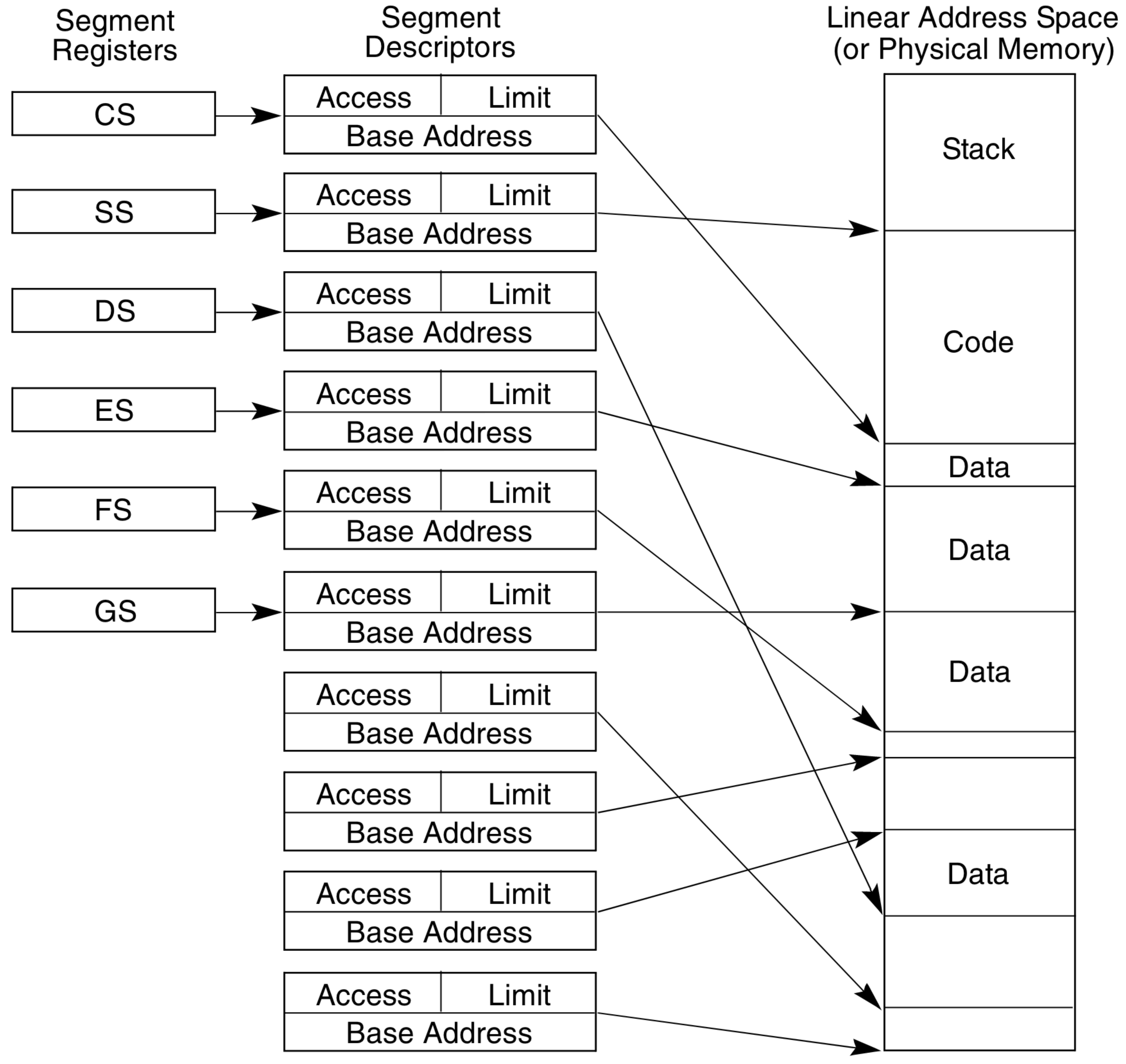


Figure 3-4. Multi-Segment Model

Limitations of segmentation

Limited flexibility:

- To support large address spaces, burden falls on programmer/compiler to manage multiple segments
- Only an entire segment can be shared. Example: cannot share some part of CS (e.g., both processes use same library)

Different sized segments, segments need to be contiguous in physical memory

- complicates physical memory allocator
- lead to external fragmentation

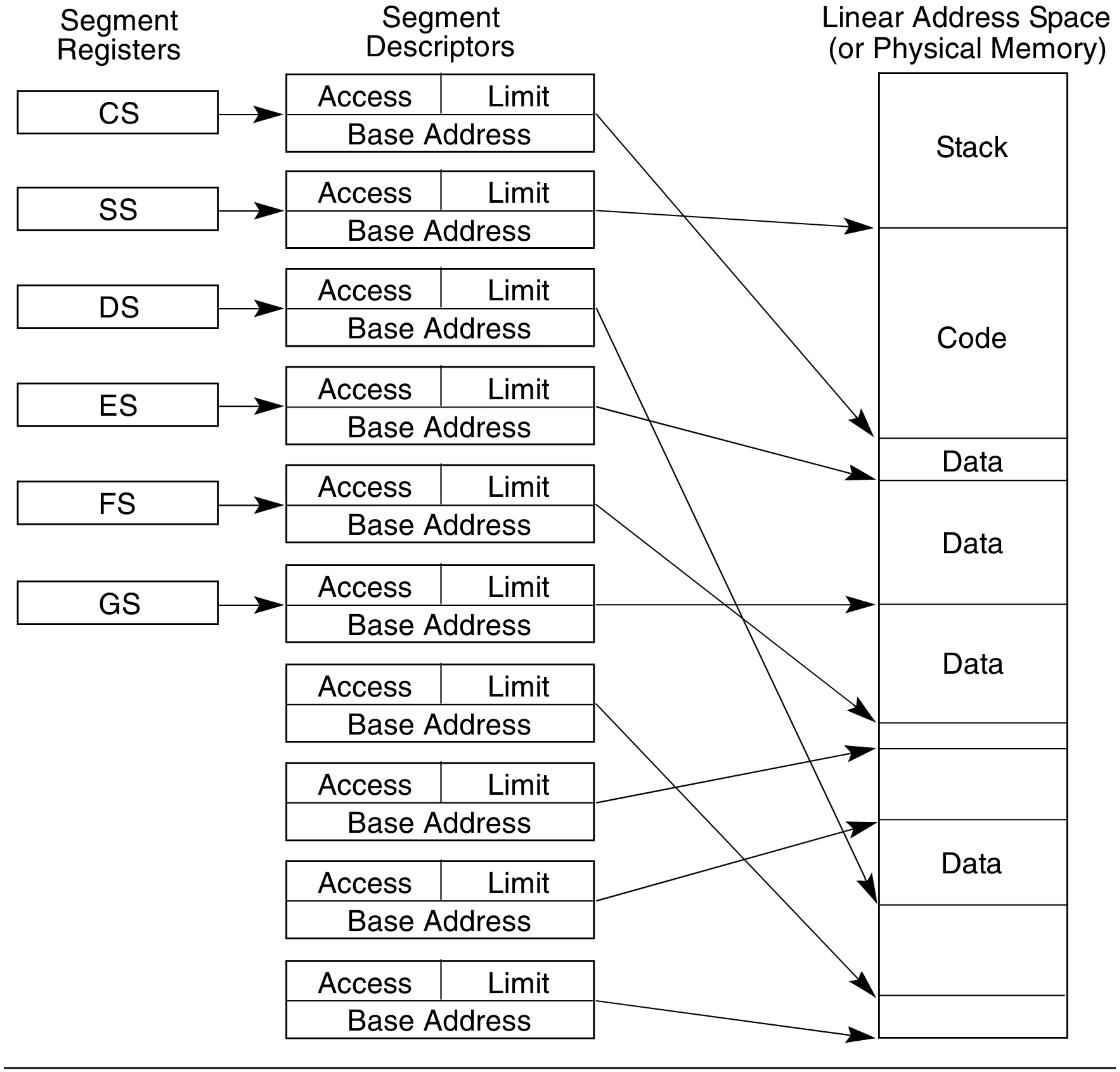


Figure 3-4. Multi-Segment Model

Limitations of segmentation

Limited flexibility:

- To support large address spaces, burden falls on programmer/compiler to manage multiple segments
- Only an entire segment can be shared. Example: cannot share some part of CS (e.g., both processes use same library)

Different sized segments, segments need to be contiguous in physical memory

- complicates physical memory allocator
- lead to external fragmentation
- growing/shrinking segments is awkward

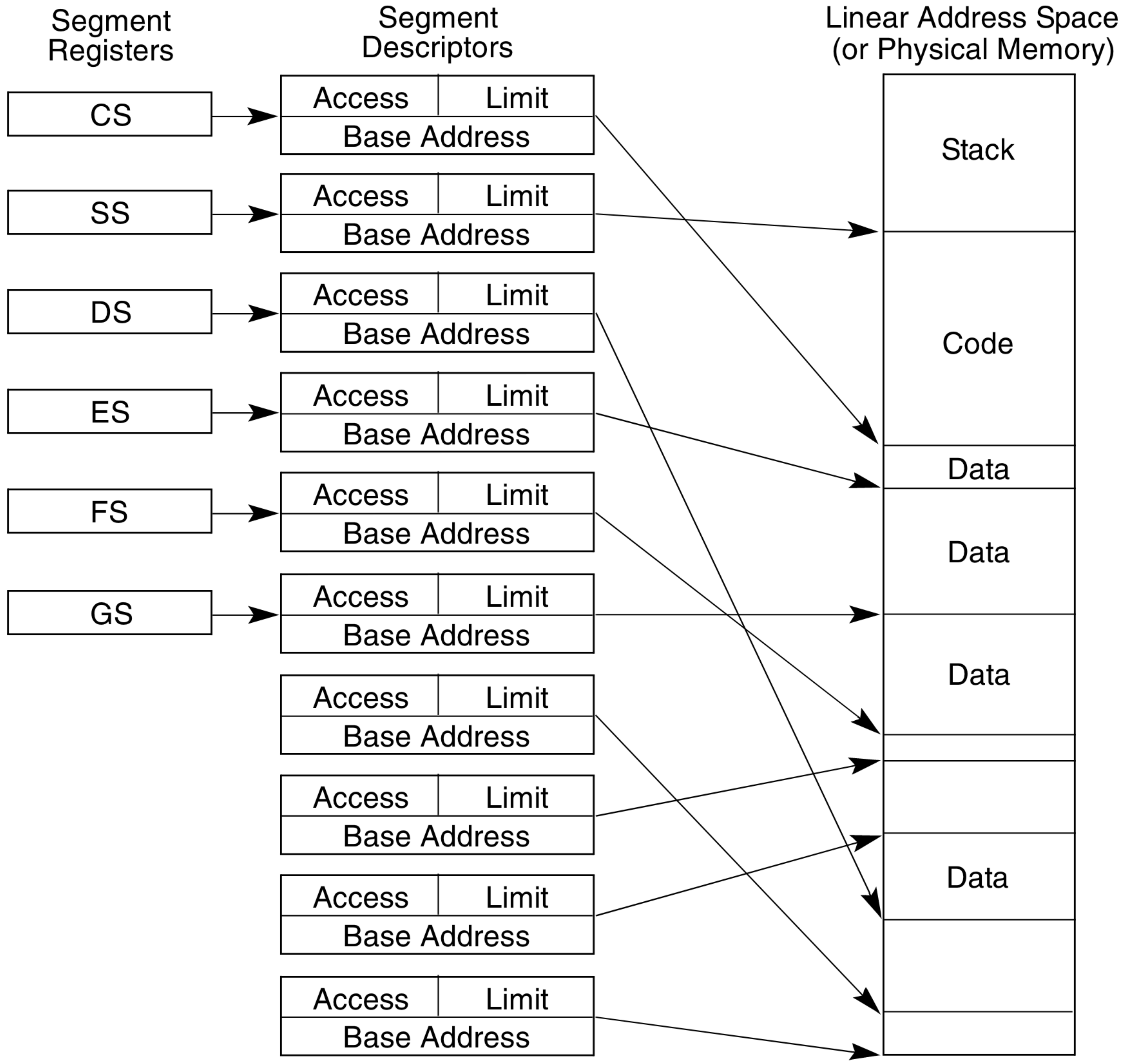


Figure 3-4. Multi-Segment Model

Paging

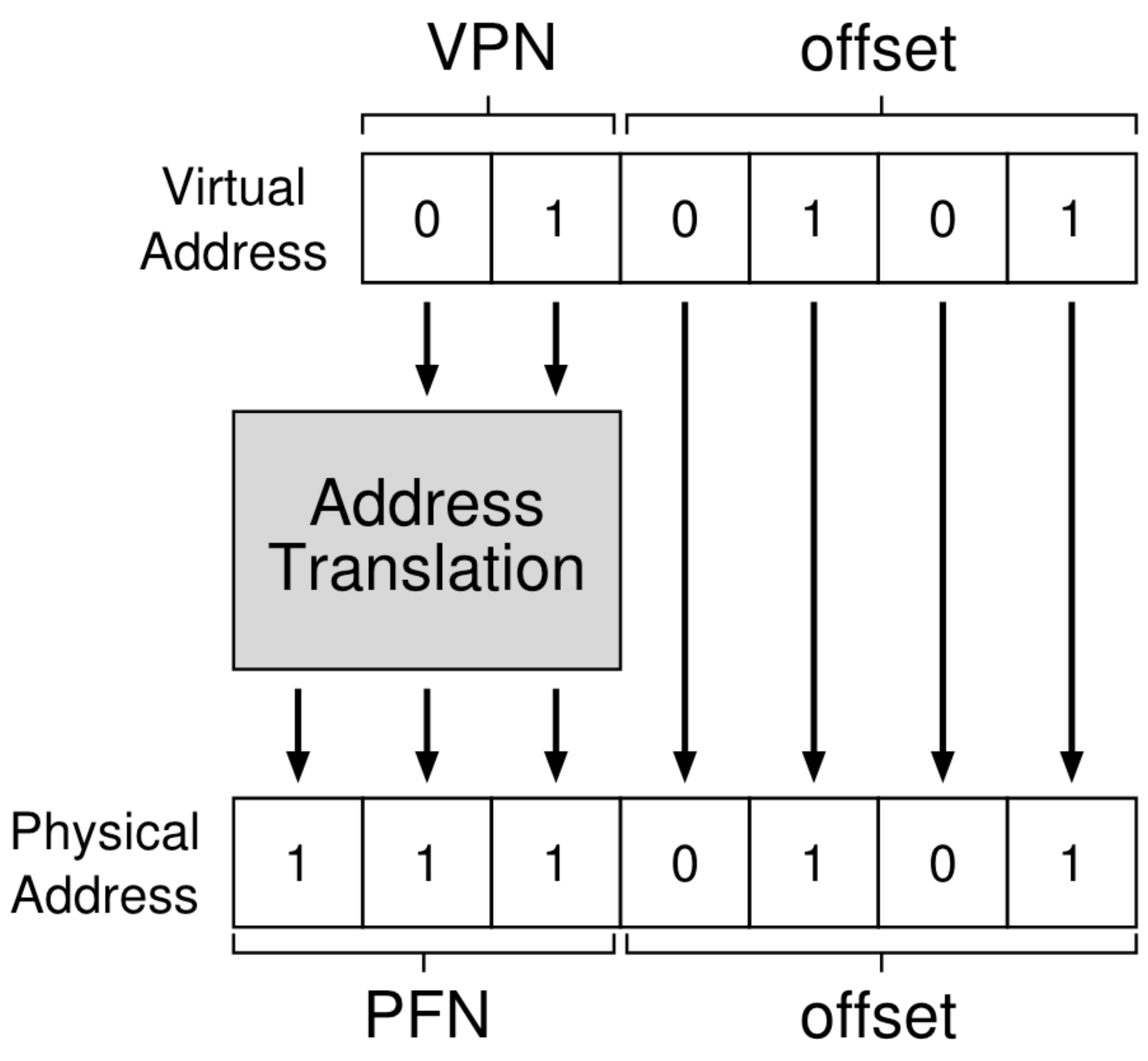
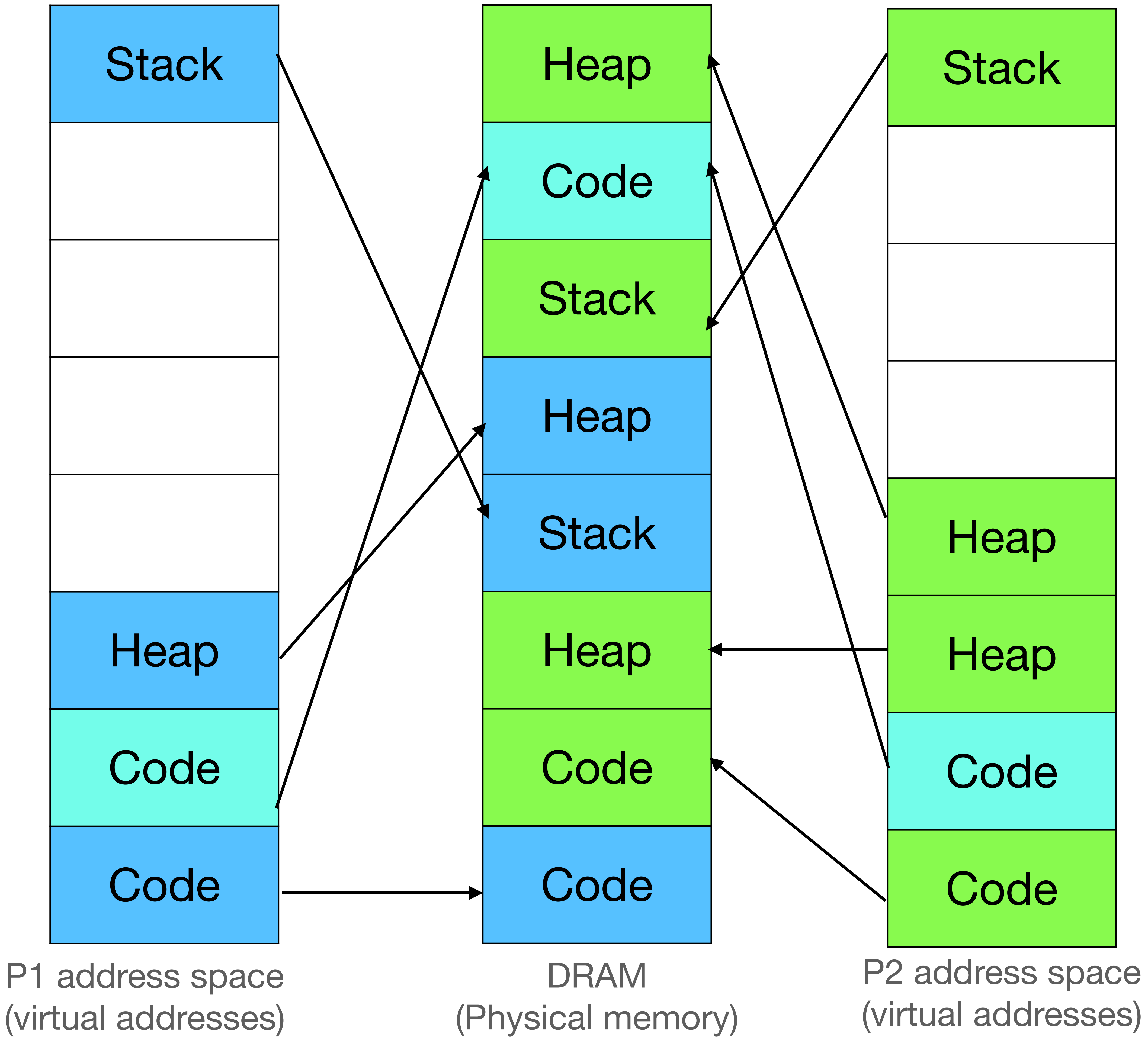


Figure 18.3: **The Address Translation Process**



Notebook analogy

Segmentation

Reading **S**

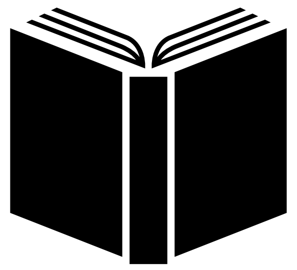
- Read second letter from 3rd page
- Read second letter from 4th page

OS

0	1	2	3	4	5
xv6	is	an	O**S**	for	x86

DB

0	1	2	3
Write	an	SQL	query



Notebook

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
OS:1 DB:7	xv6	is	an	O**S**	for	x86	Write	an	SQL	query				

Notebook analogy

Paging

Reading **S**

- Read second letter from 3rd page

- Read second letter from 8th page

OS

0	1	2	3	4	5
xv6	is	an	O**S**	for	x86

DB

0	1	2	3
Write	an	SQL	query

Notebook

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
OS:1 DB:5	0:3,1:4, 2:2,**3:8**, 4:6,5:10	an	xv6	is	0:9,1:2, 2:7,3:12	for	SQL	O**S**	Write	x86		query		

Paging

Segmentation	Paging
Large address spaces need multi segment model. Burden falls on programmer/compiler to manage multiple segments	Transparently support large address spaces. Programmer/ compiler work with a flat virtual address space

Paging

Segmentation	**Paging**
Large address spaces need multi segment model. Burden falls on programmer/compiler to manage multiple segments	Transparently support large address spaces. Programmer/ compiler work with a flat virtual address space
Different sized segments lead to external fragmentation	Fixed sized pages (4KB, 4MB)

Paging

Segmentation	**Paging**
Large address spaces need multi segment model. Burden falls on programmer/compiler to manage multiple segments	Transparently support large address spaces. Programmer/compiler work with a flat virtual address space
Different sized segments lead to external fragmentation	Fixed sized pages (4KB, 4MB)
Different sized segments complicate physical memory allocator	

Paging

Segmentation	**Paging**
Large address spaces need multi segment model. Burden falls on programmer/compiler to manage multiple segments	Transparently support large address spaces. Programmer/compiler work with a flat virtual address space
Different sized segments lead to external fragmentation	Fixed sized pages (4KB, 4MB)
Different sized segments complicate physical memory allocator	
Full segment needs to be contiguous in physical memory	Page is contiguous. Neighbouring addresses (in different pages) may not be contiguous

Paging

Segmentation	**Paging**
Large address spaces need multi segment model. Burden falls on programmer/compiler to manage multiple segments	Transparently support large address spaces. Programmer/ compiler work with a flat virtual address space
Different sized segments lead to external fragmentation	Fixed sized pages (4KB, 4MB)
Different sized segments complicate physical memory allocator	
Full segment needs to be contiguous in physical memory	Page is contiguous. Neighbouring addresses (in different pages) may not be contiguous
Growing/shrinking segments is awkward	Simple: allocate another page, free a page

Paging

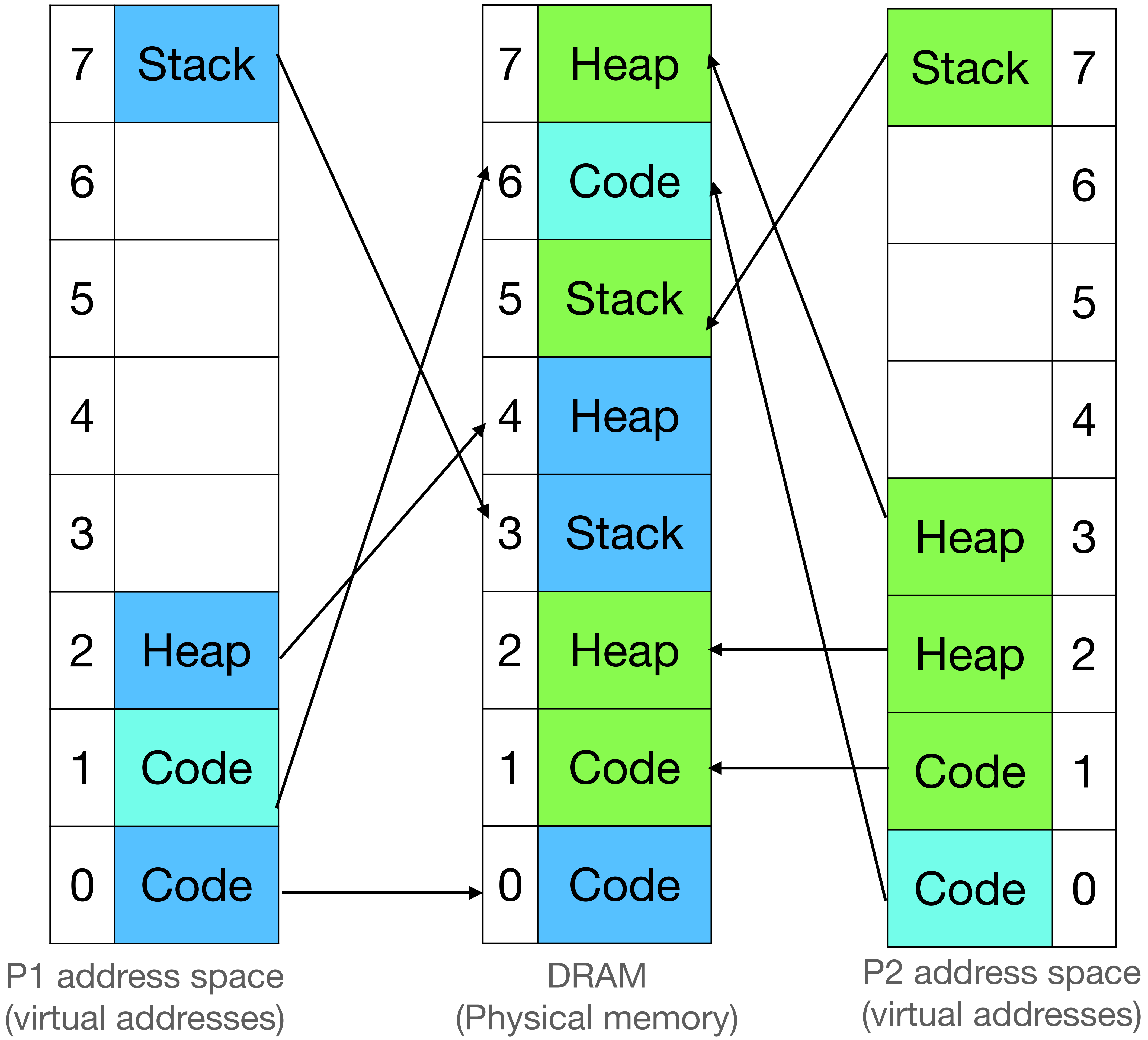
Segmentation	**Paging**
Large address spaces need multi segment model. Burden falls on programmer/compiler to manage multiple segments	Transparently support large address spaces. Programmer/ compiler work with a flat virtual address space
Different sized segments lead to external fragmentation	Fixed sized pages (4KB, 4MB)
Different sized segments complicate physical memory allocator	
Full segment needs to be contiguous in physical memory	Page is contiguous. Neighbouring addresses (in different pages) may not be contiguous
Growing/shrinking segments is awkward	Simple: allocate another page, free a page
Address translation hardware has an adder (va + base) and a comparator (va < limit)	Address translation hardware is much more complicated

How to do page-based address translation?

Page table: Maintain a lookup table for each process

Virtual page number	Physical page number
7	3
6	x
5	x
4	x
3	x
2	4
1	6
0	0

Virtual page number	Physical page number
7	5
6	x
5	x
4	x
3	7
2	2
1	1
0	6

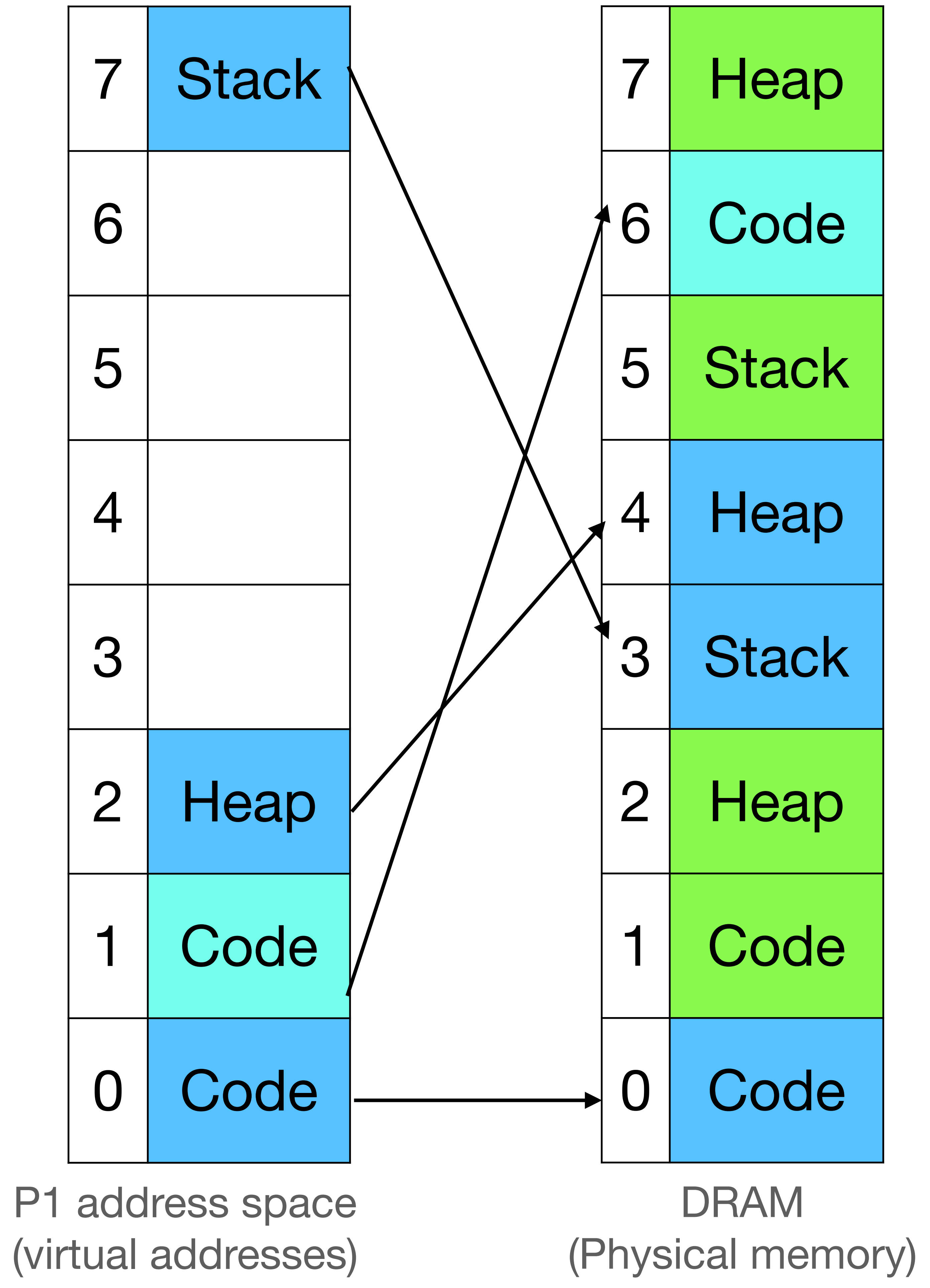


How to do page-based address translation? (2)

Page table bits

- Present bit: valid mapping
- Permission bits: read only, writeable, executable

Virtual page number	Physical page number	Present	Permissions
7	3	Y	rw
6	x	N	
5	x	N	
4	x	N	
3	x	N	
2	4	Y	rw
1	6	Y	rx
0	0	Y	rx



Page table size

- Virtual addresses: 2^{32}

Virtual page number	Physical page number
7	3
6	x
5	x
4	x
3	x
2	4
1	6
0	0

Page table size

- Virtual addresses: 2^{32}
- Size of page = (4KB) = 2^{12}

Virtual page number	Physical page number
7	3
6	x
5	x
4	x
3	x
2	4
1	6
0	0

Page table size

- Virtual addresses: 2^{32}
- Size of page = (4KB) = 2^{12}
- Number of page table entries = $2^{32} / 2^{12} = 2^{20}$

Virtual page number	Physical page number
7	3
6	x
5	x
4	x
3	x
2	4
1	6
0	0

Page table size

- Virtual addresses: 2^{32}
- Size of page = (4KB) = 2^{12}
- Number of page table entries = $2^{32} / 2^{12} = 2^{20}$
- Number of pages in a 4GB DRAM = $2^{32} / 2^{12} = 2^{20}$

Virtual page number	Physical page number
7	3
6	x
5	x
4	x
3	x
2	4
1	6
0	0

Page table size

- Virtual addresses: 2^{32}
- Size of page = (4KB) = 2^{12}
- Number of page table entries = $2^{32} / 2^{12} = 2^{20}$
- Number of pages in a 4GB DRAM = $2^{32} / 2^{12} = 2^{20}$
- Size of each page table entry = 20 bits + page table bits ~ 3 bytes

Virtual page number	Physical page number
7	3
6	x
5	x
4	x
3	x
2	4
1	6
0	0

Page table size

- Virtual addresses: 2^{32}
- Size of page = (4KB) = 2^{12}
- Number of page table entries = $2^{32} / 2^{12} = 2^{20}$
- Number of pages in a 4GB DRAM = $2^{32} / 2^{12} = 2^{20}$
- Size of each page table entry =
  20 bits + page table bits ~ 3 bytes
- Size of page table = $3*2^{20}$ = 3MB!

Virtual page number	Physical page number
7	3
6	x
5	x
4	x
3	x
2	4
1	6
0	0

Page table size

- Virtual addresses: 2^{32}
- Size of page = (4KB) = 2^{12}
- Number of page table entries = $2^{32} / 2^{12} = 2^{20}$
- Number of pages in a 4GB DRAM = $2^{32} / 2^{12} = 2^{20}$
- Size of each page table entry = 20 bits + page table bits ~ 3 bytes
- Size of page table = $3*2^{20}$ = 3MB!
- 1024 processes => ~3GB memory!

Virtual page number	Physical page number
7	3
6	x
5	x
4	x
3	x
2	4
1	6
0	0

Reducing page table size

- Bigger pages
    - Virtual addresses: 2^{32}
    - Size of page = (**4MB**) = 2^{22}

Reducing page table size

- Bigger pages
  - Virtual addresses: 2^{32}
  - Size of page = (**4MB**) = 2^{22}
  - Number of page table entries = $2^{32} / 2^{22} = 2^{10}$

Reducing page table size

- Bigger pages
  - Virtual addresses: 2^{32}
  - Size of page = (**4MB**) = 2^{22}
  - Number of page table entries = $2^{32} / 2^{22} = 2^{10}$
  - Number of pages in a 4GB DRAM = $2^{32} / 2^{22} = 2^{10}$

Reducing page table size

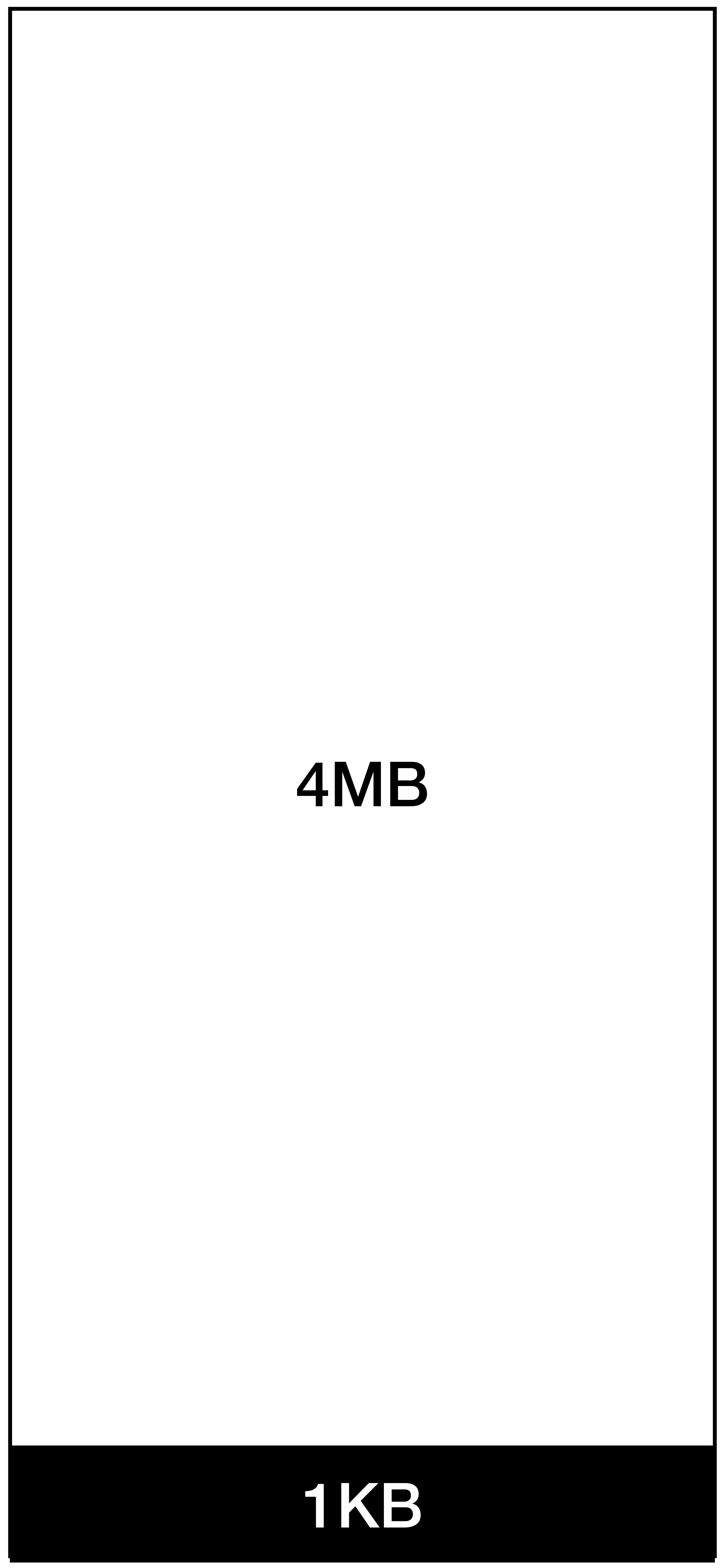
- Bigger pages
  - Virtual addresses: 2^{32}
  - Size of page = (**4MB**) = 2^{22}
  - Number of page table entries = 2^{32} / 2^{22} = 2^{10}
  - Number of pages in a 4GB DRAM = 2^{32} / 2^{22} = 2^{10}
  - Size of each page table entry = 10 bits ~ 2 bytes

Reducing page table size

- Bigger pages
  - Virtual addresses: 2^{32}
  - Size of page = (**4MB**) = 2^{22}
  - Number of page table entries = $2^{32} / 2^{22} = 2^{10}$
  - Number of pages in a 4GB DRAM = $2^{32} / 2^{22} = 2^{10}$
  - Size of each page table entry = 10 bits ~ 2 bytes
  - Size of page table = $2 * 2^{10}$ = 2KB!

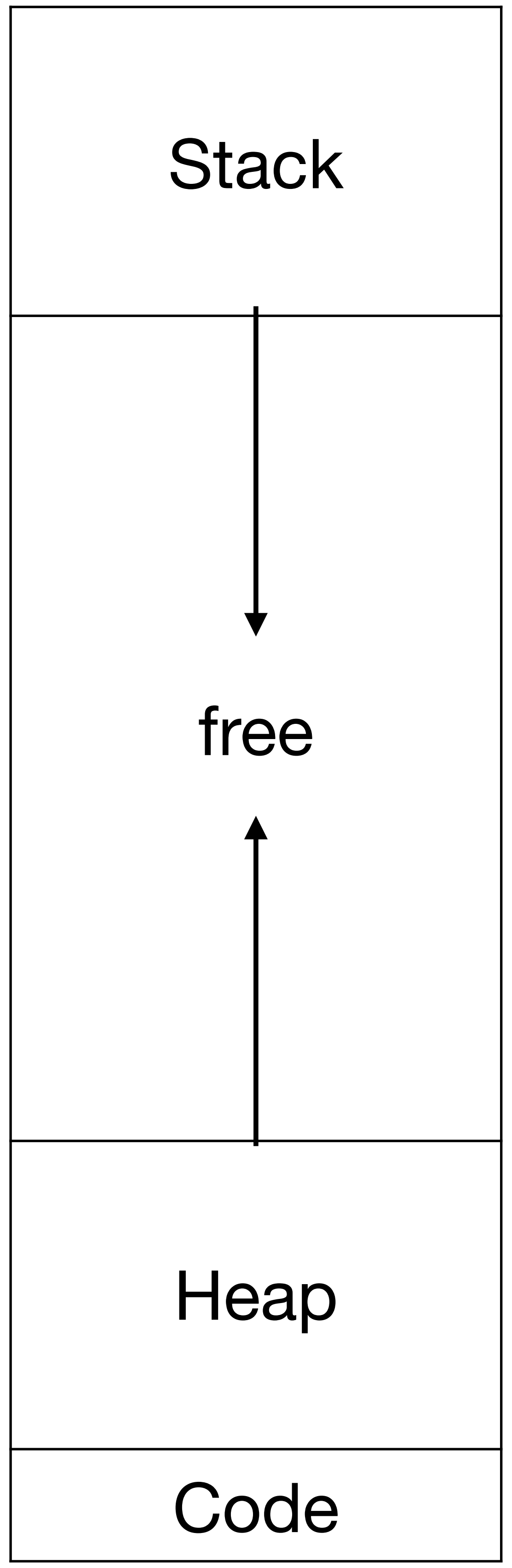
Reducing page table size

- Bigger pages
  - Virtual addresses: 2^{32}
  - Size of page = (**4MB**) = 2^{22}
  - Number of page table entries = 2^{32} / 2^{22} = 2^{10}
  - Number of pages in a 4GB DRAM = 2^{32} / 2^{22} = 2^{10}
  - Size of each page table entry = 10 bits ~ 2 bytes
  - Size of page table = 2 * 2^{10} = 2KB!
- Bigger pages increase internal fragmentation



Observation

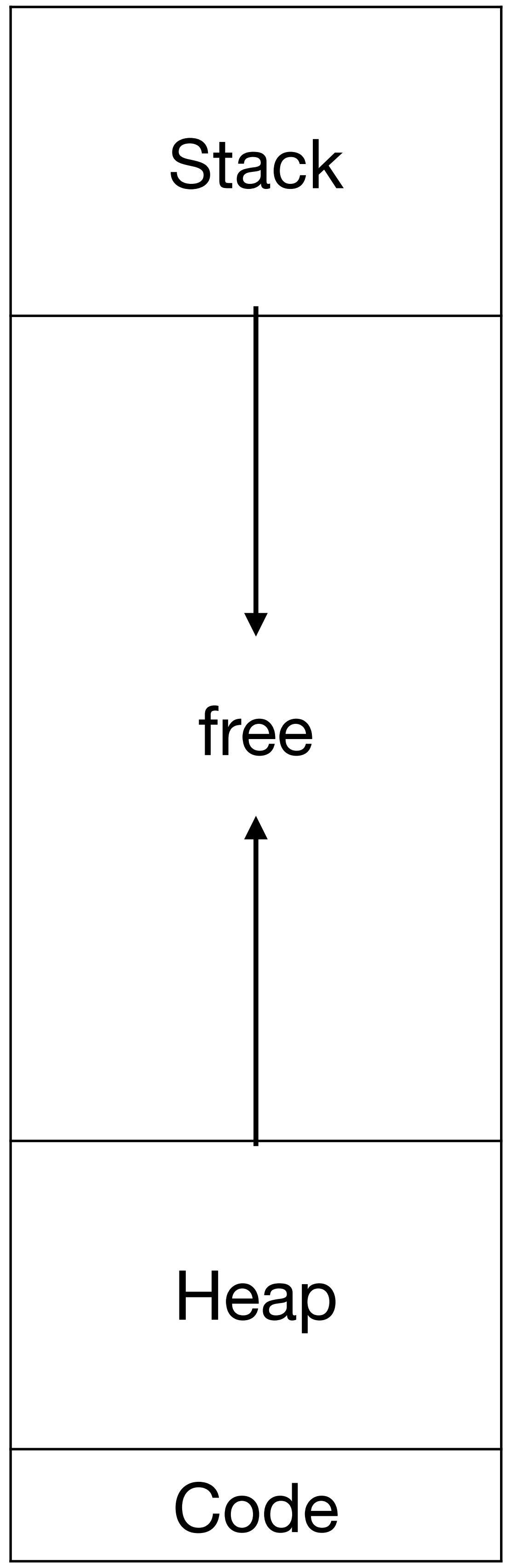
- Lot of page table entries are invalid



Virtual page number	Physical page number
7	3
6	x
5	x
4	x
3	x
2	4
1	6
0	0

Observation

- Lot of page table entries are invalid



Process 1 address space

Virtual page number	Physical page number
7	3
6	x
5	x
4	x
3	x
2	4
1	6
0	0

Multi-level page table

Virtual page number	Physical page number
9	3
8	1
7	x
6	x
5	x
4	x
3	x
2	x
1	6
0	0

PPN: 8

Virtual page number	Physical page number
8,9	9
6,7	x
4,5	x
2,3	x
0,1	10

PPN: 9

Virtual page number	Physical page number
9	3
8	1

PPN: 10

Virtual page number	Physical page number
1	6
0	0

Multi-level page table

Virtual page number	Physical page number
9	3
8	1
7	x
6	x
5	x
4	x
3	x
2	x
1	6
0	0

PPN: 8

Virtual page number	Physical page number
8,9	9
6,7	x
4,5	x
2,3	x
0,1	10

PPN: 9

Virtual page number	Physical page number
9	3
8	1

PPN: 10

Virtual page number	Physical page number
1	6
0	0

- Page directory entries point to page table pages
- Unused portions of virtual address space are skipped!

Notebook analogy

Paging

Reading **S**

- Read second letter from 3rd page
- Read second letter from 8th page

OS

0	1	2	3	4	5
xv6	is	an	O**S**	for	x86

DB

0	1	2	3
Write	an	SQL	query

Notebook

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
OS:1 DB:5	0:3,1:4, 2:2,**3:8**, 4:6,5:10	an	xv6	is	0:9,1:2, 2:7,3:12	for	SQL	O**S**	Write	x86		query		

Notebook analogy

Page directories: call 4 pages a "section"

Reading **S**

- Read second letter from 3rd page in **Section 0**

- Read second letter from 8th page

OS

0	1	2	3	0	1	2	3
xv6	is	an	O**S**	for	x86		
Section 0				Section 1			

DB

0	1	2	3
Write	an	SQL	query
Section 0			

Notebook

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
OS:1 DB:5	**0:11**, 1:14	an	xv6	is	0:13	for	SQL	O**S**	Write	x86	0:3,1:4, 2:2,**3:8**	query	0:9,1:2, 2:7,3:12	0:6, 1:10

Address translation

Simple address space

- 16KB address space has 2^{14} addresses
- Each page has 64 (= 2^6) bytes
- Number of pages = 2^{14} / 2^6 = 2^8
- First 8 bits are page number, last 6 bits are offset within the page

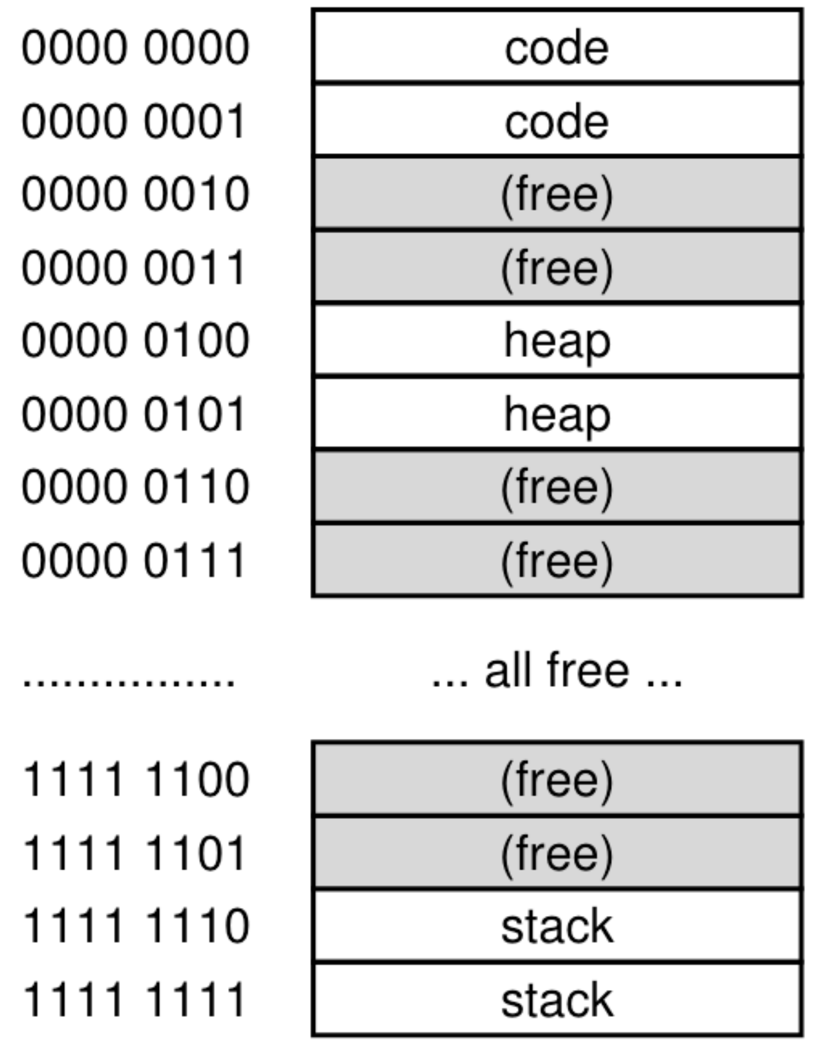
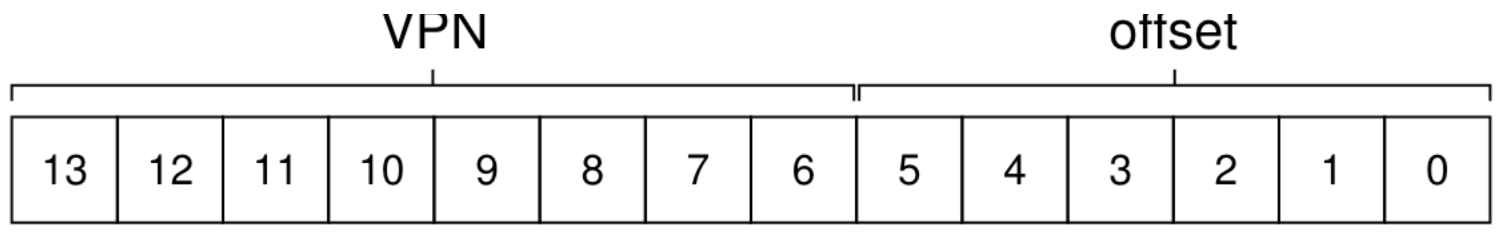
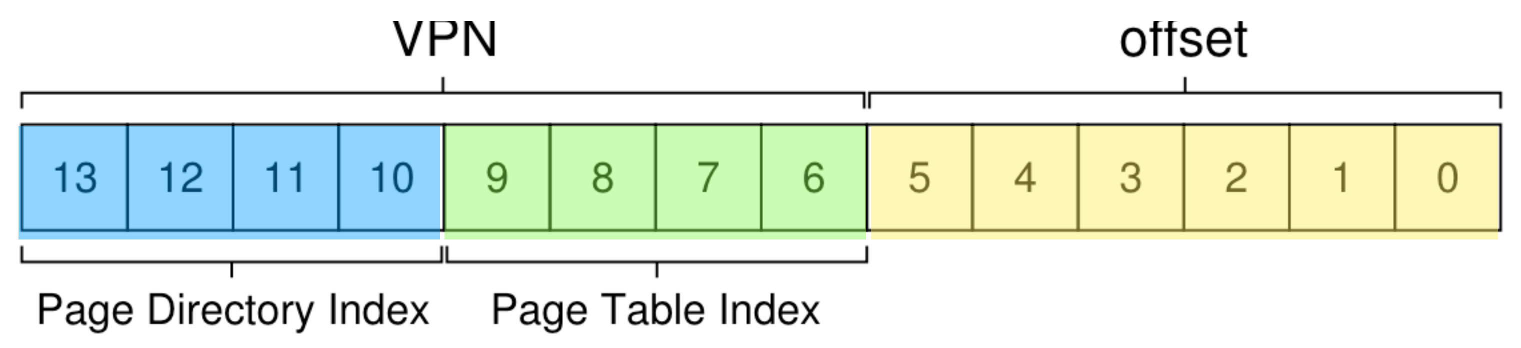


Figure 20.4: **A 16KB Address Space With 64-byte Pages**



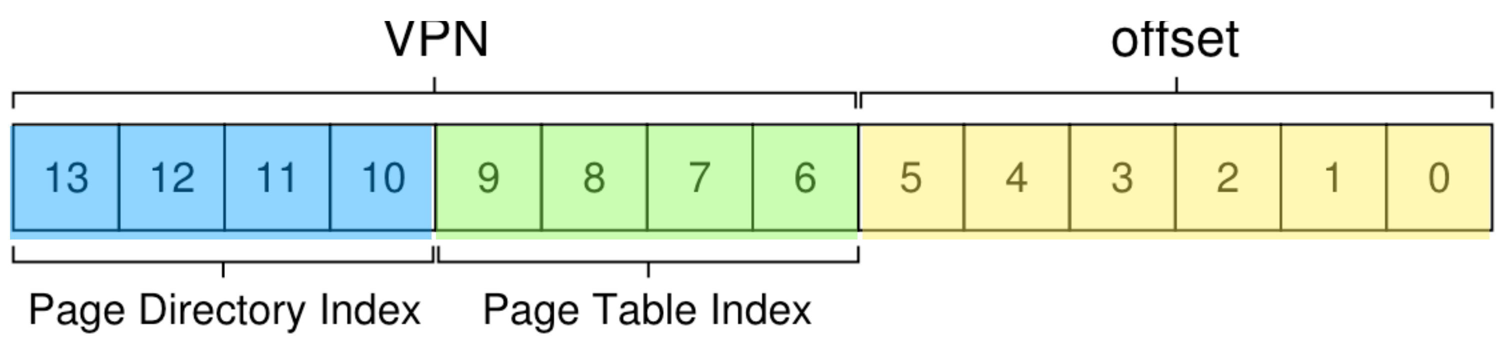
Address translation

Simple address space



Address translation

Simple address space

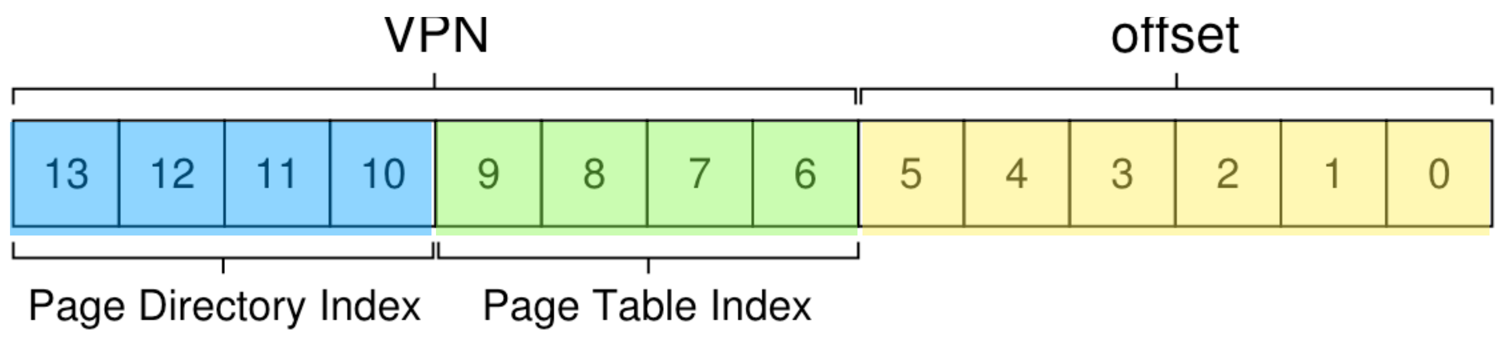


Page Directory		Page of PT (@PFN:100)			Page of PT (@PFN:101)		
PFN	valid?	PFN	valid	prot	PFN	valid	prot
100	1	10	1	r-x	—	0	—
—	0	23	1	r-x	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	80	1	rw-	—	0	—
—	0	59	1	rw-	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	55	1	rw-
101	1	—	0	—	45	1	rw-

Figure 20.5: **A Page Directory, And Pieces Of Page Table**

Address translation

Simple address space



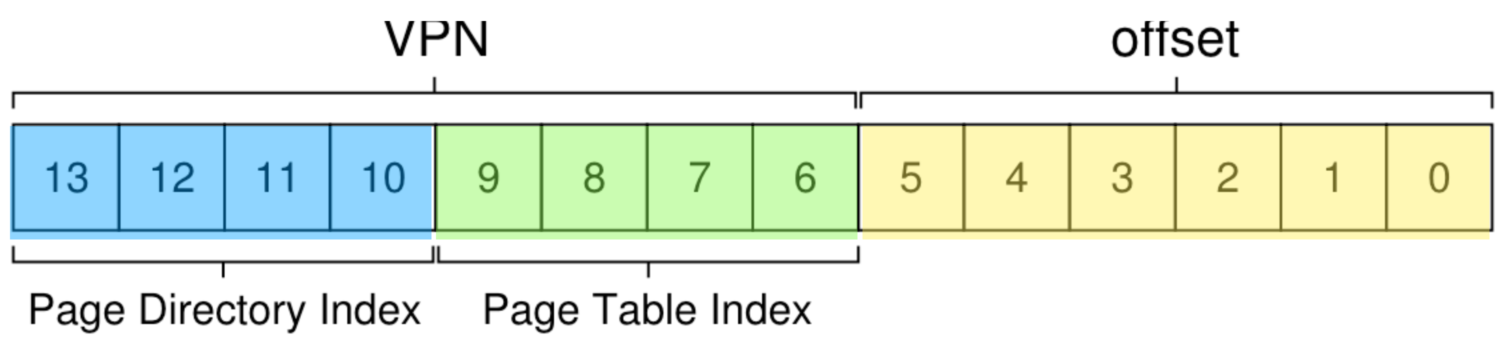
- Example: 0x3F81 (VA) => 0x0DC1 (PA)

Page Directory		Page of PT (@PFN:100)			Page of PT (@PFN:101)		
PFN	valid?	PFN	valid	prot	PFN	valid	prot
100	1	10	1	r-x	—	0	—
—	0	23	1	r-x	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	80	1	rw-	—	0	—
—	0	59	1	rw-	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	55	1	rw-
101	1	—	0	—	45	1	rw-

Figure 20.5: **A Page Directory, And Pieces Of Page Table**

Address translation

Simple address space



- Example: 0x3F81 (VA) => 0x0DC1 (PA)

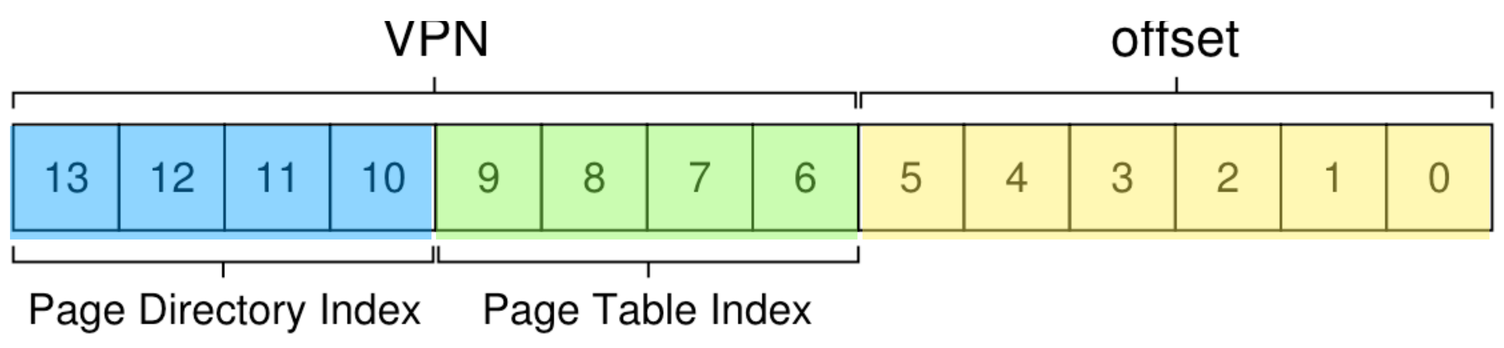
Page dir idx 15				Page table idx 14				Offset					
1	1	1	1	1	1	1	0	0	0	0	0	0	1
3		F				8				1			

Page Directory		Page of PT (@PFN:100)			Page of PT (@PFN:101)		
PFN	valid?	PFN	valid	prot	PFN	valid	prot
100	1	10	1	r-x	—	0	—
—	0	23	1	r-x	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	80	1	rw-	—	0	—
—	0	59	1	rw-	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	55	1	rw-
101	1	—	0	—	45	1	rw-

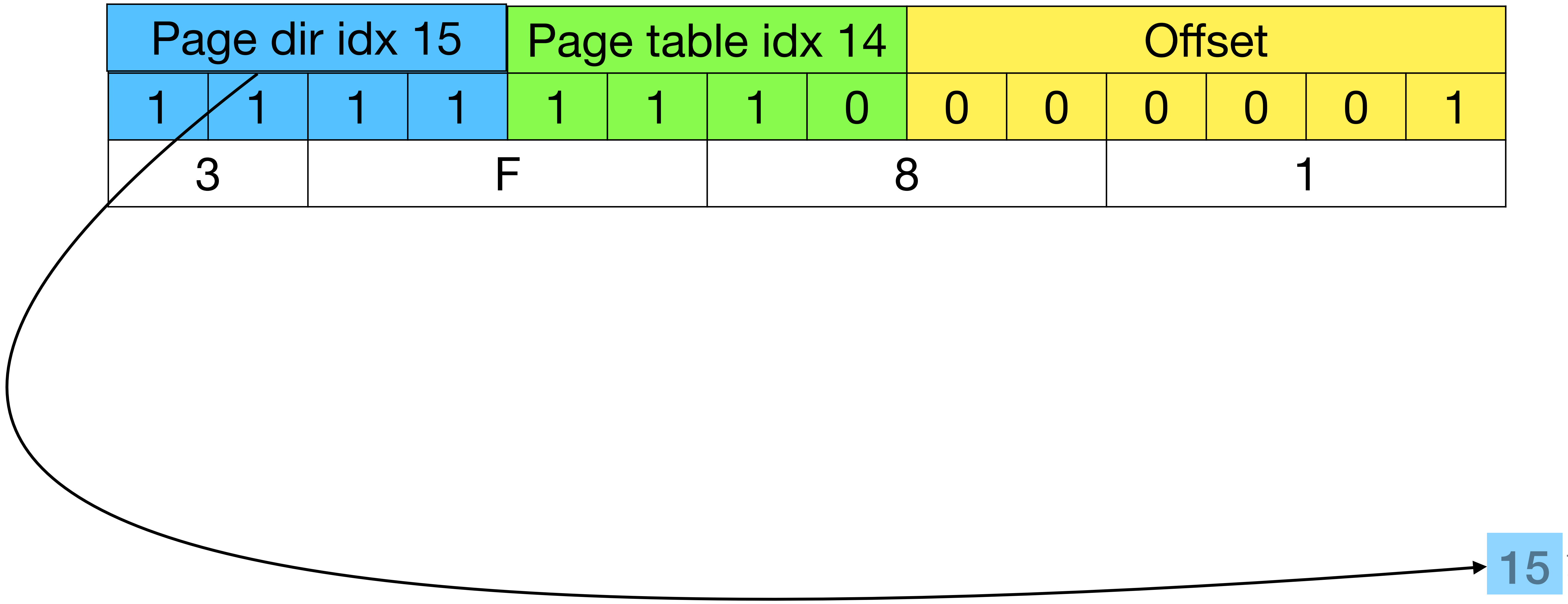
Figure 20.5: **A Page Directory, And Pieces Of Page Table**

Address translation

Simple address space



- Example: 0x3F81 (VA) => 0x0DC1 (PA)

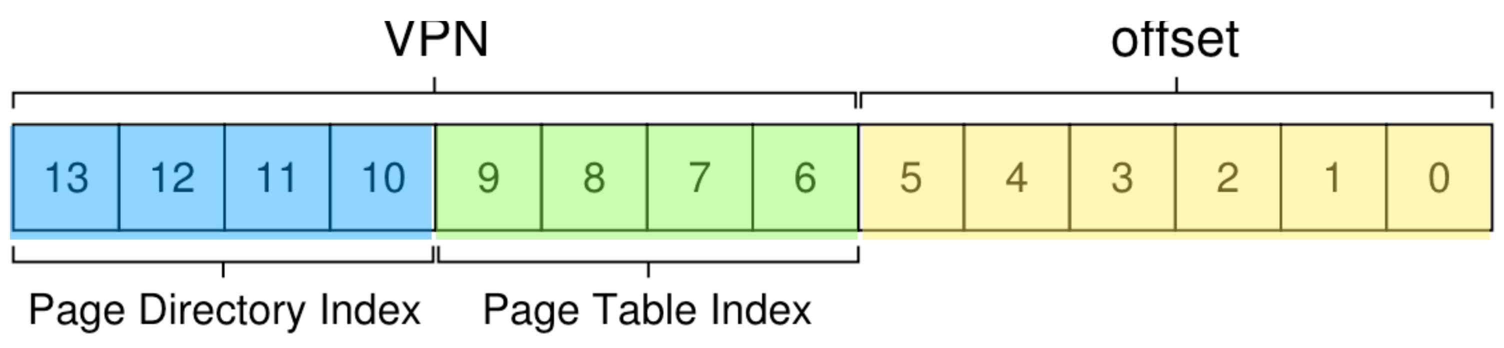


Page Directory PFN	valid?	Page of PT (@PFN:100) PFN	valid	prot	Page of PT (@PFN:101) PFN	valid	prot
100	1	10	1	r-x	—	0	—
—	0	23	1	r-x	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	80	1	rw-	—	0	—
—	0	59	1	rw-	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	55	1	rw-
101	1	—	0	—	45	1	rw-

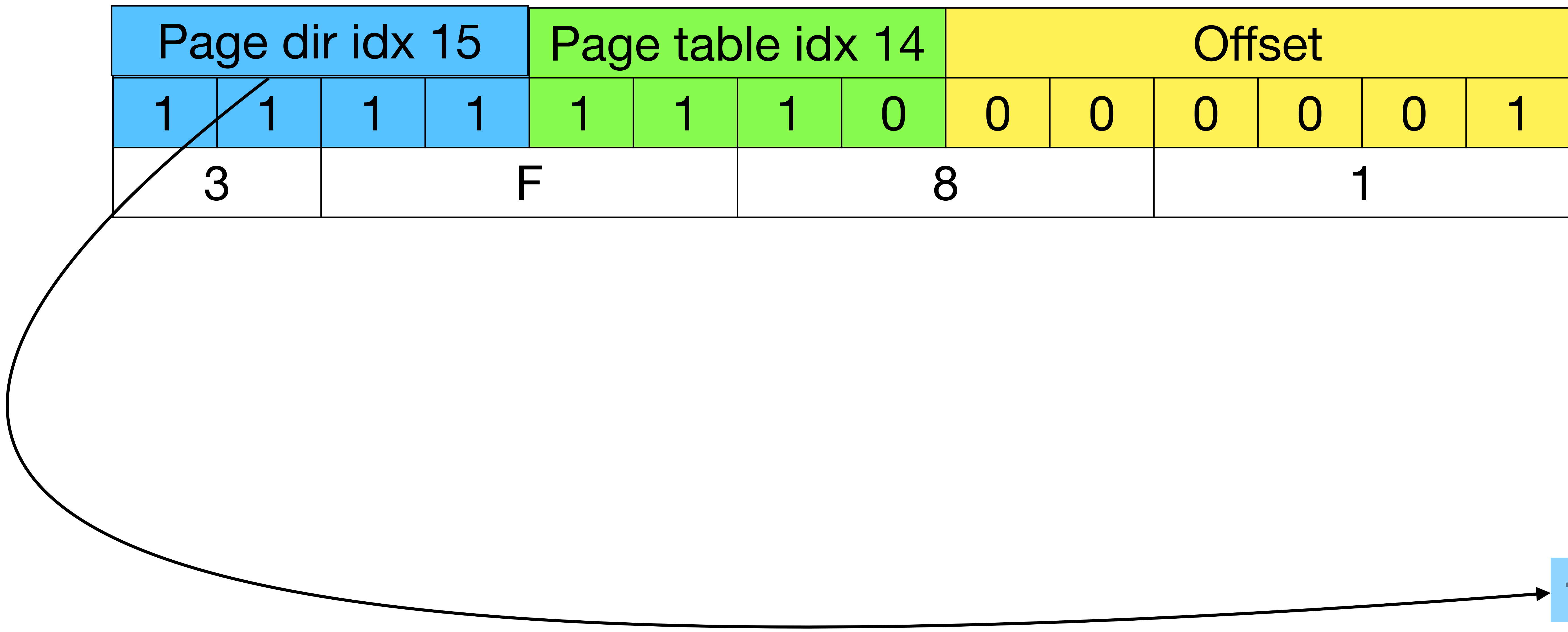
Figure 20.5: **A Page Directory, And Pieces Of Page Table**

Address translation

Simple address space



- Example: 0x3F81 (VA) => 0x0DC1 (PA)

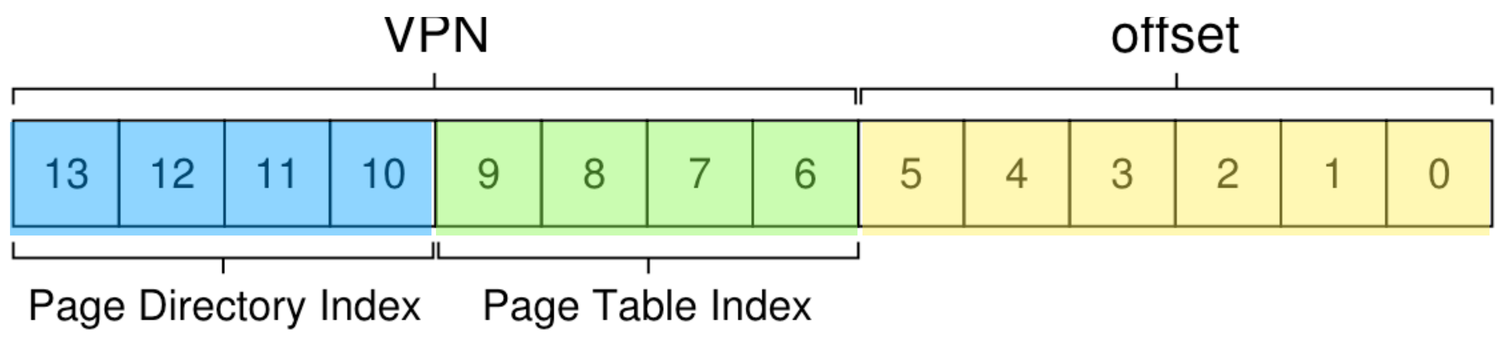


	Page Directory		Page of PT (@PFN:100)			Page of PT (@PFN:101)		
	PFN	valid?	PFN	valid	prot	PFN	valid	prot
	100	1	10	1	r-x	—	0	—
	—	0	23	1	r-x	—	0	—
	—	0	—	0	—	—	0	—
	—	0	—	0	—	—	0	—
	—	0	80	1	rw-	—	0	—
	—	0	59	1	rw-	—	0	—
	—	0	—	0	—	—	0	—
	—	0	—	0	—	—	0	—
	—	0	—	0	—	—	0	—
	—	0	—	0	—	—	0	—
	—	0	—	0	—	—	0	—
	—	0	—	0	—	—	0	—
	—	0	—	0	—	—	0	—
	—	0	—	0	—	—	0	—
	—	0	—	0	—	55	1	rw-
15	101	1	—	0	—	45	1	rw-

Figure 20.5: **A Page Directory, And Pieces Of Page Table**

Address translation

Simple address space



- Example: 0x3F81 (VA) => 0x0DC1 (PA)

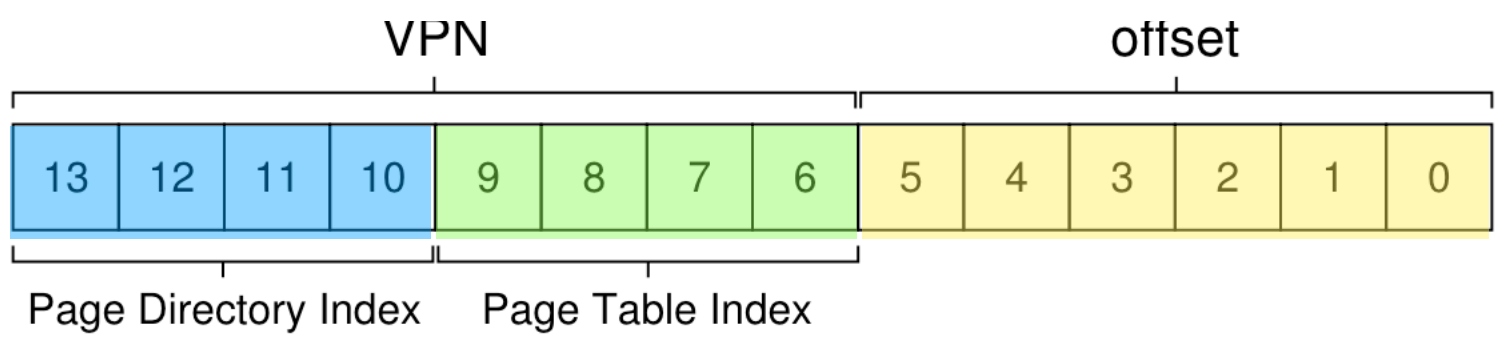
Page dir idx 15				Page table idx 14				Offset					
1	1	1	1	1	1	1	0	0	0	0	0	0	1
3		F				8				1			

	Page Directory		Page of PT (@PFN:100)			Page of PT (@PFN:101)			
	PFN	valid?	PFN	valid	prot	PFN	valid	prot	
	100	1	10	1	r-x	—	0	—	
	—	0	23	1	r-x	—	0	—	
	—	0	—	0	—	—	0	—	
	—	0	—	0	—	—	0	—	
	—	0	80	1	rw-	—	0	—	
	—	0	59	1	rw-	—	0	—	
	—	0	—	0	—	—	0	—	
	—	0	—	0	—	—	0	—	
	—	0	—	0	—	—	0	—	
	—	0	—	0	—	—	0	—	
	—	0	—	0	—	—	0	—	
	—	0	—	0	—	—	0	—	
	—	0	—	0	—	—	0	—	
	—	0	—	0	—	—	0	—	
	—	0	—	0	—	55	1	rw-	14
15	101	1	—	0	—	45	1	rw-	

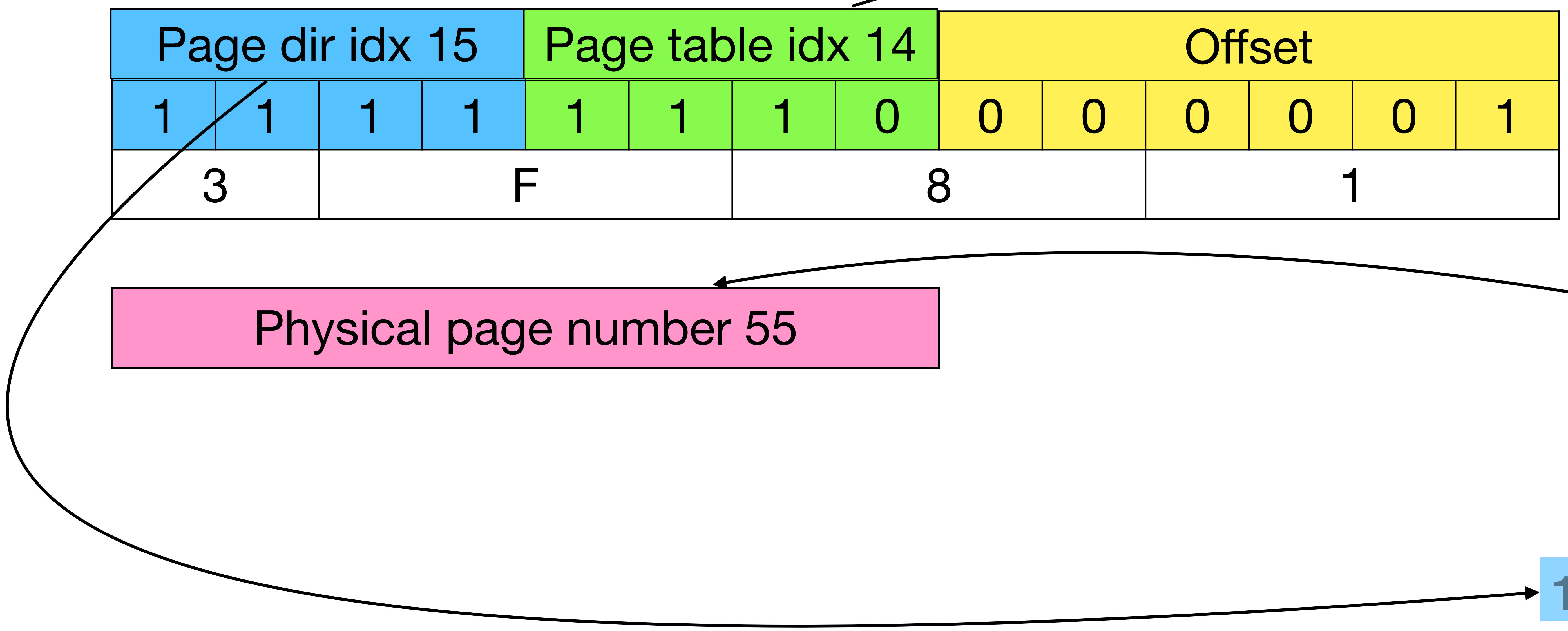
Figure 20.5: **A Page Directory, And Pieces Of Page Table**

Address translation

Simple address space



- Example: 0x3F81 (VA) => 0x0DC1 (PA)

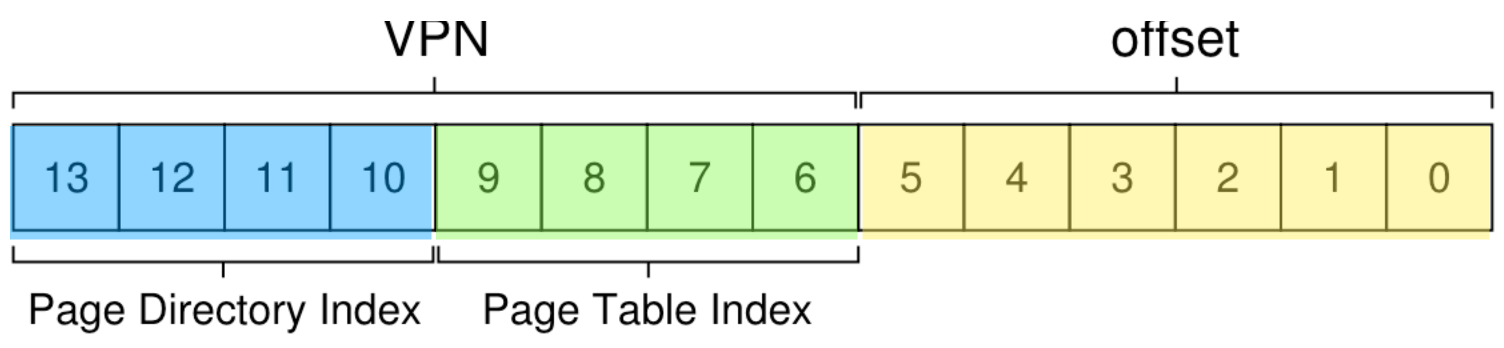


	Page Directory		Page of PT (@PFN:100)			Page of PT (@PFN:101)			
	PFN	valid?	PFN	valid	prot	PFN	valid	prot	
	100	1	10	1	r-x	—	0	—	
	—	0	23	1	r-x	—	0	—	
	—	0	—	0	—	—	0	—	
	—	0	—	0	—	—	0	—	
	—	0	80	1	rw-	—	0	—	
	—	0	59	1	rw-	—	0	—	
	—	0	—	0	—	—	0	—	
	—	0	—	0	—	—	0	—	
	—	0	—	0	—	—	0	—	
	—	0	—	0	—	—	0	—	
	—	0	—	0	—	—	0	—	
	—	0	—	0	—	—	0	—	
	—	0	—	0	—	—	0	—	
	—	0	—	0	—	—	0	—	
	—	0	—	0	—	55	1	rw-	14
15	101	1	—	0	—	45	1	rw-	

Figure 20.5: **A Page Directory, And Pieces Of Page Table**

Address translation

Simple address space



- Example: 0x3F81 (VA) => 0x0DC1 (PA)

Page dir idx 15				Page table idx 14				Offset					
1	1	1	1	1	1	1	0	0	0	0	0	0	1
3		F				8				1			

Physical page number 55								Offset					
0	0	1	1	0	1	1	1	0	0	0	0	0	1
0		D				C				1			

	Page Directory		Page of PT (@PFN:100)			Page of PT (@PFN:101)			
	PFN	valid?	PFN	valid	prot	PFN	valid	prot	
	100	1	10	1	r-x	—	0	—	
	—	0	23	1	r-x	—	0	—	
	—	0	—	0	—	—	0	—	
	—	0	—	0	—	—	0	—	
	—	0	80	1	rw-	—	0	—	
	—	0	59	1	rw-	—	0	—	
	—	0	—	0	—	—	0	—	
	—	0	—	0	—	—	0	—	
	—	0	—	0	—	—	0	—	
	—	0	—	0	—	—	0	—	
	—	0	—	0	—	—	0	—	
	—	0	—	0	—	—	0	—	
	—	0	—	0	—	—	0	—	
	—	0	—	0	—	—	0	—	
	—	0	—	0	—	55	1	rw-	14
15	101	1	—	0	—	45	1	rw-	

Figure 20.5: **A Page Directory, And Pieces Of Page Table**

x86 segmentation and paging

- Segmentation:
  - Virtual address (logical address) => "linear address"
- Paging:
  - Linear address => Physical address

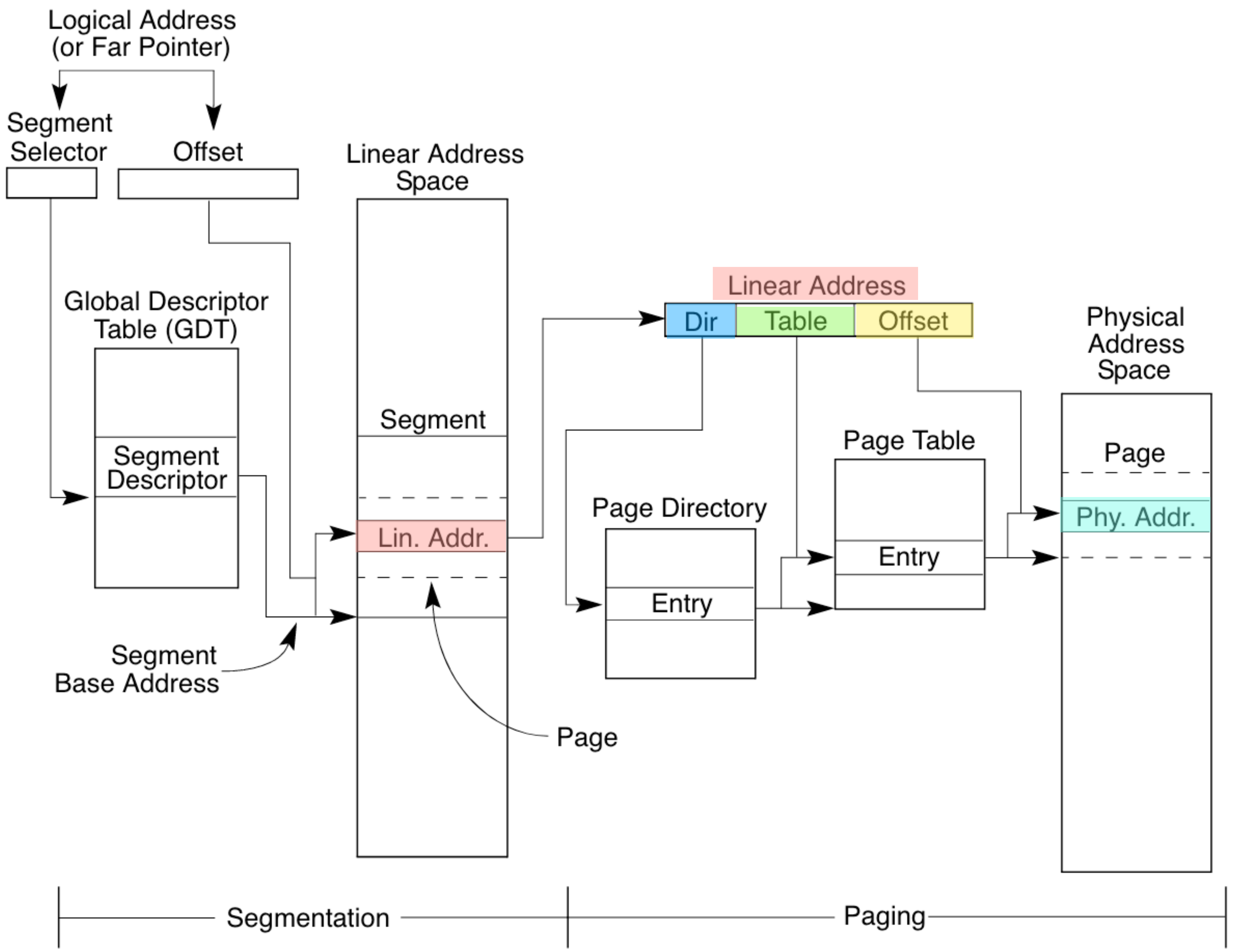
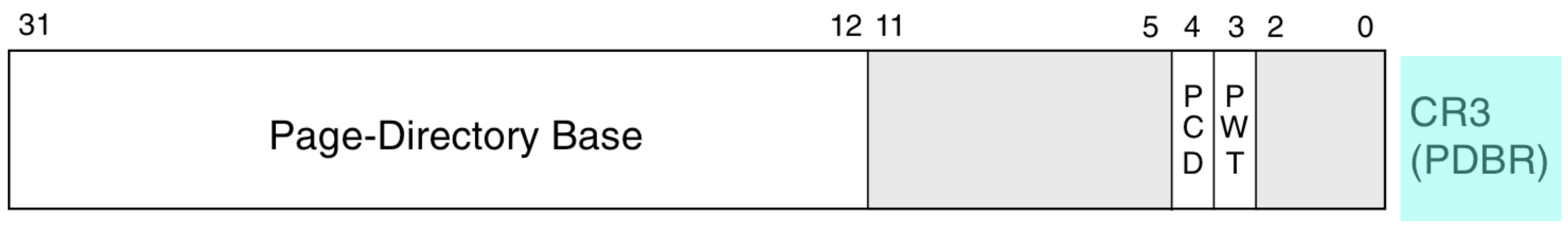


Figure 3-1. Segmentation and Paging

Address translation with paging on x86



- CR3 register contains page directory base

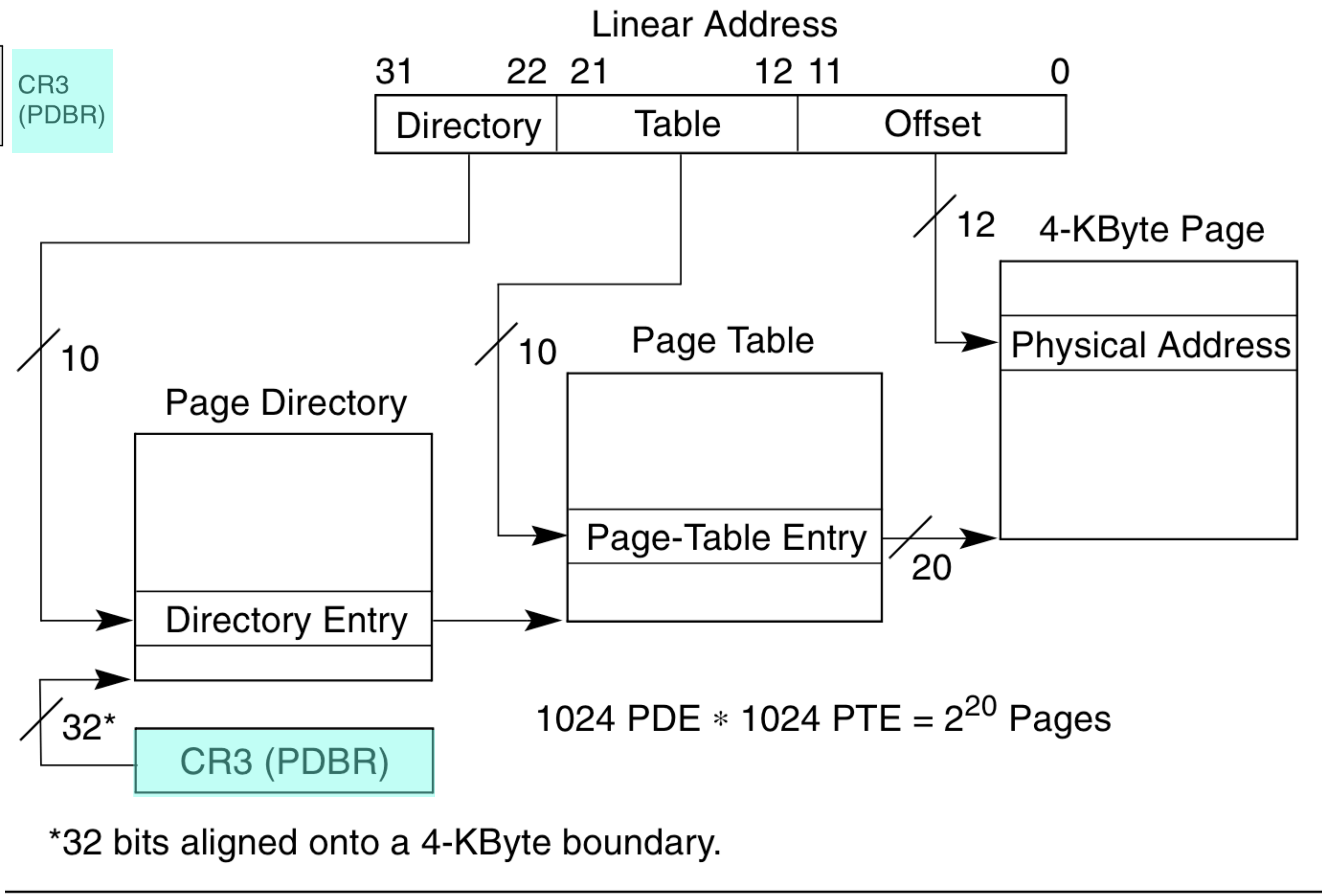
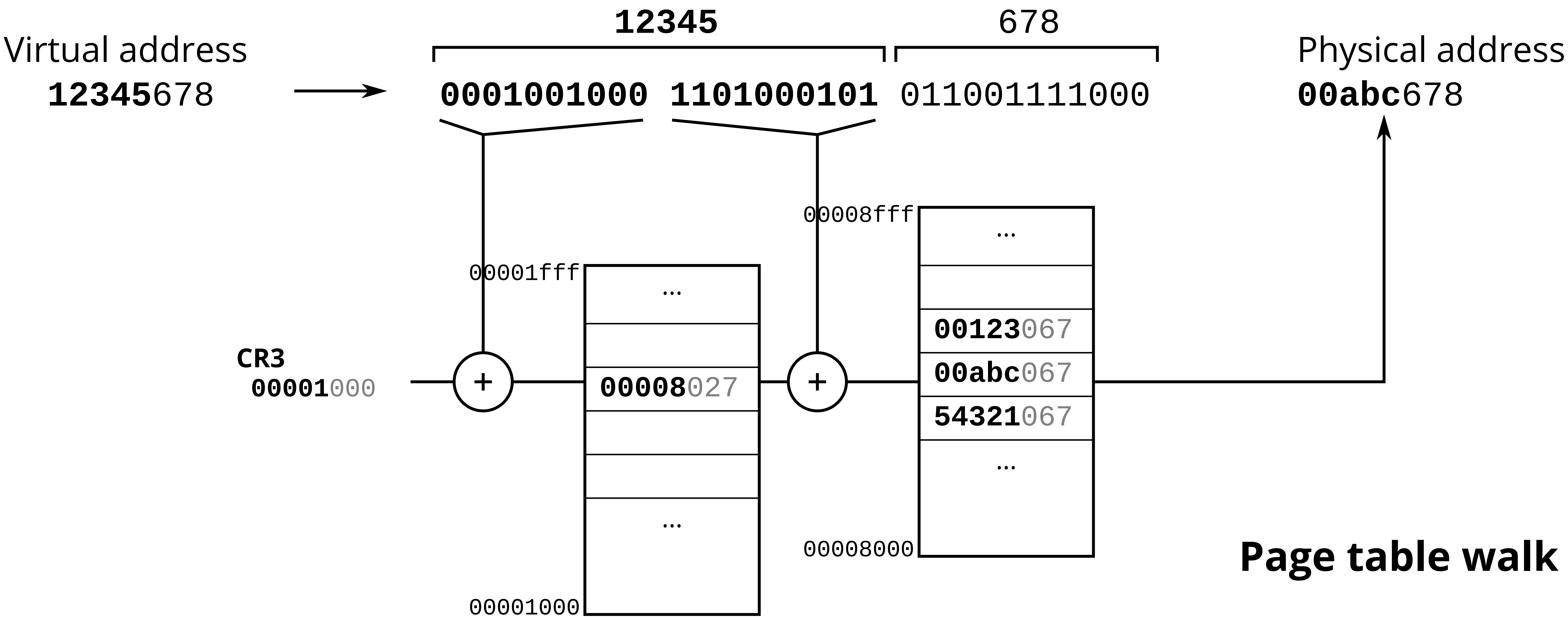


Figure 3-12. Linear Address Translation (4-KByte Pages)

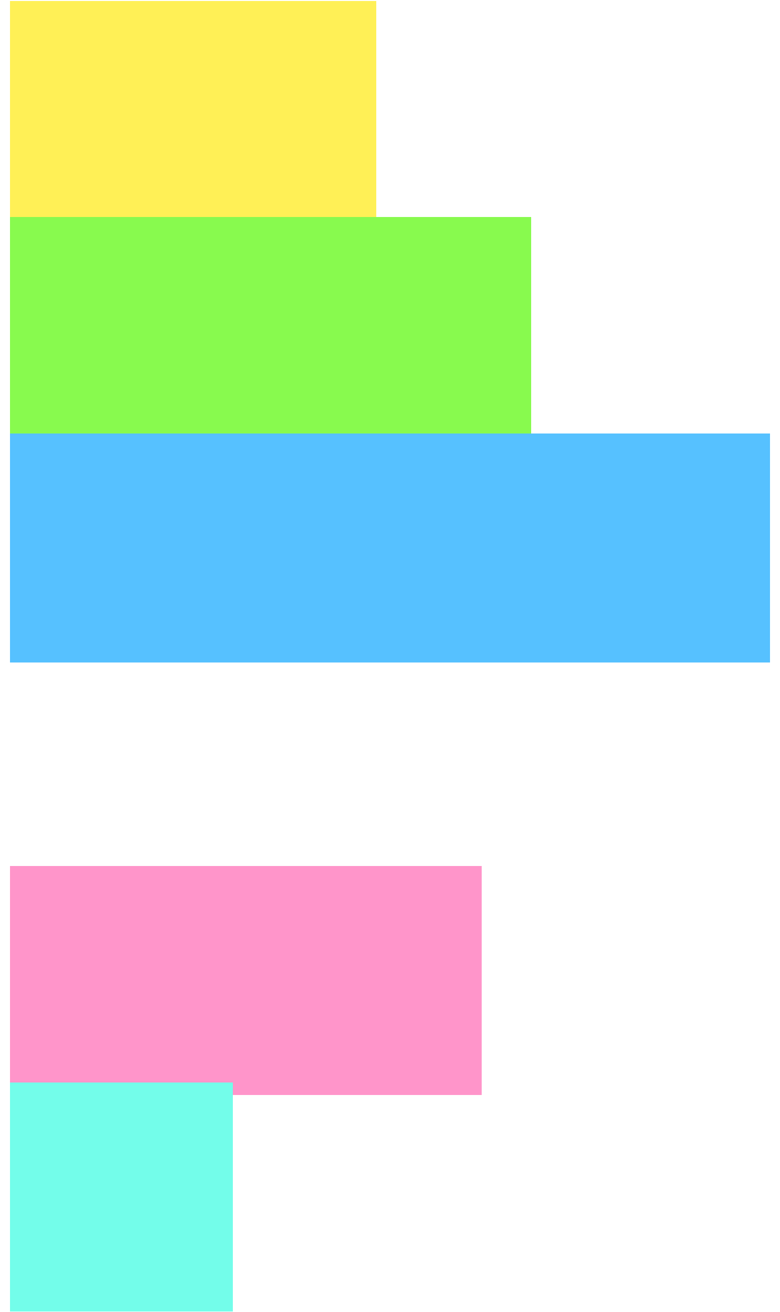
Address translation with paging on x86



Page table walk

x86 PTEs, PDEs

- 2^{20} 4KB pages in a 4GB DRAM
  - Page base address, page table base address are 20 bits



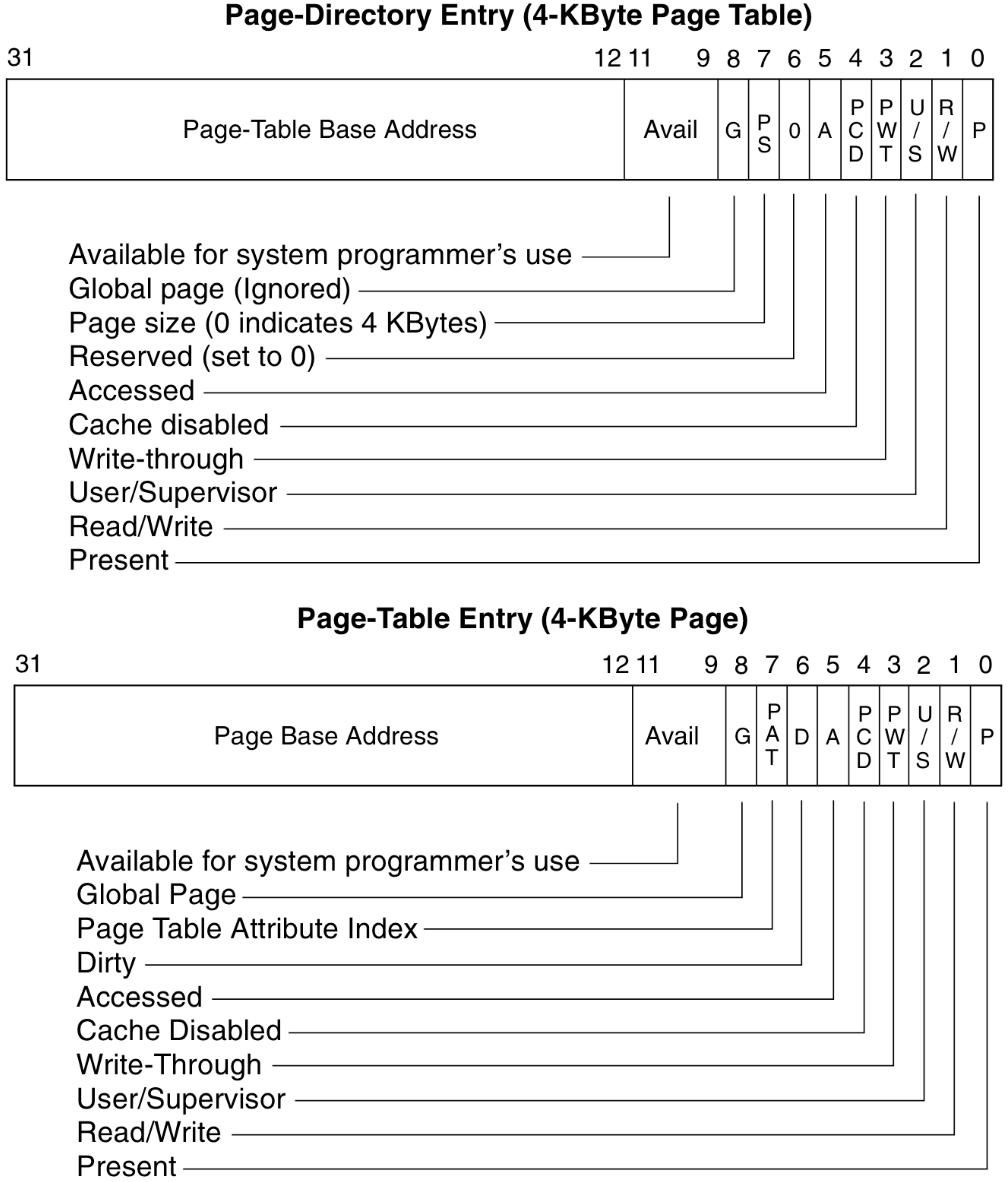


Figure 3-14. Format of Page-Directory and Page-Table Entries for 4-KByte Pages and 32-Bit Physical Addresses

x86 PTEs, PDEs

- 2^{20} 4KB pages in a 4GB DRAM
    - Page base address, page table base address are 20 bits
- Bits set by OS, used by hardware:

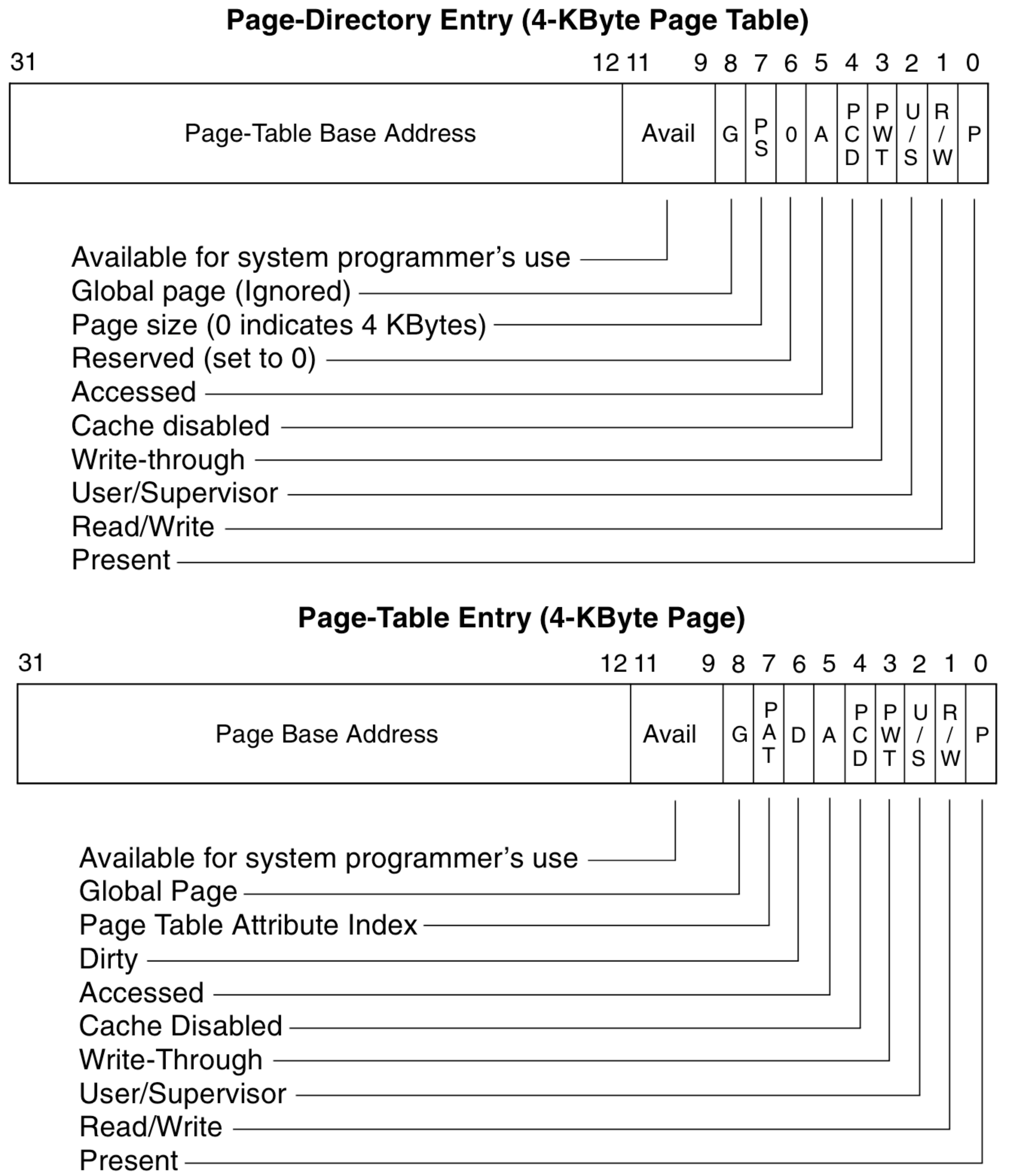


Figure 3-14. Format of Page-Directory and Page-Table Entries for 4-KByte Pages and 32-Bit Physical Addresses

x86 PTEs, PDEs

- 2^{20} 4KB pages in a 4GB DRAM
  - Page base address, page table base address are 20 bits
- Bits set by OS, used by hardware:
  - Present: It is a valid entry

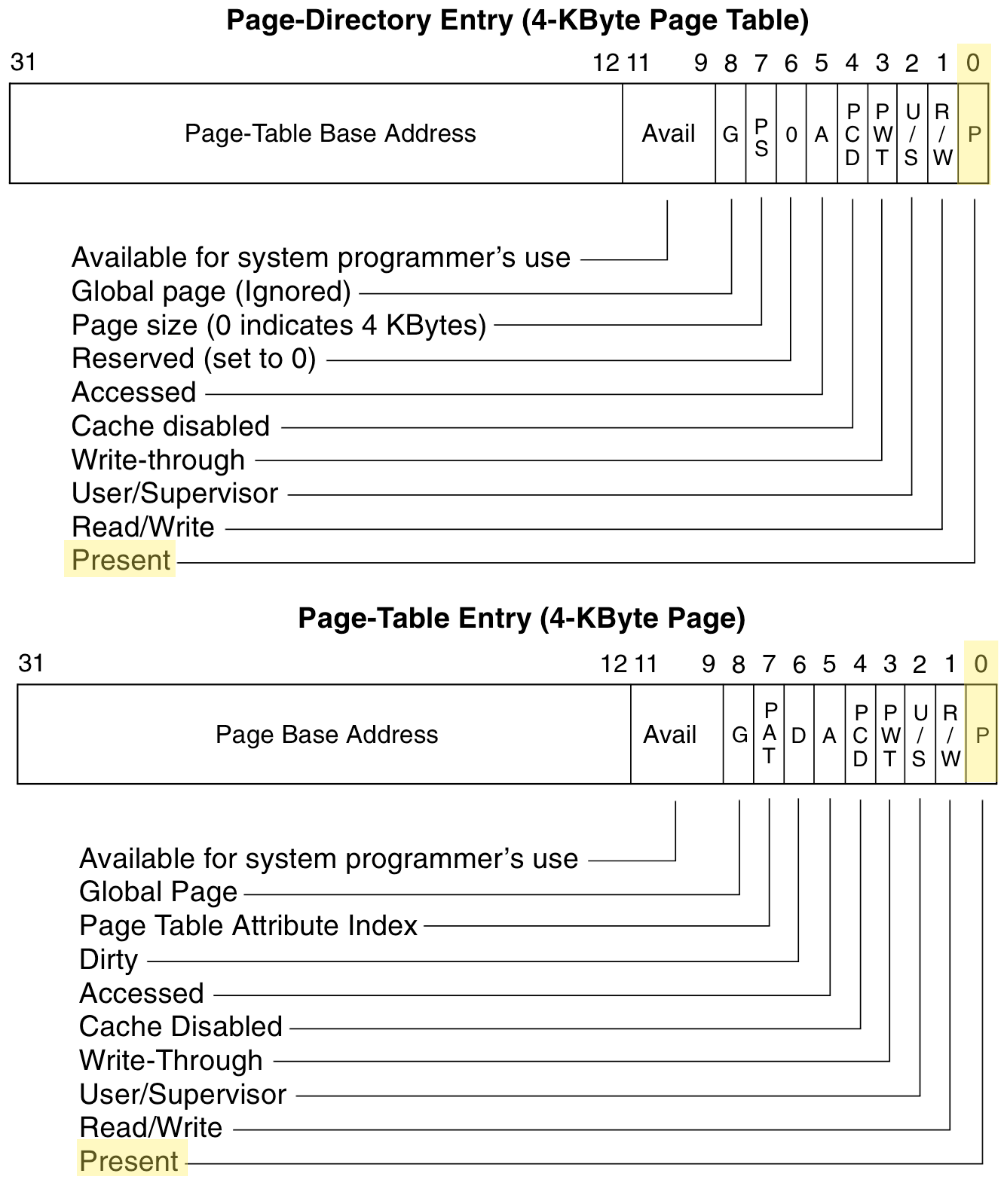


Figure 3-14. Format of Page-Directory and Page-Table Entries for 4-KByte Pages and 32-Bit Physical Addresses

x86 PTEs, PDEs

- 2^{20} 4KB pages in a 4GB DRAM
  - Page base address, page table base address are 20 bits
- Bits set by OS, used by hardware:
  - Present: It is a valid entry
  - Read/write: Can write if 1

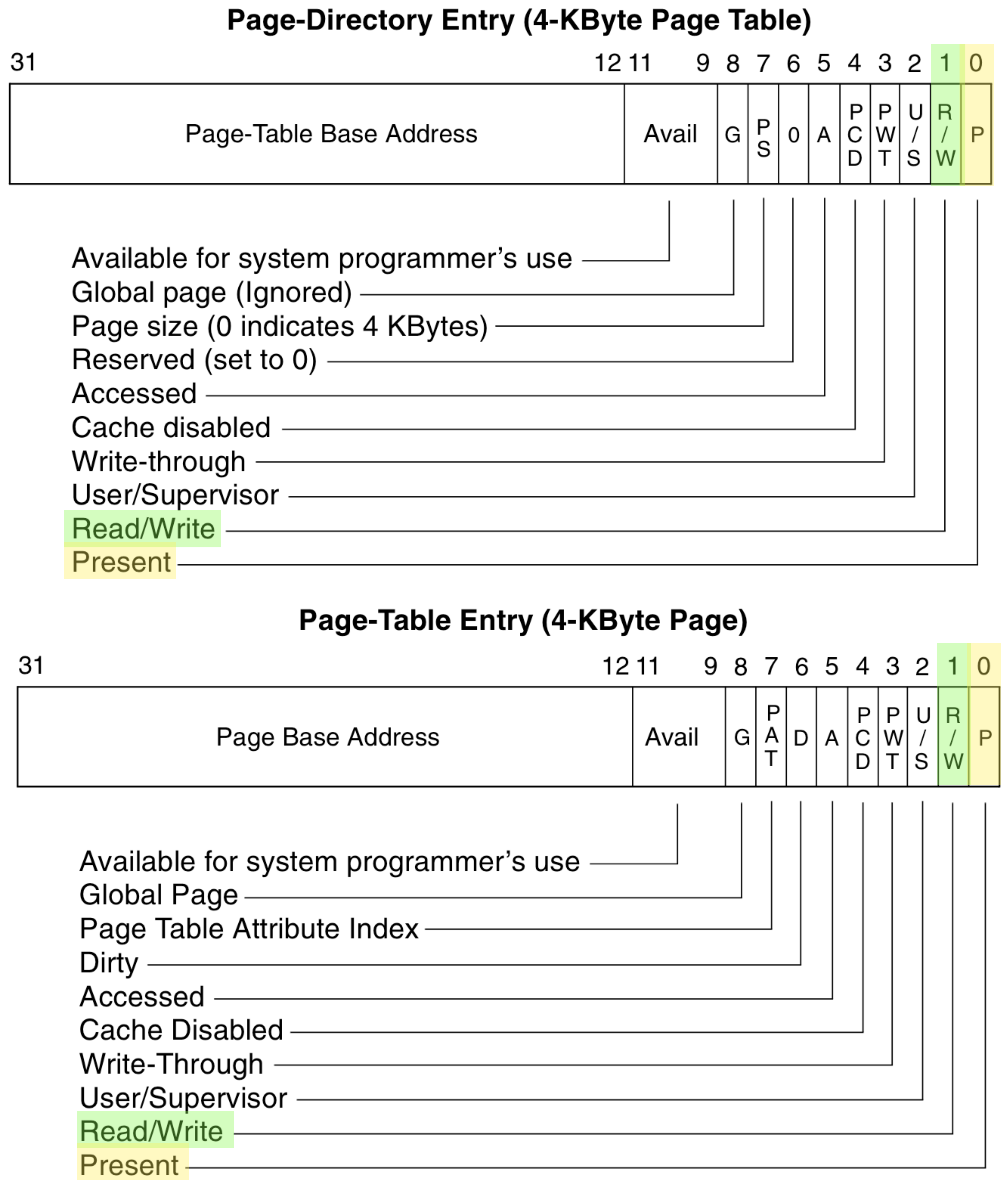


Figure 3-14. Format of Page-Directory and Page-Table Entries for 4-KByte Pages and 32-Bit Physical Addresses

x86 PTEs, PDEs

- 2^{20} 4KB pages in a 4GB DRAM
  - Page base address, page table base address are 20 bits
- Bits set by OS, used by hardware:
  - Present: It is a valid entry
  - Read/write: Can write if 1
  - User/supervisor: CPL=3 can access if 1

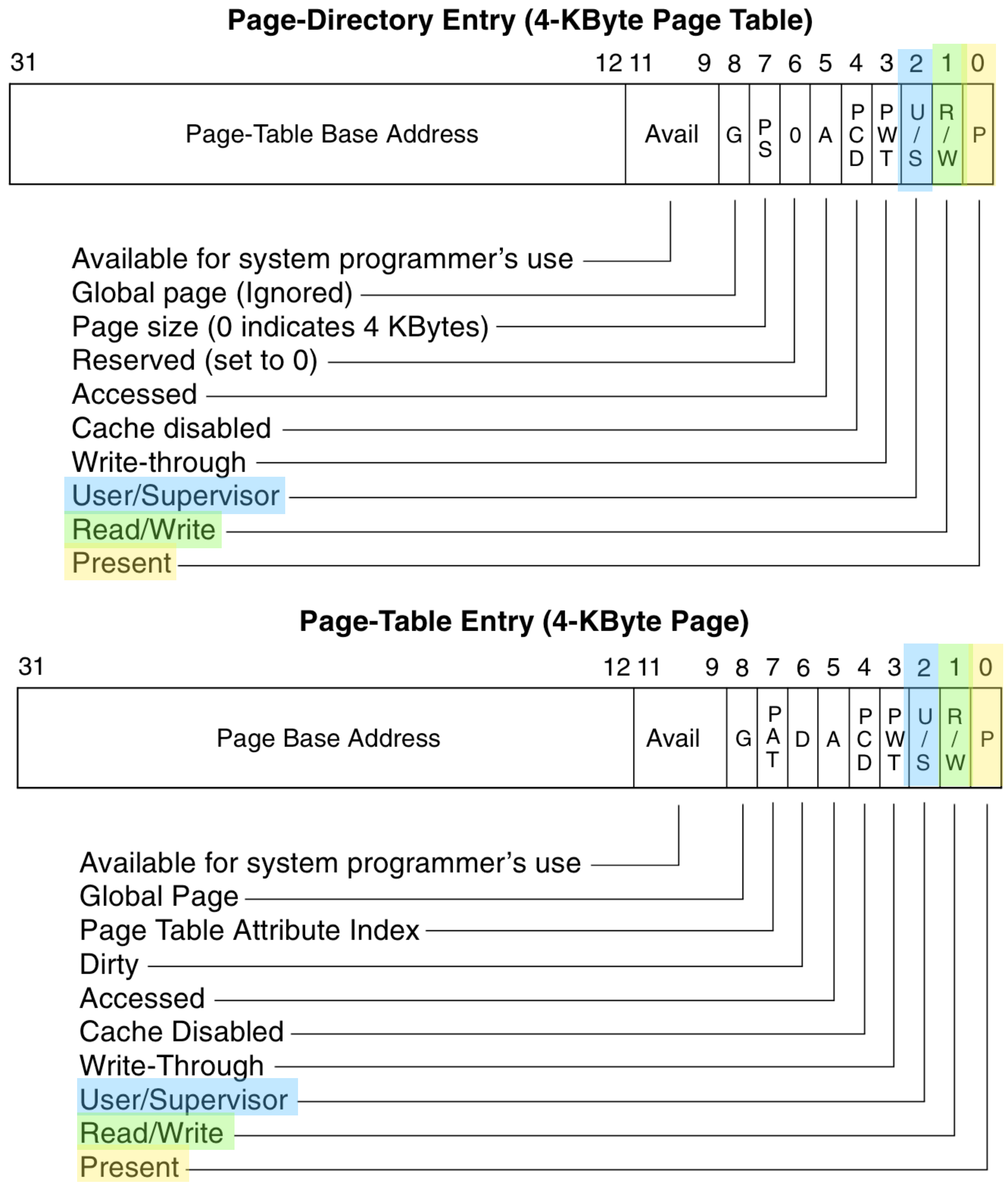


Figure 3-14. Format of Page-Directory and Page-Table Entries for 4-KByte Pages and 32-Bit Physical Addresses

x86 PTEs, PDEs

- 2^{20} 4KB pages in a 4GB DRAM
  - Page base address, page table base address are 20 bits
- Bits set by OS, used by hardware:
  - Present: It is a valid entry
  - Read/write: Can write if 1
  - User/supervisor: CPL=3 can access if 1
- Bits set by hardware, used/cleared by OS:

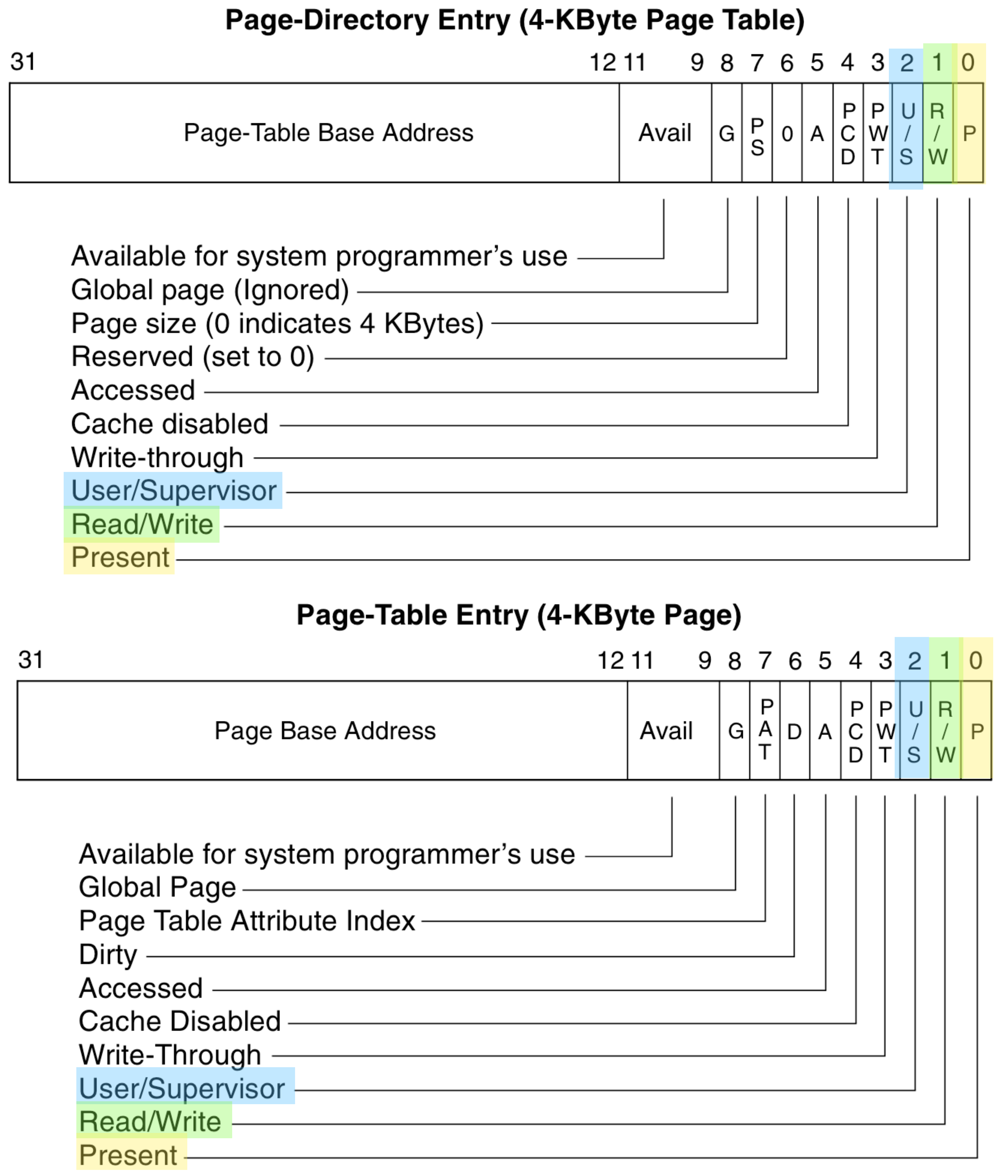


Figure 3-14. Format of Page-Directory and Page-Table Entries for 4-KByte Pages and 32-Bit Physical Addresses

x86 PTEs, PDEs

- 2^{20} 4KB pages in a 4GB DRAM
  - Page base address, page table base address are 20 bits
- Bits set by OS, used by hardware:
  - Present: It is a valid entry
  - Read/write: Can write if 1
  - User/supervisor: CPL=3 can access if 1
- Bits set by hardware, used/cleared by OS:
  - Accessed: Hardware accessed this page

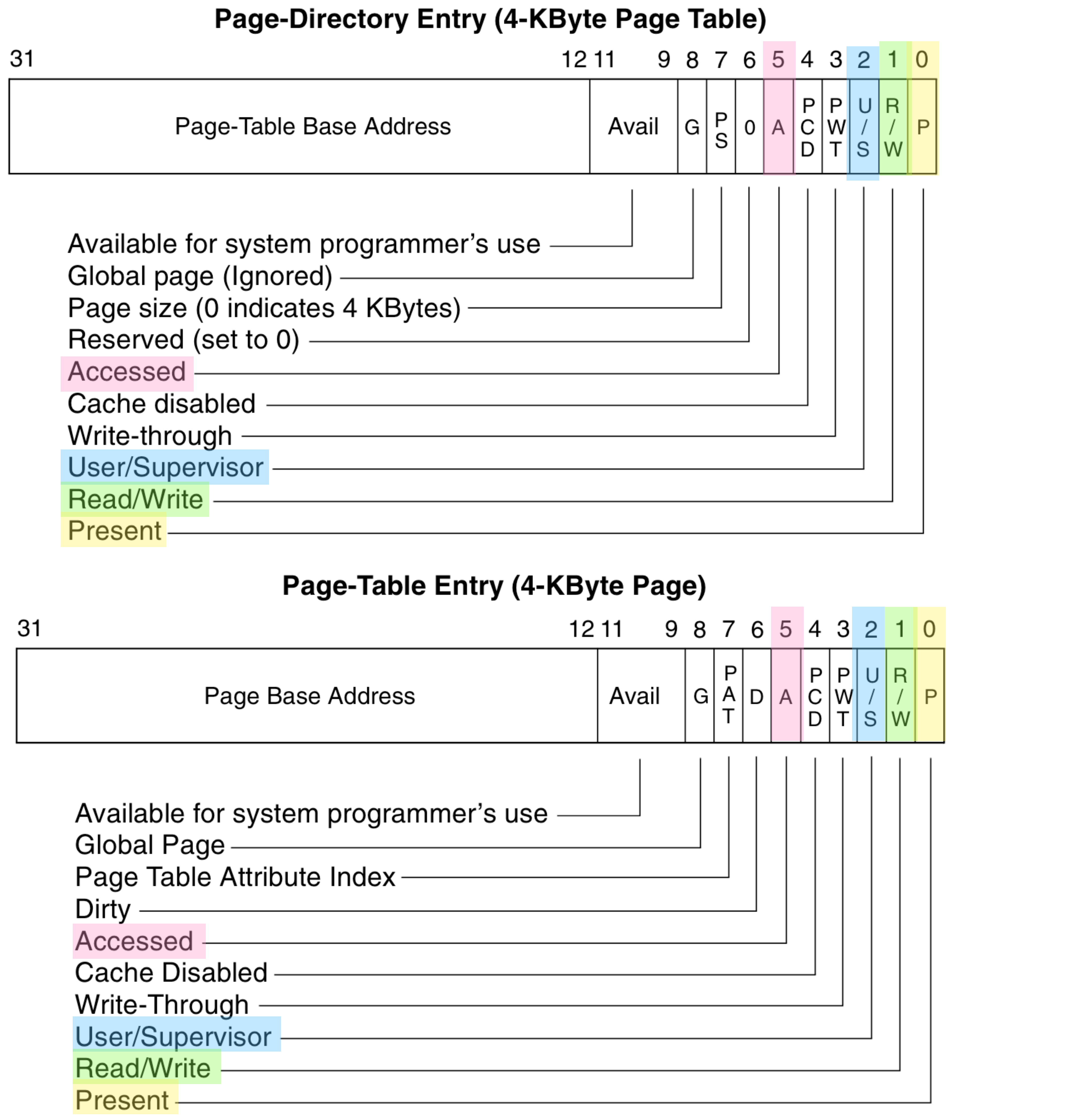


Figure 3-14. Format of Page-Directory and Page-Table Entries for 4-KByte Pages and 32-Bit Physical Addresses

x86 PTEs, PDEs

- 2^{20} 4KB pages in a 4GB DRAM
  - Page base address, page table base address are 20 bits
- Bits set by OS, used by hardware:
  - Present: It is a valid entry
  - Read/write: Can write if 1
  - User/supervisor: CPL=3 can access if 1
- Bits set by hardware, used/cleared by OS:
  - Accessed: Hardware accessed this page
  - Dirty: Hardware wrote to this page

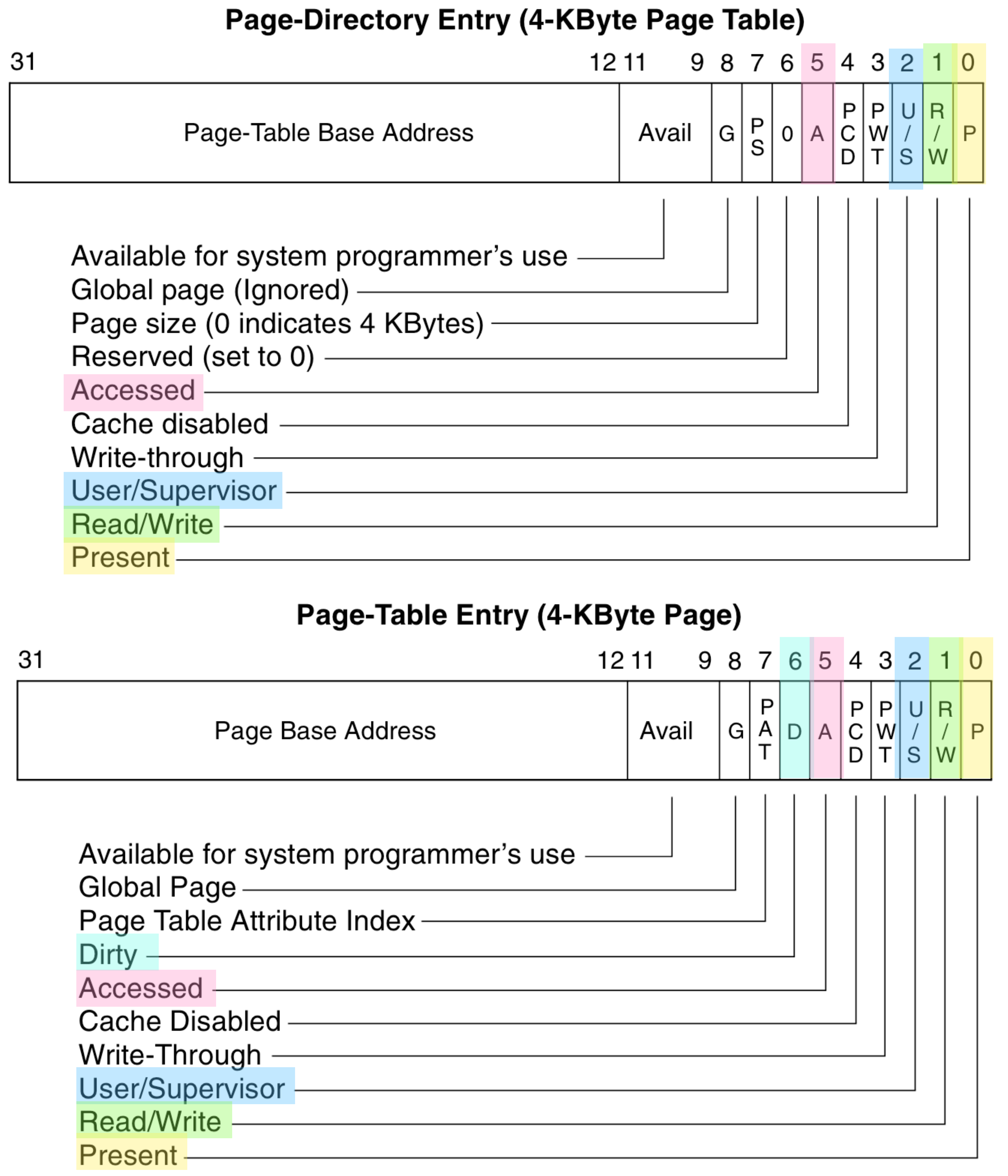
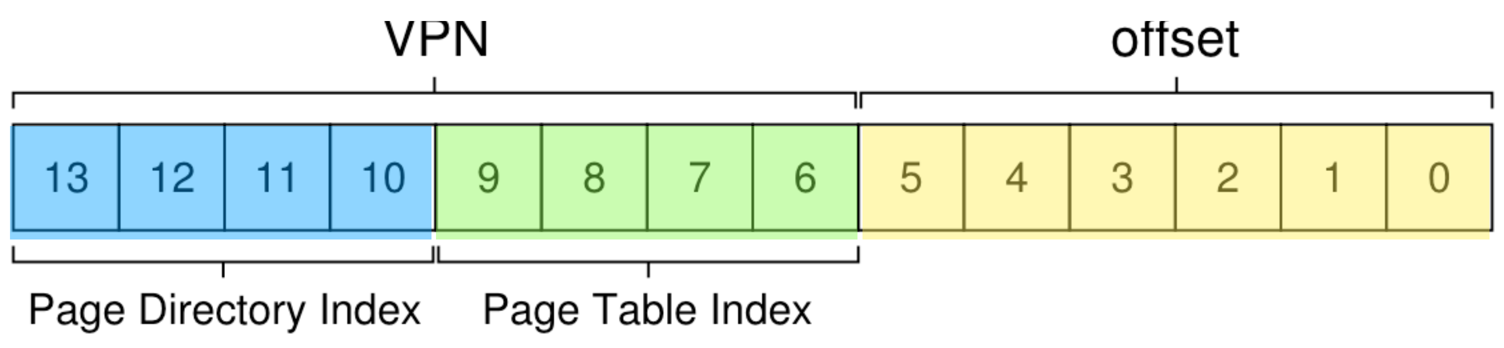
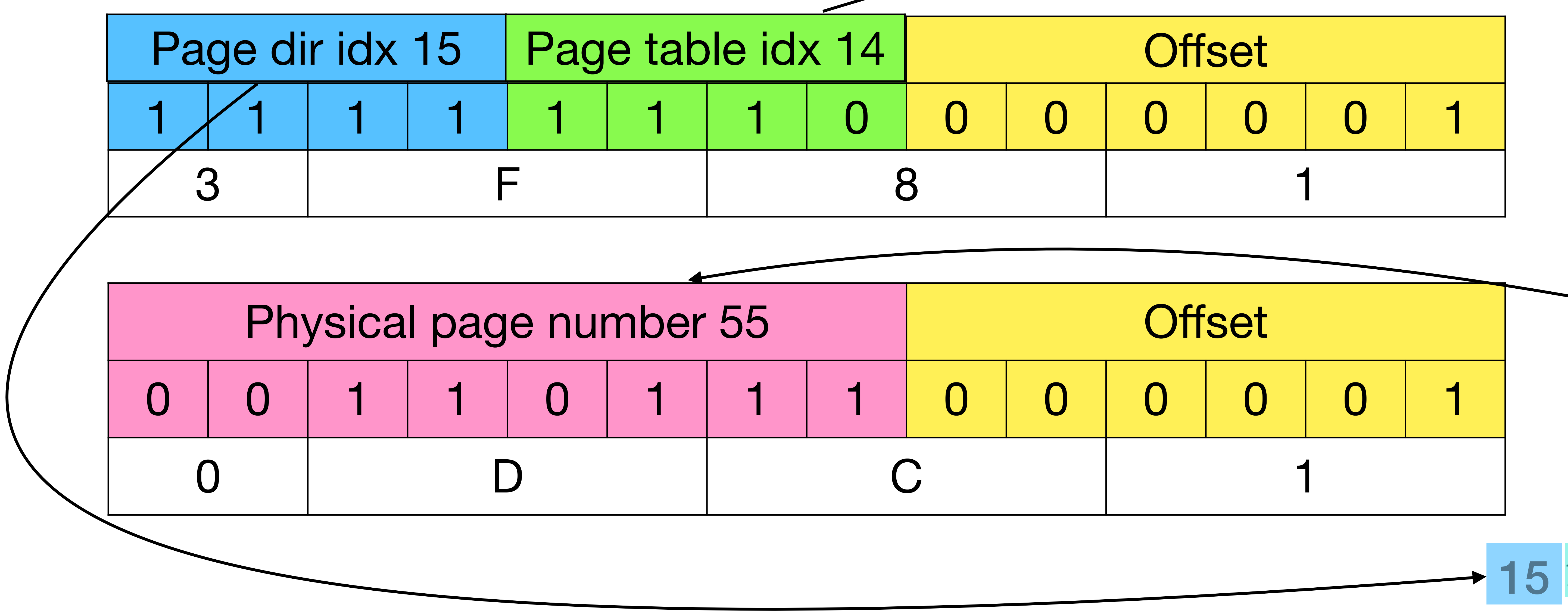


Figure 3-14. Format of Page-Directory and Page-Table Entries for 4-KByte Pages and 32-Bit Physical Addresses

Performance degradation!



- Accessing 1 memory location requires accessing 3 memory locations!



Page Directory		Page of PT (@PFN:100)			Page of PT (@PFN:101)		
PFN	valid?	PFN	valid	prot	PFN	valid	prot
100	1	10	1	r-x	—	0	—
—	0	23	1	r-x	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	80	1	rw-	—	0	—
—	0	59	1	rw-	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	55	1	rw-
101	1	—	0	—	45	1	rw-

Figure 20.5: **A Page Directory, And Pieces Of Page Table**

Translation-lookaside buffer (TLB)

- First check page translation in TLB before walking the page table
- TLB hit: ~0.5-1 cycle
- TLB miss: ~10-100 cycles

Page dir idx 15				Page table idx 14				Offset					
1	1	1	1	1	1	1	0	0	0	0	0	0	1
3		F				8				1			

Physical page number 55								Offset					
0	0	1	1	0	1	1	1	0	0	0	0	0	1
0		D				C				1			

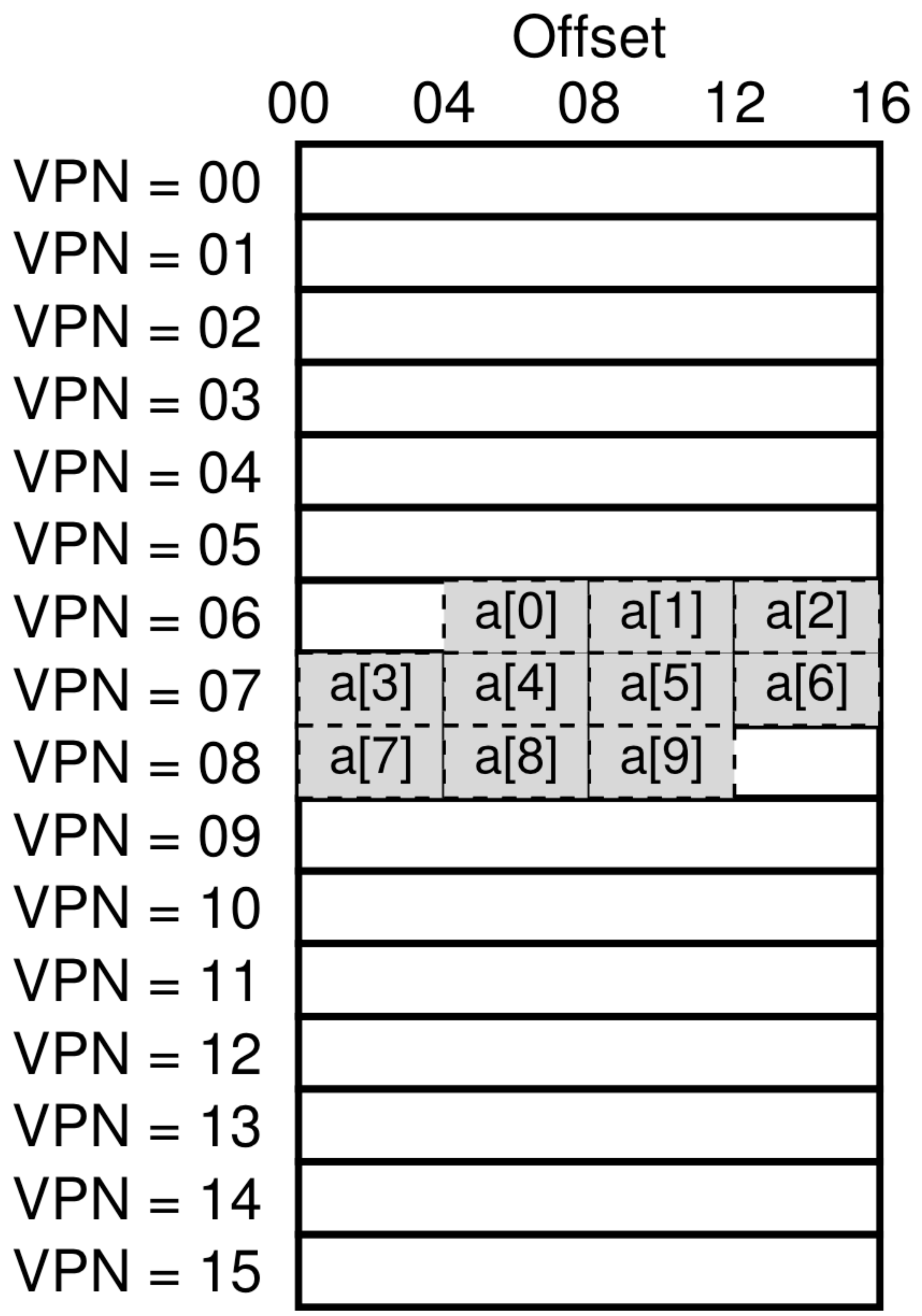
TLB

VPN								PPN							
1	1	1	1	1	1	1	0	0	0	1	1	0	1	1	1

Which programs will run faster?

Which programs will have lesser TLB misses?

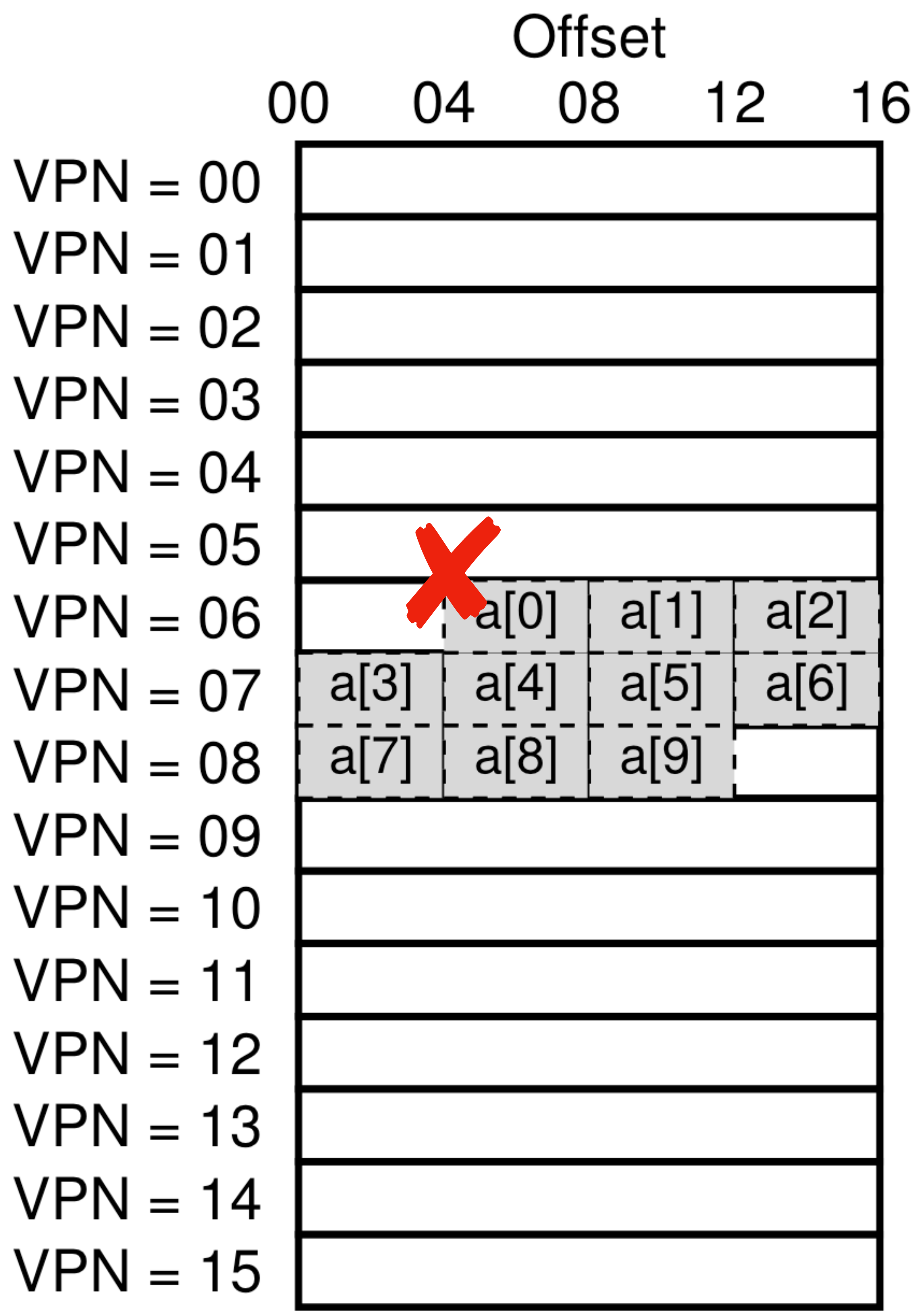
```
int sum = 0;
for (i = 0; i < 10; i++) {
    sum += a[i];
}
```



Which programs will run faster?

Which programs will have lesser TLB misses?

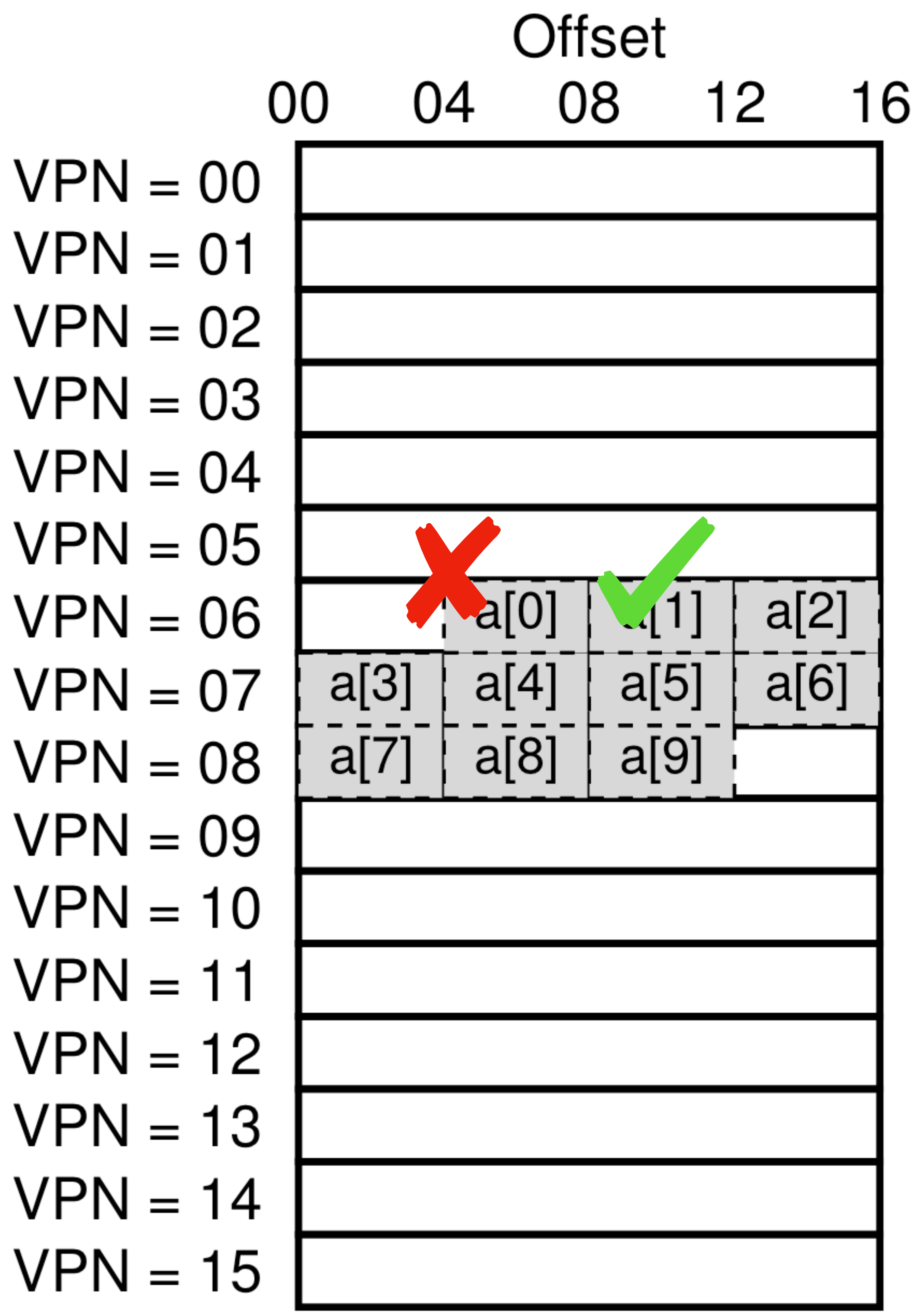
```
int sum = 0;
for (i = 0; i < 10; i++) {
    sum += a[i];
}
```



Which programs will run faster?

Which programs will have lesser TLB misses?

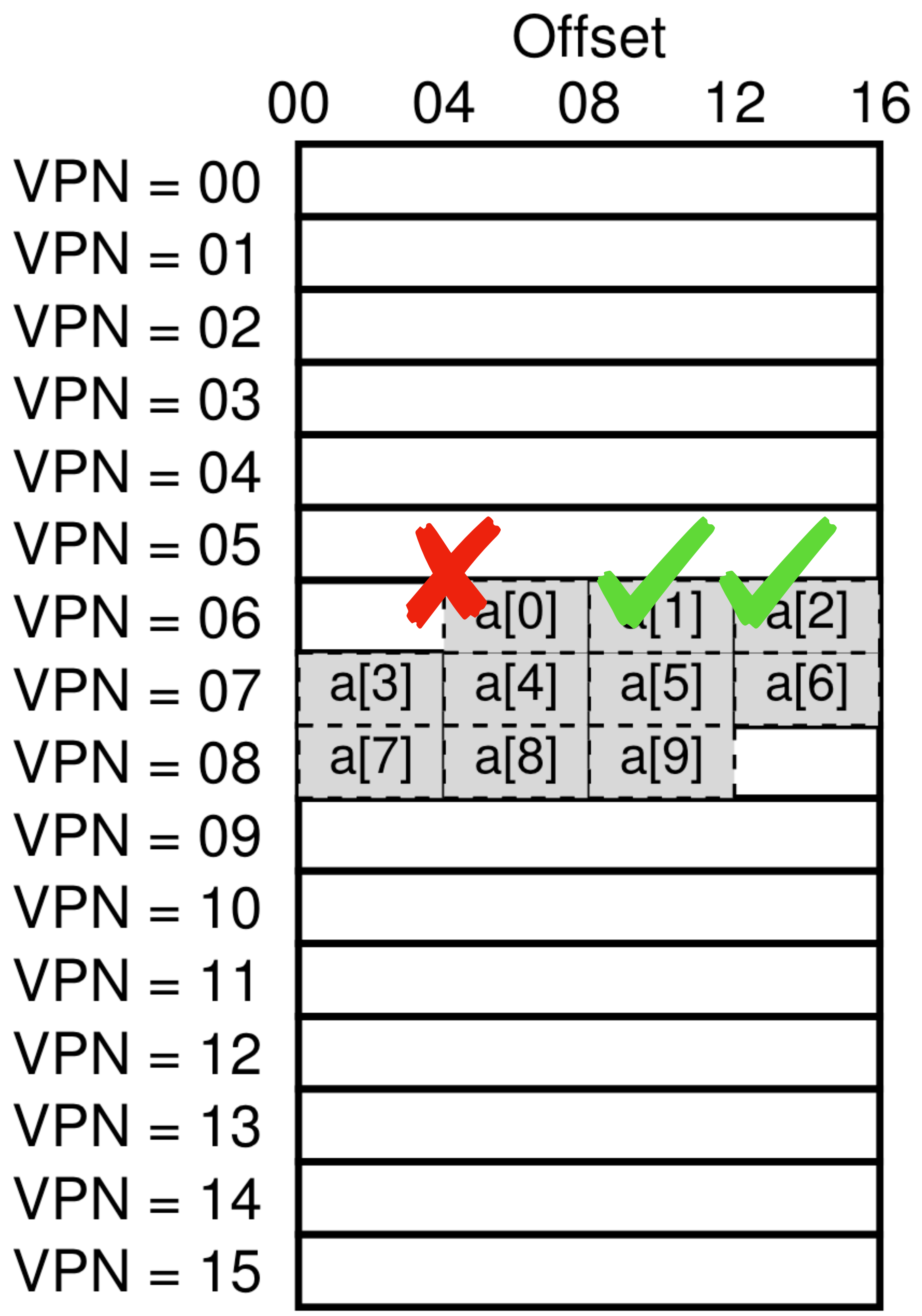
```
int sum = 0;
for (i = 0; i < 10; i++) {
    sum += a[i];
}
```



Which programs will run faster?

Which programs will have lesser TLB misses?

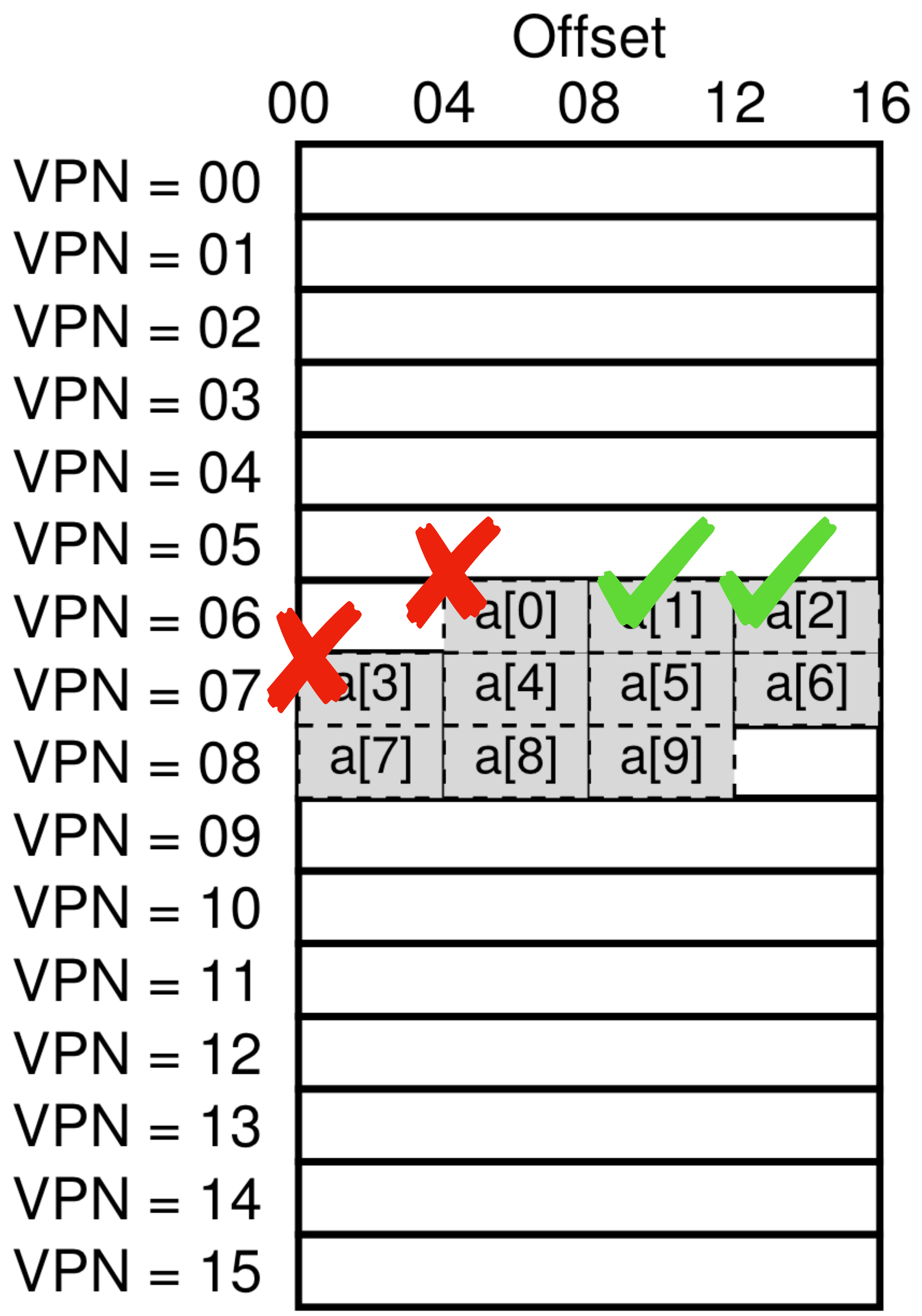
```
int sum = 0;
for (i = 0; i < 10; i++) {
     sum += a[i];
}
```



Which programs will run faster?

Which programs will have lesser TLB misses?

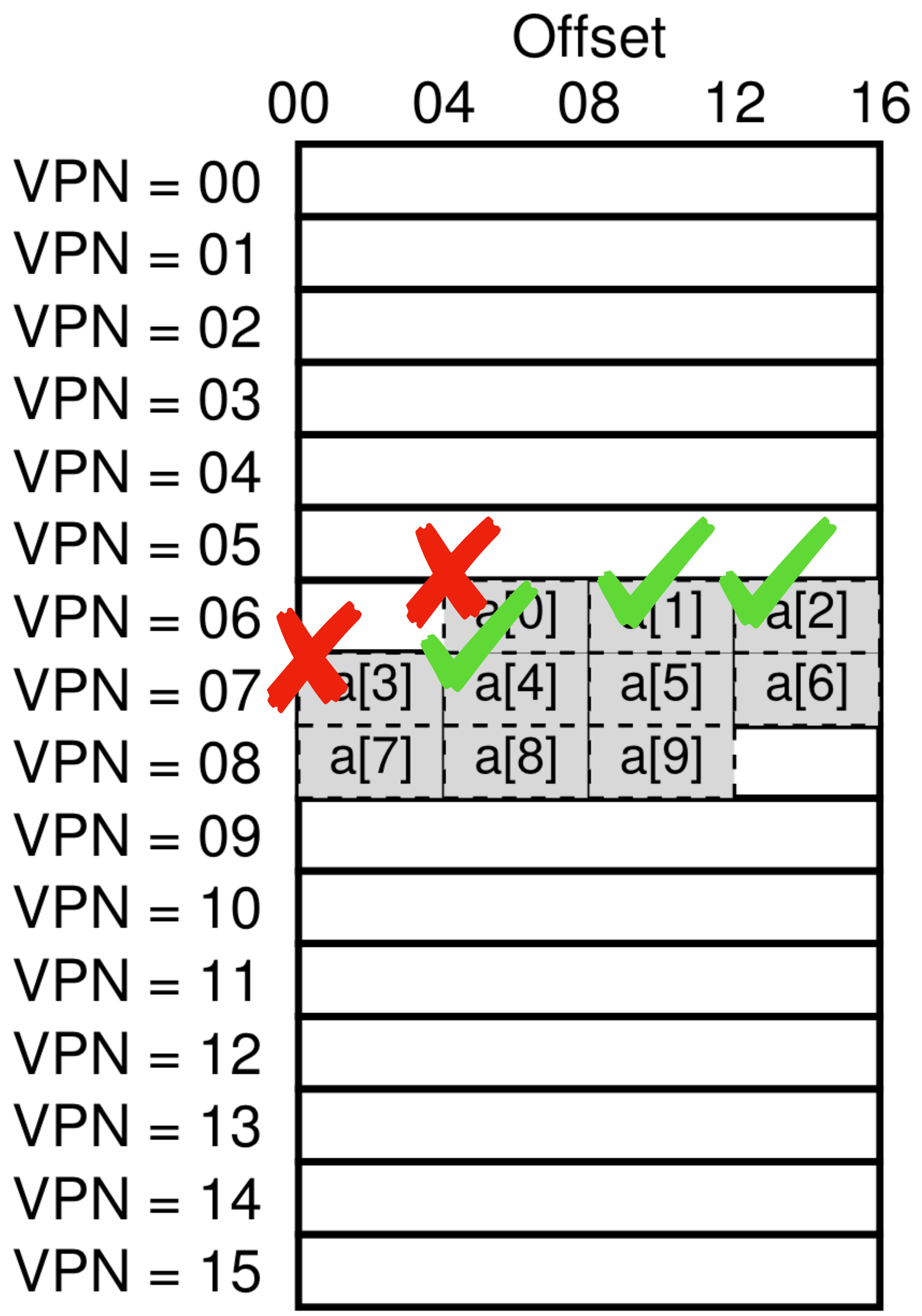
```
int sum = 0;
for (i = 0; i < 10; i++) {
    sum += a[i];
}
```



Which programs will run faster?

Which programs will have lesser TLB misses?

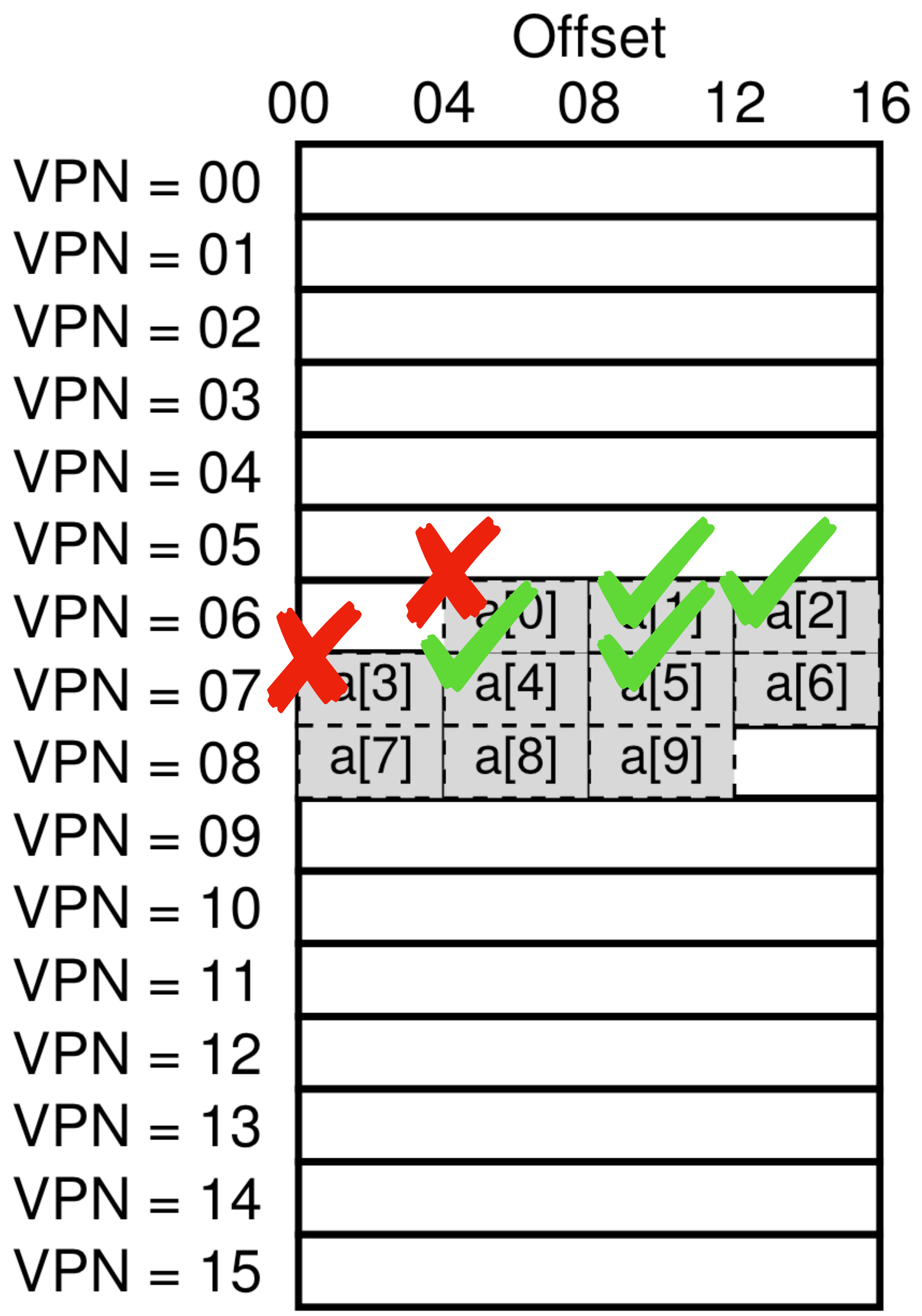
```
int sum = 0;
for (i = 0; i < 10; i++) {
    sum += a[i];
}
```



Which programs will run faster?

Which programs will have lesser TLB misses?

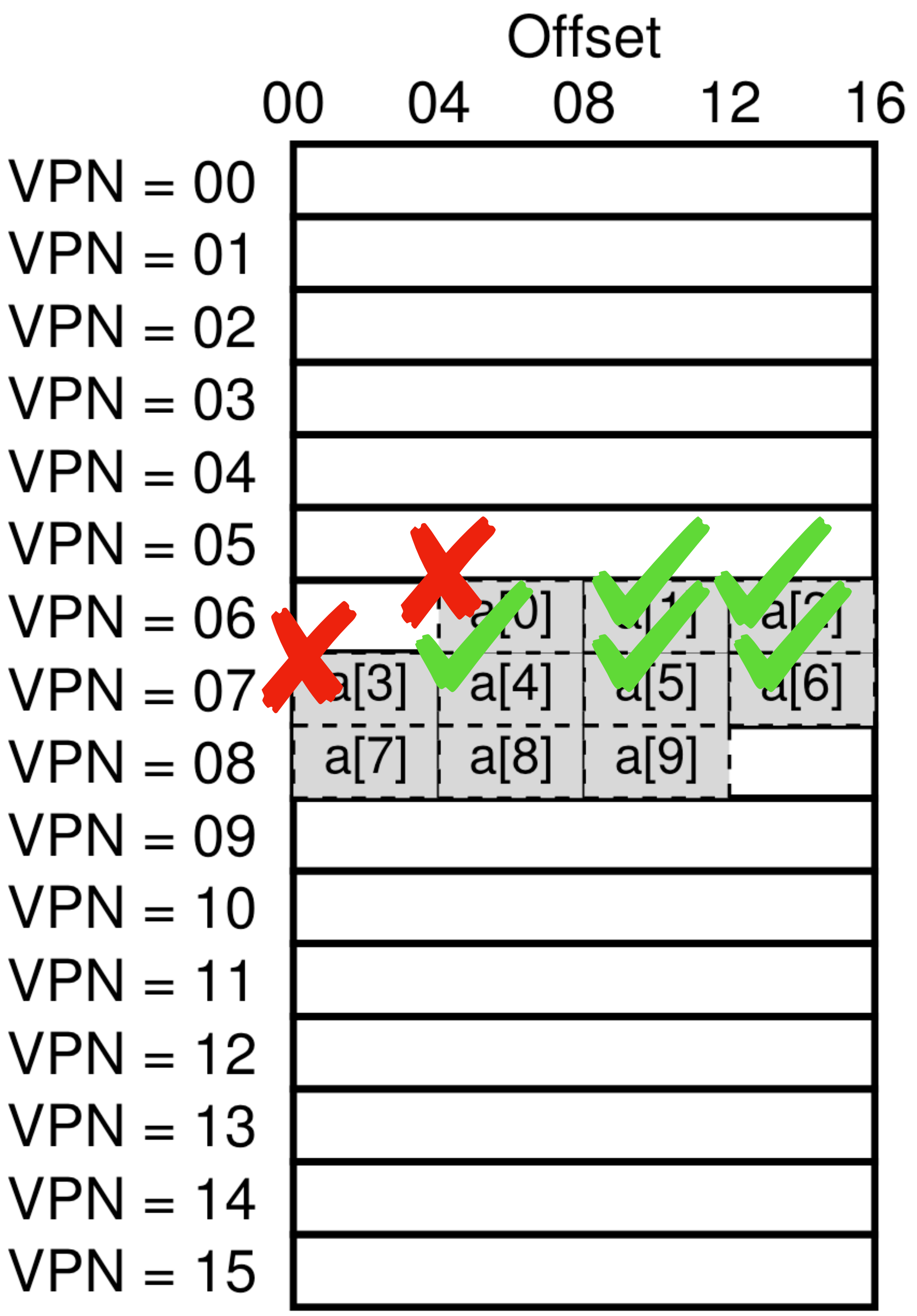
```
int sum = 0;
for (i = 0; i < 10; i++) {
    sum += a[i];
}
```



Which programs will run faster?

Which programs will have lesser TLB misses?

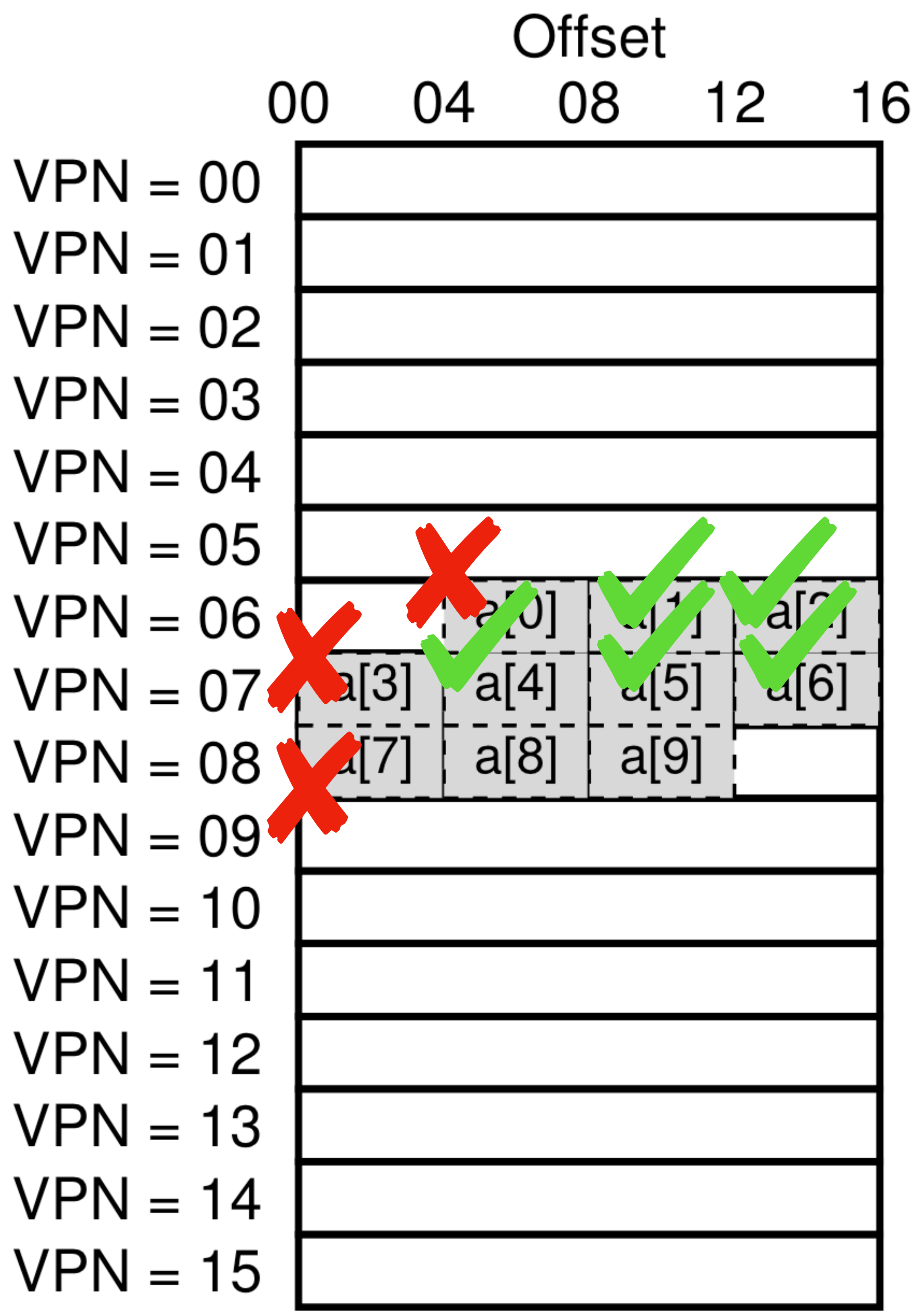
```
int sum = 0;
for (i = 0; i < 10; i++) {
    sum += a[i];
}
```



Which programs will run faster?

Which programs will have lesser TLB misses?

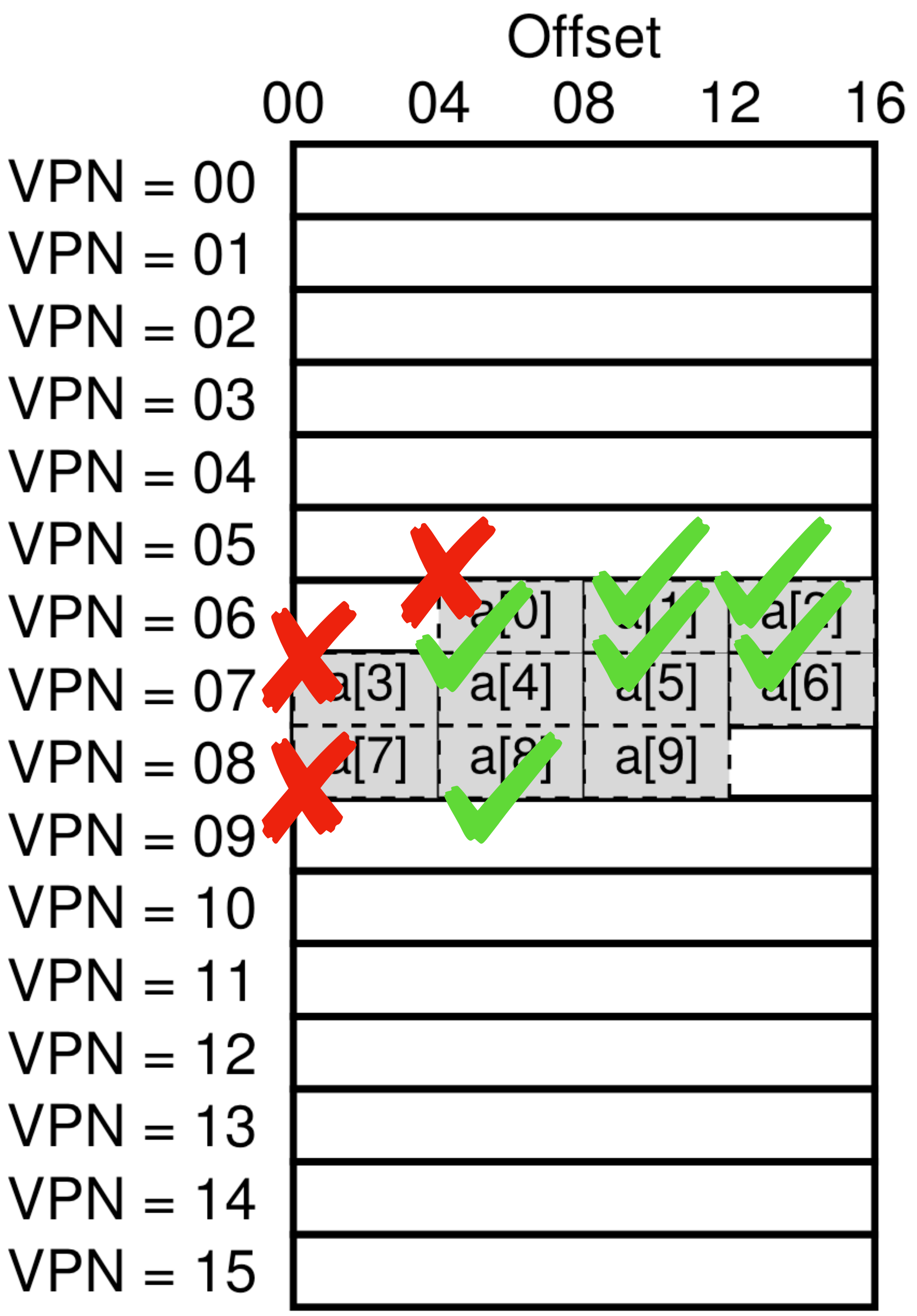
```
int sum = 0;
for (i = 0; i < 10; i++) {
     sum += a[i];
}
```



Which programs will run faster?

Which programs will have lesser TLB misses?

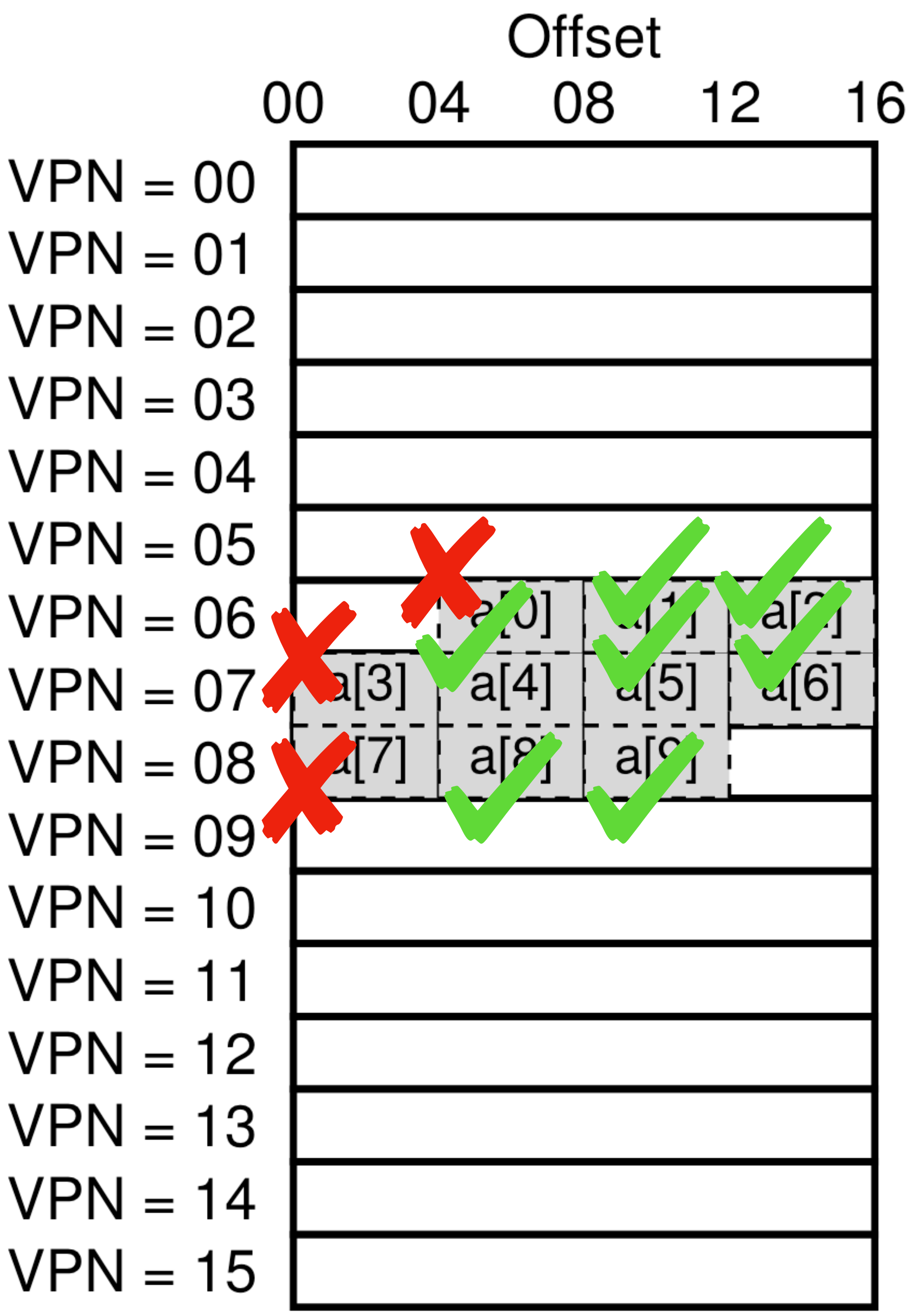
```
int sum = 0;
for (i = 0; i < 10; i++) {
    sum += a[i];
}
```



Which programs will run faster?

Which programs will have lesser TLB misses?

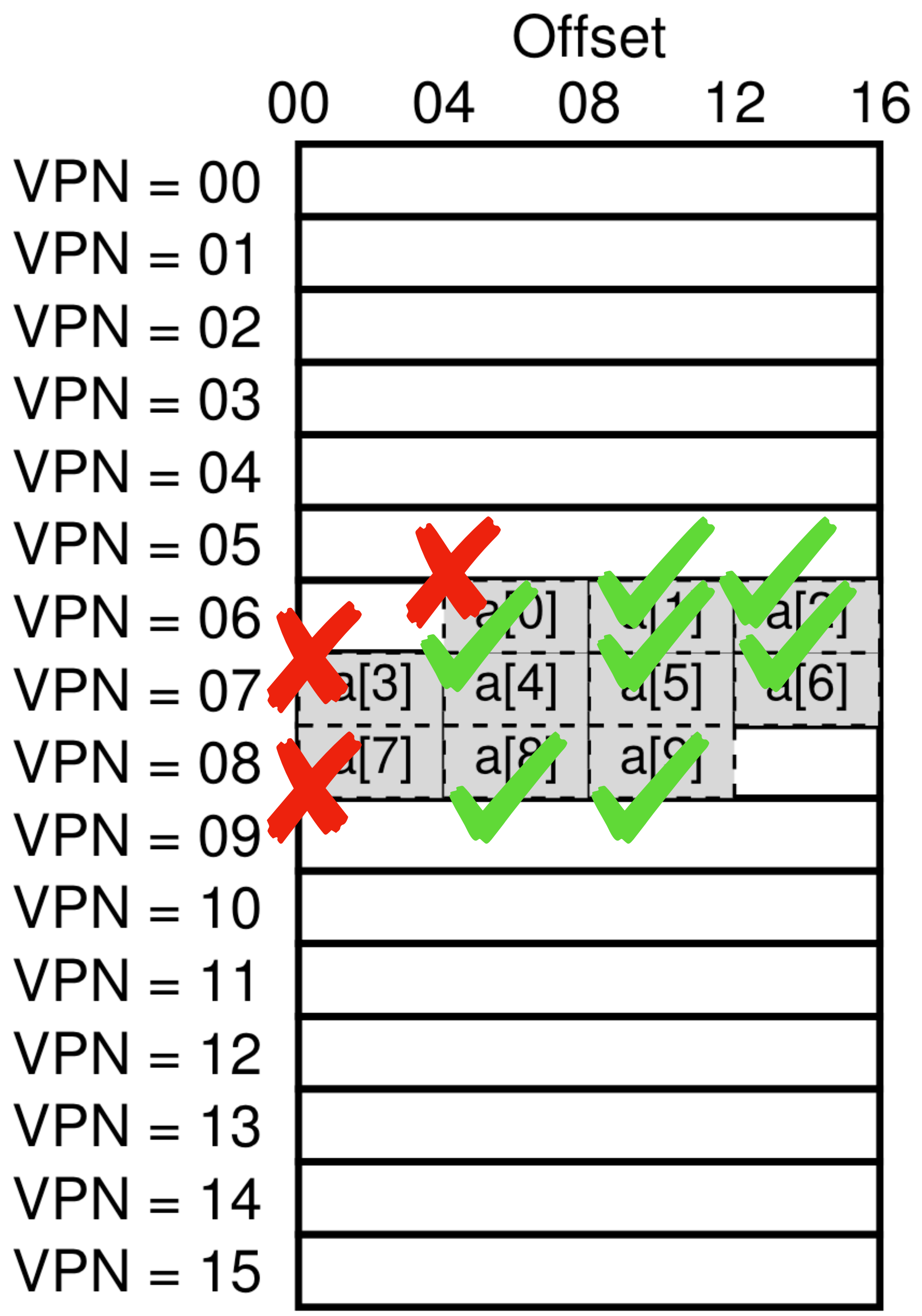
```
int sum = 0;
for (i = 0; i < 10; i++) {
    sum += a[i];
}
```



Which programs will run faster?

Which programs will have lesser TLB misses?

```
int sum = 0;
for (i = 0; i < 10; i++) {
    sum += a[i];
}
```

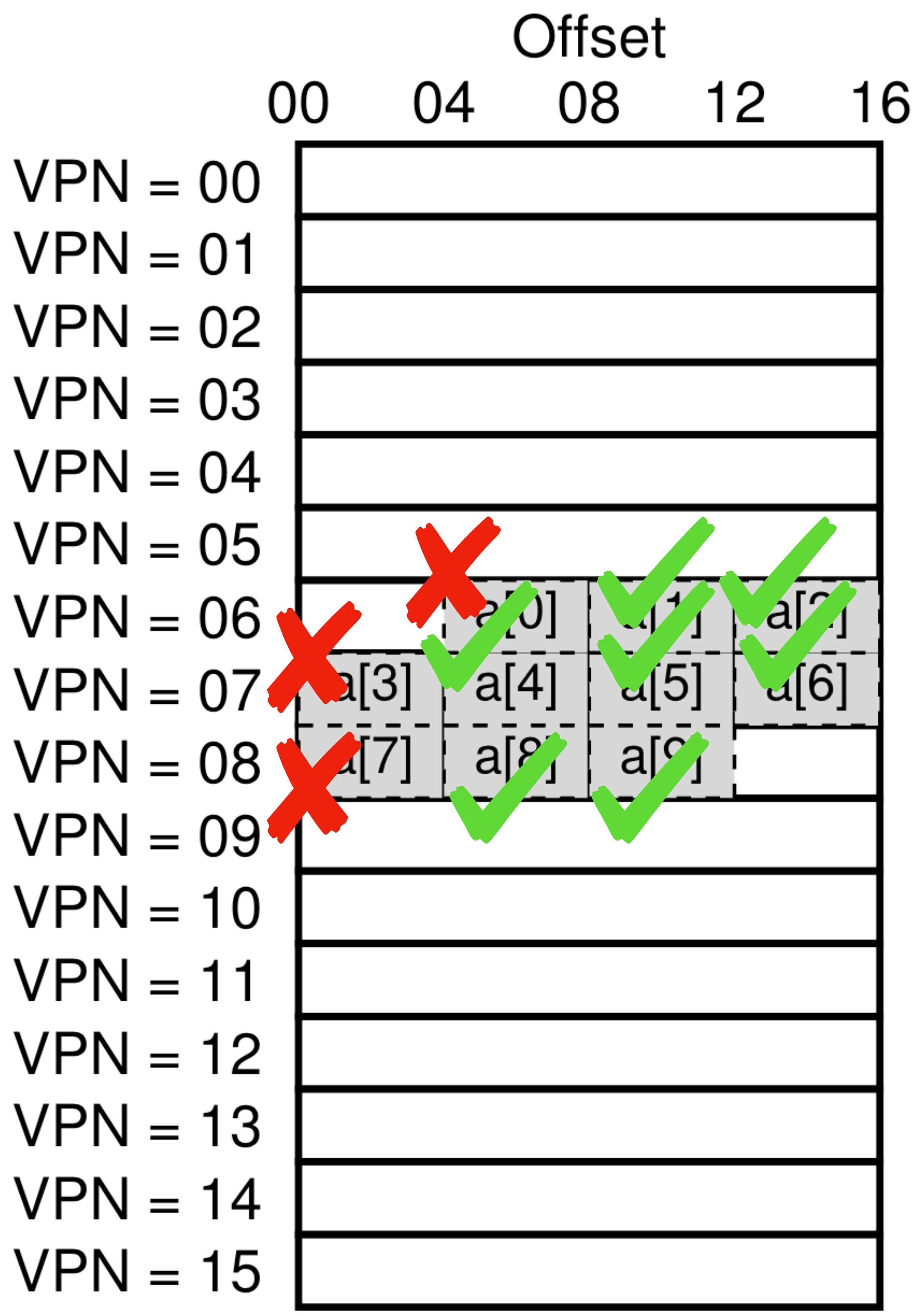


- High **spatial locality**: after the program accessed a memory location, it will access a nearby memory location

Which programs will run faster?

Which programs will have lesser TLB misses?

```
int sum = 0;
for (i = 0; i < 10; i++) {
    sum += a[i];
}
```



- High **spatial locality**: after the program accessed a memory location, it will access a nearby memory location

- High **temporal locality**: after the program accessed a memory location, it will soon access it again

Example bad programs

- Low spatial and temporal locality: most accesses lead to TLB miss

Example bad programs

- Low spatial and temporal locality: most accesses lead to TLB miss
- Large hash table with random access

Example bad programs

- Low spatial and temporal locality: most accesses lead to TLB miss
- Large hash table with random access

```
int get(int I)

  return a[I];
```

Example bad programs

- Low spatial and temporal locality: most accesses lead to TLB miss
- Large hash table with random access

```
int get(int I)

  return a[I];
```

VPN=1	a[0]	a[1]	a[2]	a[3]
VPN=2	a[4]	a[5]	a[6]	a[7]
VPN=3	a[8]	a[9]	a[10]	a[11]
VPN=4	a[12]	a[13]	a[14]	a[15]
VPN=5	a[16]	a[17]	a[18]	a[19]

Example bad programs

- Low spatial and temporal locality: most accesses lead to TLB miss

- Large hash table with random access

```
int get(int I)

  return a[I];
```

VPN=1	a[0]	a[1]	a[2]	a[3]
VPN=2	a[4]	a[5]	a[6]	a[7]
VPN=3	a[8]	a[9]	a[10]	a[11]
VPN=4	a[12]	a[13]	a[14]	a[15]
VPN=5	a[16]	a[17]	a[18]	a[19]

- Traversing large graphs: two neighbours can be on different pages

Example bad programs

- Low spatial and temporal locality: most accesses lead to TLB miss
- Large hash table with random access

```
int get(int I)

  return a[I];
```

VPN=1	a[0]	a[1]	a[2]	a[3]
VPN=2	a[4]	a[5]	a[6]	a[7]
VPN=3	a[8]	a[9]	a[10]	a[11]
VPN=4	a[12]	a[13]	a[14]	a[15]
VPN=5	a[16]	a[17]	a[18]	a[19]

- Traversing large graphs: two neighbours can be on different pages
- Ok for small hash tables, small graphs.
  - Working set of the program: Amount of memory that it touches

Example bad programs

- Low spatial and temporal locality: most accesses lead to TLB miss
- Large hash table with random access

```
int get(int I)

  return a[I];
```

VPN=1	a[0]	a[1]	a[2]	a[3]
VPN=2	a[4]	a[5]	a[6]	a[7]
VPN=3	a[8]	a[9]	a[10]	a[11]
VPN=4	a[12]	a[13]	a[14]	a[15]
VPN=5	a[16]	a[17]	a[18]	a[19]

- Traversing large graphs: two neighbours can be on different pages
- Ok for small hash tables, small graphs.
  - Working set of the program: Amount of memory that it touches
- TLB reach = (number of TLB entries) * (page size)

Example bad programs

- Low spatial and temporal locality: most accesses lead to TLB miss
- Large hash table with random access

```
int get(int I)

  return a[I];
```

VPN=1	a[0]	a[1]	a[2]	a[3]
VPN=2	a[4]	a[5]	a[6]	a[7]
VPN=3	a[8]	a[9]	a[10]	a[11]
VPN=4	a[12]	a[13]	a[14]	a[15]
VPN=5	a[16]	a[17]	a[18]	a[19]

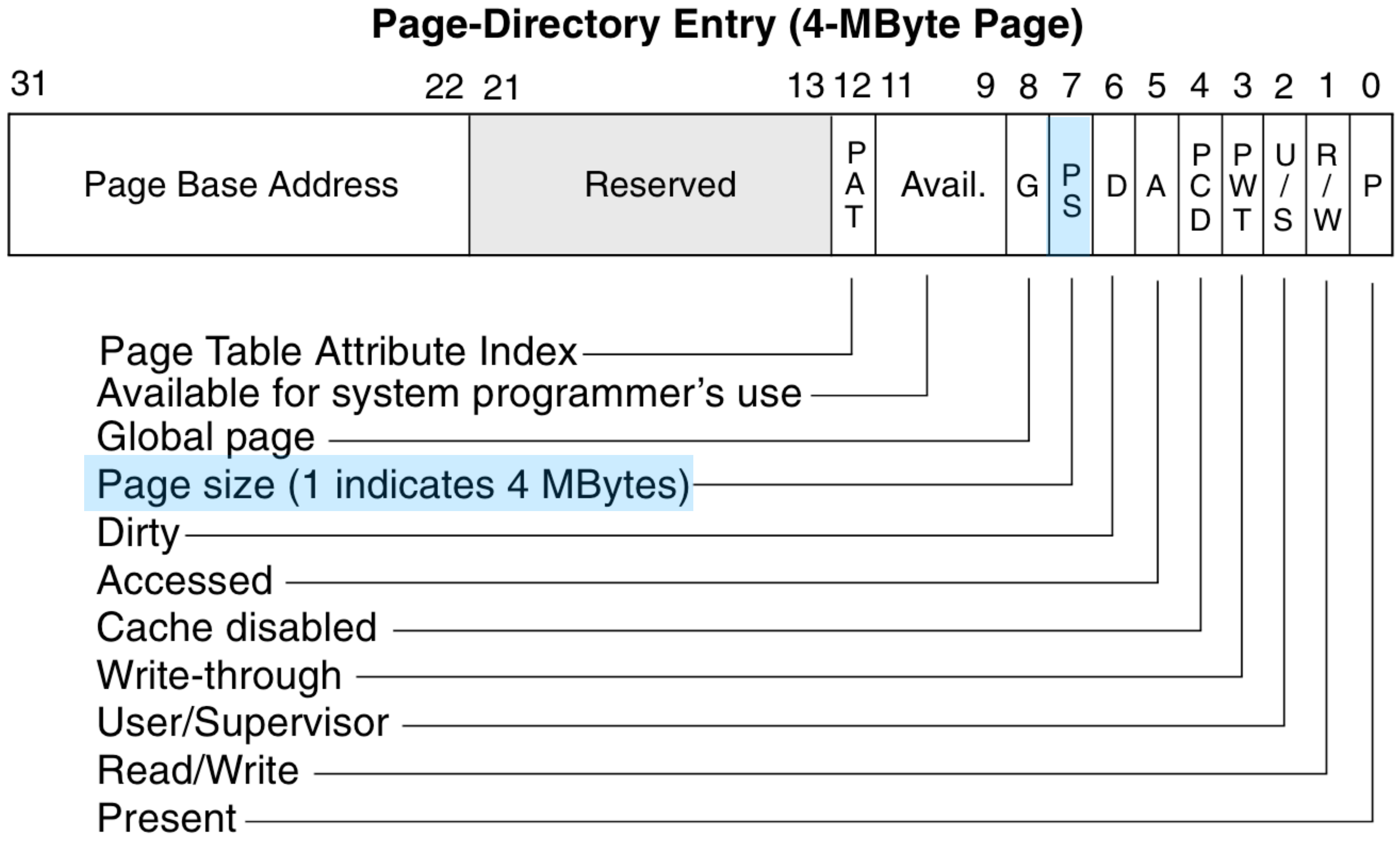
- Traversing large graphs: two neighbours can be on different pages
- Ok for small hash tables, small graphs.
  - Working set of the program: Amount of memory that it touches
- TLB reach = (number of TLB entries) * (page size)
- Bad performance when Working set > TLB reach

Increasing TLB reach

- Larger TLBs
    - 32 -> 64 entries.
    - Larger caches => slower hits

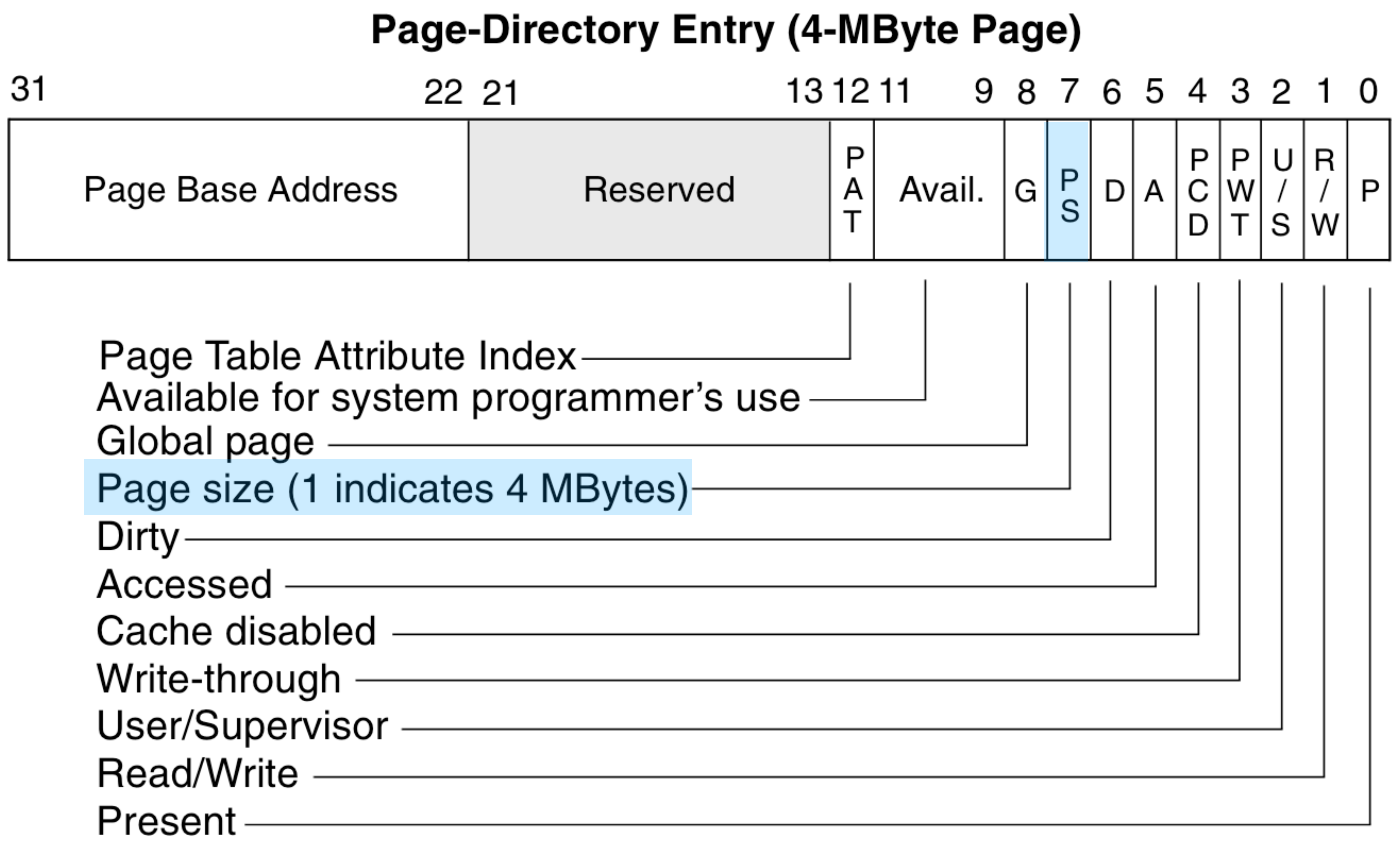
Increasing TLB reach

- Larger TLBs
    - 32 -> 64 entries.
    - Larger caches => slower hits
- Larger pages
    - 1 TLB entry for 4MB of addresses (with a 4MB page)
    - 1024 TLB entries for 4MB of addresses (with 4KB pages)



Increasing TLB reach

- Larger TLBs
  - 32 -> 64 entries.
  - Larger caches => slower hits
- Larger pages
  - 1 TLB entry for 4MB of addresses (with a 4MB page)
  - 1024 TLB entries for 4MB of addresses (with 4KB pages)
- Allocating large pages on Linux:




```
posix_memalign(void **memptr, size_t alignment, size_t size);
int madvise(void *addr, size_t length, int advice= MADV_HUGEPAGE);
```

TLB on my x86-64 machine

```
$ cpuid
  ..
  (simple synth)  = Intel Core (unknown type) (Kaby Lake / Coffee Lake) {Skylake}, 14nm
  ..
   cache and TLB information (2):
      0x63: data TLB: 2M/4M pages, 4-way, 32 entries
            data TLB: 1G pages, 4-way, 4 entries
      0x03: data TLB: 4K pages, 4-way, 64 entries
      0x76: instruction TLB: 2M/4M pages, fully, 8 entries
      0xb5: instruction TLB: 4K, 8-way, 64 entries
      0xc3: L2 TLB: 4K/2M pages, 6-way, 1536 entries
 ..
```

TLB on my x86-64 machine

```
$ cpuid
  ..
  (simple synth)  = Intel Core (unknown type) (Kaby Lake / Coffee Lake) {Skylake}, 14nm
  ..
   cache and TLB information (2):
      0x63: data TLB: 2M/4M pages, 4-way, 32 entries
            data TLB: 1G pages, 4-way, 4 entries
      0x03: data TLB: 4K pages, 4-way, 64 entries
      0x76: instruction TLB: 2M/4M pages, fully, 8 entries
      0xb5: instruction TLB: 4K, 8-way, 64 entries
      0xc3: L2 TLB: 4K/2M pages, 6-way, 1536 entries
 ..
```

Separate TLBs for different page sizes

TLB on my x86-64 machine

```
$ cpuid
  ..
  (simple synth)  = Intel Core (unknown type) (Kaby Lake / Coffee Lake) {Skylake}, 14nm
  ..
   cache and TLB information (2):
      0x63: data TLB: 2M/4M pages, 4-way, 32 entries
            data TLB: 1G pages, 4-way, 4 entries
      0x03: data TLB: 4K pages, 4-way, 64 entries
      0x76: instruction TLB: 2M/4M pages, fully, 8 entries
      0xb5: instruction TLB: 4K, 8-way, 64 entries
      0xc3: L2 TLB: 4K/2M pages, 6-way, 1536 entries
 ..
```

Separate TLBs for different page sizes

Data TLB reach:

- 4KB * 64 = 256KB
- 4MB * 32 = 128MB
- 1GB * 4 = 4GB

TLB on my x86-64 machine

```
$ cpuid
  ..
  (simple synth)  = Intel Core (unknown type) (Kaby Lake / Coffee Lake) {Skylake}, 14nm
  ..
   cache and TLB information (2):
      0x63: data TLB: 2M/4M pages, 4-way, 32 entries
            data TLB: 1G pages, 4-way, 4 entries
      0x03: data TLB: 4K pages, 4-way, 64 entries
      0x76: instruction TLB: 2M/4M pages, fully, 8 entries
      0xb5: instruction TLB: 4K, 8-way, 64 entries
      0xc3: L2 TLB: 4K/2M pages, 6-way, 1536 entries
 ..
```

Separate TLBs for different page sizes

Separate TLBs for instruction and data

Data TLB reach:

- 4KB * 64 = 256KB
- 4MB * 32 = 128MB
- 1GB * 4 = 4GB

TLB on my x86-64 machine

```
$ cpuid
  ..
  (simple synth)  = Intel Core (unknown type) (Kaby Lake / Coffee Lake) {Skylake}, 14nm
  ..
   cache and TLB information (2):
      0x63: data TLB: 2M/4M pages, 4-way, 32 entries
            data TLB: 1G pages, 4-way, 4 entries
      0x03: data TLB: 4K pages, 4-way, 64 entries
      0x76: instruction TLB: 2M/4M pages, fully, 8 entries
      0xb5: instruction TLB: 4K, 8-way, 64 entries
      0xc3: L2 TLB: 4K/2M pages, 6-way, 1536 entries
 ..
```

Separate TLBs for different page sizes

Separate TLBs for instruction and data

Data TLB reach:

- 4KB * 64 = 256KB
- 4MB * 32 = 128MB
- 1GB * 4 = 4GB

Instruction TLB reach:

- 4KB * 64 = 256KB
- 4MB * 8 = 32MB

TLB on my x86-64 machine

```
$ cpuid
  ..
  (simple synth)  = Intel Core (unknown type) (Kaby Lake / Coffee Lake) {Skylake}, 14nm
  ..
   cache and TLB information (2):
      0x63: data TLB: 2M/4M pages, 4-way, 32 entries
            data TLB: 1G pages, 4-way, 4 entries
      0x03: data TLB: 4K pages, 4-way, 64 entries
      0x76: instruction TLB: 2M/4M pages, fully, 8 entries
      0xb5: instruction TLB: 4K, 8-way, 64 entries
      0xc3: L2 TLB: 4K/2M pages, 6-way, 1536 entries
 ..
```

Separate TLBs for different page sizes

Separate TLBs for instruction and data

Much larger (slower) L2 TLB

Data TLB reach:

- 4KB * 64 = 256KB
- 4MB * 32 = 128MB
- 1GB * 4 = 4GB

Instruction TLB reach:

- 4KB * 64 = 256KB
- 4MB * 8 = 32MB

TLB on my x86-64 machine

```
$ cpuid
  ..
  (simple synth)  = Intel Core (unknown type) (Kaby Lake / Coffee Lake) {Skylake}, 14nm
  ..
   cache and TLB information (2):
      0x63: data TLB: 2M/4M pages, 4-way, 32 entries
            data TLB: 1G pages, 4-way, 4 entries
      0x03: data TLB: 4K pages, 4-way, 64 entries
      0x76: instruction TLB: 2M/4M pages, fully, 8 entries
      0xb5: instruction TLB: 4K, 8-way, 64 entries
      0xc3: L2 TLB: 4K/2M pages, 6-way, 1536 entries
 ..
```

Separate TLBs for different page sizes

Separate TLBs for instruction and data

Much larger (slower) L2 TLB

Data TLB reach:

- 4KB * 64 = 256KB
- 4MB * 32 = 128MB
- 1GB * 4 = 4GB

Instruction TLB reach:

- 4KB * 64 = 256KB
- 4MB * 8 = 32MB

L2 TLB reach:

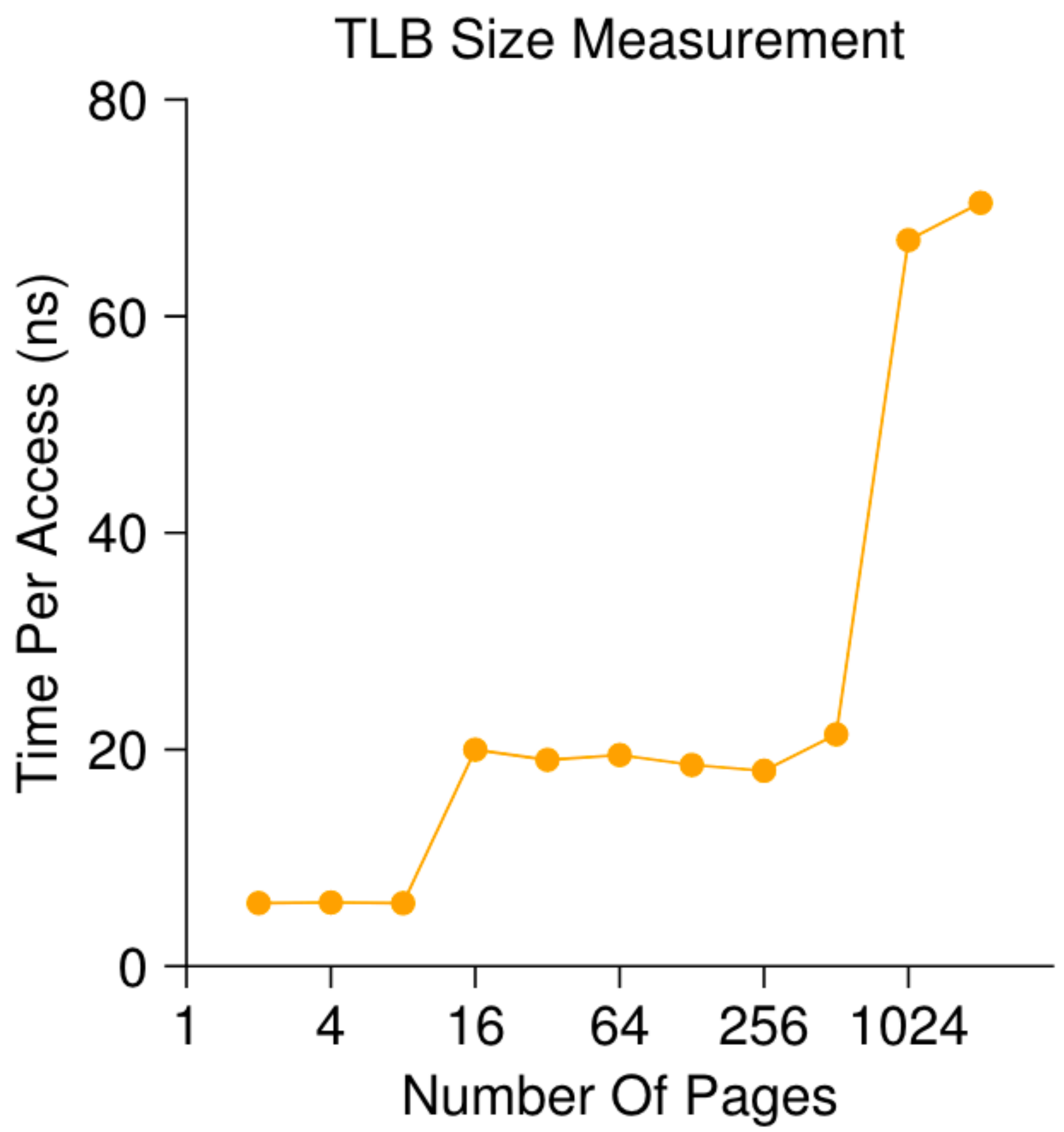
- 4KB * 1536 = 6MB
- 2MB * 1536 = 3GB

TLB size, multiple TLBs example

```
int jump = PAGESIZE / sizeof(int);
for (i = 0; i < NUMPAGES * jump; i += jump) {
      a[i] += 1;
}
```

TLB size, multiple TLBs example

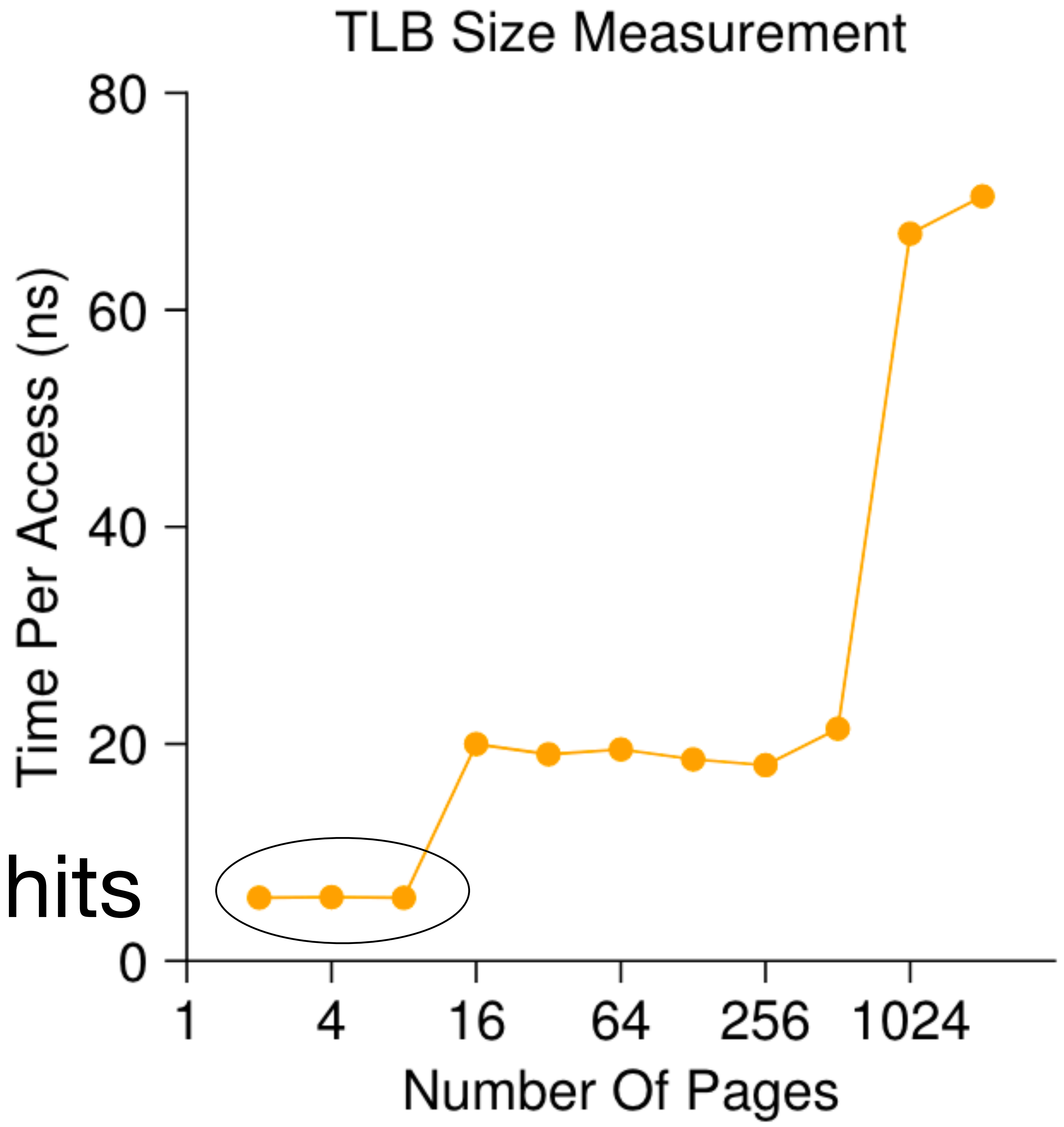
```
int jump = PAGESIZE / sizeof(int);
for (i = 0; i < NUMPAGES * jump; i += jump) {
      a[i] += 1;
}
```



TLB size, multiple TLBs example

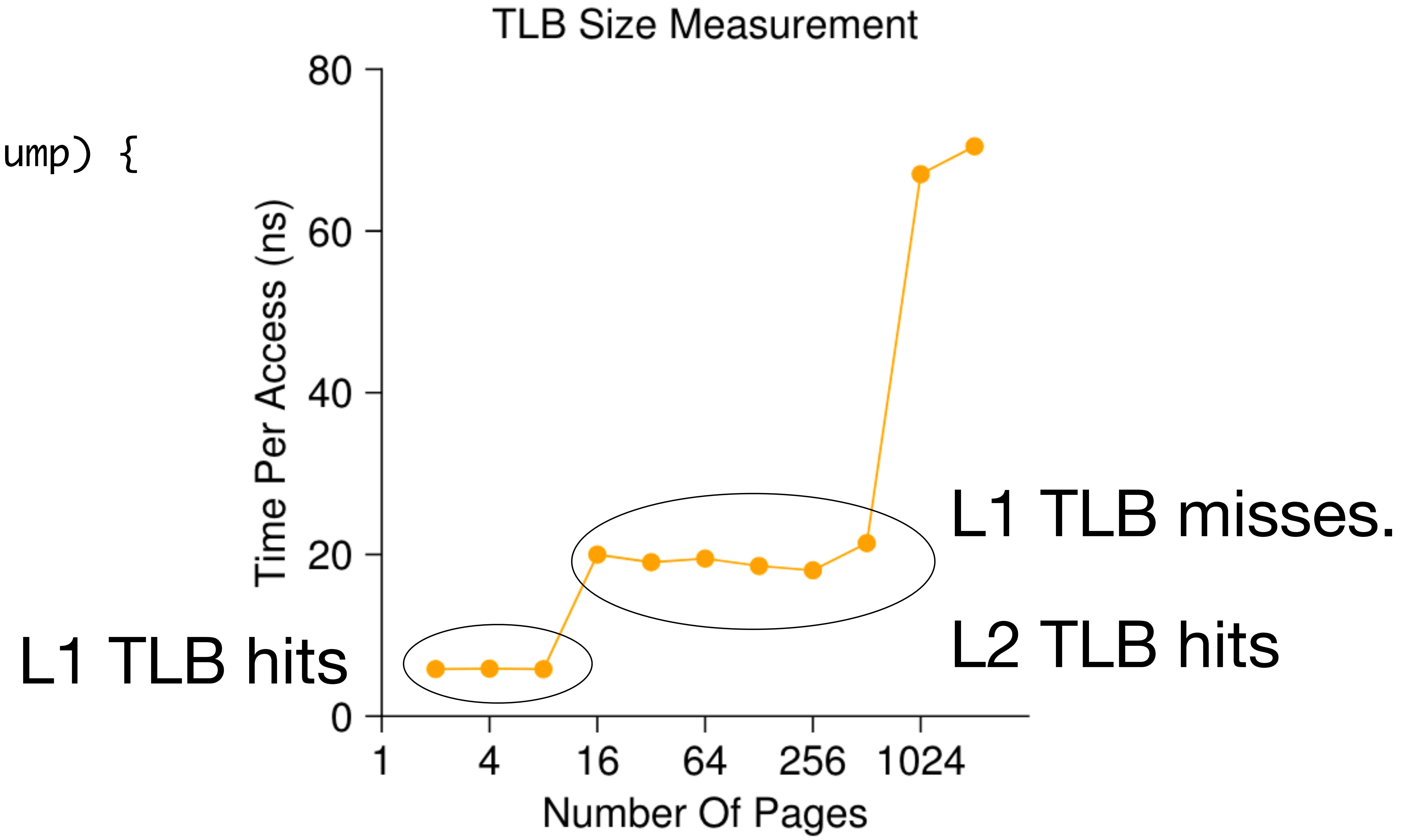
```
int jump = PAGESIZE / sizeof(int);
for (i = 0; i < NUMPAGES * jump; i += jump) {
      a[i] += 1;
}
```

L1 TLB hits



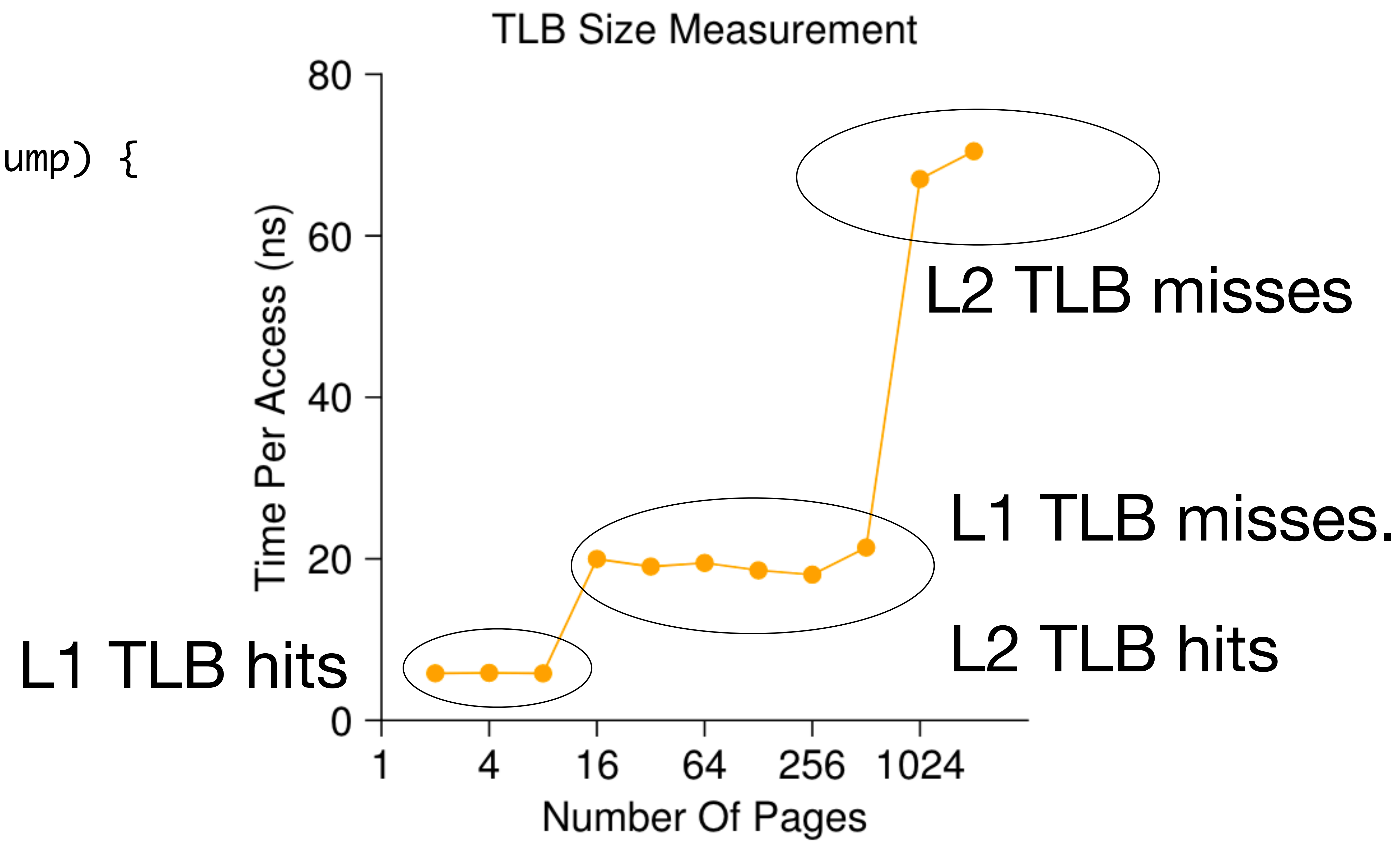
TLB size, multiple TLBs example

```
int jump = PAGESIZE / sizeof(int);
for (i = 0; i < NUMPAGES * jump; i += jump) {
      a[i] += 1;
}
```



TLB size, multiple TLBs example

```
int jump = PAGESIZE / sizeof(int);
for (i = 0; i < NUMPAGES * jump; i += jump) {
      a[i] += 1;
}
```



TLB replacement policies

- Need to replace an entry after TLB is full. Which entry to replace to minimise TLB miss rate?

TLB replacement policies

- Need to replace an entry after TLB is full. Which entry to replace to minimise TLB miss rate?
- Least recently used (LRU):
  - If an entry hasn’t been used recently, it is unlikely to be used soon => assumes spatial and temporal locality of access.
  - Corner case behaviours: Program cycling over N+1 pages.

TLB replacement policies

- Need to replace an entry after TLB is full. Which entry to replace to minimise TLB miss rate?
- Least recently used (LRU):
  - If an entry hasn't been used recently, it is unlikely to be used soon => assumes spatial and temporal locality of access.
  - Corner case behaviours: Program cycling over N+1 pages.
- Hardware typically implements something simple
  - FIFO
  - Random replacement: Just pick an entry randomly

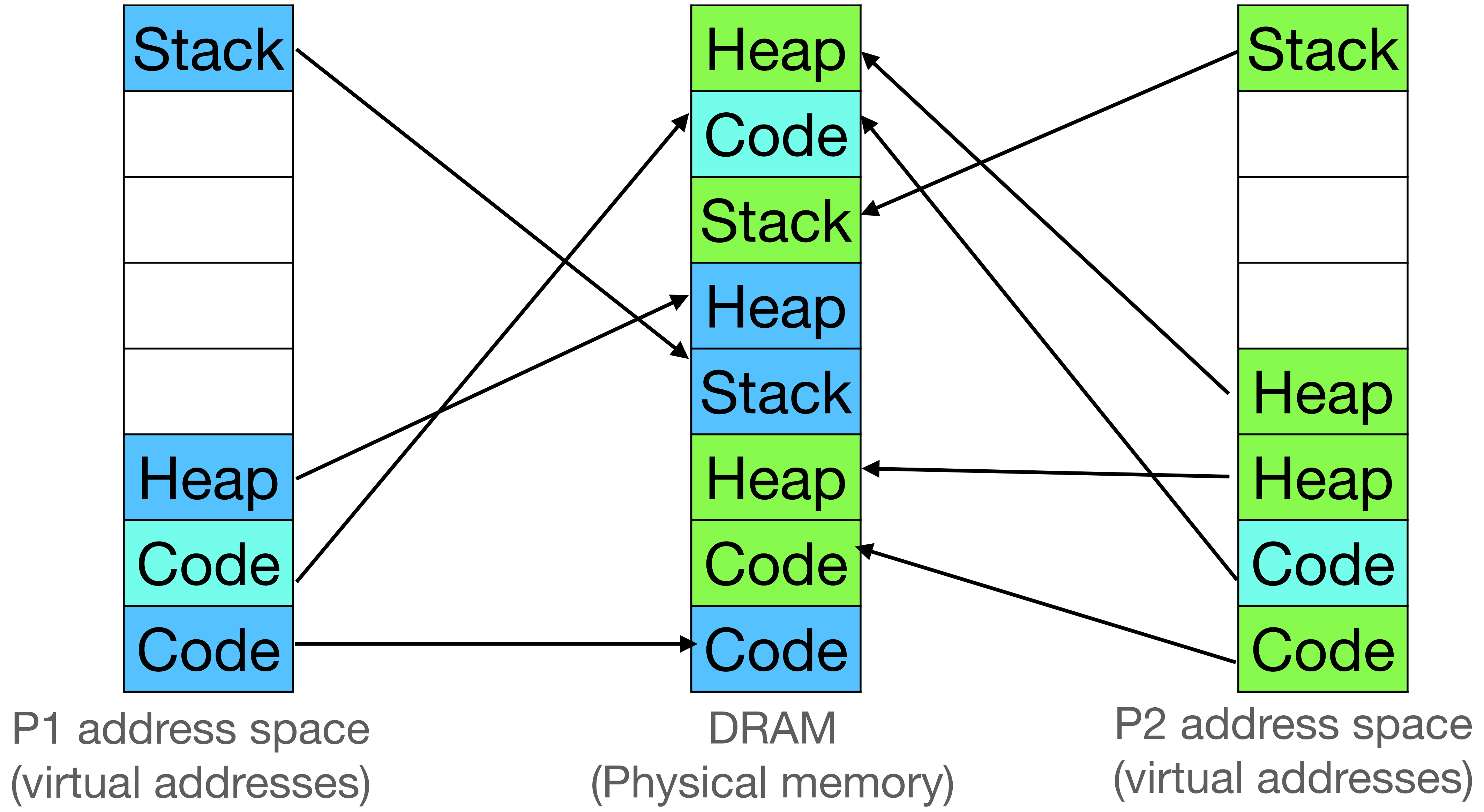
Context switching

- During context switch, OS changes CR3 register to change page table
- Privileged operation

Virtual page number	Physical page number
7	3
6	x
5	x
4	x
3	x
2	4
1	6
0	0

Virtual page number	Physical page number
7	5
6	x
5	x
4	x
3	7
2	3
1	6
0	1

Virtual page number	Physical page number
0	0
2	4



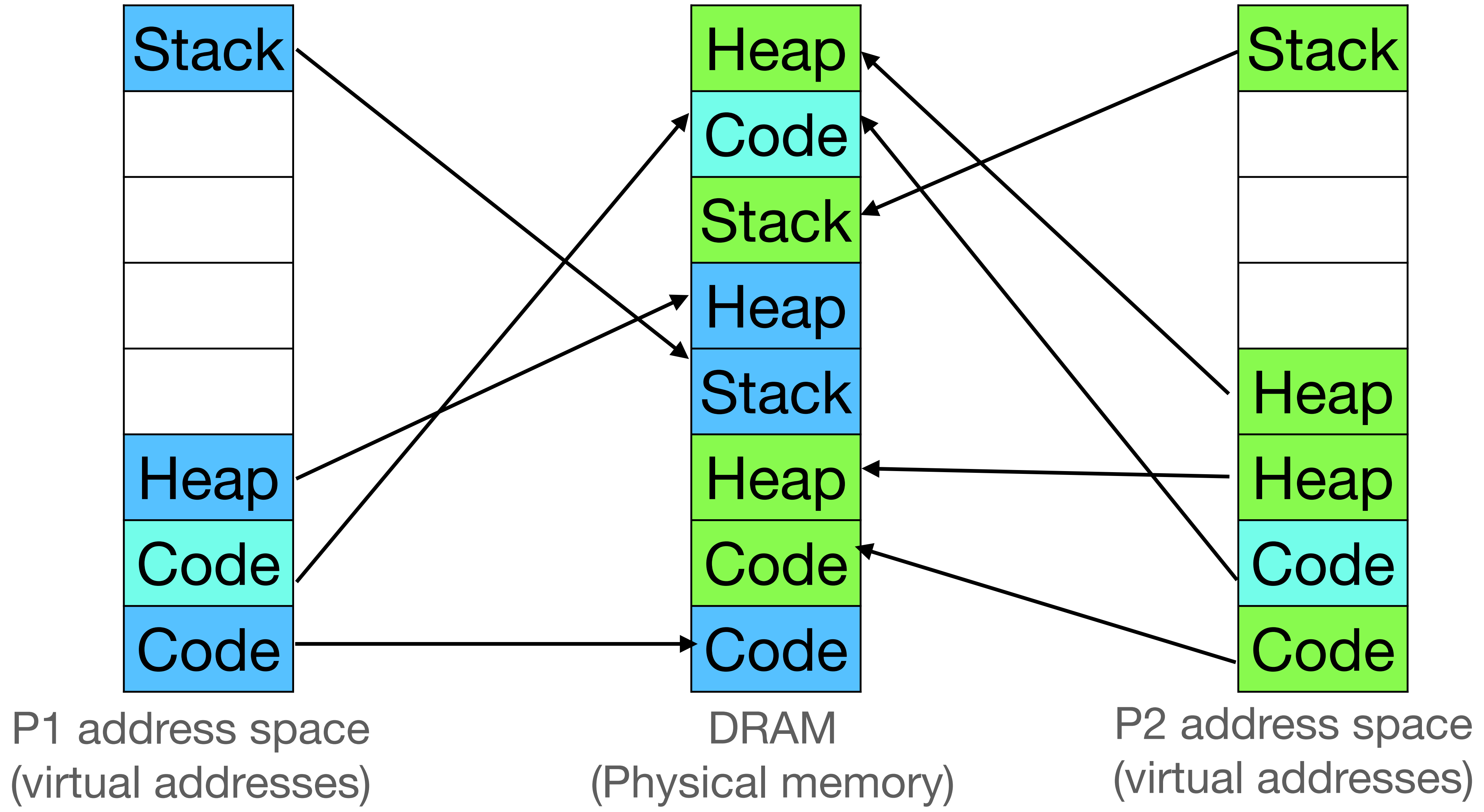
Context switching

- During context switch, OS changes CR3 register to change page table
- Privileged operation
- Marks each TLB entry invalid. Every memory access after context switch causes TLB miss!

Virtual page number	Physical page number
7	3
6	x
5	x
4	x
3	x
2	4
1	6
0	0

Virtual page number	Physical page number
7	5
6	x
5	x
4	x
3	7
2	3
1	6
0	1

Virtual page number	Physical page number
0	0
2	4



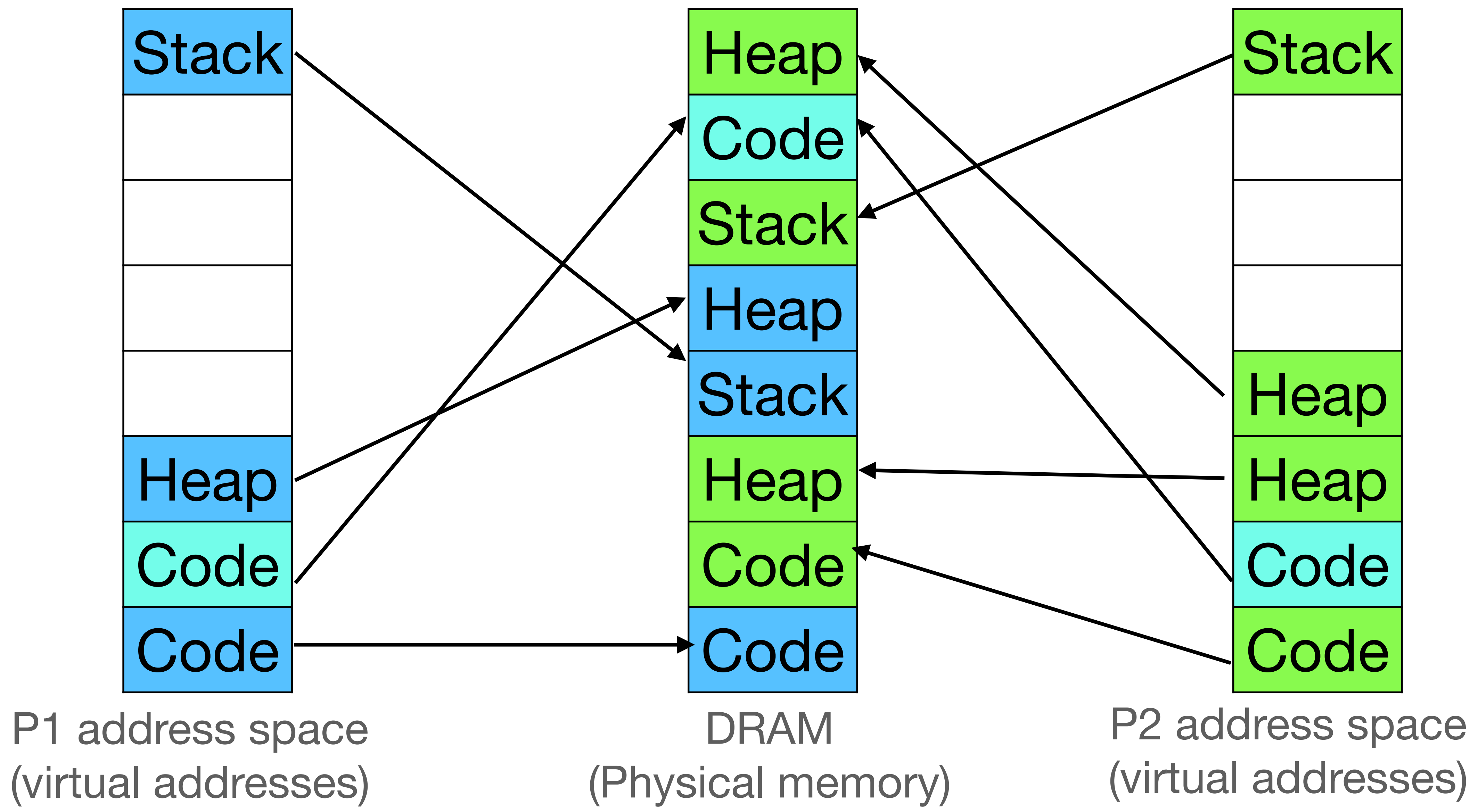
Context switching

- During context switch, OS changes CR3 register to change page table
- Privileged operation
- Marks each TLB entry invalid. Every memory access after context switch causes TLB miss!

Virtual page number	Physical page number
7	3
6	x
5	x
4	x
3	x
2	4
1	6
0	0

Virtual page number	Physical page number
7	5
6	x
5	x
4	x
3	7
2	3
1	6
0	1

Virtual page number	Physical page number
~~0~~	~~0~~
2	4



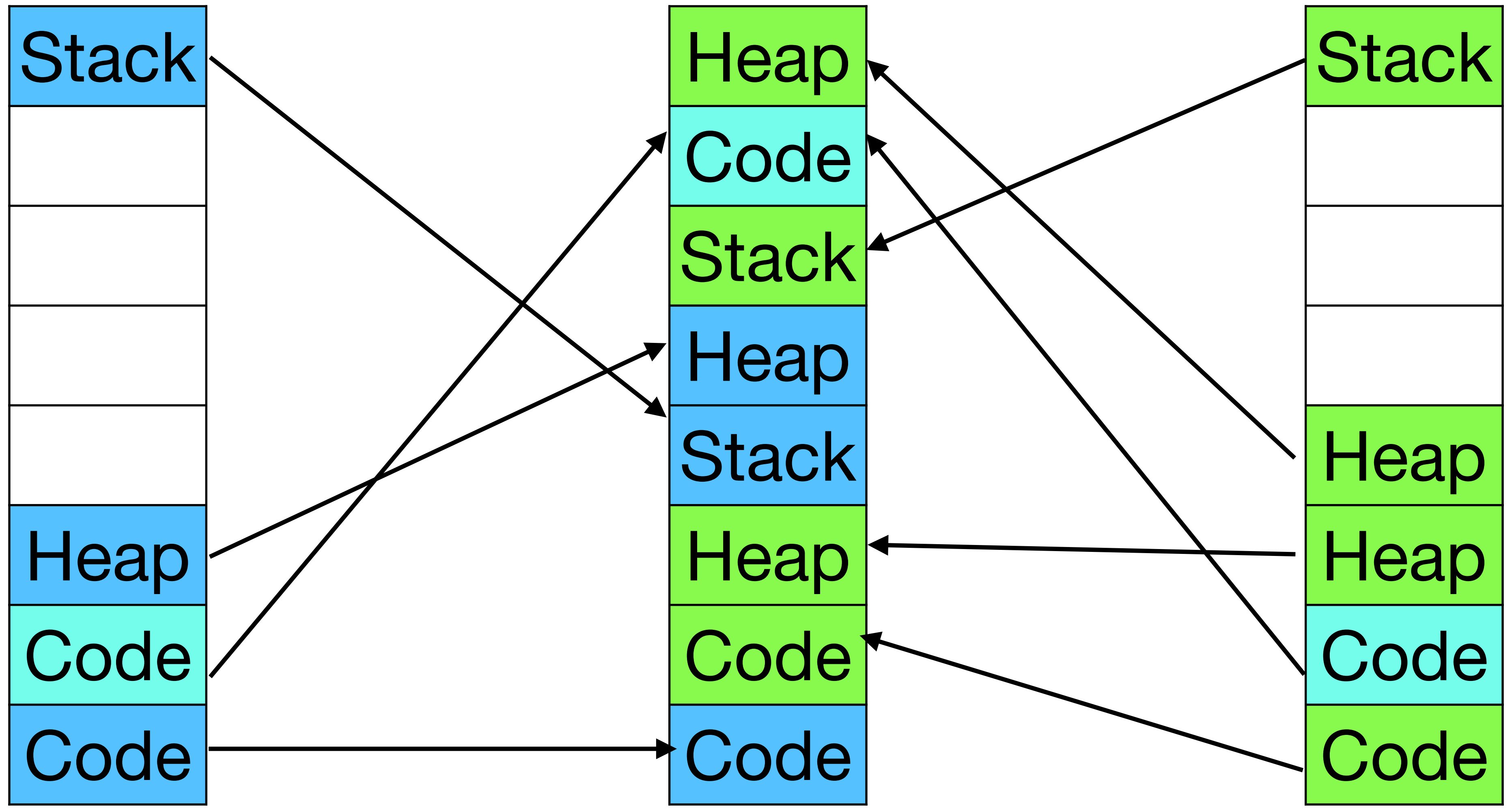
Context switching

- During context switch, OS changes CR3 register to change page table
- Privileged operation
- Marks each TLB entry invalid. Every memory access after context switch causes TLB miss!

Virtual page number	Physical page number
~~0~~	~~0~~
~~2~~	~~4~~

Virtual page number	Physical page number
7	3
6	x
5	x
4	x
3	x
2	4
1	6
0	0

Virtual page number	Physical page number
7	5
6	x
5	x
4	x
3	7
2	3
1	6
0	1



P1 address space (virtual addresses)

DRAM (Physical memory)

P2 address space (virtual addresses)

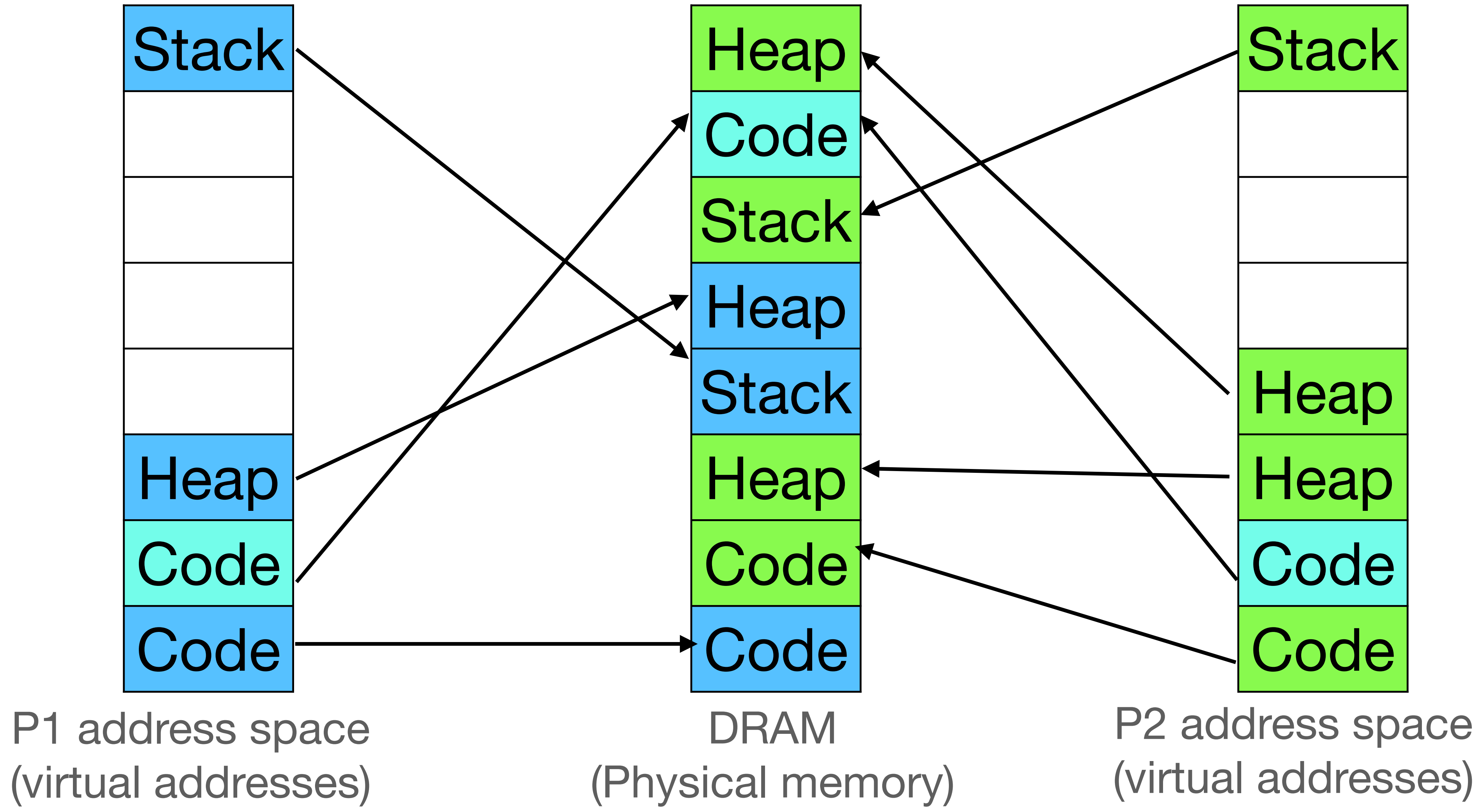
Context switching

- During context switch, OS changes CR3 register to change page table
- Privileged operation
- Marks each TLB entry invalid. Every memory access after context switch causes TLB miss!

Virtual page number	Physical page number
7	3
6	x
5	x
4	x
3	x
2	4
1	6
0	0

Virtual page number	Physical page number
7	5
6	x
5	x
4	x
3	7
2	3
1	6
0	1

Virtual page number	Physical page number
~~0~~	~~0~~
~~2~~	~~4~~
0	1



INVLPG instruction

- TLB is neither write-back, nor write-through cache
  - Need to run INVLPG <virtual page number> when a page table entry is modified
- Similar to how OS needs to reload segment selector when corresponding GDT entry is changed

Tagged TLBs

- TLB entries are “tagged” with a *process context identifier*

TLB

Virtual page number	Physical page number	Process context identifier
0	0	P1
2	4	P1
0	1	P2

Tagged TLBs

- TLB entries are "tagged" with a *process context identifier*
- Additional bits in CR3 register tells hardware about the current PCID (only available in x86-64)

TLB

Virtual page number	Physical page number	Process context identifier
0	0	P1
2	4	P1
0	1	P2

Tagged TLBs

- TLB entries are "tagged" with a *process context identifier*
- Additional bits in CR3 register tells hardware about the current PCID (only available in x86-64)
- Upon context switch:

TLB

Virtual page number	Physical page number	Process context identifier
0	0	P1
2	4	P1
0	1	P2

Tagged TLBs

- TLB entries are "tagged" with a *process context identifier*
- Additional bits in CR3 register tells hardware about the current PCID (only available in x86-64)
- Upon context switch:
  - OS changes CR3: PCID, page directory base

TLB

Virtual page number	Physical page number	Process context identifier
0	0	P1
2	4	P1
0	1	P2

Tagged TLBs

- TLB entries are “tagged” with a *process context identifier*
- Additional bits in CR3 register tells hardware about the current PCID (only available in x86-64)
- Upon context switch:
  - OS changes CR3: PCID, page directory base
  - Hardware *need not* invalidate TLB entries. ~5us

TLB

Virtual page number	Physical page number	Process context identifier
0	0	P1
2	4	P1
0	1	P2

Tagged TLBs

- TLB entries are “tagged” with a *process context identifier*
- Additional bits in CR3 register tells hardware about the current PCID (only available in x86-64)
- Upon context switch:
  - OS changes CR3: PCID, page directory base
  - Hardware *need not* invalidate TLB entries. ~5us
  - When process gets back control, some of its TLB entries might still be present!

TLB

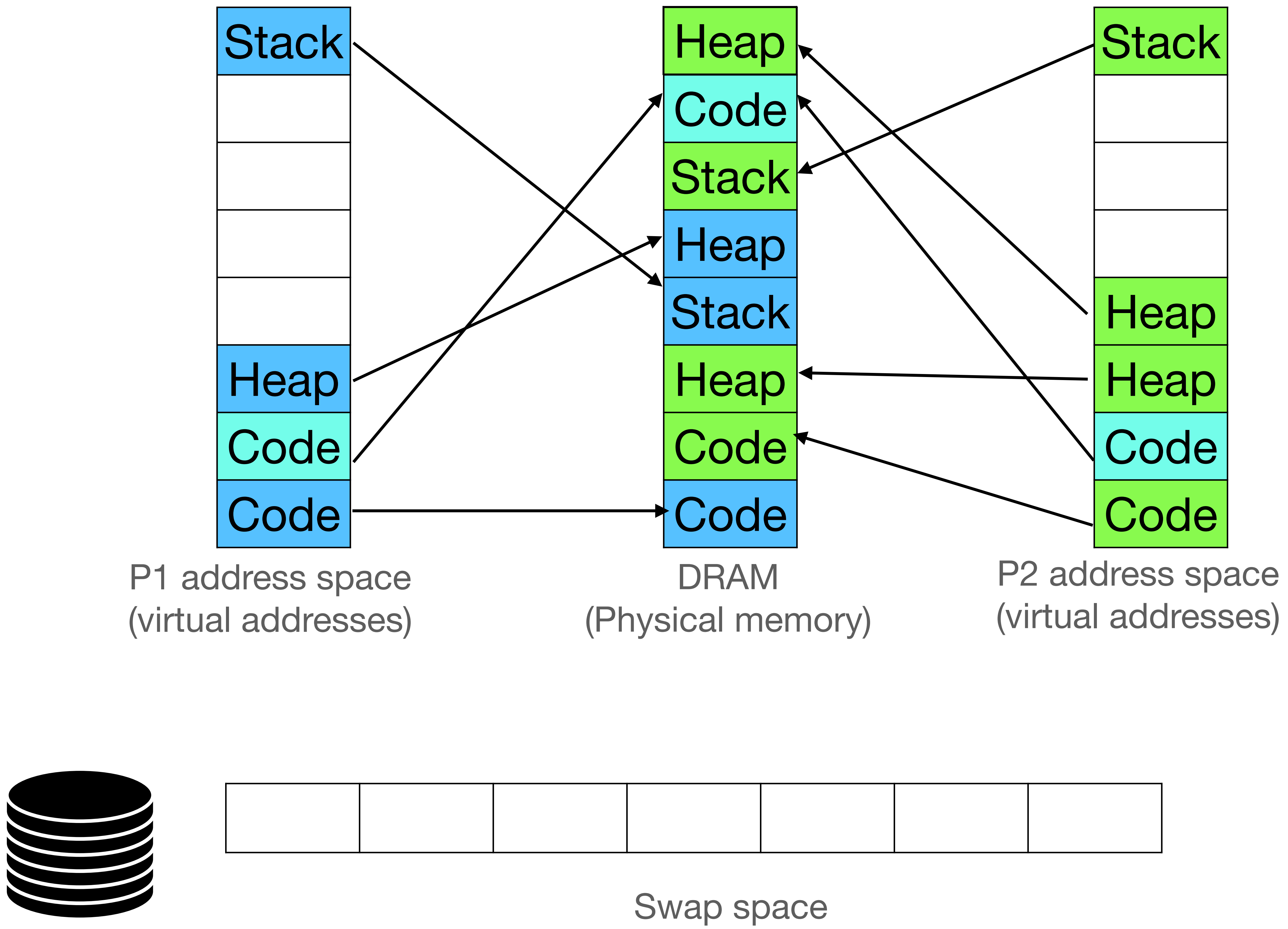
Virtual page number	Physical page number	Process context identifier
0	0	P1
2	4	P1
0	1	P2

Demand Paging

OSTEP Ch 21-22

Demand paging

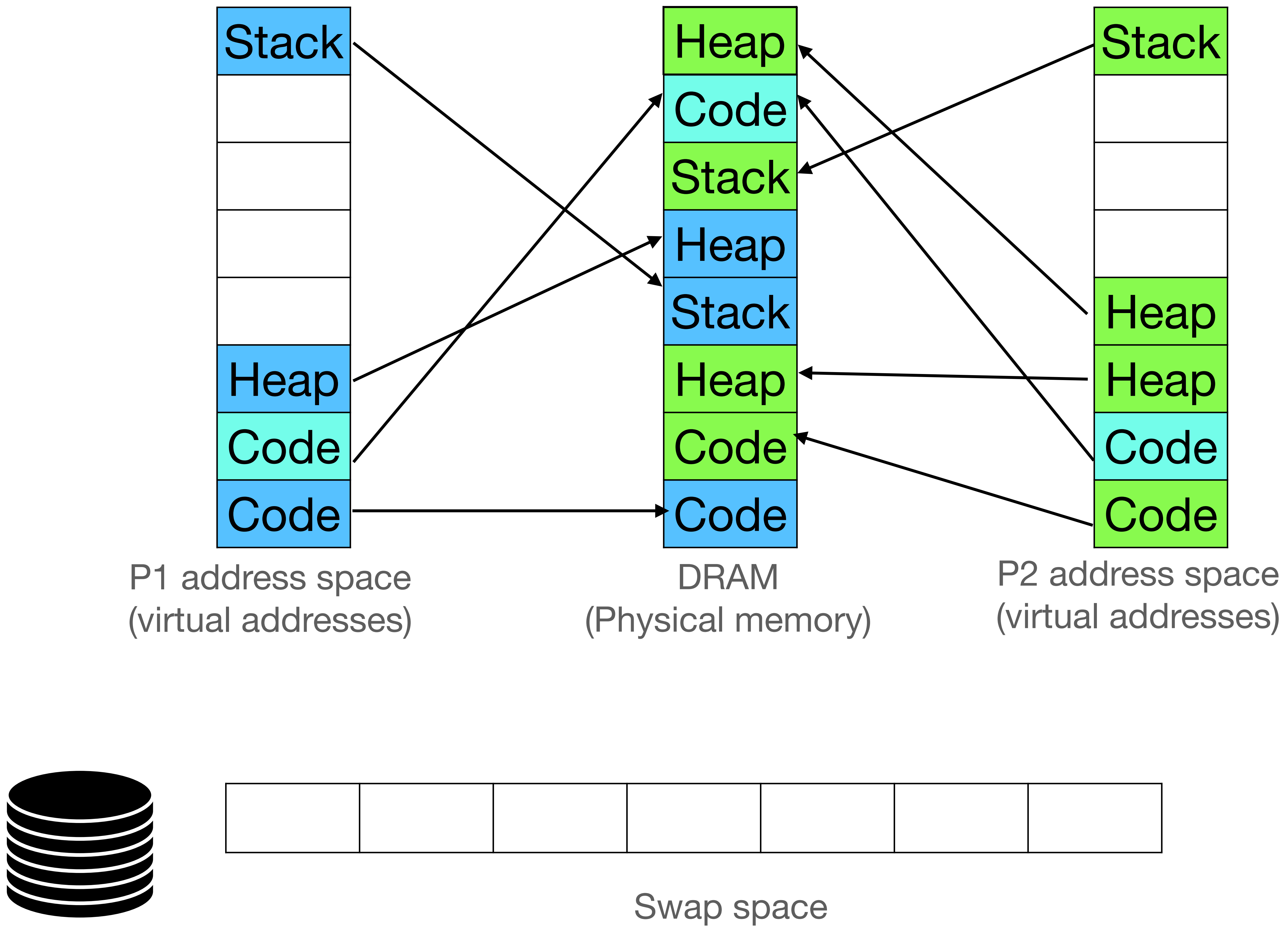
Providing illusion of large virtual address spaces



Demand paging

Providing illusion of large virtual address spaces

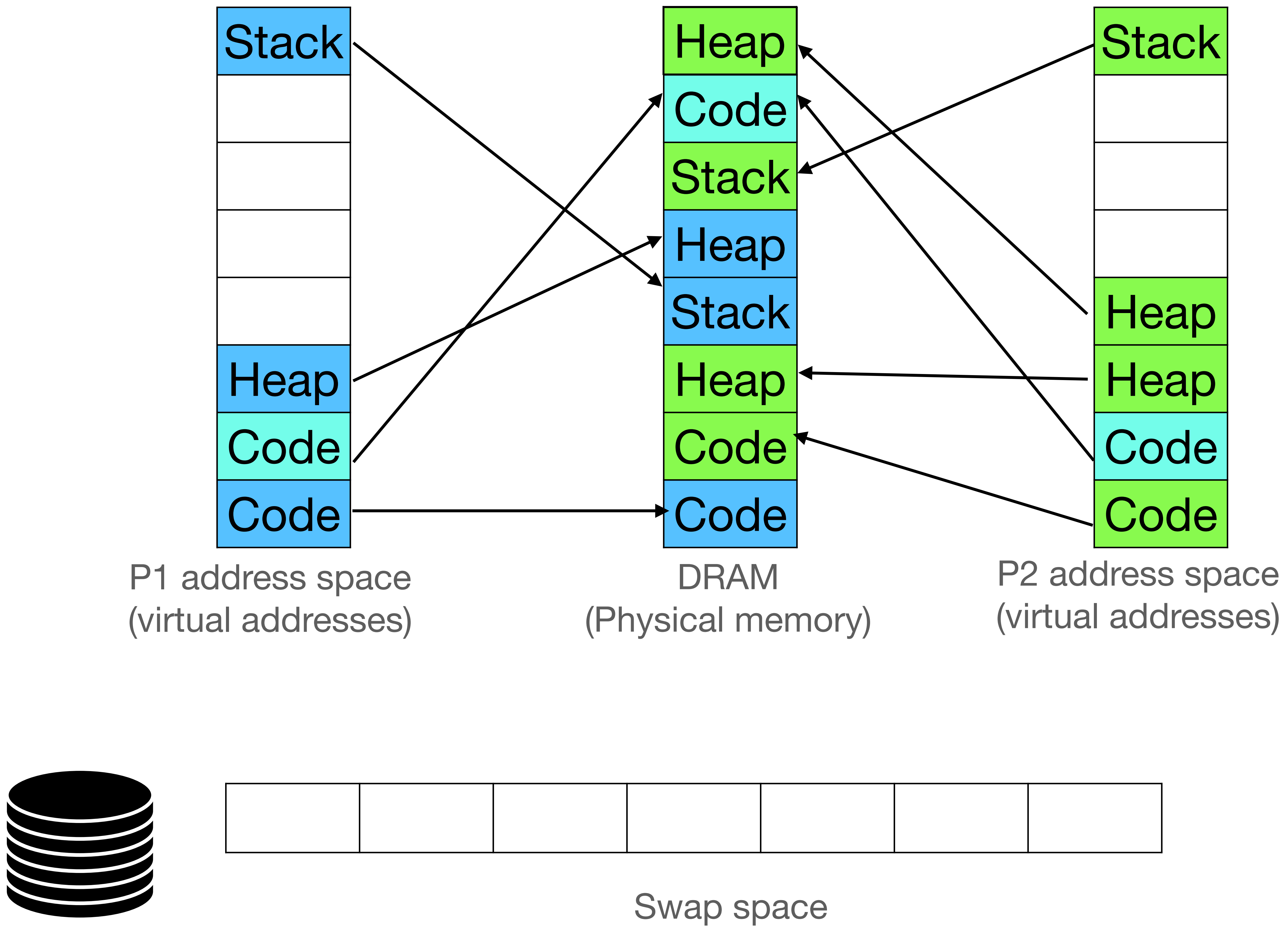
- Swap out a page to disk when physical memory is about to run out of space:



Demand paging

Providing illusion of large virtual address spaces

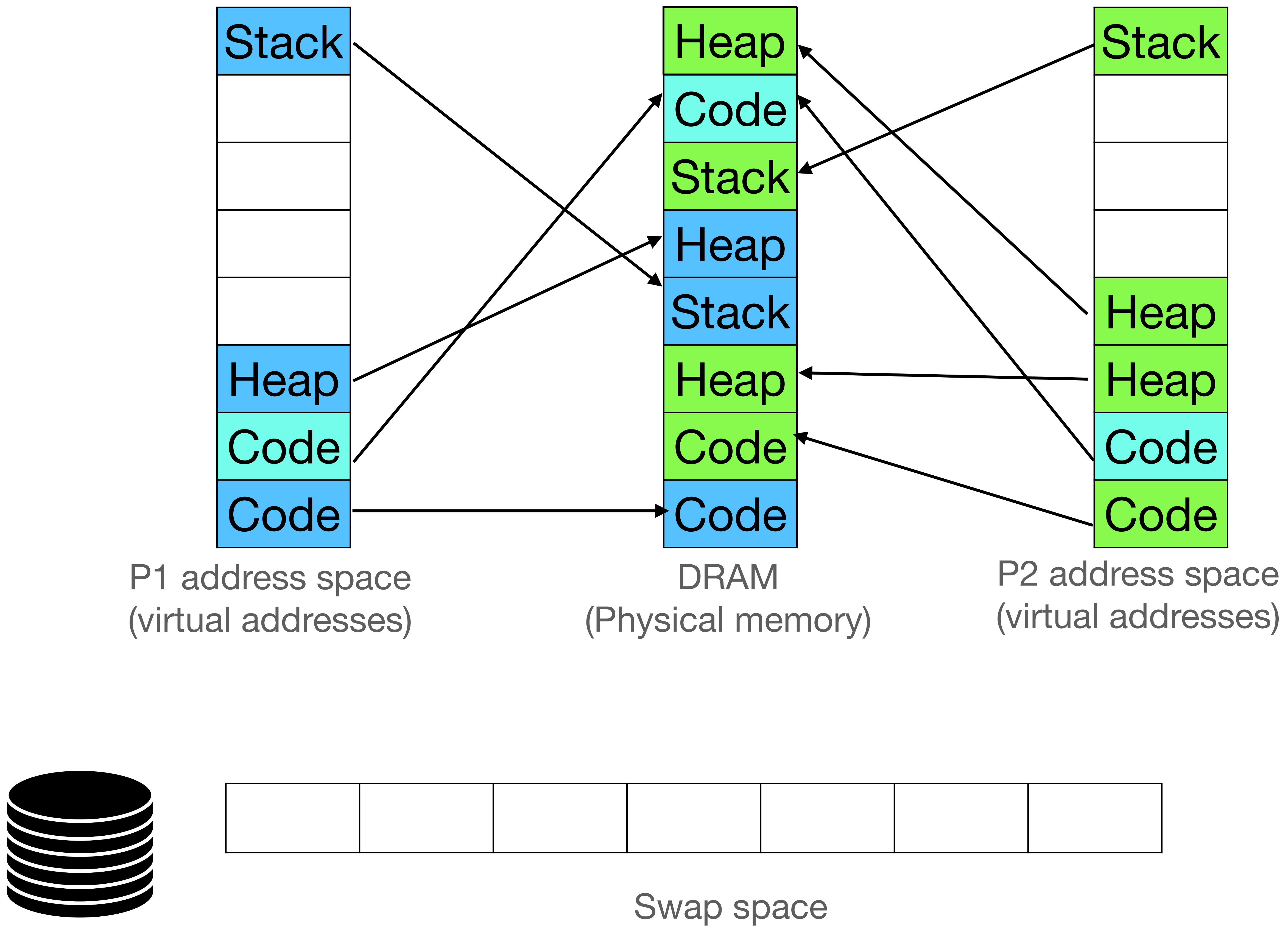
- Swap out a page to disk when physical memory is about to run out of space:
  - Mark page as not present in page table



Demand paging

Providing illusion of large virtual address spaces

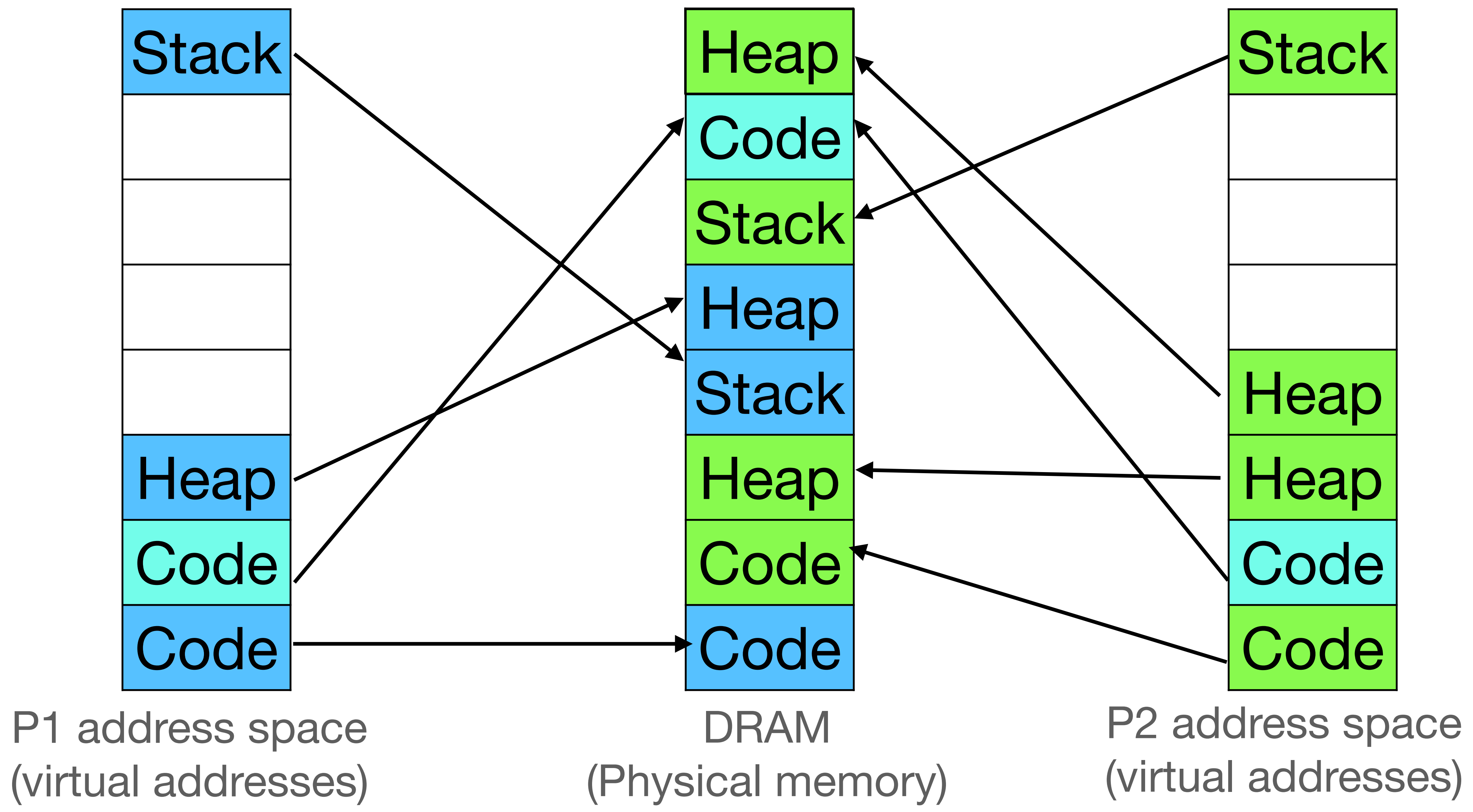
- Swap out a page to disk when physical memory is about to run out of space:
    - Mark page as not present in page table
    - Remember where page is swapped out on disk

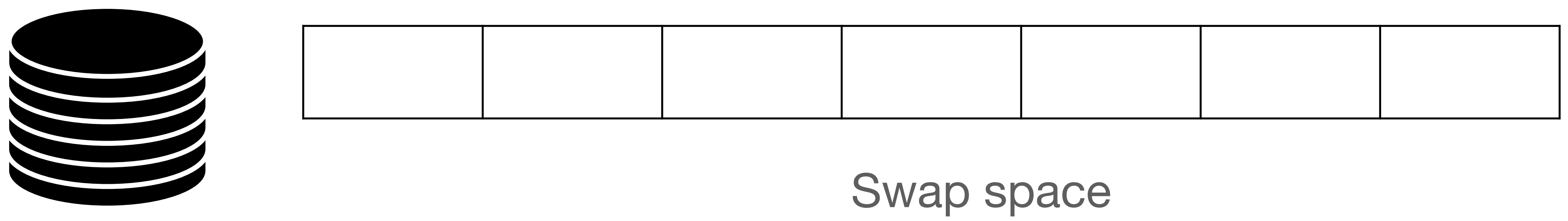


Demand paging

Providing illusion of large virtual address spaces

- Swap out a page to disk when physical memory is about to run out of space:
  - Mark page as not present in page table
  - Remember where page is swapped out on disk
- Add swapped out physical page to free list

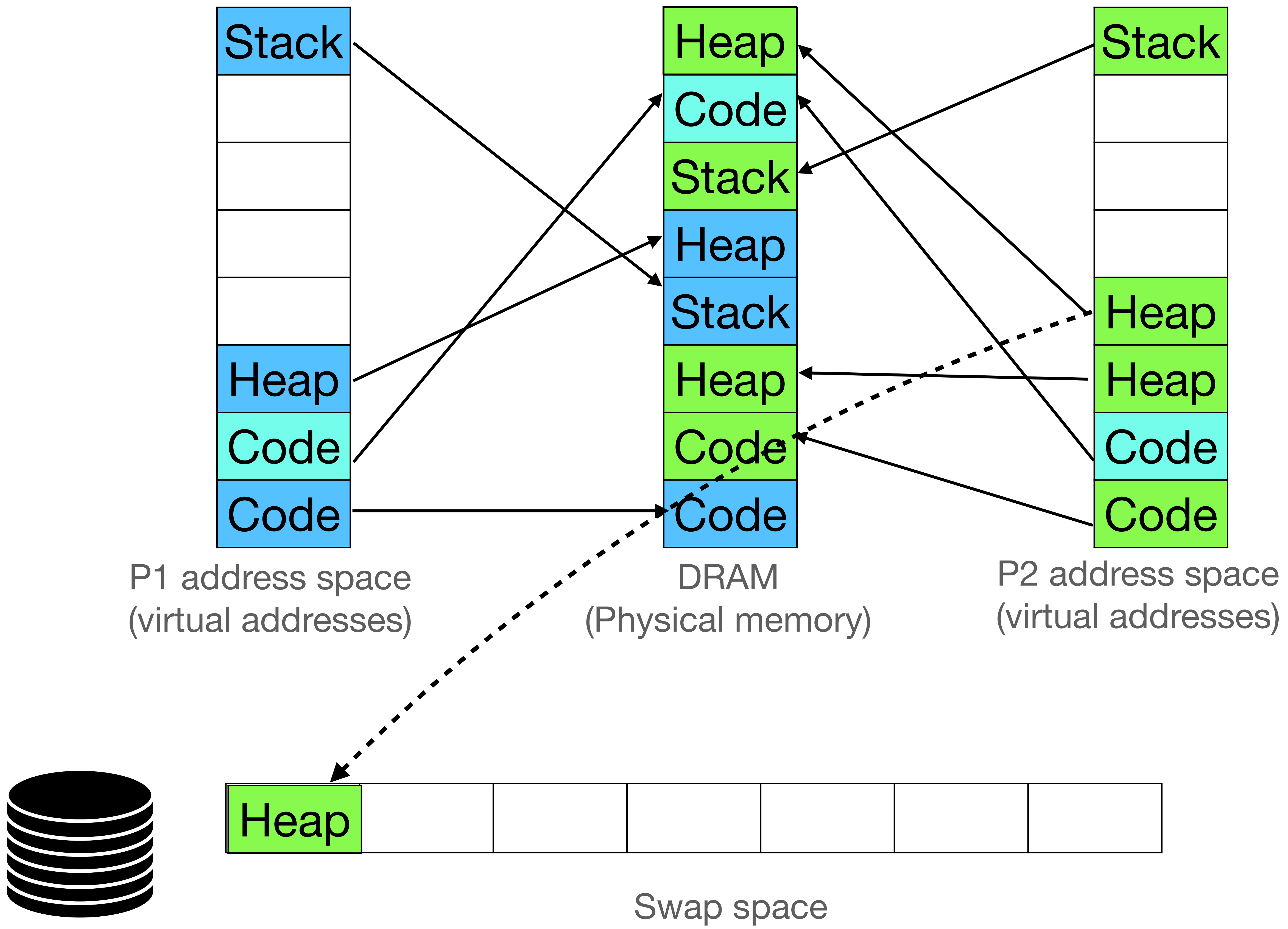




Demand paging

Providing illusion of large virtual address spaces

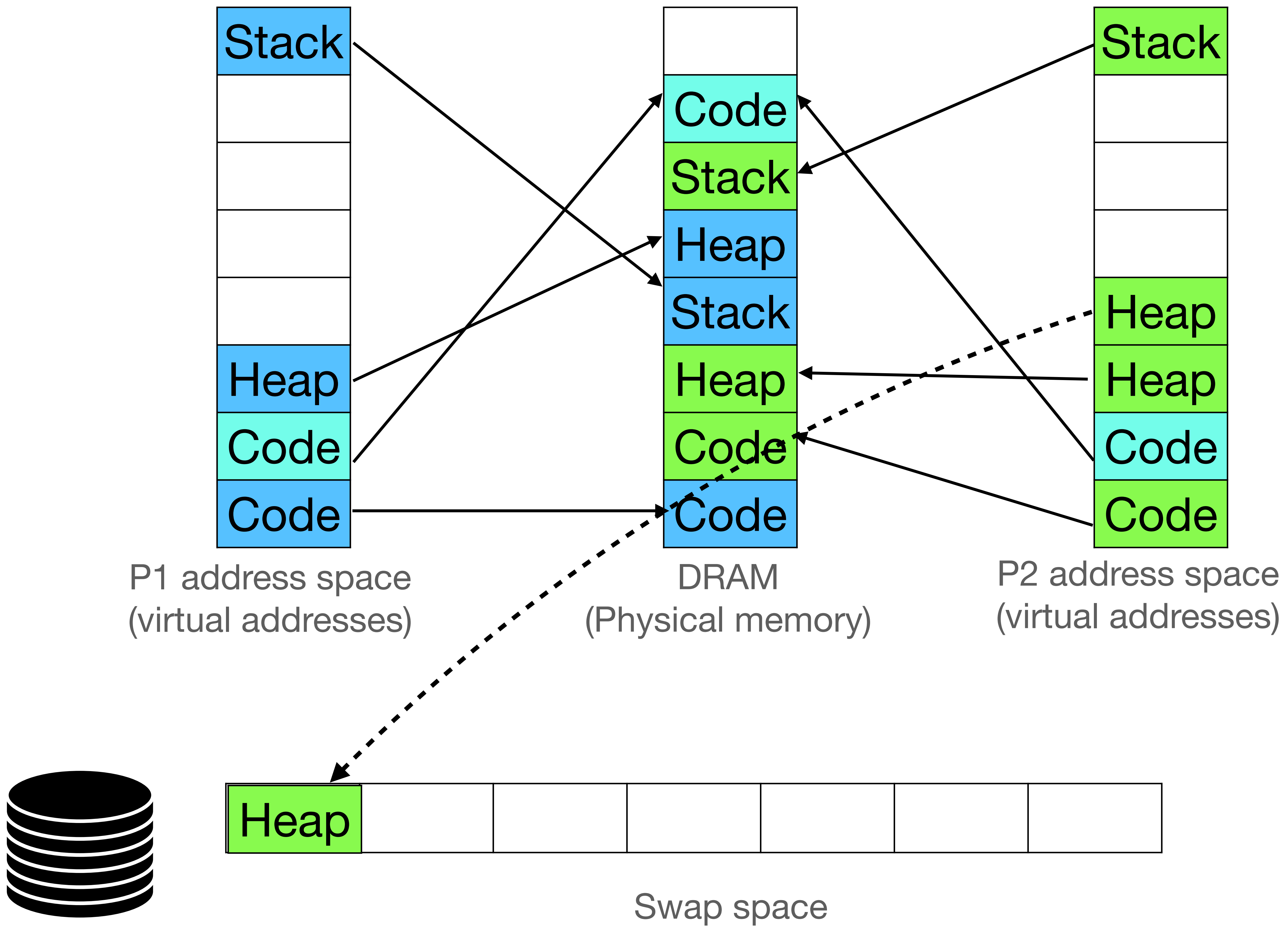
- Swap out a page to disk when physical memory is about to run out of space:
    - Mark page as not present in page table
    - Remember where page is swapped out on disk
- Add swapped out physical page to free list



Demand paging

Providing illusion of large virtual address spaces

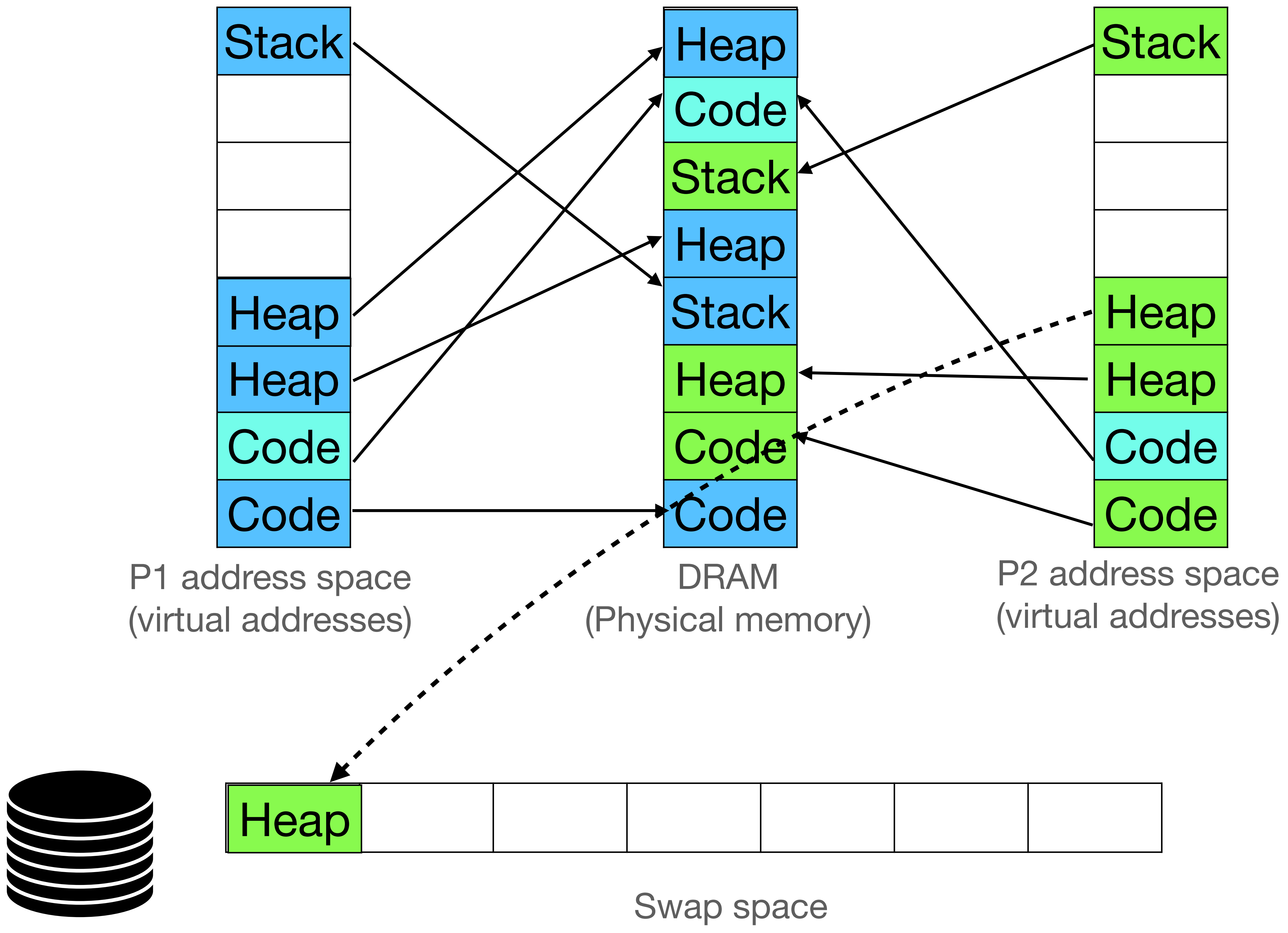
- Swap out a page to disk when physical memory is about to run out of space:
  - Mark page as not present in page table
  - Remember where page is swapped out on disk
- Add swapped out physical page to free list



Demand paging

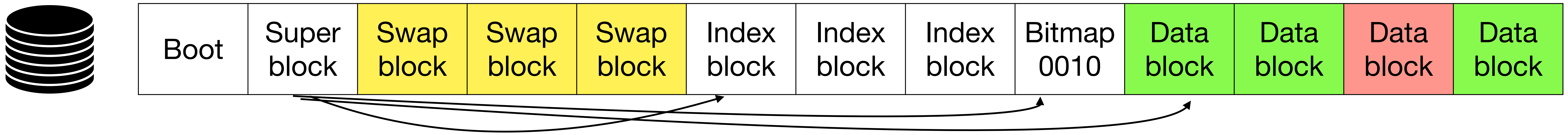
Providing illusion of large virtual address spaces

- Swap out a page to disk when physical memory is about to run out of space:
  - Mark page as not present in page table
  - Remember where page is swapped out on disk
- Add swapped out physical page to free list



Disk layout with swap space

- Reserved swap blocks, not touched by file system
- Swap space does not require crash consistency: anyways garbage after restart (all processes are dead)



Swapping out a page

- Find a free swap block on disk

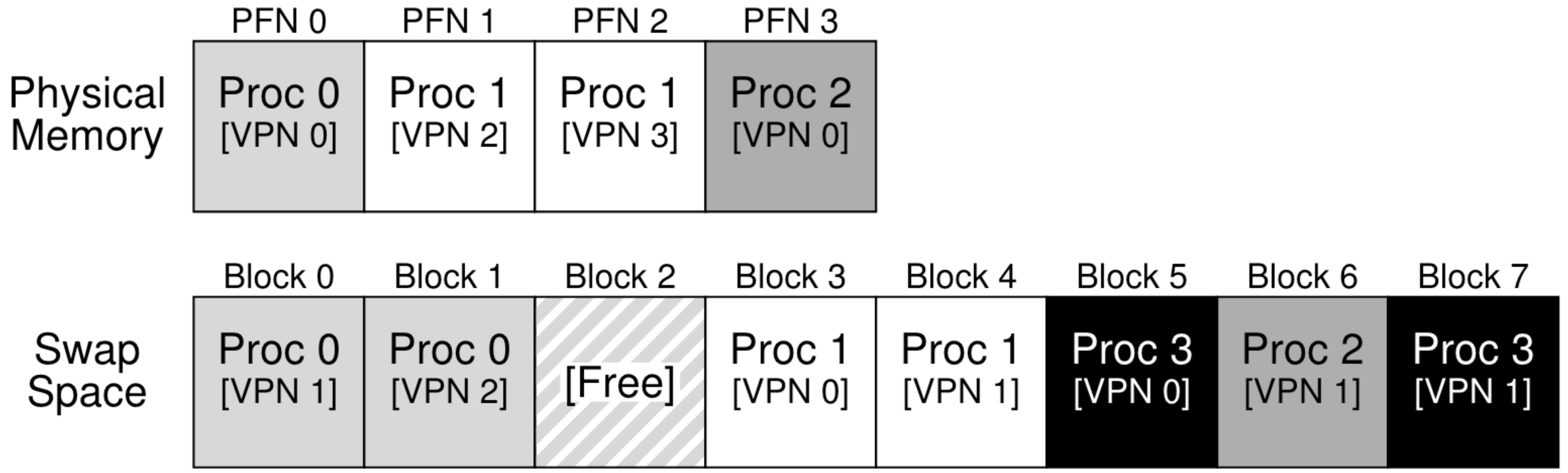


Figure 21.1: **Physical Memory and Swap Space**

Page table entry for Proc 1 [VPN2]

Swapping out a page

- Find a free swap block on disk
- Mark not present, remember swap block number in PTE

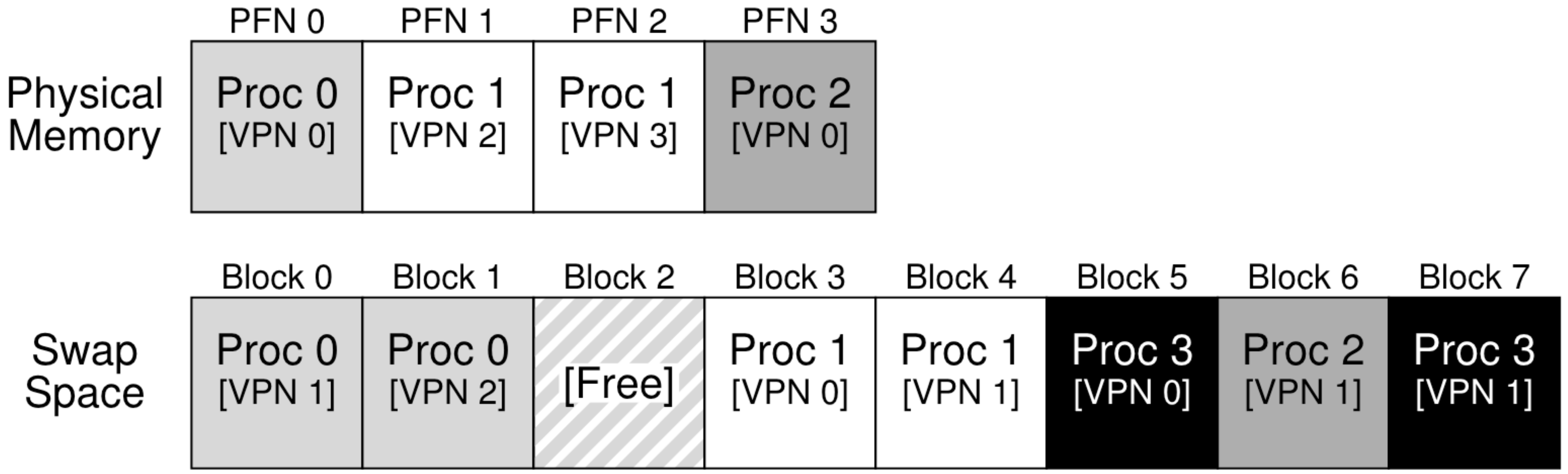
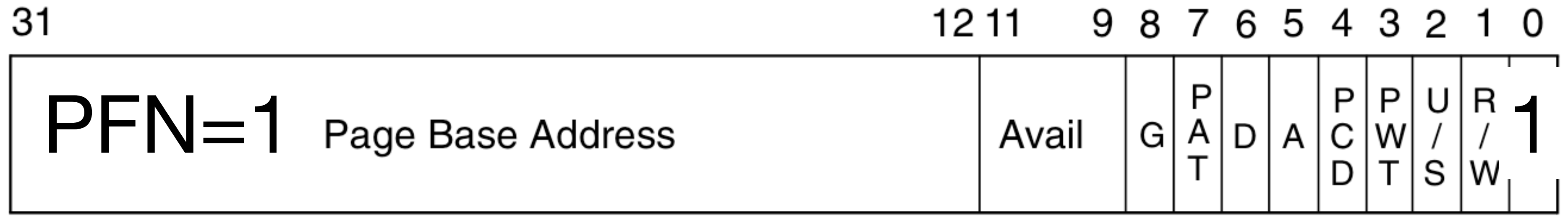


Figure 21.1: **Physical Memory and Swap Space**

Page table entry for Proc 1 [VPN2]



Swapping out a page

- Find a free swap block on disk
- Mark not present, remember swap block number in PTE

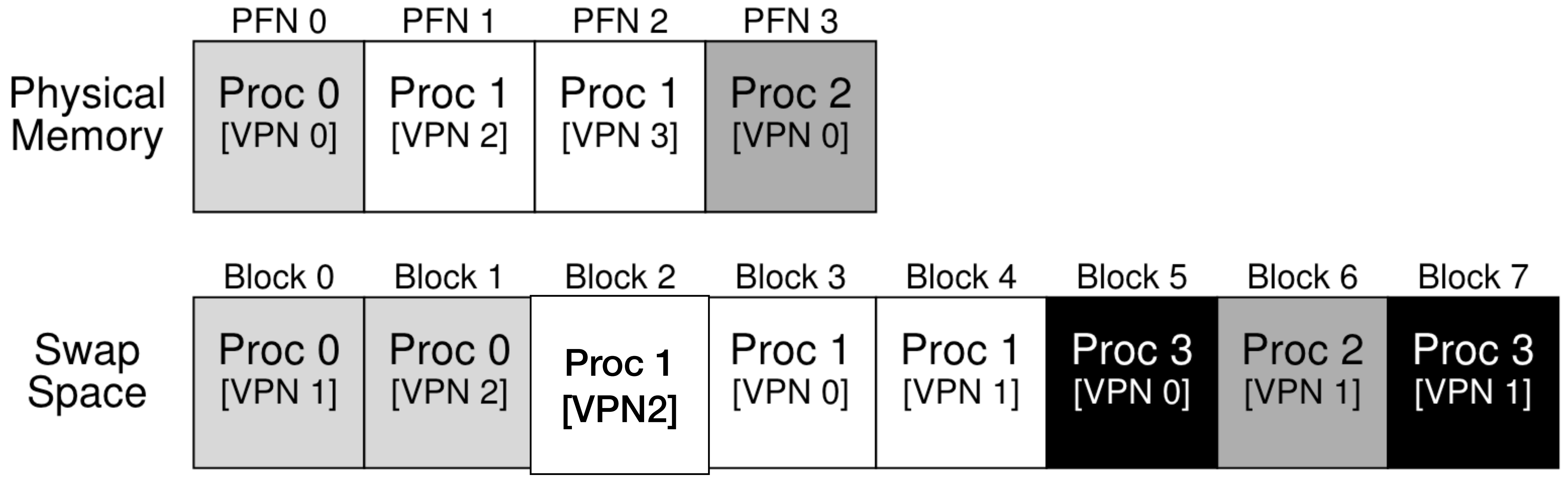


Figure 21.1: **Physical Memory and Swap Space**

Page table entry for Proc 1 [VPN2]

Swapping out a page

- Find a free swap block on disk
- Mark not present, remember swap block number in PTE
- Run INVLPG instruction to remove page from TLB

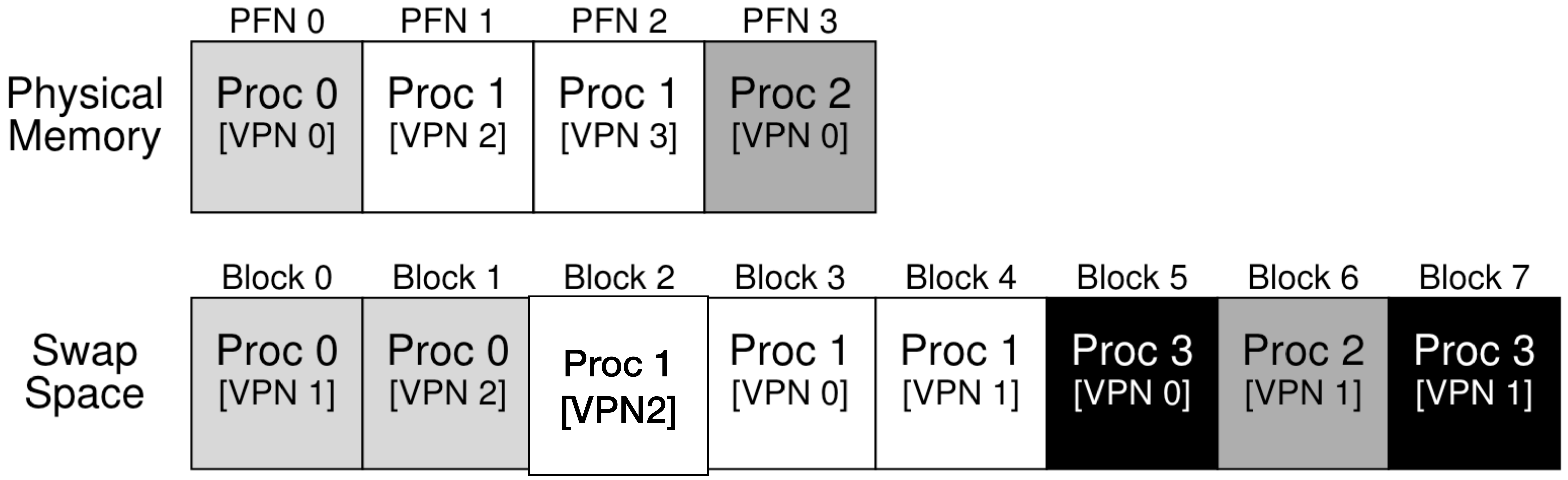


Figure 21.1: **Physical Memory and Swap Space**

Page table entry for Proc 1 [VPN2]

Swapping out a page

- Find a free swap block on disk
- Mark not present, remember swap block number in PTE
- Run INVLPG instruction to remove page from TLB
- Copy page to the free block

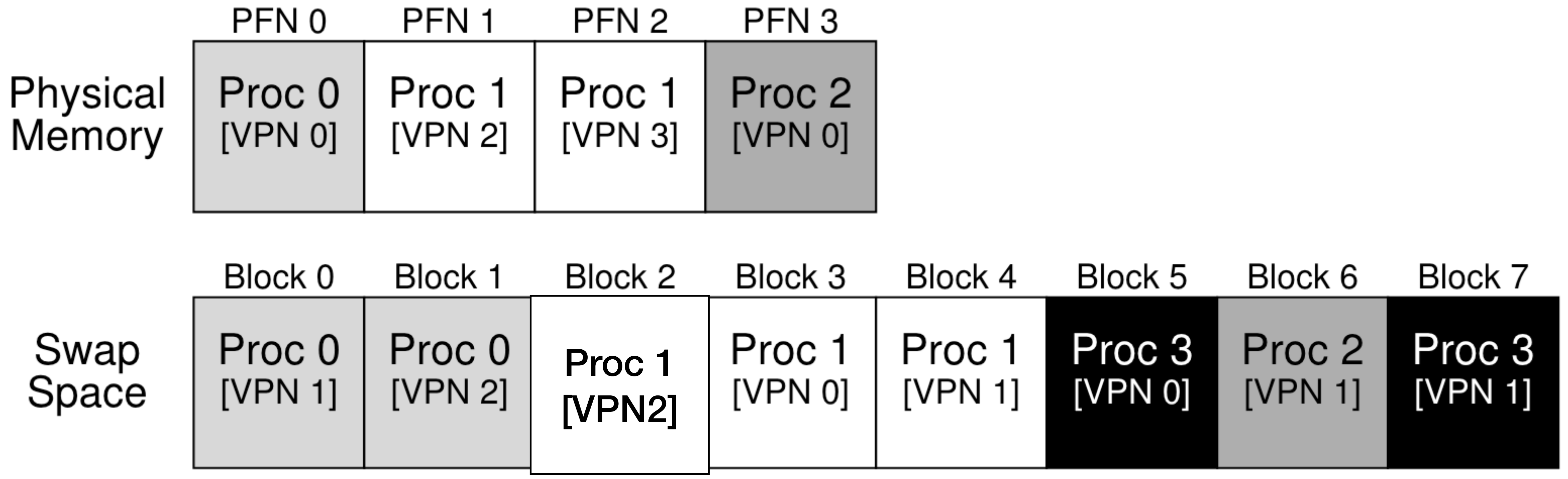


Figure 21.1: **Physical Memory and Swap Space**

Page table entry for Proc 1 [VPN2]

Swapping out a page

- Find a free swap block on disk
- Mark not present, remember swap block number in PTE
- Run INVLPG instruction to remove page from TLB
- Copy page to the free block
- Add page to free list

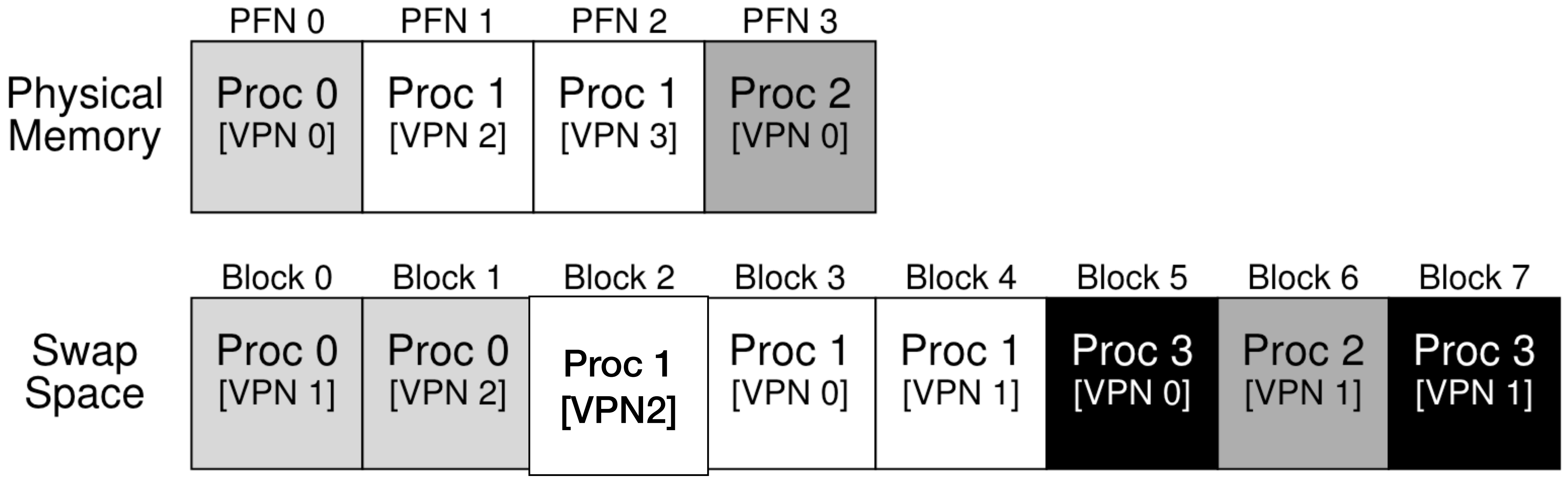
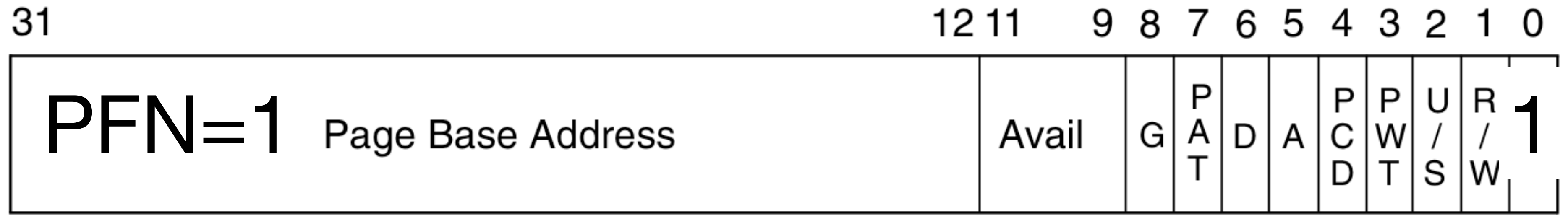


Figure 21.1: **Physical Memory and Swap Space**

Page table entry for Proc 1 [VPN2]



Swapping out a page

- Find a free swap block on disk
- Mark not present, remember swap block number in PTE
- Run INVLPG instruction to remove page from TLB
- Copy page to the free block
- Add page to free list

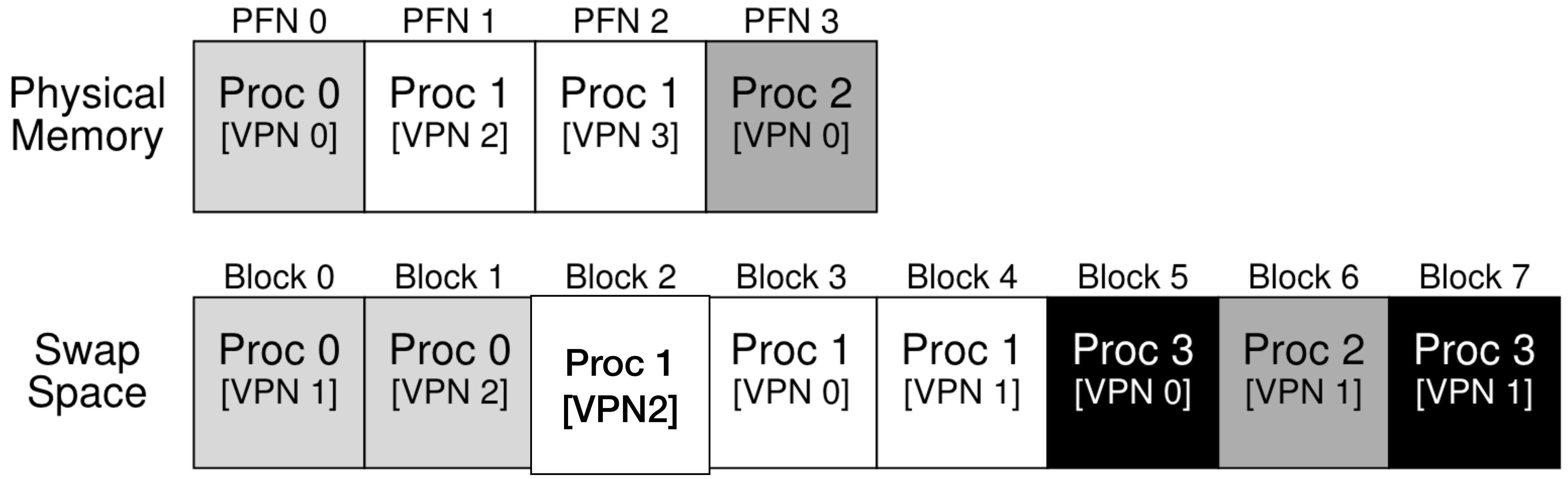


Figure 21.1: **Physical Memory and Swap Space**

Page table entry for Proc 1 [VPN2]

Swapping out a page

- Find a free swap block on disk
- Mark not present, remember swap block number in PTE
- Run INVLPG instruction to remove page from TLB
- Copy page to the free block
- Add page to free list

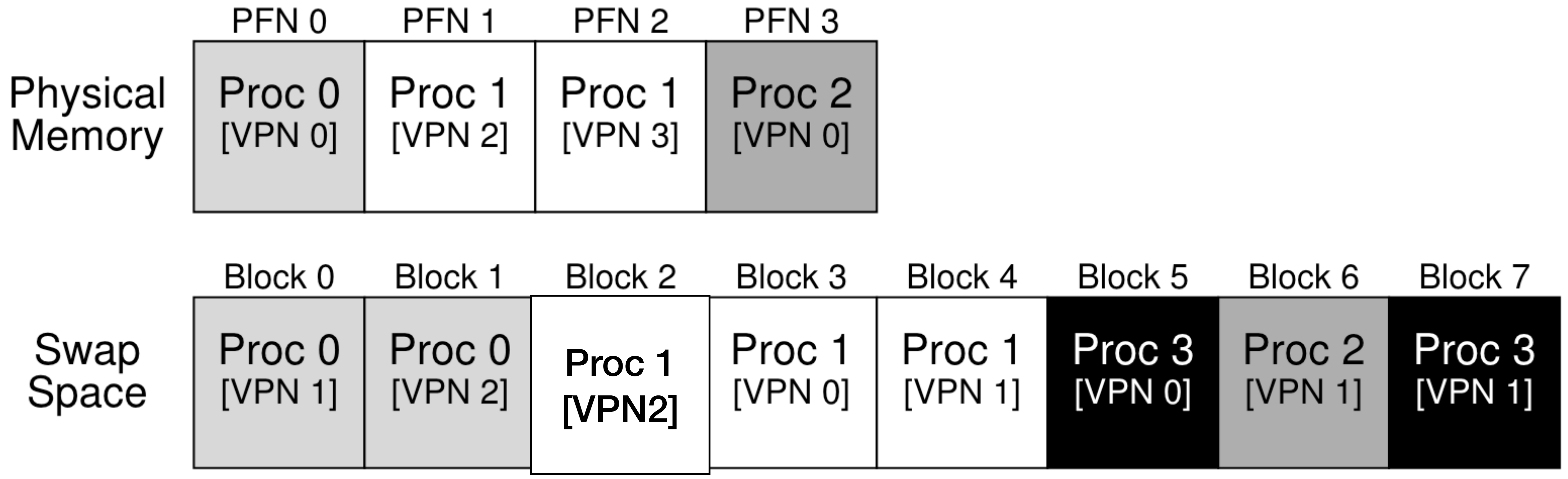


Figure 21.1: **Physical Memory and Swap Space**

Page table entry for Proc 1 [VPN2]

Swapping out a page

- Find a free swap block on disk
- Mark not present, remember swap block number in PTE
- Run INVLPG instruction to remove page from TLB
- Copy page to the free block
- Add page to free list

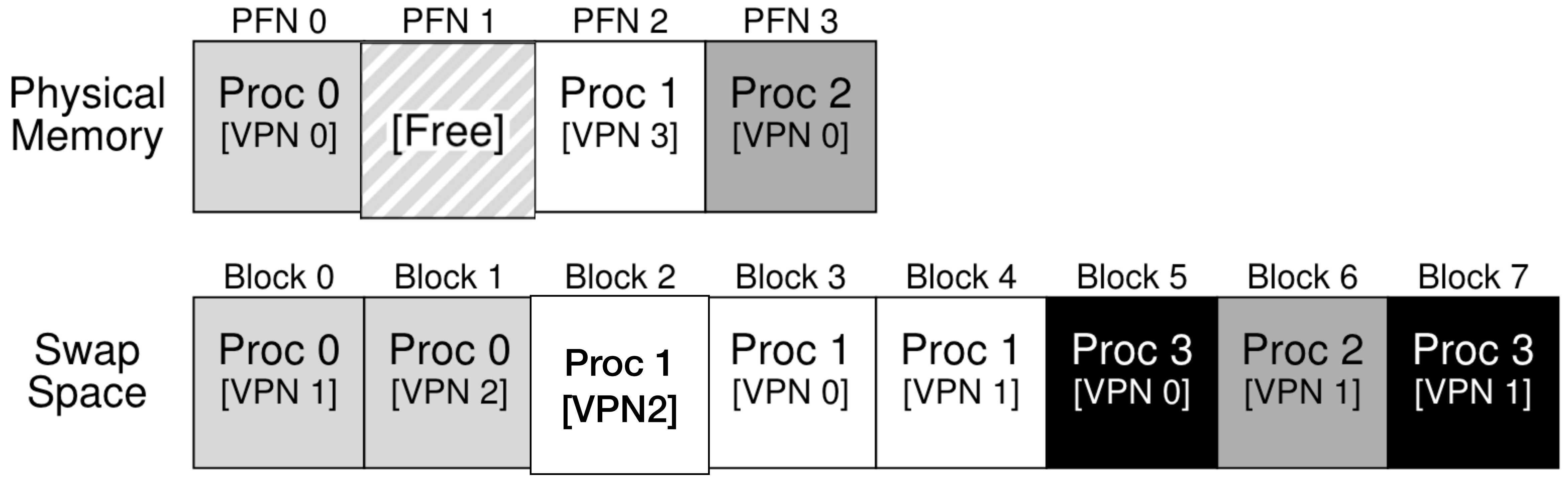
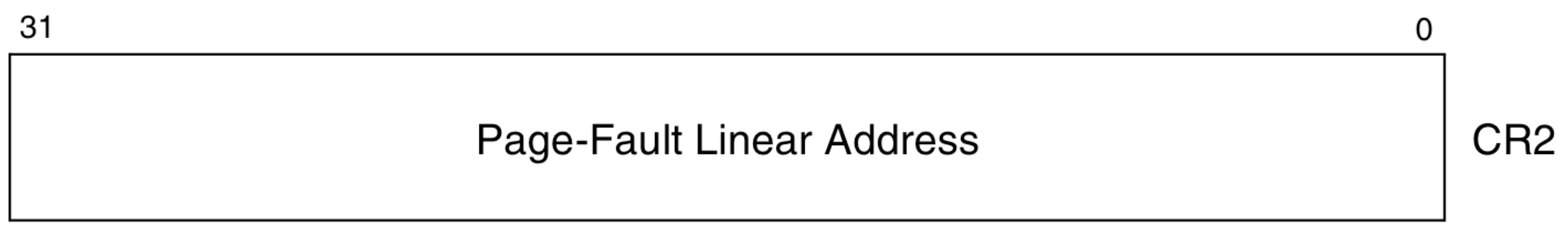


Figure 21.1: **Physical Memory and Swap Space**

Page table entry for Proc 1 [VPN2]

Swapping in a page



- Hardware does page table walk to find that the page is not present. It raise page fault with linear address in CR2.

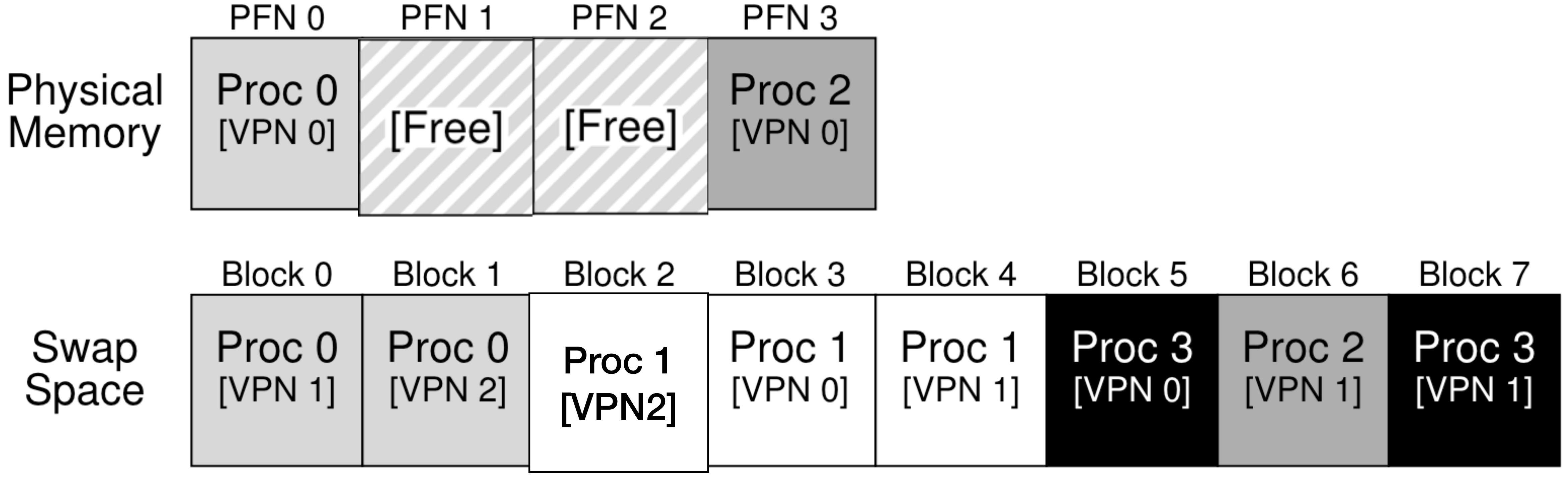
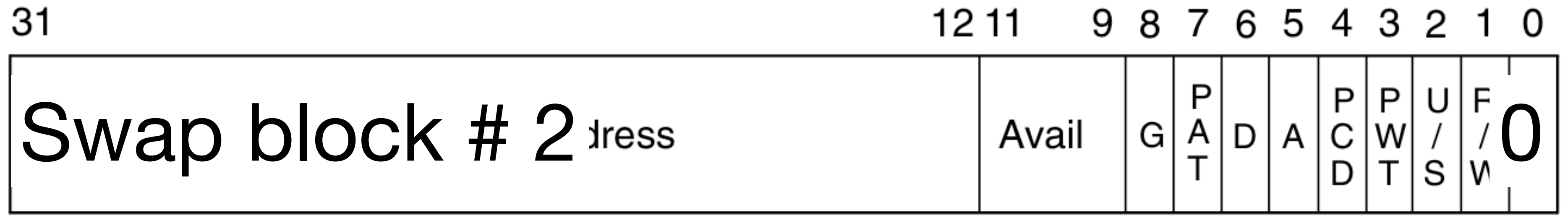
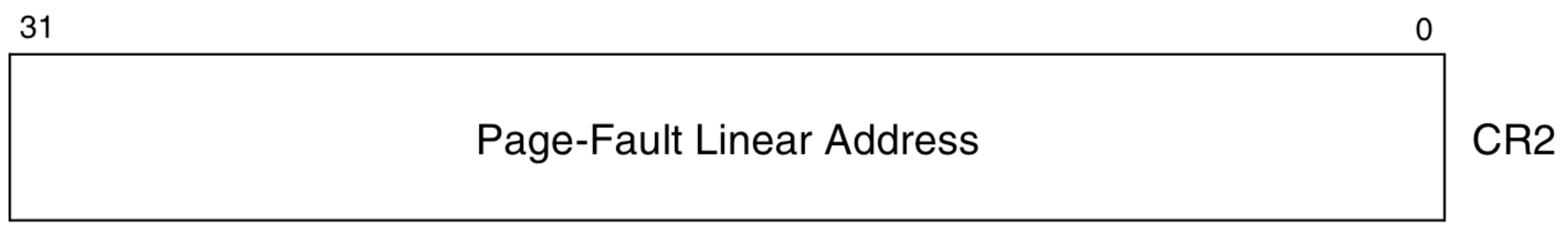


Figure 21.1: **Physical Memory and Swap Space**

Page table entry for Proc 1 [VPN2]



Swapping in a page



- Hardware does page table walk to find that the page is not present. It raise page fault with linear address in CR2.

- OS handles page fault:

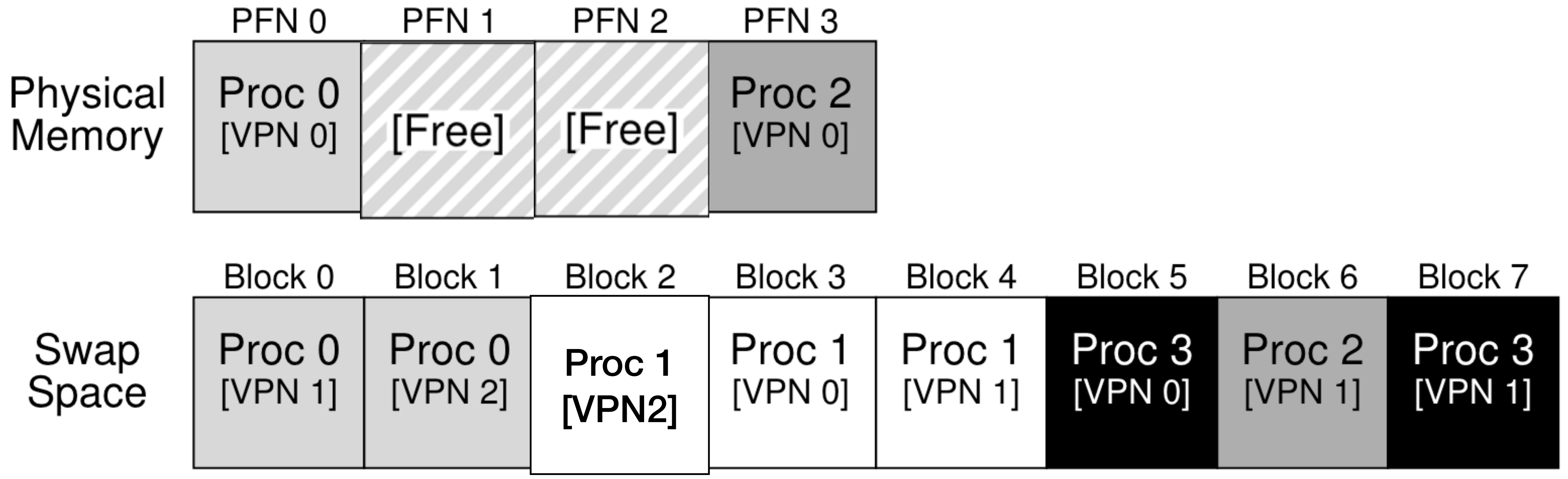
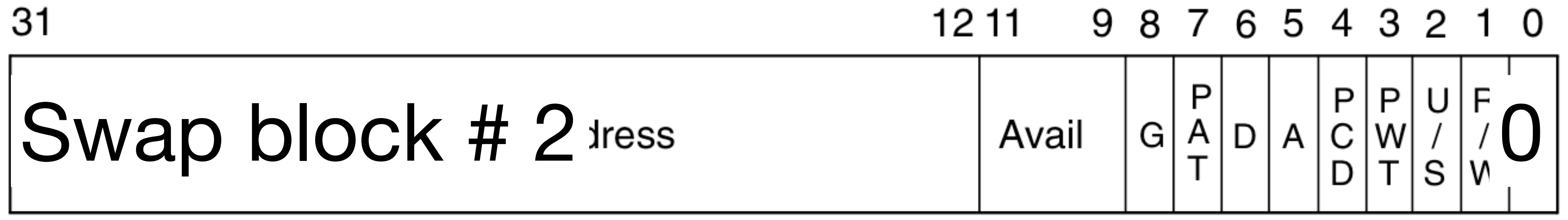
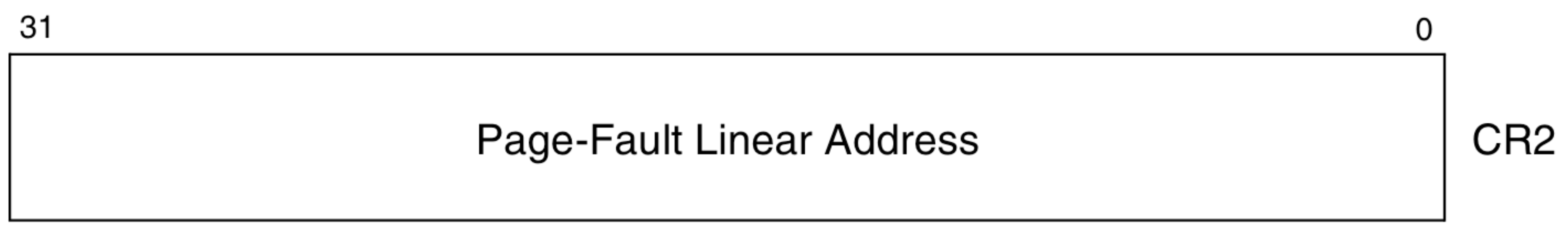


Figure 21.1: **Physical Memory and Swap Space**

Page table entry for Proc 1 [VPN2]



Swapping in a page



- Hardware does page table walk to find that the page is not present. It raise page fault with linear address in CR2.
- OS handles page fault:
    - Copies page to a free physical page

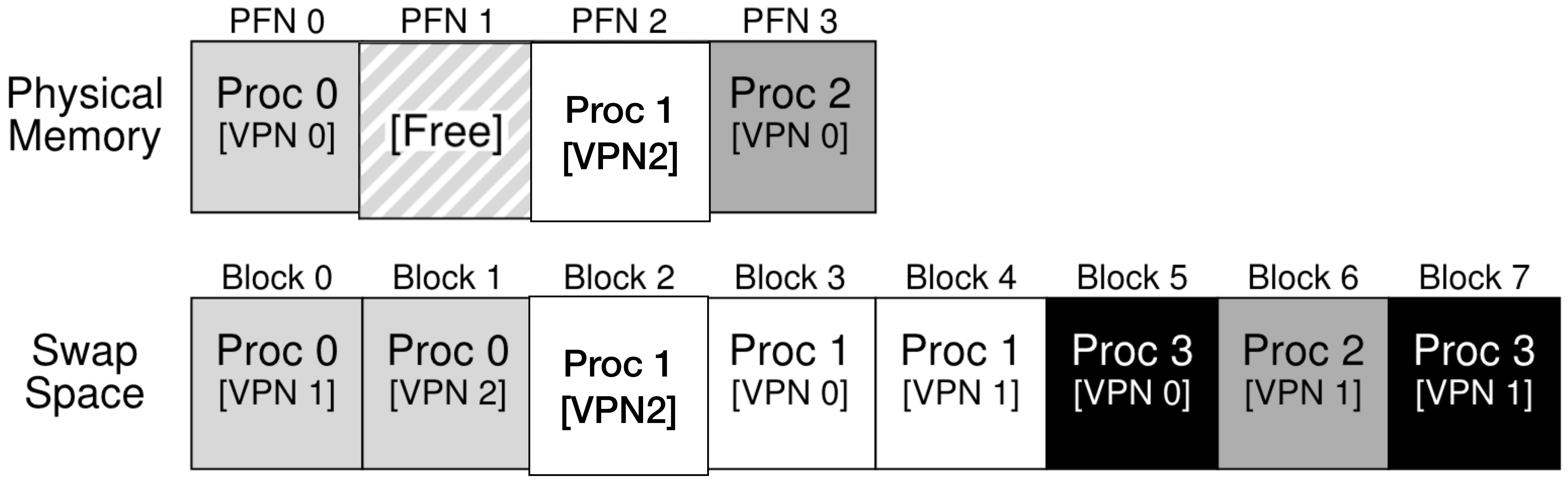
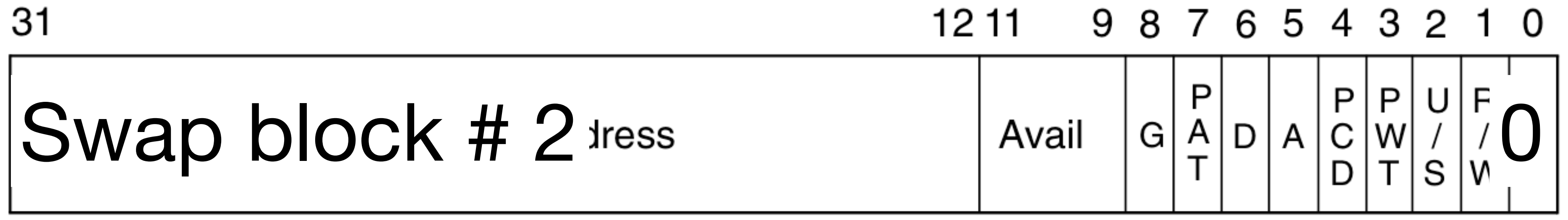
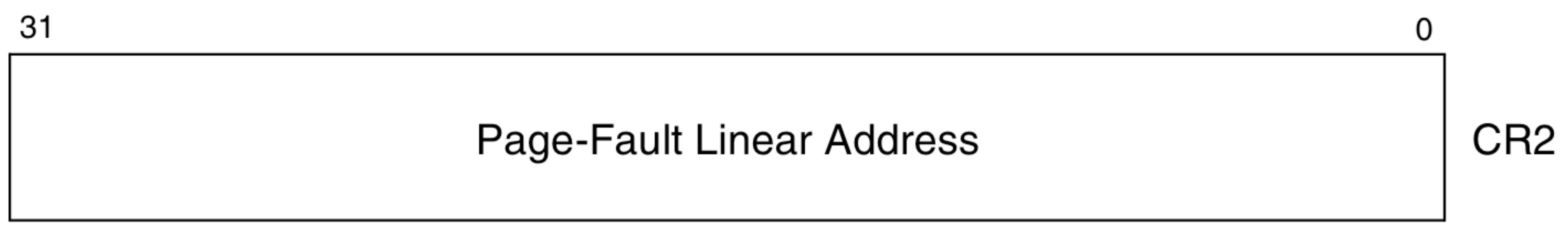


Figure 21.1: **Physical Memory and Swap Space**

Page table entry for Proc 1 [VPN2]



Swapping in a page



- Hardware does page table walk to find that the page is not present. It raise page fault with linear address in CR2.
- OS handles page fault:
    - Copies page to a free physical page
    - Updates page table entry

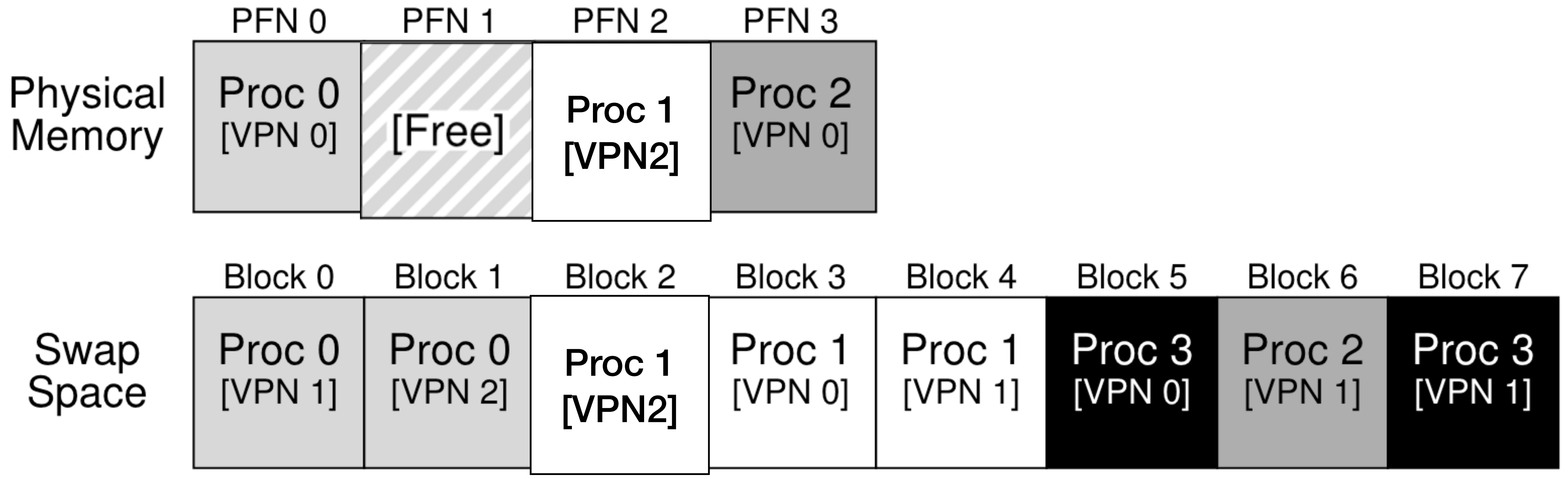
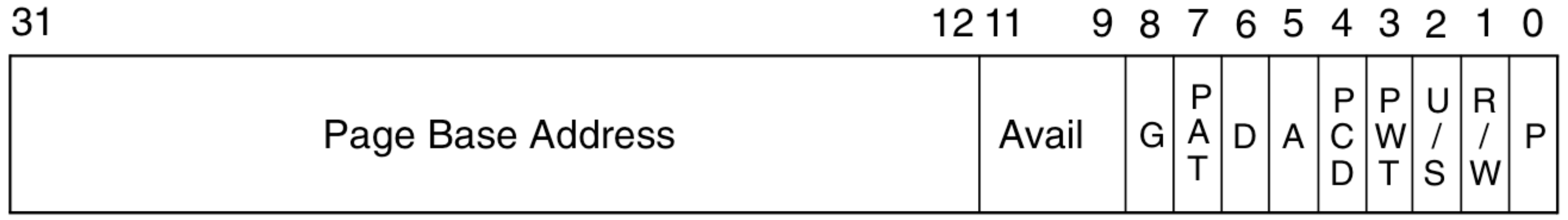
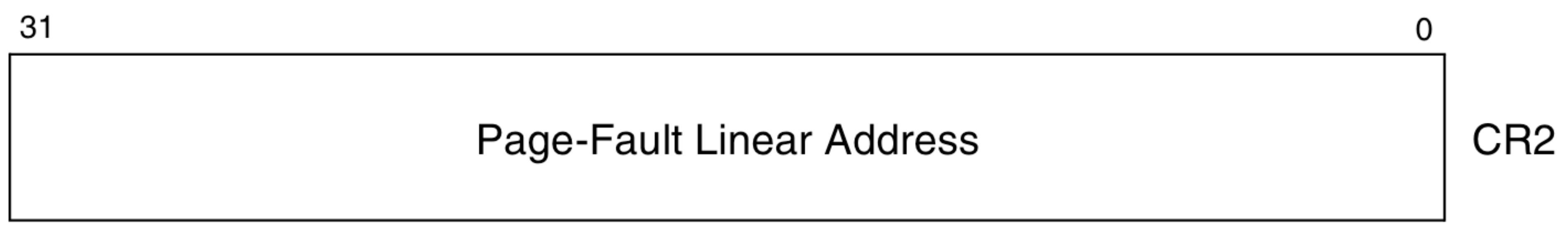


Figure 21.1: **Physical Memory and Swap Space**

Page table entry for Proc 1 [VPN2]



Swapping in a page



- Hardware does page table walk to find that the page is not present. It raise page fault with linear address in CR2.
- OS handles page fault:
    - Copies page to a free physical page
    - Updates page table entry

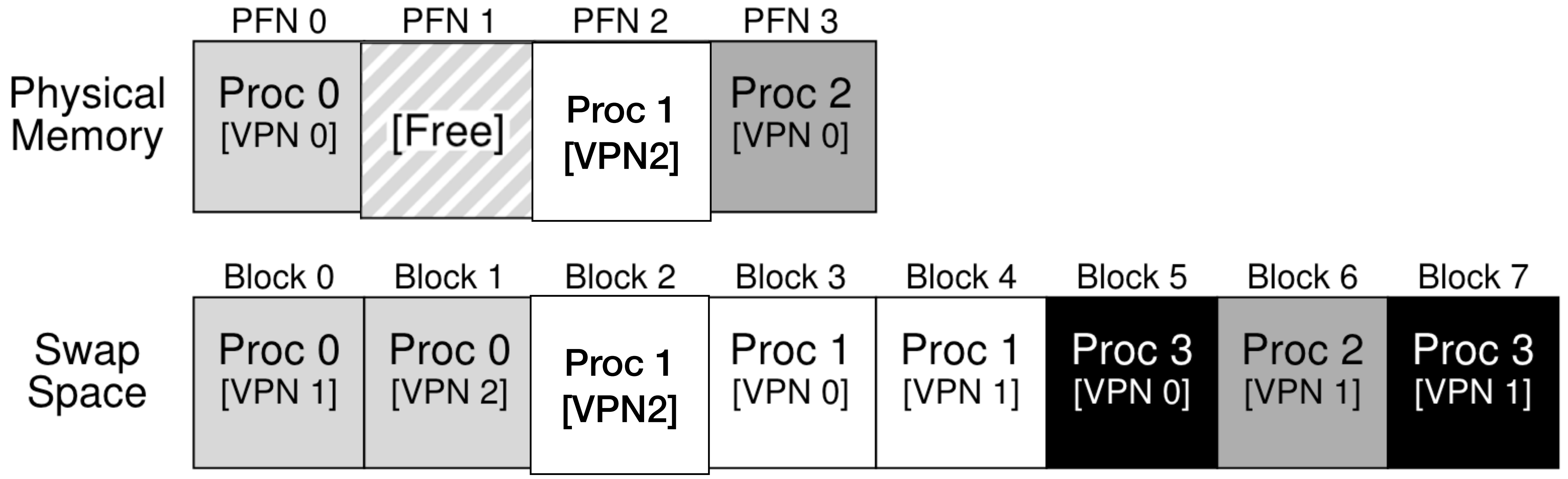
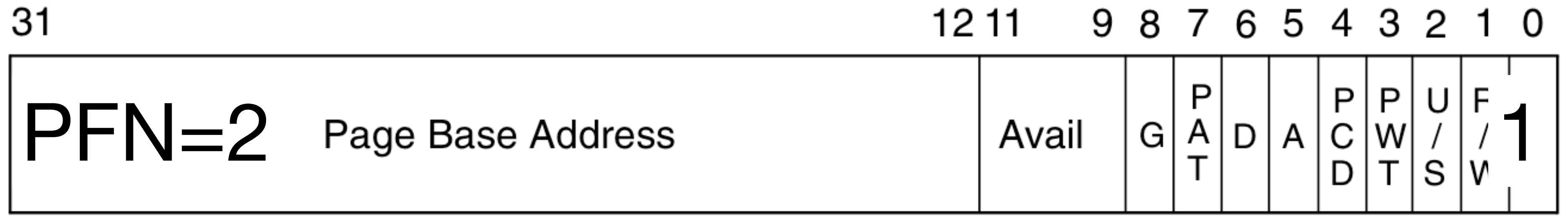
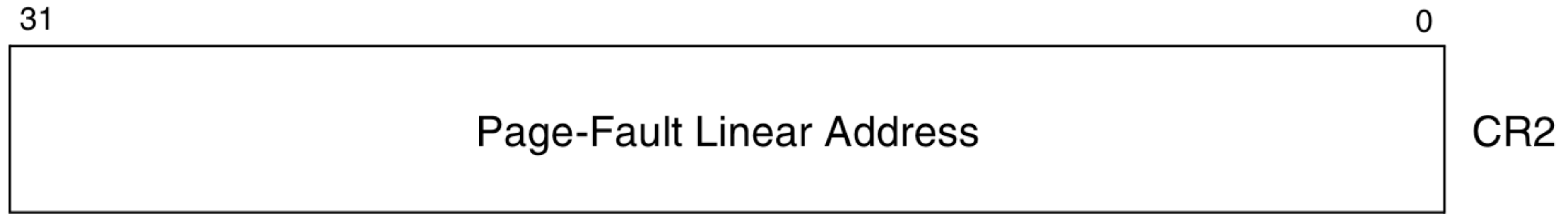


Figure 21.1: **Physical Memory and Swap Space**

Page table entry for Proc 1 [VPN2]



Swapping in a page



- Hardware does page table walk to find that the page is not present. It raise page fault with linear address in CR2.
- OS handles page fault:
    - Copies page to a free physical page
    - Updates page table entry
- Hardware retries instruction. This time finds the page, adds entry to TLB, continues as normal

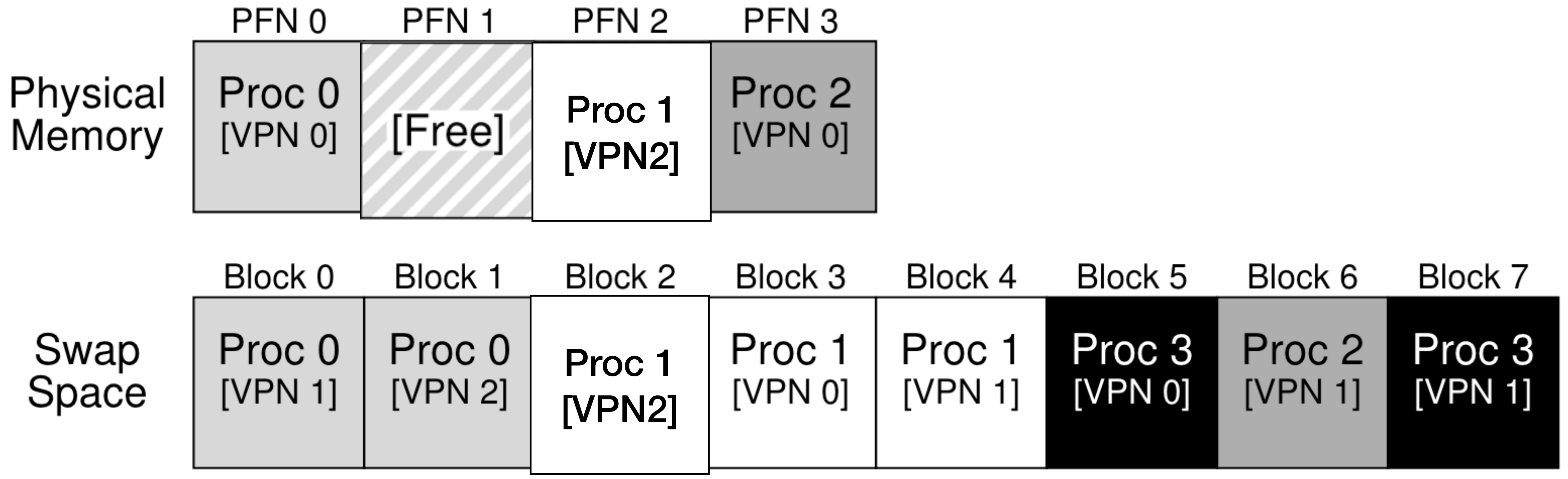
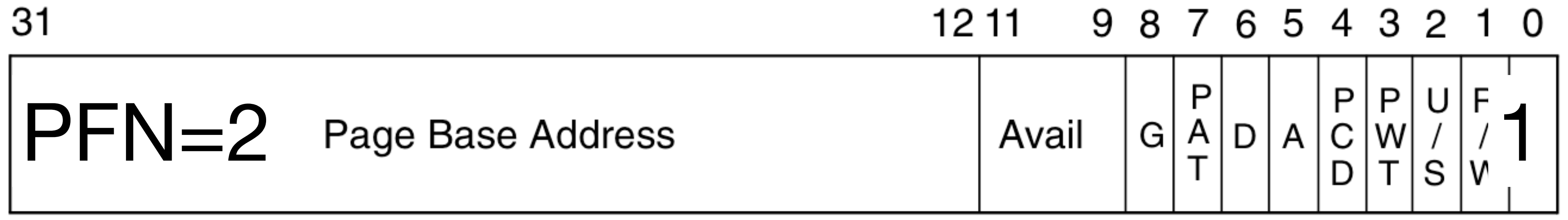


Figure 21.1: **Physical Memory and Swap Space**

Page table entry for Proc 1 [VPN2]



Page replacement policies

- When to evict pages?
- How many pages to evict?
- Which page to evict?

When to evict pages? How many pages to evict?

- Swap out one page when we completely run out of physical memory
    - What if OS itself needed a new page?

When to evict pages? How many pages to evict?

- Swap out one page when we completely run out of physical memory
  - What if OS itself needed a new page?
- Start swapping out before we completely run out
  - When there are less than N free pages left

When to evict pages? How many pages to evict?

- Swap out one page when we completely run out of physical memory
  - What if OS itself needed a new page?
- Start swapping out before we completely run out
  - When there are less than N free pages left
- Swap out multiple pages in one shot until we have M (> N) free pages left
  - Sends multiple disk writes in one shot reduces seek delay

Which page to evict?

- Goal: minimize number of swap ins/outs

Page access sequence:

0, 1, 2, 0, 1, 3, 0, 3, 1, 2, 1

Which page to evict?

- Goal: minimize number of swap ins/outs
- Belady's algorithm for optimal page replacement
  - Evict the page required furthest in the future
  - Optimal because all other pages will be required sooner

Page access sequence:

0, 1, 2, 0, 1, 3, 0, 3, 1, 2, 1

Which page to evict?

- Goal: minimize number of swap ins/outs
- Belady's algorithm for optimal page replacement
  - Evict the page required furthest in the future
  - Optimal because all other pages will be required sooner

Page access sequence:

0, 1, 2, 0, 1, 3, 0, 3, 1, 2, 1

Access	Hit/Miss?	Evict	Resulting Cache State
0	Miss		0

Which page to evict?

- Goal: minimize number of swap ins/outs
- Belady's algorithm for optimal page replacement
  - Evict the page required furthest in the future
  - Optimal because all other pages will be required sooner

Page access sequence:

0, 1, 2, 0, 1, 3, 0, 3, 1, 2, 1

Access	Hit/Miss?	Evict	Resulting Cache State
0	Miss		0
1	Miss		0, 1
2	Miss		0, 1, 2
0	Hit		0, 1, 2
1	Hit		0, 1, 2

Which page to evict?

- Goal: minimize number of swap ins/outs
- Belady's algorithm for optimal page replacement
  - Evict the page required furthest in the future
  - Optimal because all other pages will be required sooner

Page access sequence:

0, 1, 2, 0, 1, 3, 0, 3, 1, 2, 1

Access	Hit/Miss?	Evict	Resulting Cache State
0	Miss		0
1	Miss		0, 1
2	Miss		0, 1, 2
0	Hit		0, 1, 2
1	Hit		0, 1, 2
3	Miss	2	0, 1, 3

Which page to evict?

- Goal: minimize number of swap ins/outs
- Belady's algorithm for optimal page replacement
    - Evict the page required furthest in the future
    - Optimal because all other pages will be required sooner

Page access sequence:

0, 1, 2, 0, 1, 3, 0, 3, 1, 2, 1

Access	Hit/Miss?	Evict	Resulting Cache State
0	Miss		0
1	Miss		0, 1
2	Miss		0, 1, 2
0	Hit		0, 1, 2
1	Hit		0, 1, 2
3	Miss	2	0, 1, 3
0	Hit		0, 1, 3
3	Hit		0, 1, 3
1	Hit		0, 1, 3
2	Miss	3	0, 1, 2

Which page to evict?

- Goal: minimize number of swap ins/outs
- Belady's algorithm for optimal page replacement
    - Evict the page required furthest in the future
    - Optimal because all other pages will be required sooner

Page access sequence:

0, 1, 2, 0, 1, 3, 0, 3, 1, 2, 1

Access	Hit/Miss?	Evict	Resulting Cache State
0	Miss		0
1	Miss		0, 1
2	Miss		0, 1, 2
0	Hit		0, 1, 2
1	Hit		0, 1, 2
3	Miss	2	0, 1, 3
0	Hit		0, 1, 3
3	Hit		0, 1, 3
1	Hit		0, 1, 3
2	Miss	3	0, 1, 2
1	Hit		0, 1, 2

Figure 22.1: **Tracing The Optimal Policy**

Which page to evict?

- Goal: minimize number of swap ins/outs
- Belady's algorithm for optimal page replacement
  - Evict the page required furthest in the future
  - Optimal because all other pages will be required sooner
- Future is unknown!

Page access sequence:

0, 1, 2, 0, 1, 3, 0, 3, 1, 2, 1

Access	Hit/Miss?	Evict	Resulting Cache State
0	Miss		0
1	Miss		0, 1
2	Miss		0, 1, 2
0	Hit		0, 1, 2
1	Hit		0, 1, 2
3	Miss	2	0, 1, 3
0	Hit		0, 1, 3
3	Hit		0, 1, 3
1	Hit		0, 1, 3
2	Miss	3	0, 1, 2
1	Hit		0, 1, 2

Figure 22.1: **Tracing The Optimal Policy**

FIFO

- Evict the page that came first to the cache
- OS appends the page to a queue when it swaps in a page (or when it allocates a new page)

Memory access sequence:

0, 1, 2, 0, 1, 3, 0, 3, 1, 2, 1

Access	Hit/Miss?	Evict	Resulting Cache State	
0	Miss		First-in→	0

FIFO

- Evict the page that came first to the cache
- OS appends the page to a queue when it swaps in a page (or when it allocates a new page)

Memory access sequence:

0, 1, 2, 0, 1, 3, 0, 3, 1, 2, 1

Access	Hit/Miss?	Evict	Resulting Queue Cache State	
0	Miss		First-in→	0
1	Miss		First-in→	0, 1
2	Miss		First-in→	0, 1, 2
0	Hit		First-in→	0, 1, 2
1	Hit		First-in→	0, 1, 2

FIFO

- Evict the page that came first to the cache
- OS appends the page to a queue when it swaps in a page (or when it allocates a new page)

Memory access sequence:

0, 1, 2, 0, 1, 3, 0, 3, 1, 2, 1

Access	Hit/Miss?	Evict	Resulting Cache State	
0	Miss		First-in→	0
1	Miss		First-in→	0, 1
2	Miss		First-in→	0, 1, 2
0	Hit		First-in→	0, 1, 2
1	Hit		First-in→	0, 1, 2
3	Miss	0	First-in→	1, 2, 3

FIFO

- Evict the page that came first to the cache

- OS appends the page to a queue when it swaps in a page (or when it allocates a new page)

Memory access sequence:

0, 1, 2, 0, 1, 3, 0, 3, 1, 2, 1

Access	Hit/Miss?	Evict	Resulting Cache State	
0	Miss		First-in→	0
1	Miss		First-in→	0, 1
2	Miss		First-in→	0, 1, 2
0	Hit		First-in→	0, 1, 2
1	Hit		First-in→	0, 1, 2
3	Miss	0	First-in→	1, 2, 3
0	Miss	1	First-in→	2, 3, 0

FIFO

- Evict the page that came first to the cache
- OS appends the page to a queue when it swaps in a page (or when it allocates a new page)

Memory access sequence:

0, 1, 2, 0, 1, 3, 0, 3, 1, 2, 1

Access	Hit/Miss?	Evict	Resulting Cache State	
0	Miss		First-in→	0
1	Miss		First-in→	0, 1
2	Miss		First-in→	0, 1, 2
0	Hit		First-in→	0, 1, 2
1	Hit		First-in→	0, 1, 2
3	Miss	0	First-in→	1, 2, 3
0	Miss	1	First-in→	2, 3, 0
3	Hit		First-in→	2, 3, 0

FIFO

- Evict the page that came first to the cache
- OS appends the page to a queue when it swaps in a page (or when it allocates a new page)

Memory access sequence:

0, 1, 2, 0, 1, 3, 0, 3, 1, 2, 1

Access	Hit/Miss?	Evict	Resulting Cache State	
0	Miss		First-in→	0
1	Miss		First-in→	0, 1
2	Miss		First-in→	0, 1, 2
0	Hit		First-in→	0, 1, 2
1	Hit		First-in→	0, 1, 2
3	Miss	0	First-in→	1, 2, 3
0	Miss	1	First-in→	2, 3, 0
3	Hit		First-in→	2, 3, 0
1	Miss	2	First-in→	3, 0, 1
2	Miss	3	First-in→	0, 1, 2
1	Hit		First-in→	0, 1, 2

Figure 22.2: **Tracing The FIFO Policy**

Belady’s anomaly

Bigger caches can have lower hit rates!

Access	Hit/miss	Resulting cache
1	Miss	1
2	Miss	1, 2
3	Miss	1, 2, 3
4	Miss	2, 3, 4
1	Miss	3, 4, 1
2	Miss	4, 1, 2
5	Miss	1, 2, 5
1	**Hit**	1, 2, 5
2	**Hit**	1, 2, 5
3	Miss	2, 5, 3
4	Miss	5, 3, 4
5	**Hit**	5, 3, 4

Access	Hit/miss	Resulting cache
1	Miss	1
2	Miss	1, 2
3	Miss	1, 2, 3
4	Miss	1, 2, 3, 4
1	**Hit**	1, 2, 3, 4
2	**Hit**	1, 2, 3, 4
5	Miss	2, 3, 4, 5
1	Miss	3, 4, 5, 1
2	Miss	4, 5, 1, 2
3	Miss	5, 1, 2, 3
4	Miss	1, 2, 3, 4
5	Miss	2, 3, 4, 5

Belady's anomaly

Bigger caches can have lower hit rates!

Access	Hit/miss	Resulting cache	Access	Hit/miss	Resulting cache
1	Miss	1	**1**	Miss	1
2	Miss	1, 2	**2**	Miss	1, 2
3	Miss	1, 2, 3	**3**	Miss	1, 2, 3
4	Miss	2, 3, 4	**4**	Miss	1, 2, 3, 4
1	Miss	3, 4, 1	**1**	**Hit**	1, 2, 3, 4
2	Miss	4, 1, 2	**2**	**Hit**	1, 2, 3, 4
5	Miss	1, 2, 5	**5**	Miss	2, 3, 4, 5
1	**Hit**	1, 2, 5	**1**	Miss	3, 4, 5, 1
2	**Hit**	1, 2, 5	**2**	Miss	4, 5, 1, 2
3	Miss	2, 5, 3	**3**	Miss	5, 1, 2, 3
4	Miss	5, 3, 4	**4**	Miss	1, 2, 3, 4
5	**Hit**	5, 3, 4	**5**	Miss	2, 3, 4, 5

FIFO does not follow "stack property". Cache of size 4 may not contain elements in cache of size 3.

Belady’s anomaly

Bigger caches can have lower hit rates!

Access	Hit/miss	Resulting cache	Access	Hit/miss	Resulting cache
1	Miss	1	**1**	Miss	1
2	Miss	1, 2	**2**	Miss	1, 2
3	Miss	1, 2, 3	**3**	Miss	1, 2, 3
4	Miss	2, 3, 4	**4**	Miss	1, 2, 3, 4
1	Miss	3, 4, 1	**1**	**Hit**	1, 2, 3, 4
2	Miss	4, 1, 2	**2**	**Hit**	1, 2, 3, 4
5	Miss	1, 2, 5	**5**	Miss	2, 3, 4, 5
1	**Hit**	1, 2, 5	**1**	Miss	3, 4, 5, 1
2	**Hit**	1, 2, 5	**2**	Miss	4, 5, 1, 2
3	Miss	2, 5, 3	**3**	Miss	5, 1, 2, 3
4	Miss	5, 3, 4	**4**	Miss	1, 2, 3, 4
5	**Hit**	5, 3, 4	**5**	Miss	2, 3, 4, 5

FIFO does not follow “stack property”. Cache of size 4 may not contain elements in cache of size 3.

Fairness in page replacement

- Someone had lots of pages. I had very little. My page was evicted

Fairness in page replacement

- Someone had lots of pages. I had very little. My page was evicted
- OS maintains "resident size" per process: 1, 7, 9

Fairness in page replacement

- Someone had lots of pages. I had very little. My page was evicted
- OS maintains “resident size” per process: 1, 7, 9
- First select a victim process with highest resident size, remove its pages

Least Recently Used (LRU)

- Most programs exhibit temporal locality:
  - If a page was accessed recently, it shall be accessed soon
  - Keep list according to recency

Access	Hit/Miss?	Evict	Resulting Cache State	
0	Miss		LRU→	0

Least Recently Used (LRU)

- Most programs exhibit temporal locality:
  - If a page was accessed recently, it shall be accessed soon
  - Keep list according to recency

Access	Hit/Miss?	Evict	Resulting Cache State	
0	Miss		LRU→	0
1	Miss		LRU→	0, 1
2	Miss		LRU→	0, 1, 2
0	Hit		LRU→	1, 2, 0
1	Hit		LRU→	2, 0, 1

Least Recently Used (LRU)

- Most programs exhibit temporal locality:
  - If a page was accessed recently, it shall be accessed soon
  - Keep list according to recency

Access	Hit/Miss?	Evict	Resulting Cache State	
0	Miss		LRU→	0
1	Miss		LRU→	0, 1
2	Miss		LRU→	0, 1, 2
0	Hit		LRU→	1, 2, 0
1	Hit		LRU→	2, 0, 1
3	Miss	2	LRU→	0, 1, 3

Least Recently Used (LRU)

- Most programs exhibit temporal locality:
  - If a page was accessed recently, it shall be accessed soon
  - Keep list according to recency

Access	Hit/Miss?	Evict	Resulting Cache State	
0	Miss		LRU→	0
1	Miss		LRU→	0, 1
2	Miss		LRU→	0, 1, 2
0	Hit		LRU→	1, 2, 0
1	Hit		LRU→	2, 0, 1
3	Miss	2	LRU→	0, 1, 3
0	Hit		LRU→	1, 3, 0
3	Hit		LRU→	1, 0, 3
1	Hit		LRU→	0, 3, 1

Least Recently Used (LRU)

- Most programs exhibit temporal locality:
  - If a page was accessed recently, it shall be accessed soon
  - Keep list according to recency

Access	Hit/Miss?	Evict	Resulting Cache State	
0	Miss		LRU→	0
1	Miss		LRU→	0, 1
2	Miss		LRU→	0, 1, 2
0	Hit		LRU→	1, 2, 0
1	Hit		LRU→	2, 0, 1
3	Miss	2	LRU→	0, 1, 3
0	Hit		LRU→	1, 3, 0
3	Hit		LRU→	1, 0, 3
1	Hit		LRU→	0, 3, 1
2	Miss	0	LRU→	3, 1, 2

Least Recently Used (LRU)

- Most programs exhibit temporal locality:
    - If a page was accessed recently, it shall be accessed soon
    - Keep list according to recency

Access	Hit/Miss?	Evict	Resulting Cache State	
0	Miss		LRU→	0
1	Miss		LRU→	0, 1
2	Miss		LRU→	0, 1, 2
0	Hit		LRU→	1, 2, 0
1	Hit		LRU→	2, 0, 1
3	Miss	2	LRU→	0, 1, 3
0	Hit		LRU→	1, 3, 0
3	Hit		LRU→	1, 0, 3
1	Hit		LRU→	0, 3, 1
2	Miss	0	LRU→	3, 1, 2
1	Hit		LRU→	3, 2, 1

Figure 22.5: **Tracing The LRU Policy**

Difficulty in implementing LRU

- In FIFO, list is updated by the OS when a new page is allocated, or when a page is swapped out

Access	Hit/Miss?	Evict	Resulting Cache State	
0	Miss		LRU→	0
1	Miss		LRU→	0, 1
2	Miss		LRU→	0, 1, 2
0	Hit		LRU→	1, 2, 0
1	Hit		LRU→	2, 0, 1
3	Miss	2	LRU→	0, 1, 3
0	Hit		LRU→	1, 3, 0
3	Hit		LRU→	1, 0, 3
1	Hit		LRU→	0, 3, 1
2	Miss	0	LRU→	3, 1, 2
1	Hit		LRU→	3, 2, 1

Figure 22.5: **Tracing The LRU Policy**

Difficulty in implementing LRU

- In FIFO, list is updated by the OS when a new page is allocated, or when a page is swapped out
- In LRU, list needs to be updated at every access
  - OS is not running during page accesses :-/

Access	Hit/Miss?	Evict	Resulting Cache State	
0	Miss		LRU→	0
1	Miss		LRU→	0, 1
2	Miss		LRU→	0, 1, 2
0	Hit		LRU→	1, 2, 0
1	Hit		LRU→	2, 0, 1
3	Miss	2	LRU→	0, 1, 3
0	Hit		LRU→	1, 3, 0
3	Hit		LRU→	1, 0, 3
1	Hit		LRU→	0, 3, 1
2	Miss	0	LRU→	3, 1, 2
1	Hit		LRU→	3, 2, 1

Figure 22.5: **Tracing The LRU Policy**

Implementation options

- Option 1: Hardware maintains the LRU list

Implementation options

- Option 1: Hardware maintains the LRU list
  - 4GB / 4KB ~ 2^{20} pages

Implementation options

- Option 1: Hardware maintains the LRU list
  - 4GB / 4KB ~ 2^{20} pages
  - List size: 20 bits * 2^{20} pages ~ 3MB

Implementation options

- Option 1: Hardware maintains the LRU list
  - 4GB / 4KB ~ 2^{20} pages
  - List size: 20 bits * 2^{20} pages ~ 3MB
  - List cannot be in CPU => must be in memory

Implementation options

- Option 1: Hardware maintains the LRU list
    - 4GB / 4KB ~ 2^{20} pages
    - List size: 20 bits * 2^{20} pages ~ 3MB
    - List cannot be in CPU => must be in memory
    - Each memory access causes another set of memory accesses to update list

Implementation options

- Option 2: Hardware updates timestamp in page table entries.
  OS scans PTEs to find page with oldest timestamp

Implementation options

- Option 2: Hardware updates timestamp in page table entries. OS scans PTEs to find page with oldest timestamp

Page table

Physical page number	Access Timestamp
10	8:11am
11	7:05am
15	7:00am
12	8:21am

Implementation options

- Option 2: Hardware updates timestamp in page table entries. OS scans PTEs to find page with oldest timestamp
- PTEs live in memory. Updating timestamp again touches memory at every access. Defeats TLB.

Page table

Physical page number	Access Timestamp
10	8:11am
11	7:05am
15	7:00am
12	8:21am

Implementation options

- Option 2: Hardware updates timestamp in page table entries. OS scans PTEs to find page with oldest timestamp

- PTEs live in memory. Updating timestamp again touches memory at every access. Defeats TLB.

Page table

Physical page number	Access Timestamp
10	8:11am
11	7:05am
15	7:00am
12	8:21am

TLB

Virtual page number	Physical page number
0	10
1	11
2	15
3	12

Implementation options

- Option 2: Hardware updates timestamp in page table entries. OS scans PTEs to find page with oldest timestamp
- PTEs live in memory. Updating timestamp again touches memory at every access. Defeats TLB.
- Lazily update timestamp:

Page table

Physical page number	Access Timestamp
10	8:11am
11	7:05am
15	7:00am
12	8:21am

TLB

Virtual page number	Physical page number
0	10
1	11
2	15
3	12

Implementation options

- Option 2: Hardware updates timestamp in page table entries. OS scans PTEs to find page with oldest timestamp
- PTEs live in memory. Updating timestamp again touches memory at every access. Defeats TLB.
- Lazily update timestamp:
    - when mapping is brought to TLB

Page table

Physical page number	Access Timestamp
10	8:11am
11	7:05am
15	7:00am
12	8:21am

TLB

Virtual page number	Physical page number
0	10
1	11
2	15
3	12

Implementation options

- Option 2: Hardware updates timestamp in page table entries. OS scans PTEs to find page with oldest timestamp
- PTEs live in memory. Updating timestamp again touches memory at every access. Defeats TLB.
- Lazily update timestamp:
  - when mapping is brought to TLB
  - when mapping is evicted from TLB

Page table

Physical page number	Access Timestamp
10	8:11am
11	7:05am
15	7:00am
12	8:21am

TLB

Virtual page number	Physical page number
0	10
1	11
2	15
3	12

Implementation options

- Option 2: Hardware updates timestamp in page table entries. OS scans PTEs to find page with oldest timestamp
- PTEs live in memory. Updating timestamp again touches memory at every access. Defeats TLB.
- Lazily update timestamp:
  - when mapping is brought to TLB
  - when mapping is evicted from TLB
  - Once in a while

Page table

Physical page number	Access Timestamp
10	8:11am
11	7:05am
15	7:00am
12	8:21am

TLB

Virtual page number	Physical page number
0	10
1	11
2	15
3	12

Implementation options

- Option 2: Hardware updates timestamp in page table entries. OS scans PTEs to find page with oldest timestamp
- PTEs live in memory. Updating timestamp again touches memory at every access. Defeats TLB.
- Lazily update timestamp:
    - when mapping is brought to TLB
    - when mapping is evicted from TLB
    - Once in a while
- Need many more bits in PTE to store timestamp

Page table

Physical page number	**Access Timestamp**
10	8:11am
11	7:05am
15	7:00am
12	8:21am

TLB

Virtual page number	**Physical page number**
0	10
1	11
2	15
3	12

Implementation options

- Option 2: Hardware updates timestamp in page table entries. OS scans PTEs to find page with oldest timestamp
- PTEs live in memory. Updating timestamp again touches memory at every access. Defeats TLB.
- Lazily update timestamp:
  - when mapping is brought to TLB
  - when mapping is evicted from TLB
  - Once in a while
- Need many more bits in PTE to store timestamp
- Victim process has highest resident size. Scanning (worst case 2^{20}) PTEs will be slow

Page table

Physical page number	**Access Timestamp**
10	8:11am
11	7:05am
15	7:00am
12	8:21am

TLB

Virtual page number	**Physical page number**
0	10
1	11
2	15
3	12

Approximating LRU

- Give up on finding *least* recently used. OK to evict a *less* recently used

Page table

Physical page number	Access Timestamp
10	8:11am
11	7:05am
15	7:00am
12	8:21am

Approximating LRU

- Give up on finding *least* recently used. OK to evict a *less* recently used

Page table

Physical page number	Access Timestamp
10	8:11am
11	7:05am
15	7:00am
12	8:21am

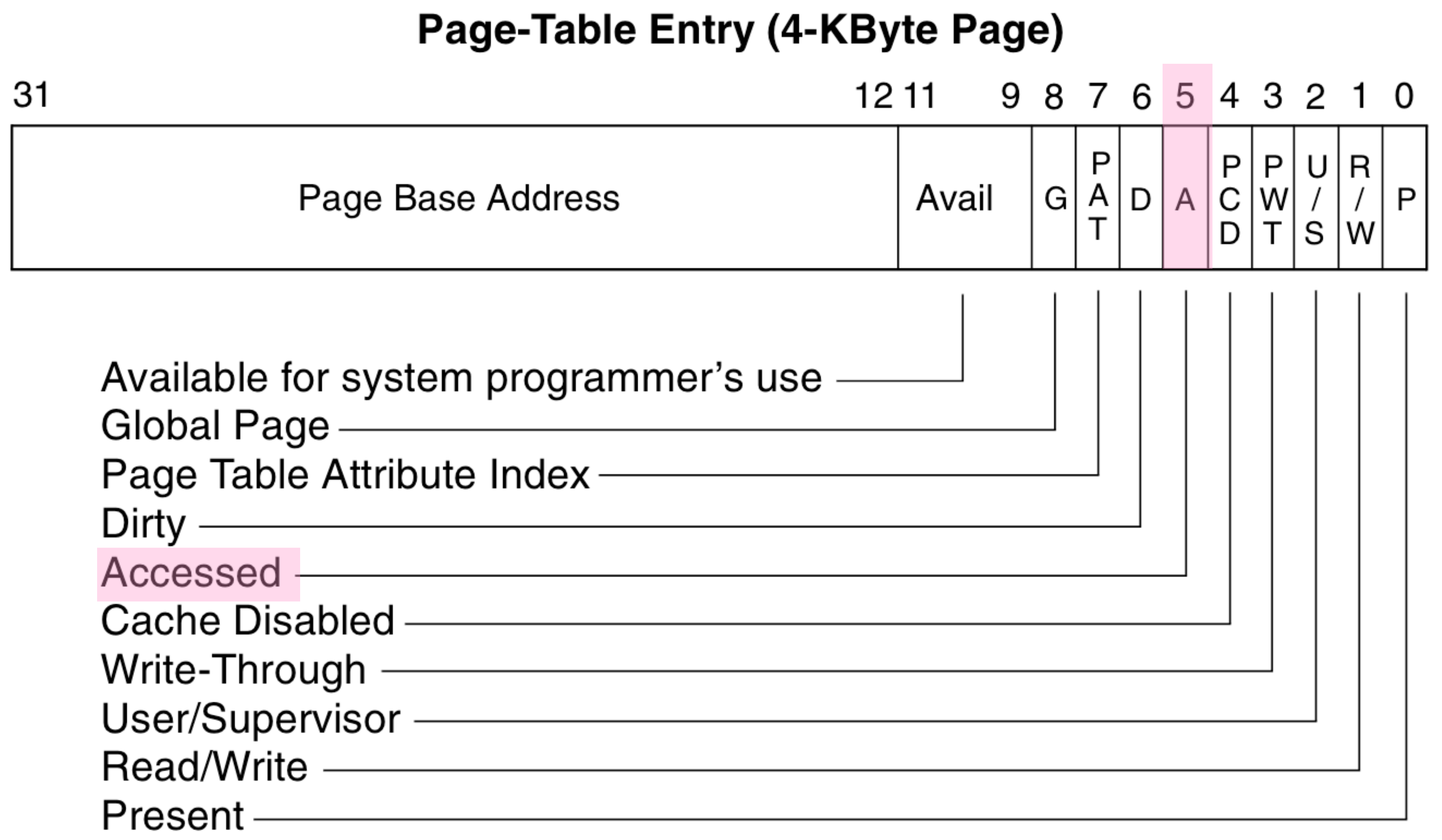


Figure 3-14. Format of Page-Directory and Page-Table Entries for 4-KByte Pages and 32-Bit Physical Addresses

Approximating LRU

- Give up on finding *least* recently used. OK to evict a *less* recently used
- Hardware just lazily sets 1 access bit

Page table

Physical page number	Access Timestamp
10	8:11am
11	7:05am
15	7:00am
12	8:21am

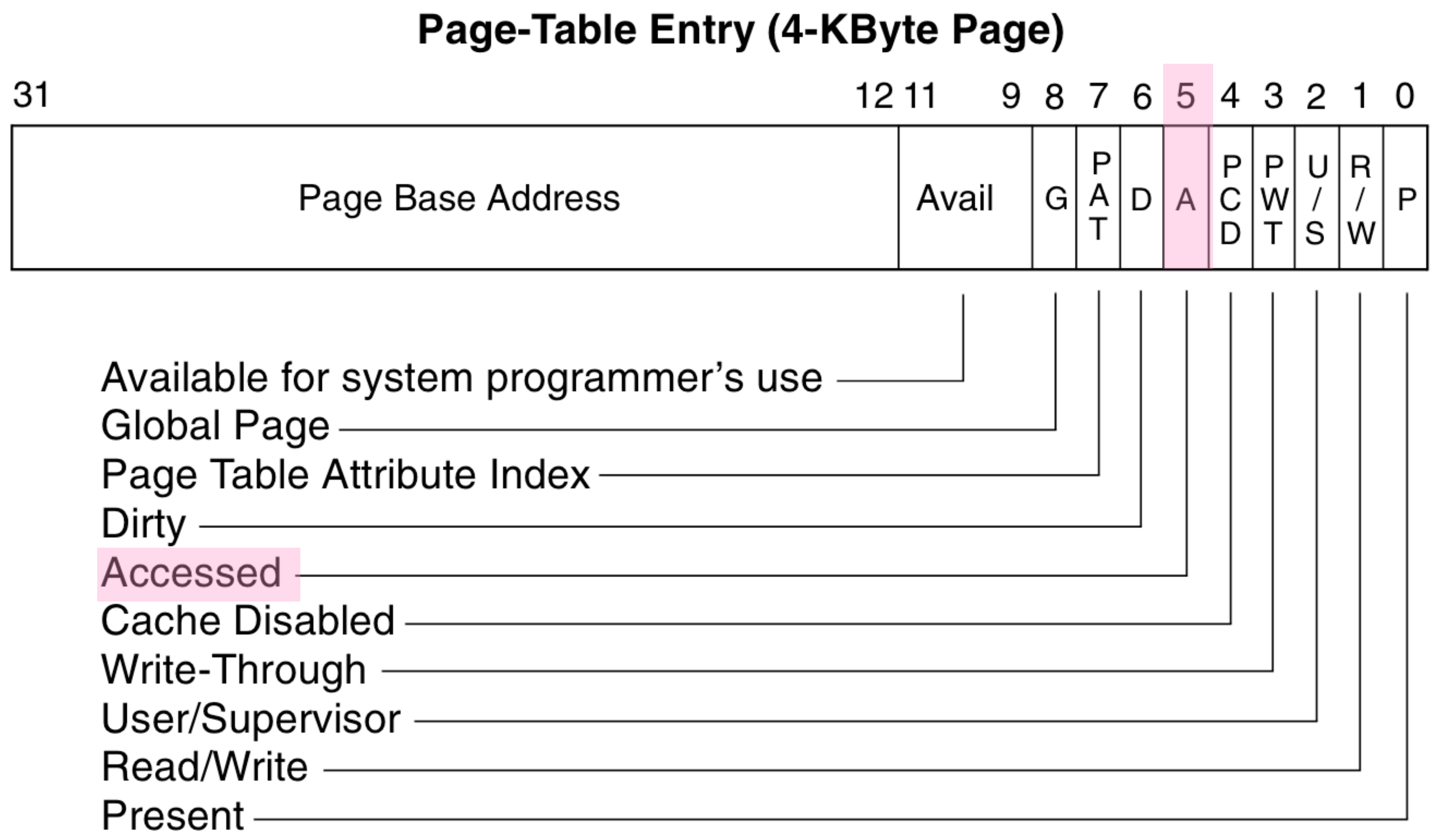
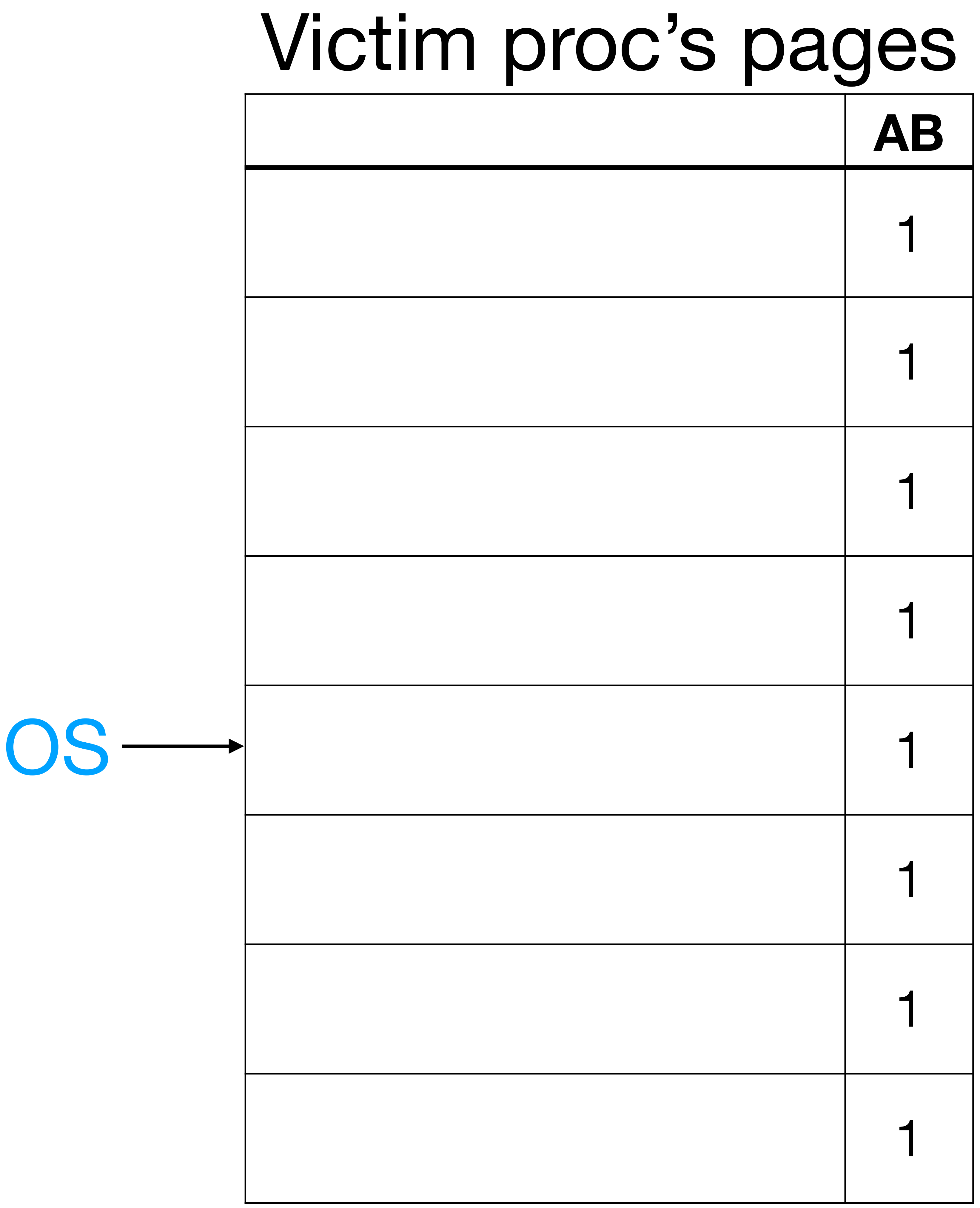


Figure 3-14. Format of Page-Directory and Page-Table Entries for 4-KByte Pages and 32-Bit Physical Addresses

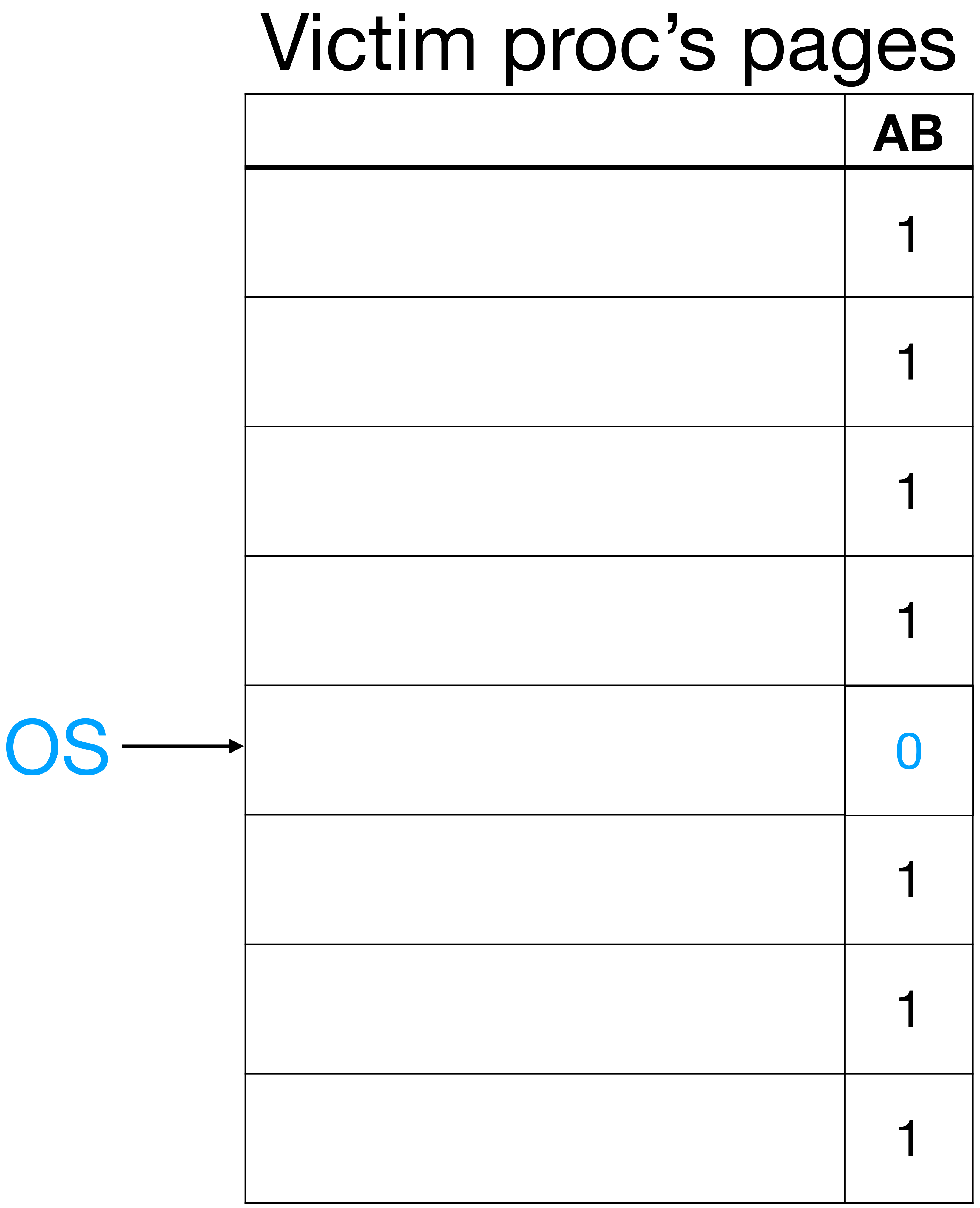
Clock algorithm

- OS clears access bit. Evicts page with access bit = 0
- Hardware sets access bit to 1
- Evicted page was “not recently used”



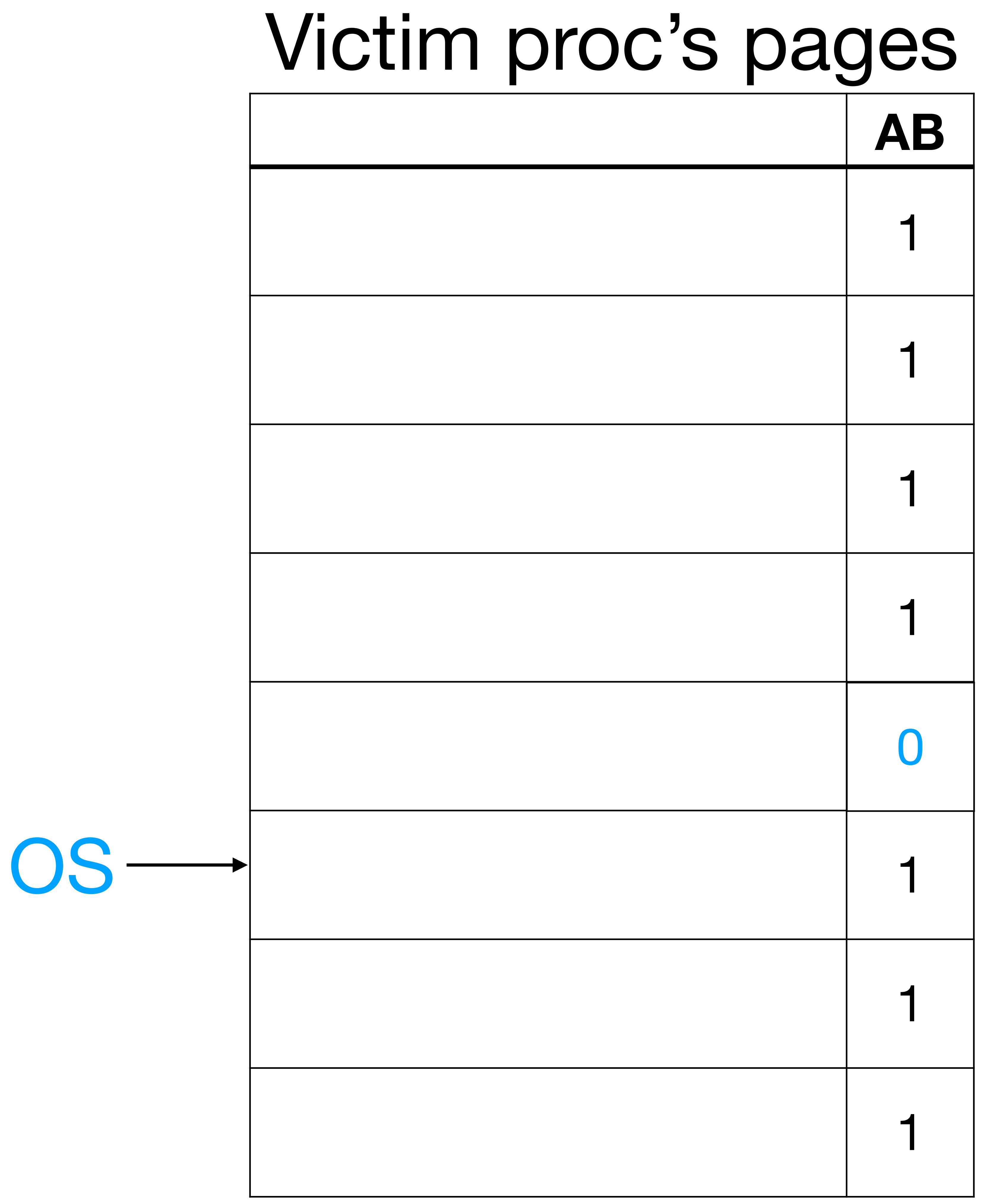
Clock algorithm

- OS clears access bit. Evicts page with access bit = 0
- Hardware sets access bit to 1
- Evicted page was “not recently used”



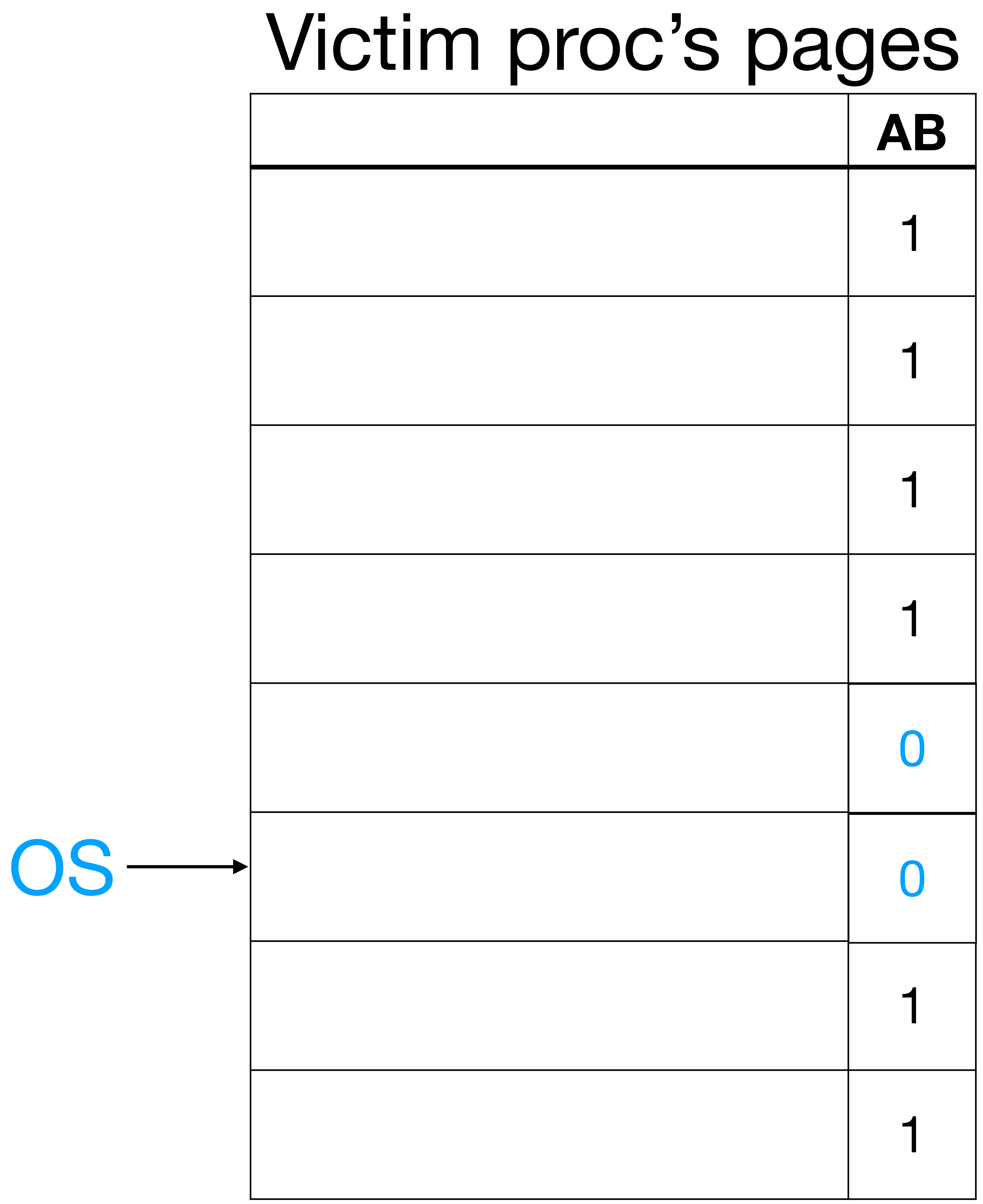
Clock algorithm

- OS clears access bit. Evicts page with access bit = 0
- Hardware sets access bit to 1
- Evicted page was “not recently used”



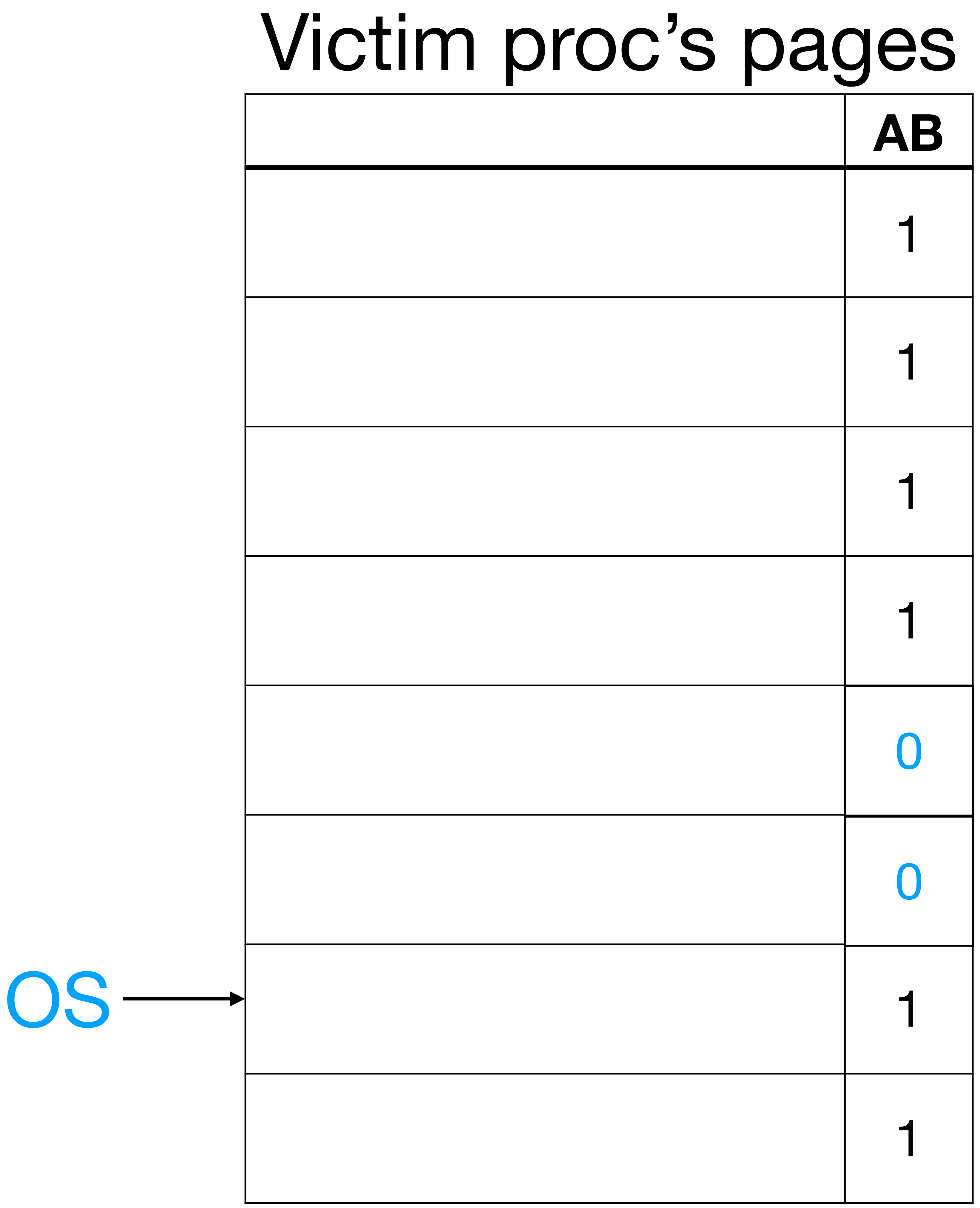
Clock algorithm

- OS clears access bit. Evicts page with access bit = 0
- Hardware sets access bit to 1
- Evicted page was “not recently used”



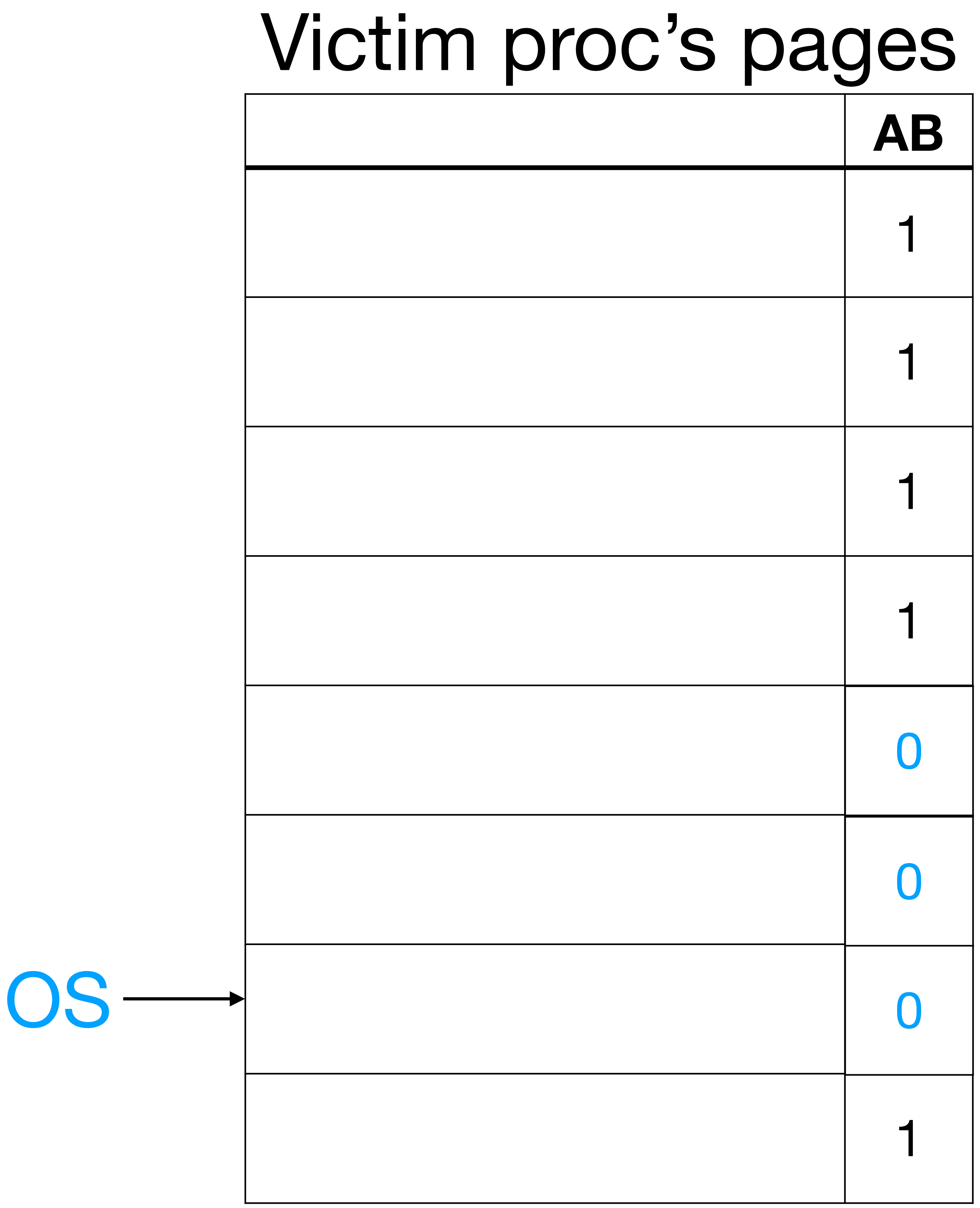
Clock algorithm

- OS clears access bit. Evicts page with access bit = 0
- Hardware sets access bit to 1
- Evicted page was “not recently used”



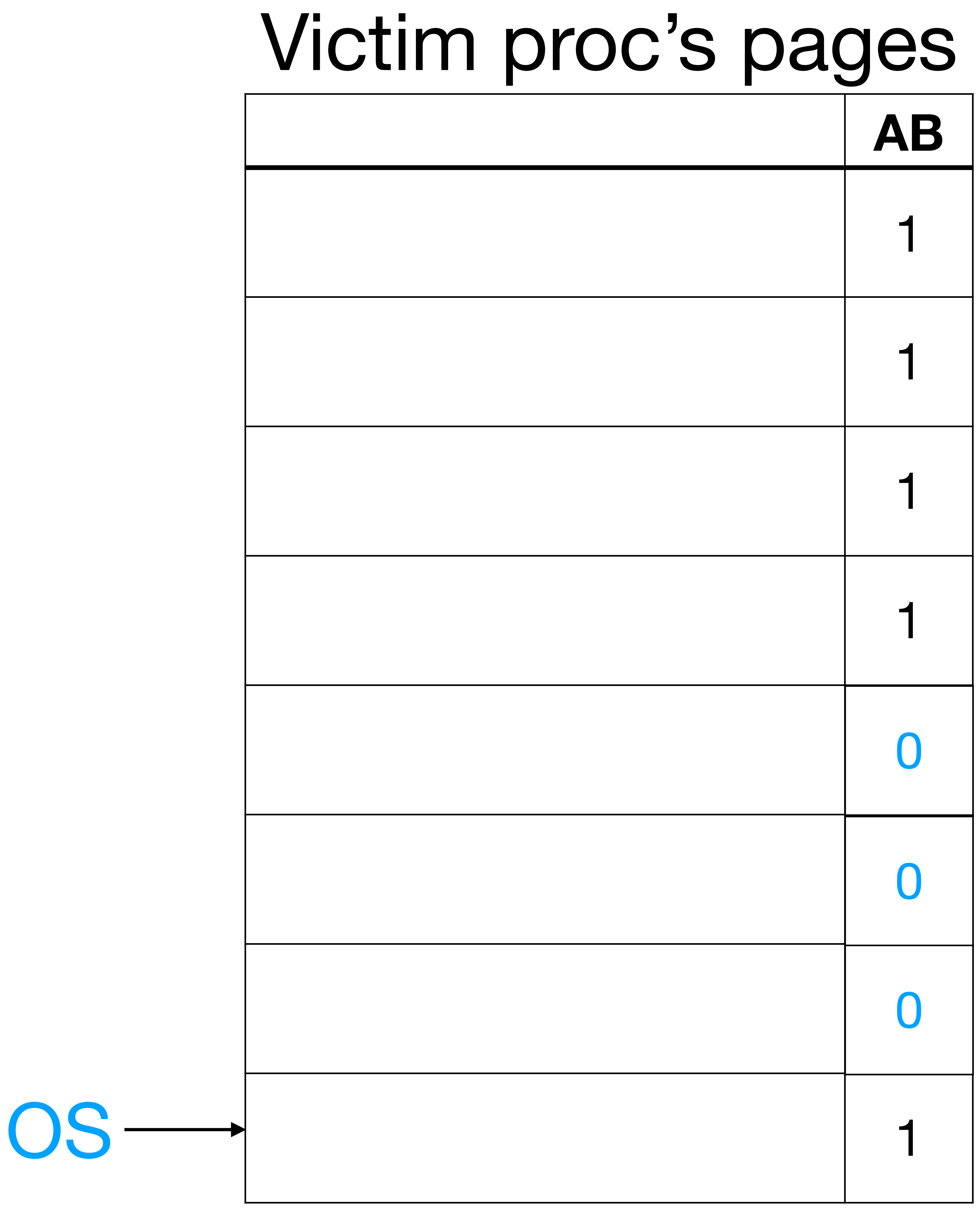
Clock algorithm

- OS clears access bit. Evicts page with access bit = 0
- Hardware sets access bit to 1
- Evicted page was “not recently used”



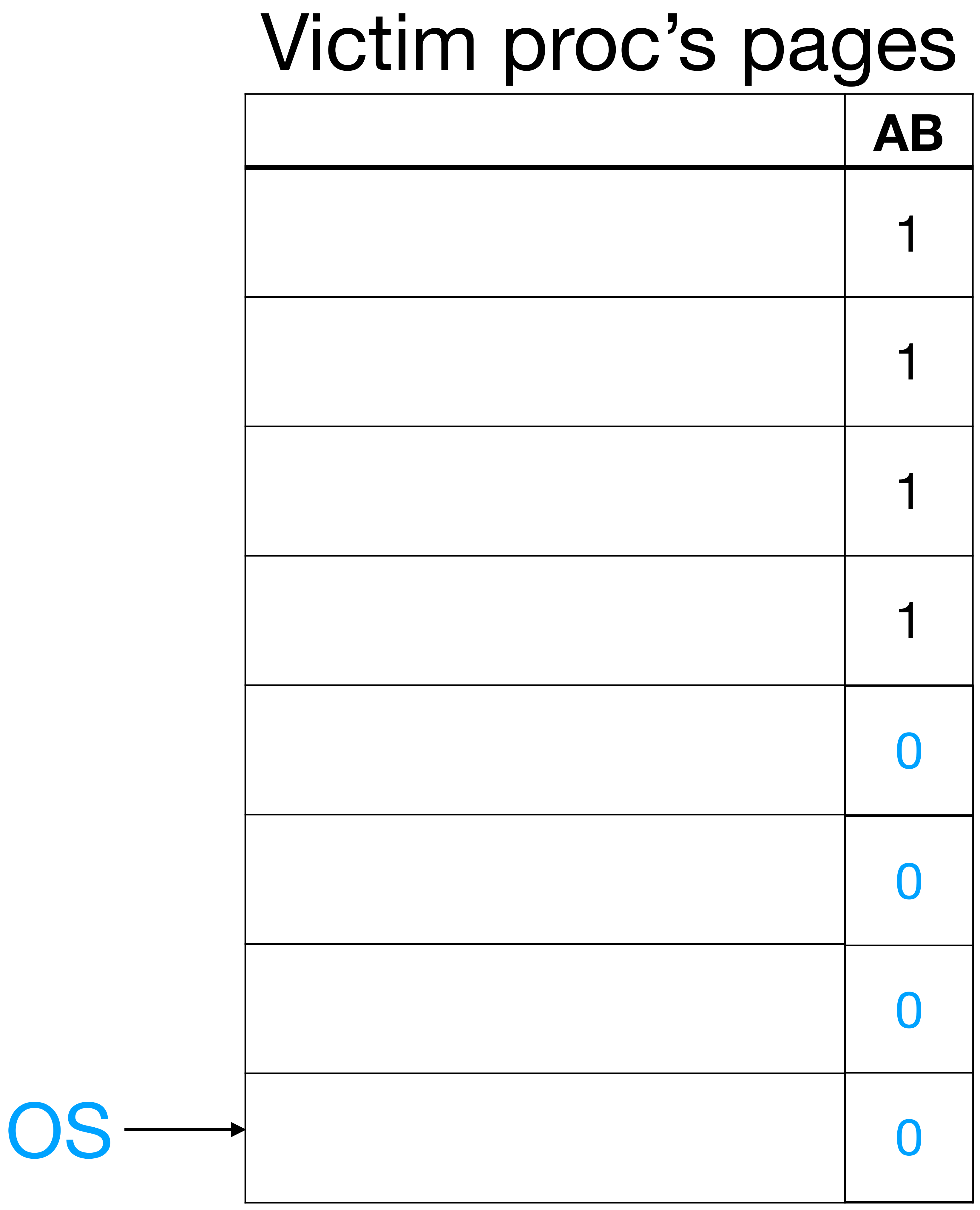
Clock algorithm

- OS clears access bit. Evicts page with access bit = 0
- Hardware sets access bit to 1
- Evicted page was “not recently used”



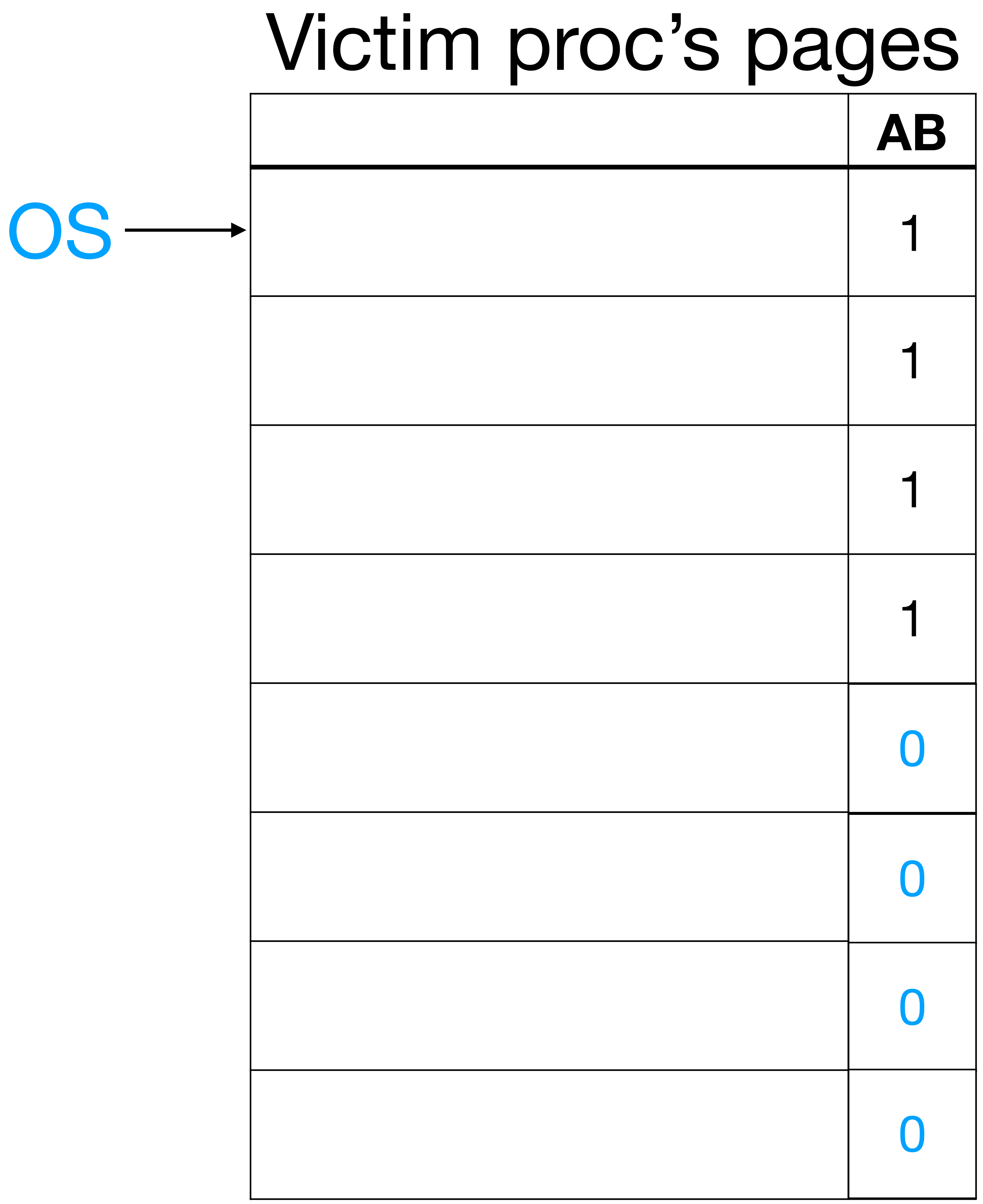
Clock algorithm

- OS clears access bit. Evicts page with access bit = 0
- Hardware sets access bit to 1
- Evicted page was “not recently used”



Clock algorithm

- OS clears access bit. Evicts page with access bit = 0
- Hardware sets access bit to 1
- Evicted page was "not recently used"



Clock algorithm

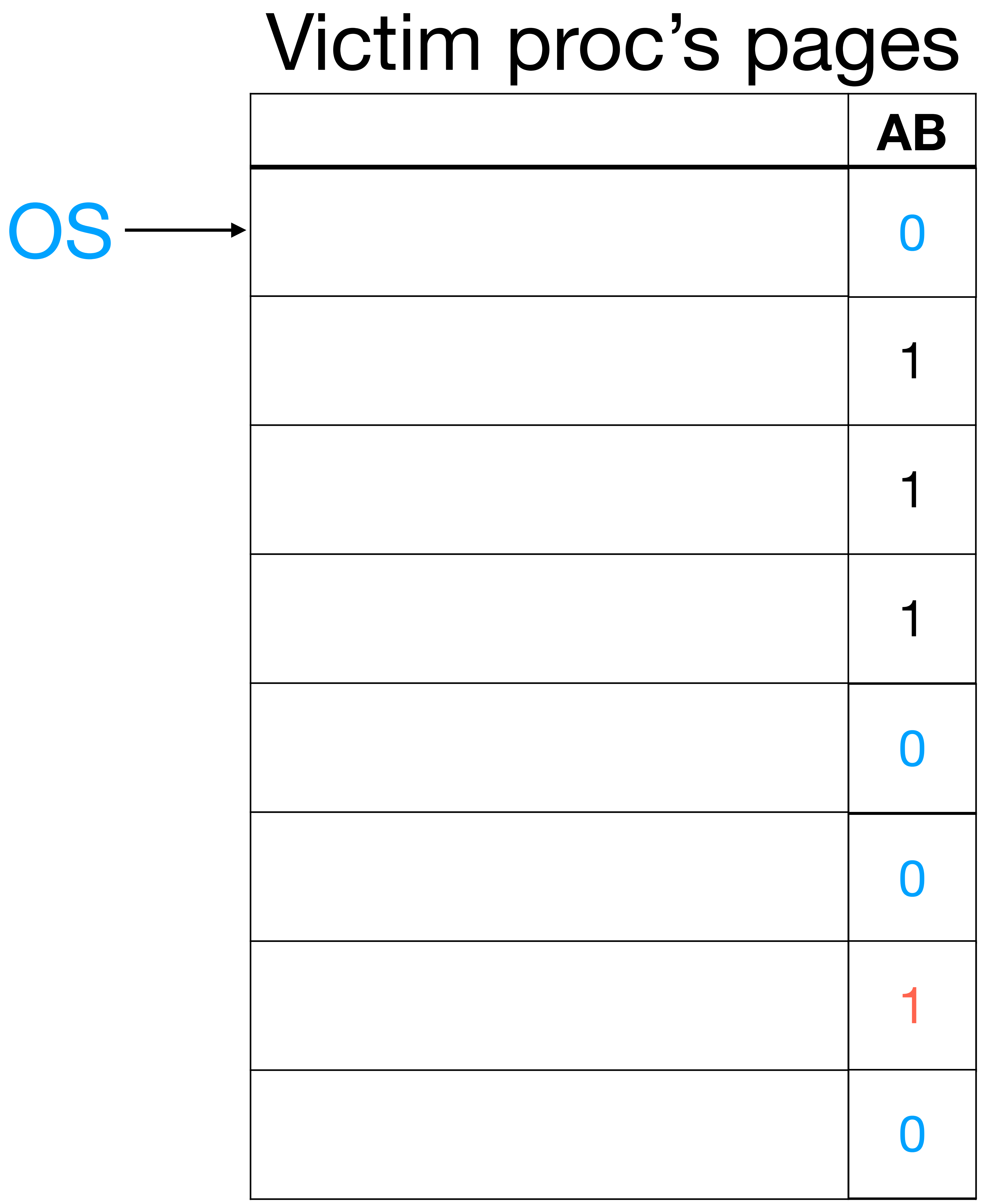
- OS clears access bit. Evicts page with access bit = 0
- Hardware sets access bit to 1
- Evicted page was “not recently used”

Victim proc’s pages

		AB
OS →		1
		1
		1
		1
		0
		0
		1
		0

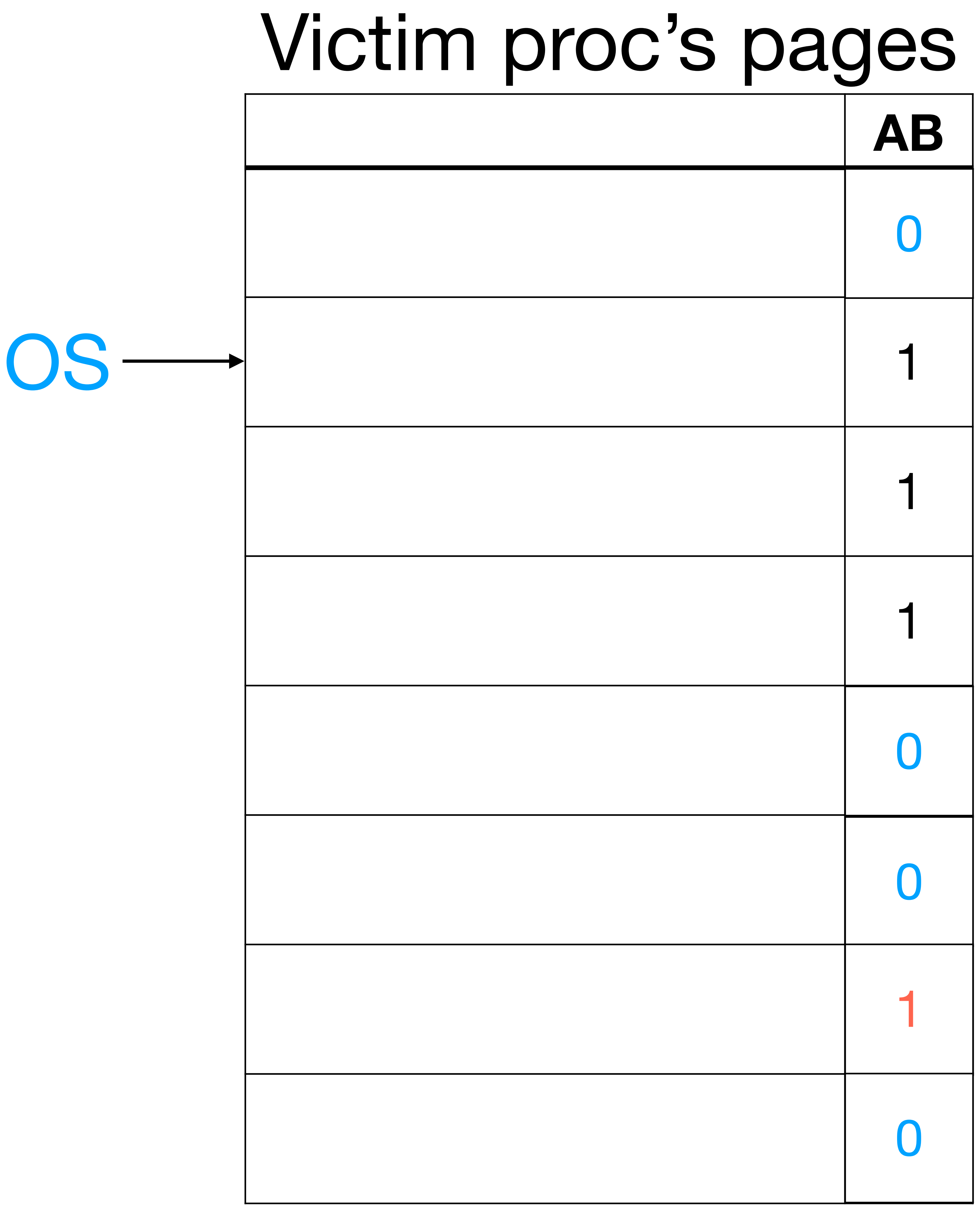
Clock algorithm

- OS clears access bit. Evicts page with access bit = 0
- Hardware sets access bit to 1
- Evicted page was "not recently used"



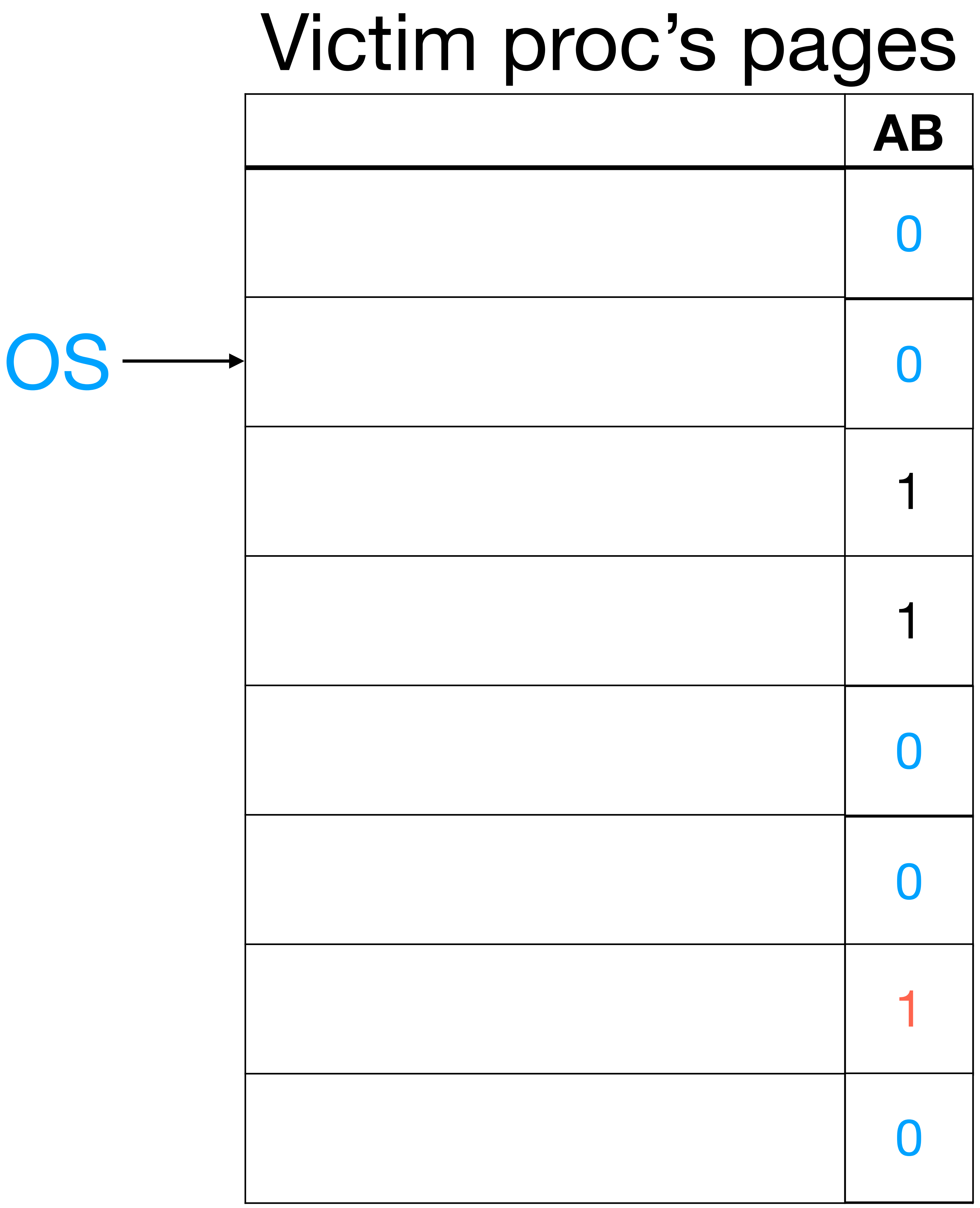
Clock algorithm

- OS clears access bit. Evicts page with access bit = 0
- Hardware sets access bit to 1
- Evicted page was “not recently used”



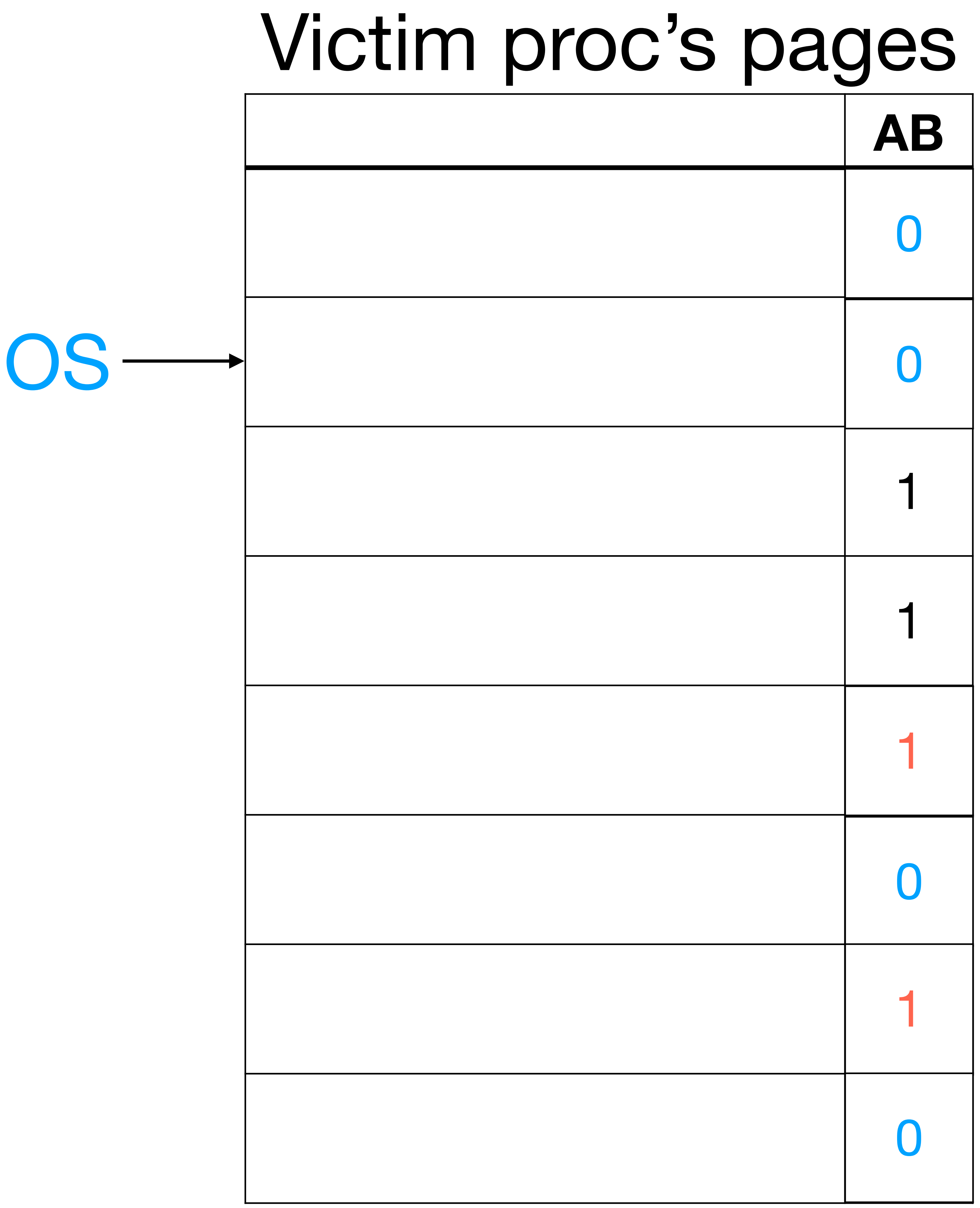
Clock algorithm

- OS clears access bit. Evicts page with access bit = 0
- Hardware sets access bit to 1
- Evicted page was “not recently used”



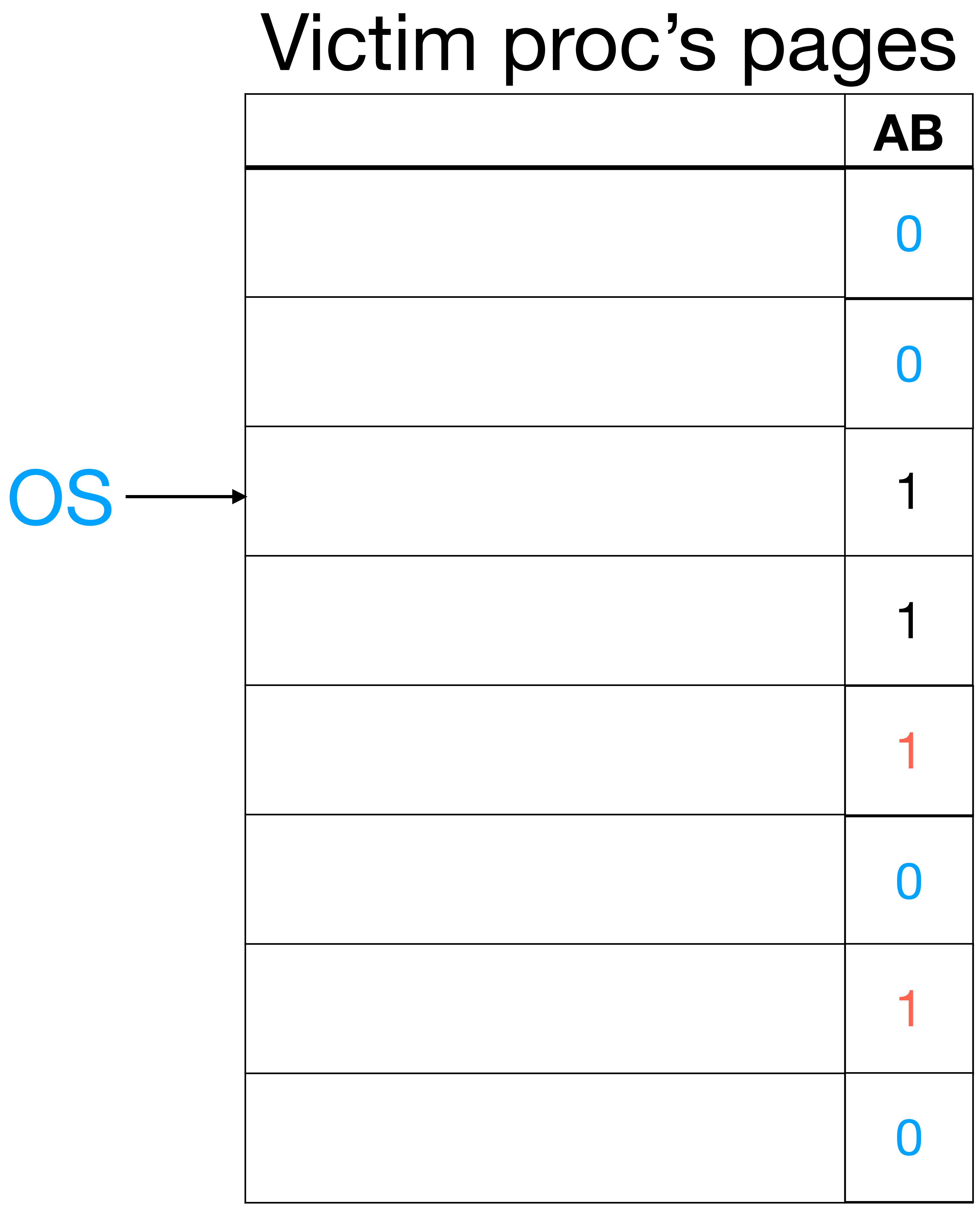
Clock algorithm

- OS clears access bit. Evicts page with access bit = 0
- Hardware sets access bit to 1
- Evicted page was "not recently used"



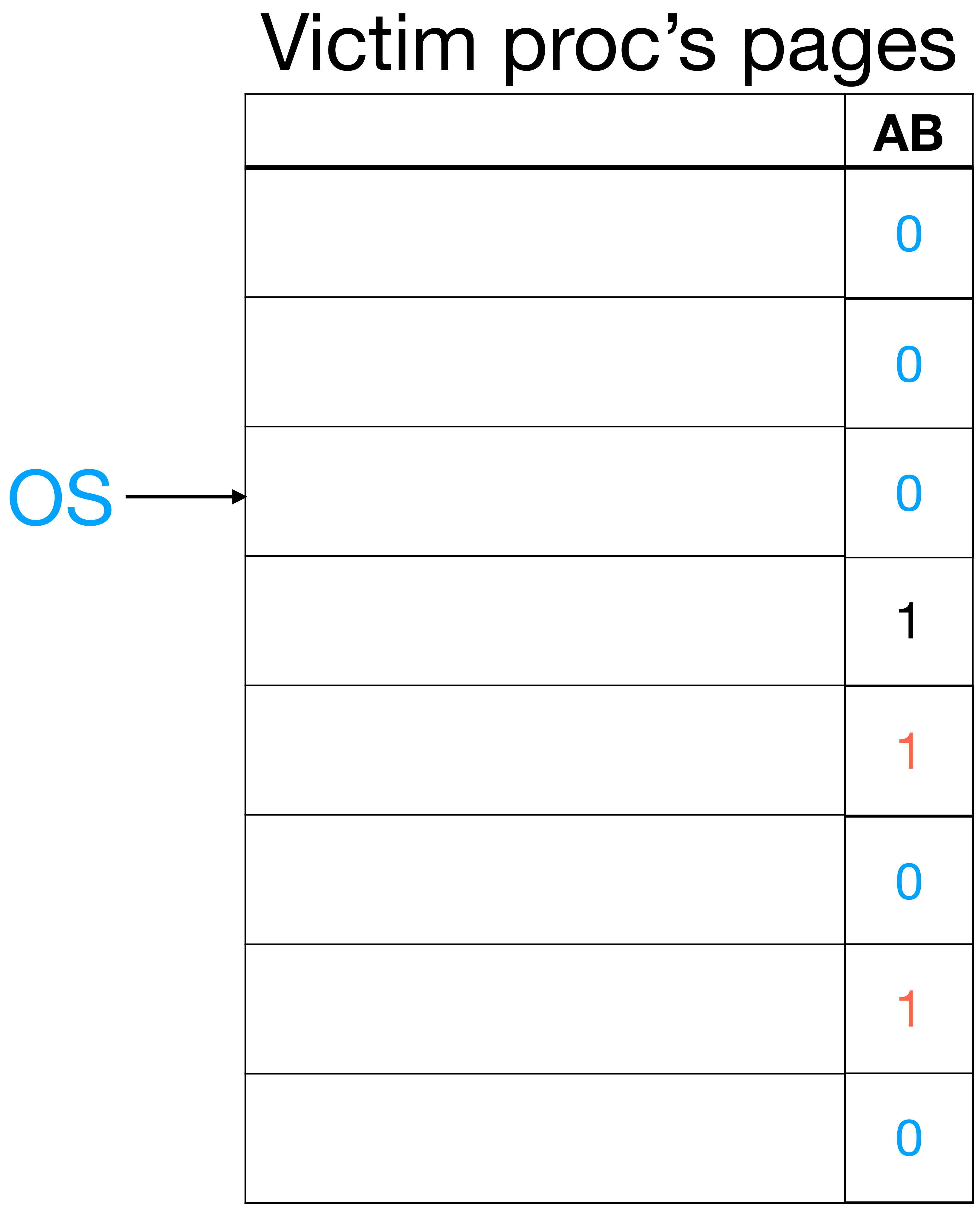
Clock algorithm

- OS clears access bit. Evicts page with access bit = 0
- Hardware sets access bit to 1
- Evicted page was “not recently used”



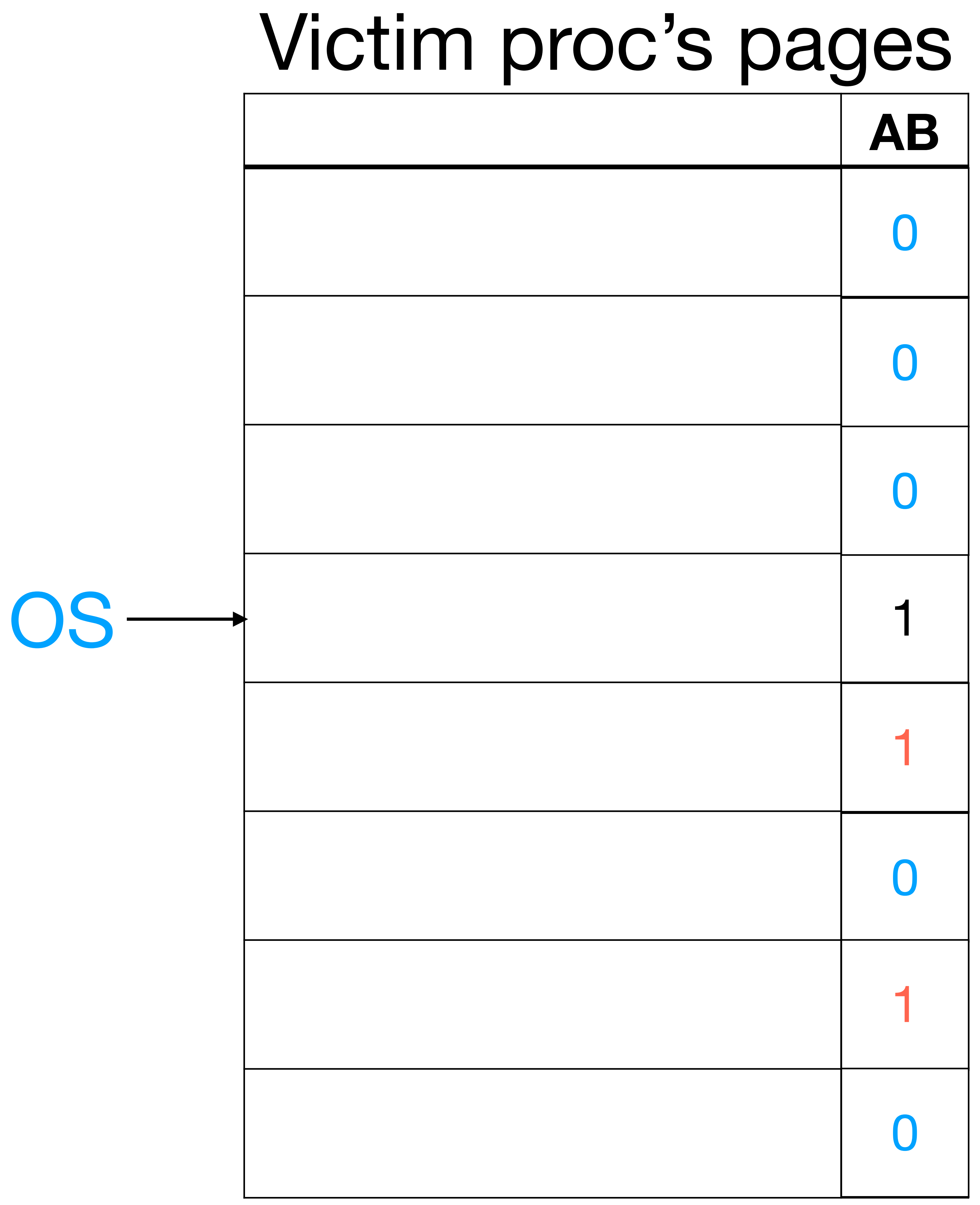
Clock algorithm

- OS clears access bit. Evicts page with access bit = 0
- Hardware sets access bit to 1
- Evicted page was “not recently used”



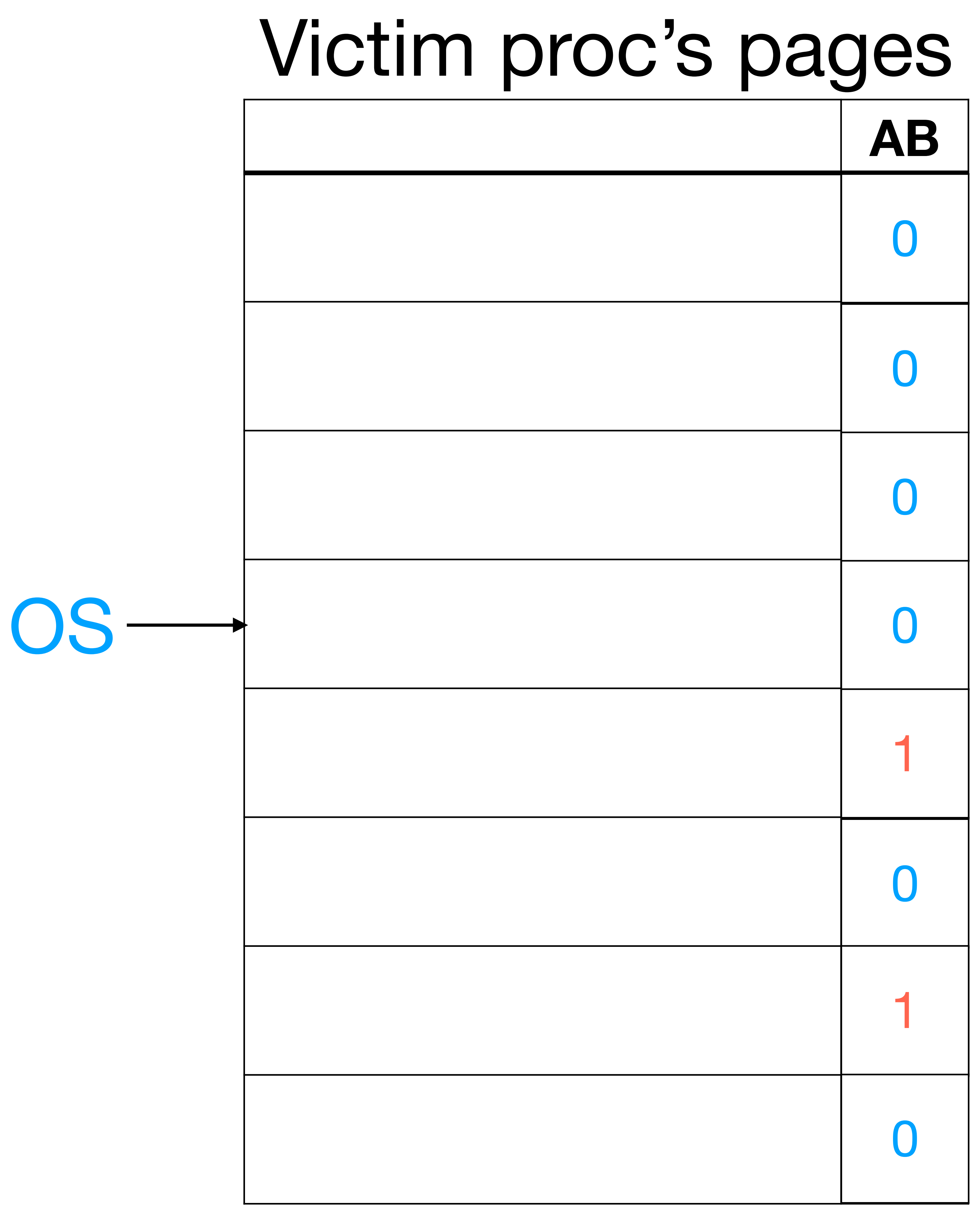
Clock algorithm

- OS clears access bit. Evicts page with access bit = 0
- Hardware sets access bit to 1
- Evicted page was “not recently used”



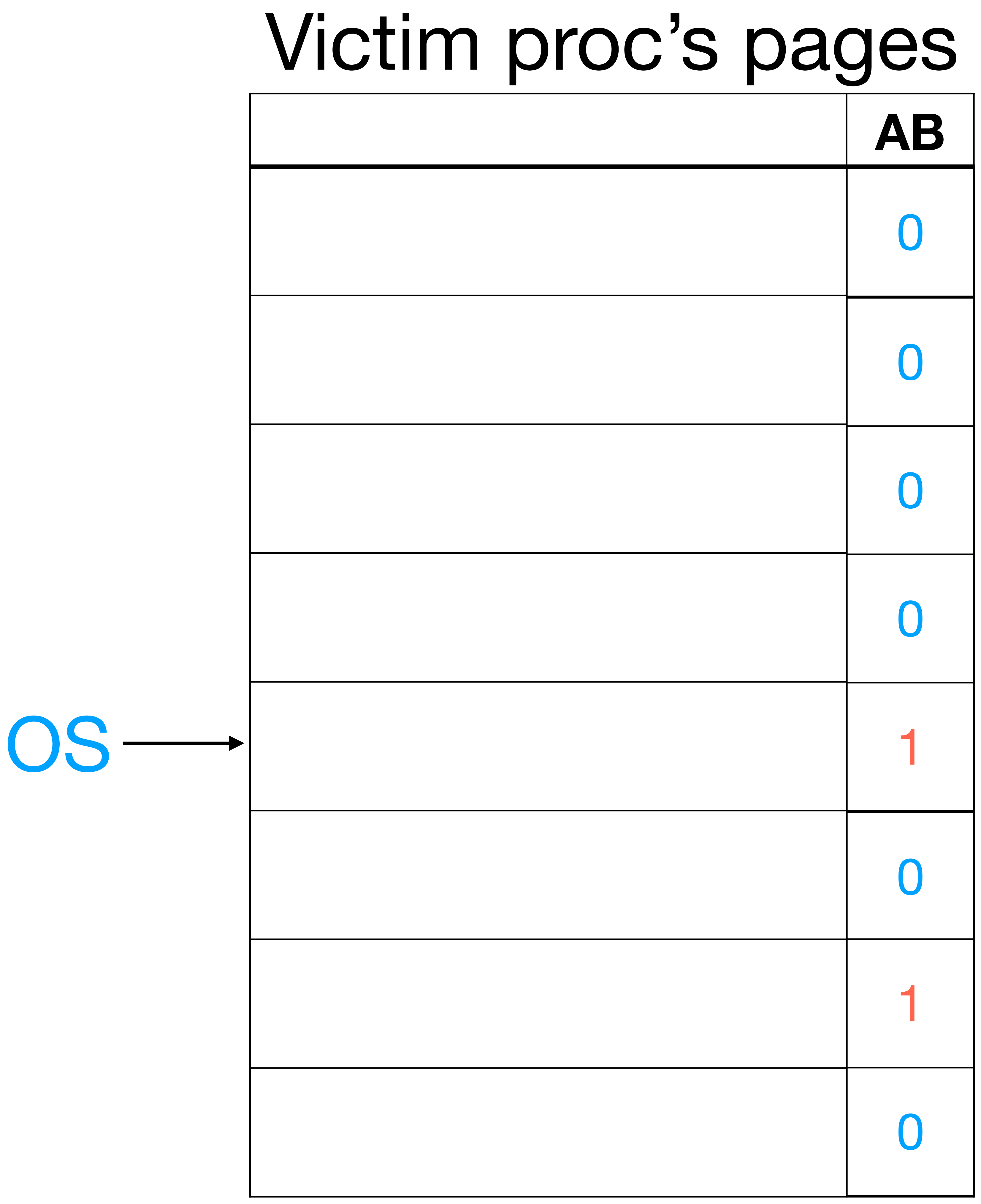
Clock algorithm

- OS clears access bit. Evicts page with access bit = 0
- Hardware sets access bit to 1
- Evicted page was “not recently used”



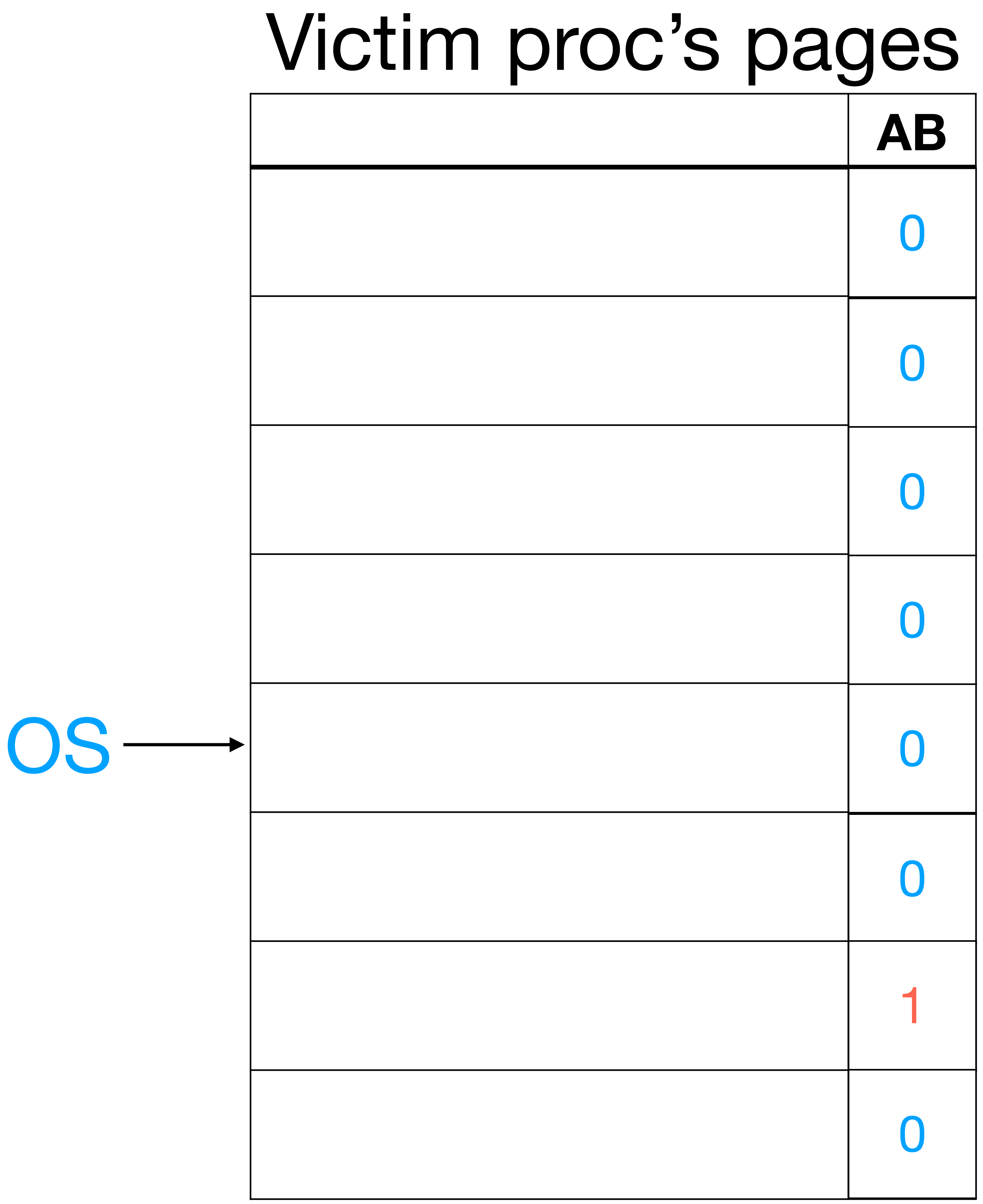
Clock algorithm

- OS clears access bit. Evicts page with access bit = 0
- Hardware sets access bit to 1
- Evicted page was “not recently used”



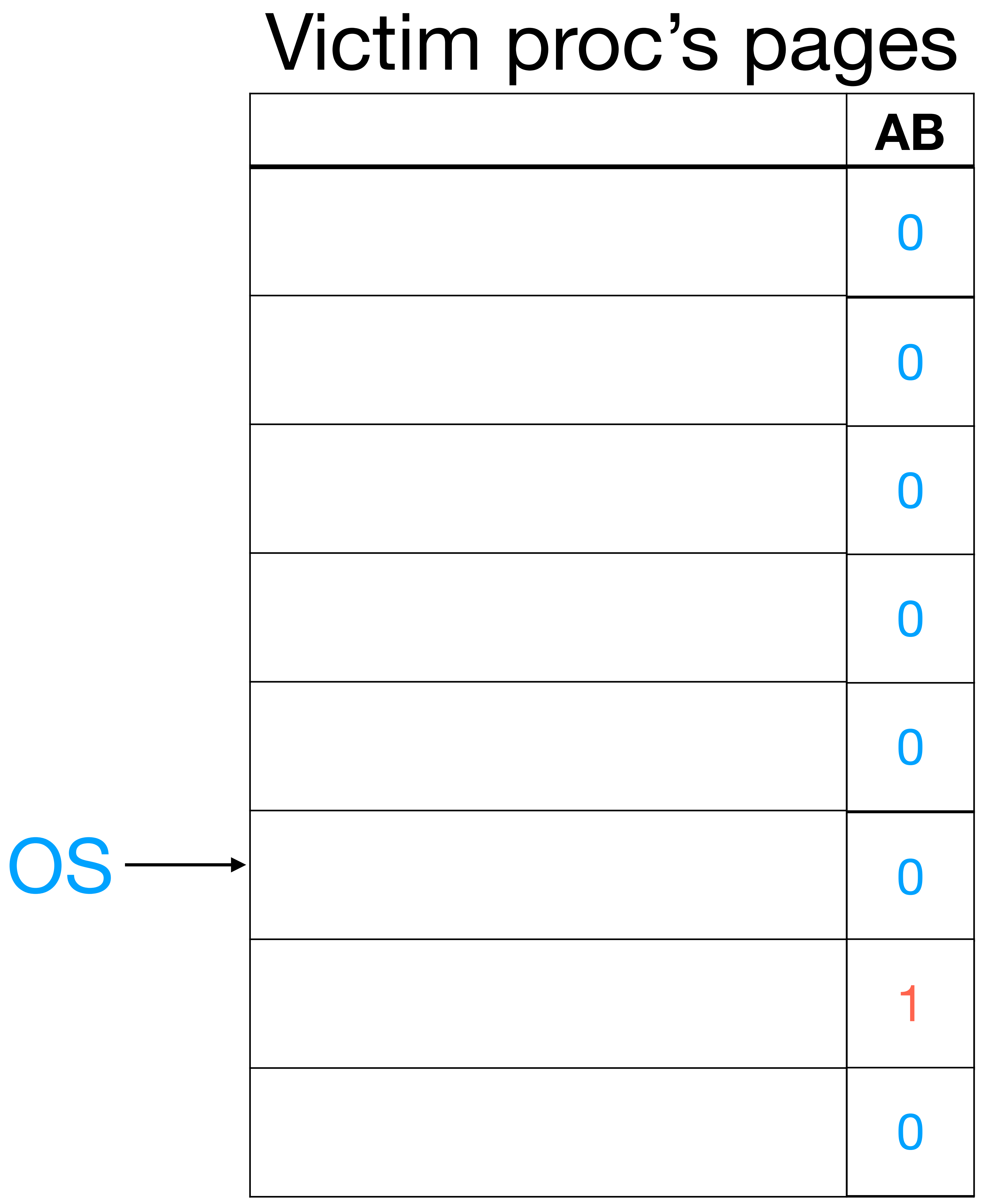
Clock algorithm

- OS clears access bit. Evicts page with access bit = 0
- Hardware sets access bit to 1
- Evicted page was “not recently used”



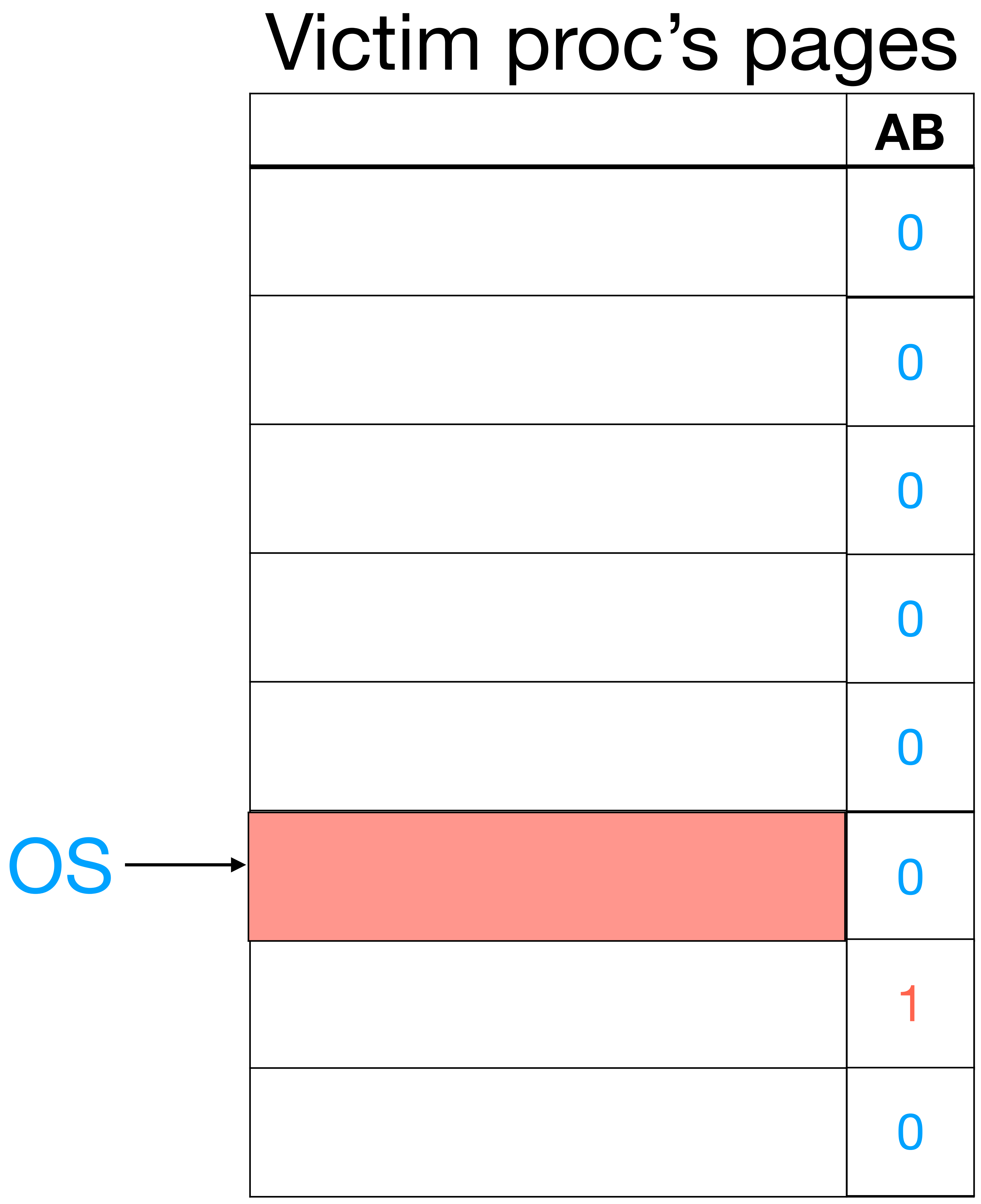
Clock algorithm

- OS clears access bit. Evicts page with access bit = 0
- Hardware sets access bit to 1
- Evicted page was “not recently used”



Clock algorithm

- OS clears access bit. Evicts page with access bit = 0
- Hardware sets access bit to 1
- Evicted page was “not recently used”



Clock algorithm (2)

- Optimisation:
  - We should prefer evicting page that has not changed since it was brought back from disk

Victim proc’s pages

		DB	AB
		0	1
		0	1
		0	1
		1	1
OS →		0	1
		1	1
		1	1
		0	1

Clock algorithm (2)

- Optimisation:
  - We should prefer evicting page that has not changed since it was brought back from disk
  - Such pages can just be deleted without doing a copy

Victim proc's pages

		DB	**AB**
		0	1
		0	1
		0	1
		1	1
OS		0	1
		1	1
		1	1
		0	1

Clock algorithm (2)

- Optimisation:
  - We should prefer evicting page that has not changed since it was brought back from disk
  - Such pages can just be deleted without doing a copy
- OS clears dirty bit when it brings a page to memory from disk

Victim proc's pages

		DB	AB
		0	1
		0	1
		0	1
		1	1
OS →		0	1
		1	1
		1	1
		0	1

Clock algorithm (2)

- Optimisation:
  - We should prefer evicting page that has not changed since it was brought back from disk
  - Such pages can just be deleted without doing a copy
- OS clears dirty bit when it brings a page to memory from disk
- Evict dirty pages iff not able to find a clean page

Victim proc’s pages

		DB	AB
		0	1
		0	1
		0	1
		1	1
OS		0	1
		1	1
		1	1
		0	1

Clock algorithm (2)

- Optimisation:
    - We should prefer evicting page that has not changed since it was brought back from disk
    - Such pages can just be deleted without doing a copy
- OS clears dirty bit when it brings a page to memory from disk
- Evict dirty pages iff not able to find a clean page

Victim proc's pages

		DB	AB
		0	1
		0	1
		0	1
		1	1
OS →		0	0
		1	1
		1	1
		0	1

Clock algorithm (2)

- Optimisation:
  - We should prefer evicting page that has not changed since it was brought back from disk
  - Such pages can just be deleted without doing a copy
- OS clears dirty bit when it brings a page to memory from disk
- Evict dirty pages iff not able to find a clean page

Victim proc's pages

	DB	AB
	0	1
	0	1
	0	1
	1	1
	0	0
OS →	1	1
	1	1
	0	1

Clock algorithm (2)

- Optimisation:
  - We should prefer evicting page that has not changed since it was brought back from disk
  - Such pages can just be deleted without doing a copy
- OS clears dirty bit when it brings a page to memory from disk
- Evict dirty pages iff not able to find a clean page

Victim proc's pages

		DB	AB
		0	1
		0	1
		0	1
		1	1
		0	0
OS →		1	0
		1	1
		0	1

Clock algorithm (2)

- Optimisation:
  - We should prefer evicting page that has not changed since it was brought back from disk
  - Such pages can just be deleted without doing a copy
- OS clears dirty bit when it brings a page to memory from disk
- Evict dirty pages iff not able to find a clean page

Victim proc's pages

		DB	AB
		0	1
		0	1
		0	1
		1	1
		0	0
		1	0
OS		1	1
		0	1

Clock algorithm (2)

- Optimisation:
  - We should prefer evicting page that has not changed since it was brought back from disk
  - Such pages can just be deleted without doing a copy
- OS clears dirty bit when it brings a page to memory from disk
- Evict dirty pages iff not able to find a clean page

Victim proc's pages

		DB	AB
		0	1
		0	1
		0	1
		1	1
		0	0
		1	0
OS →		1	0
		0	1

Clock algorithm (2)

- Optimisation:
  - We should prefer evicting page that has not changed since it was brought back from disk
  - Such pages can just be deleted without doing a copy
- OS clears dirty bit when it brings a page to memory from disk
- Evict dirty pages iff not able to find a clean page

Victim proc's pages

		DB	AB
		0	1
		0	1
		0	1
		1	1
		0	0
		1	0
		1	0
OS →		0	1

Clock algorithm (2)

- Optimisation:
  - We should prefer evicting page that has not changed since it was brought back from disk
  - Such pages can just be deleted without doing a copy
- OS clears dirty bit when it brings a page to memory from disk
- Evict dirty pages iff not able to find a clean page

Victim proc's pages

		DB	AB
		0	1
		0	1
		0	1
		1	1
		0	0
		1	0
		1	0
OS →		0	0

Clock algorithm (2)

- Optimisation:
  - We should prefer evicting page that has not changed since it was brought back from disk
  - Such pages can just be deleted without doing a copy
- OS clears dirty bit when it brings a page to memory from disk
- Evict dirty pages iff not able to find a clean page

Victim proc's pages

		DB	AB
OS →		0	1
		0	1
		0	1
		1	1
		0	0
		1	0
		1	0
		0	0

Clock algorithm (2)

- Optimisation:
  - We should prefer evicting page that has not changed since it was brought back from disk
  - Such pages can just be deleted without doing a copy
- OS clears dirty bit when it brings a page to memory from disk
- Evict dirty pages iff not able to find a clean page

Victim proc’s pages

		DB	AB
OS →		0	1
		0	1
		0	1
		1	1
		0	0
		1	0
		1	1
		0	0

Clock algorithm (2)

- Optimisation:
  - We should prefer evicting page that has not changed since it was brought back from disk
  - Such pages can just be deleted without doing a copy
- OS clears dirty bit when it brings a page to memory from disk
- Evict dirty pages iff not able to find a clean page

Victim proc's pages

		DB	AB
OS →		0	0
		0	1
		0	1
		1	1
		0	0
		1	0
		1	1
		0	0

Clock algorithm (2)

- Optimisation:
  - We should prefer evicting page that has not changed since it was brought back from disk
  - Such pages can just be deleted without doing a copy
- OS clears dirty bit when it brings a page to memory from disk
- Evict dirty pages iff not able to find a clean page

Victim proc’s pages

		DB	AB
		0	0
OS →		0	1
		0	1
		1	1
		0	0
		1	0
		1	1
		0	0

Clock algorithm (2)

- Optimisation:
  - We should prefer evicting page that has not changed since it was brought back from disk
  - Such pages can just be deleted without doing a copy
- OS clears dirty bit when it brings a page to memory from disk
- Evict dirty pages iff not able to find a clean page

Victim proc’s pages

		DB	AB
		0	0
OS →		0	0
		0	1
		1	1
		0	0
		1	0
		1	1
		0	0

Clock algorithm (2)

- Optimisation:
    - We should prefer evicting page that has not changed since it was brought back from disk
    - Such pages can just be deleted without doing a copy
- OS clears dirty bit when it brings a page to memory from disk
- Evict dirty pages iff not able to find a clean page

Victim proc's pages

	DB	AB
	0	0
	0	0
	0	1
	1	1
	0	1
	1	0
	1	1
	0	0

OS

Clock algorithm (2)

- Optimisation:
  - We should prefer evicting page that has not changed since it was brought back from disk
  - Such pages can just be deleted without doing a copy
- OS clears dirty bit when it brings a page to memory from disk
- Evict dirty pages iff not able to find a clean page

Victim proc's pages

		DB	AB
		0	0
		0	0
OS →		0	1
		1	1
		0	1
		1	0
		1	1
		0	0

Clock algorithm (2)

- Optimisation:
  - We should prefer evicting page that has not changed since it was brought back from disk
  - Such pages can just be deleted without doing a copy
- OS clears dirty bit when it brings a page to memory from disk
- Evict dirty pages iff not able to find a clean page

Victim proc's pages

		DB	AB
		0	0
		0	0
OS →		0	0
		1	1
		0	1
		1	0
		1	1
		0	0

Clock algorithm (2)

- Optimisation:
  - We should prefer evicting page that has not changed since it was brought back from disk
  - Such pages can just be deleted without doing a copy
- OS clears dirty bit when it brings a page to memory from disk
- Evict dirty pages iff not able to find a clean page

Victim proc's pages

		DB	AB
		0	0
		0	0
		0	0
OS		1	1
		0	1
		1	0
		1	1
		0	0

Clock algorithm (2)

- Optimisation:
    - We should prefer evicting page that has not changed since it was brought back from disk
    - Such pages can just be deleted without doing a copy
- OS clears dirty bit when it brings a page to memory from disk
- Evict dirty pages iff not able to find a clean page

Victim proc's pages

		DB	AB
		0	0
		0	0
		0	0
OS		1	0
		0	1
		1	0
		1	1
		0	0

Clock algorithm (2)

- Optimisation:
  - We should prefer evicting page that has not changed since it was brought back from disk
  - Such pages can just be deleted without doing a copy
- OS clears dirty bit when it brings a page to memory from disk
- Evict dirty pages iff not able to find a clean page

Victim proc's pages

		DB	AB
		0	0
		0	0
		0	0
		1	0
OS →		0	1
		1	0
		1	1
		0	0

Clock algorithm (2)

- Optimisation:
    - We should prefer evicting page that has not changed since it was brought back from disk
    - Such pages can just be deleted without doing a copy
- OS clears dirty bit when it brings a page to memory from disk
- Evict dirty pages iff not able to find a clean page

Victim proc's pages

		DB	AB
		0	0
		0	0
		0	0
		1	0
OS →		0	0
		1	0
		1	1
		0	0

Clock algorithm (2)

- Optimisation:
  - We should prefer evicting page that has not changed since it was brought back from disk
  - Such pages can just be deleted without doing a copy
- OS clears dirty bit when it brings a page to memory from disk
- Evict dirty pages iff not able to find a clean page

Victim proc's pages

		DB	AB
		0	0
		0	0
		0	0
		1	0
		0	0
OS →		1	0
		1	1
		0	0

Clock algorithm (2)

- Optimisation:
  - We should prefer evicting page that has not changed since it was brought back from disk
  - Such pages can just be deleted without doing a copy
- OS clears dirty bit when it brings a page to memory from disk
- Evict dirty pages iff not able to find a clean page

Victim proc’s pages

		DB	AB
		0	0
		0	0
		0	0
		1	0
		0	0
		1	0
OS →		1	1
		0	0

Clock algorithm (2)

- Optimisation:
  - We should prefer evicting page that has not changed since it was brought back from disk
  - Such pages can just be deleted without doing a copy
- OS clears dirty bit when it brings a page to memory from disk
- Evict dirty pages iff not able to find a clean page

Victim proc’s pages

		DB	**AB**
		0	0
		0	0
		0	0
		1	0
		0	0
		1	0
OS →		1	0
		0	0

Clock algorithm (2)

- Optimisation:
  - We should prefer evicting page that has not changed since it was brought back from disk
  - Such pages can just be deleted without doing a copy
- OS clears dirty bit when it brings a page to memory from disk
- Evict dirty pages iff not able to find a clean page

Victim proc's pages

		DB	AB
		0	0
		0	0
		0	0
		1	0
		0	0
		1	0
		1	0
OS →		0	0

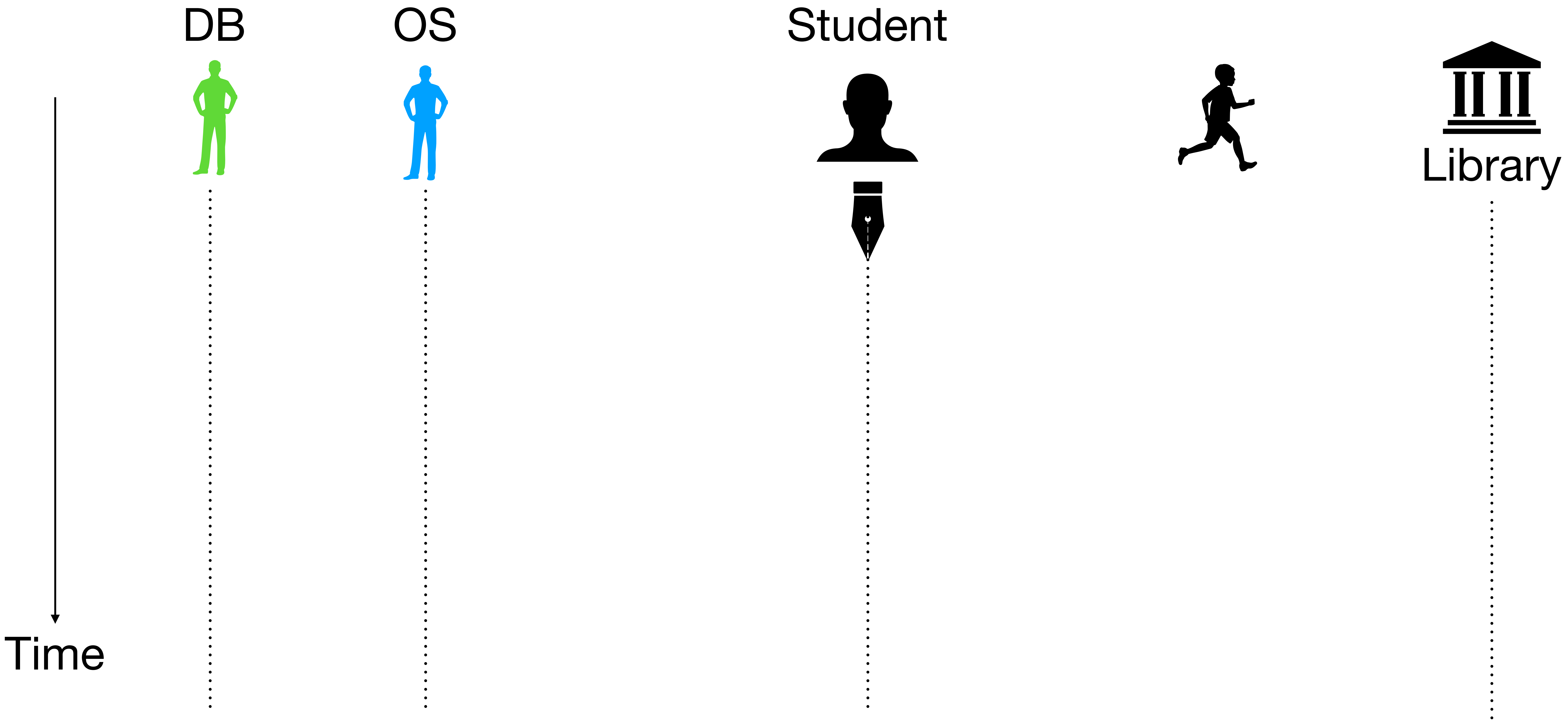
Clock algorithm (2)

- Optimisation:
  - We should prefer evicting page that has not changed since it was brought back from disk
  - Such pages can just be deleted without doing a copy
- OS clears dirty bit when it brings a page to memory from disk
- Evict dirty pages iff not able to find a clean page

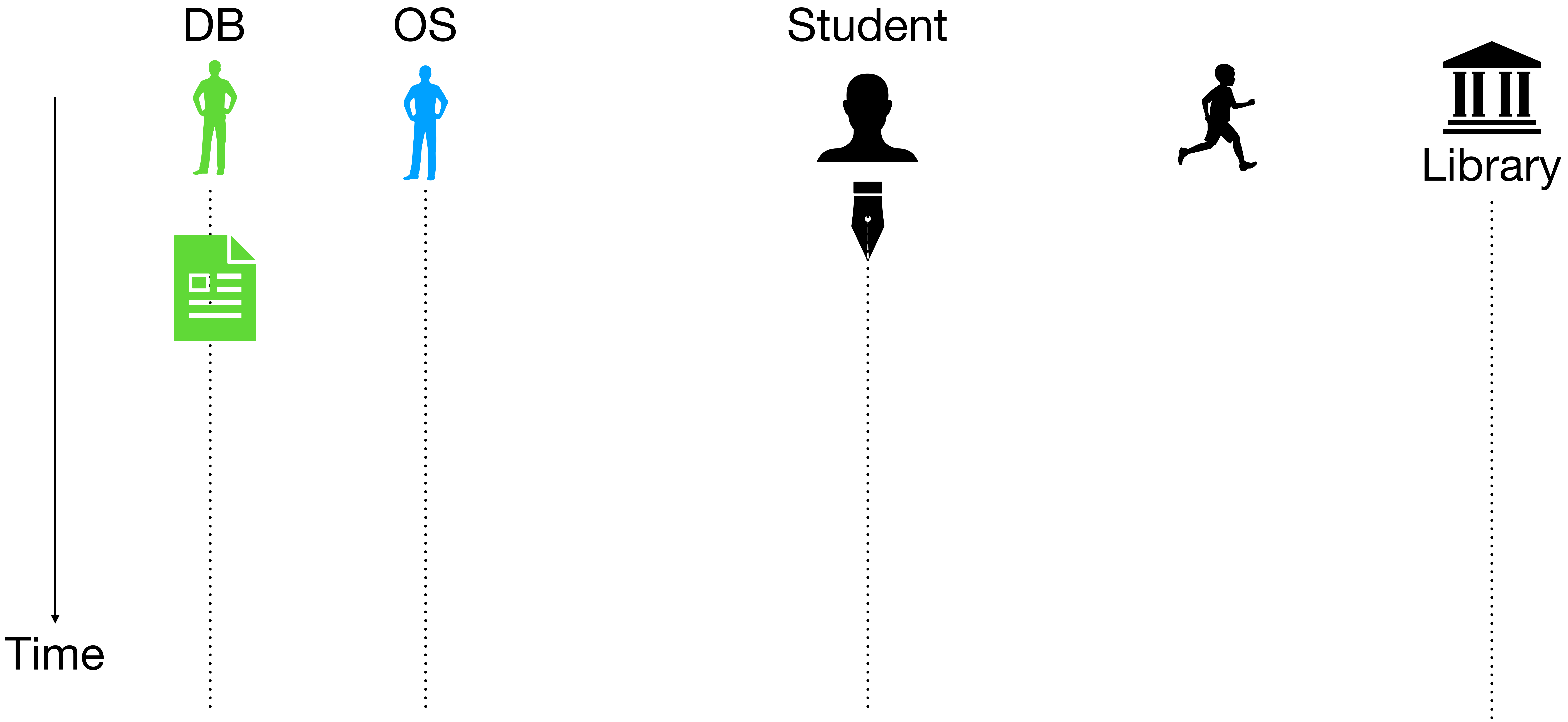
Victim proc's pages

		DB	AB
		0	0
		0	0
		0	0
		1	0
		0	0
		1	0
		1	0
OS		0	0

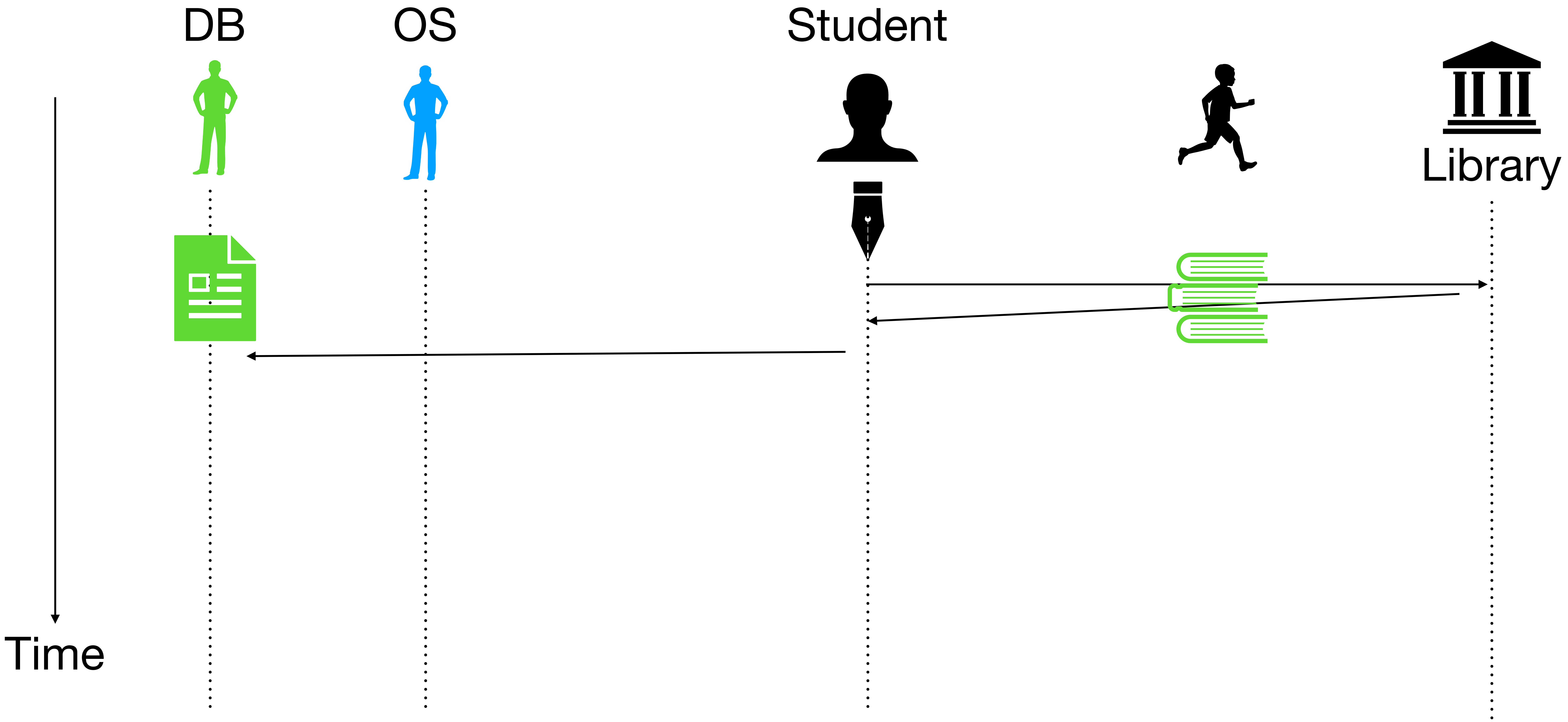
Thrashing: Library analogy



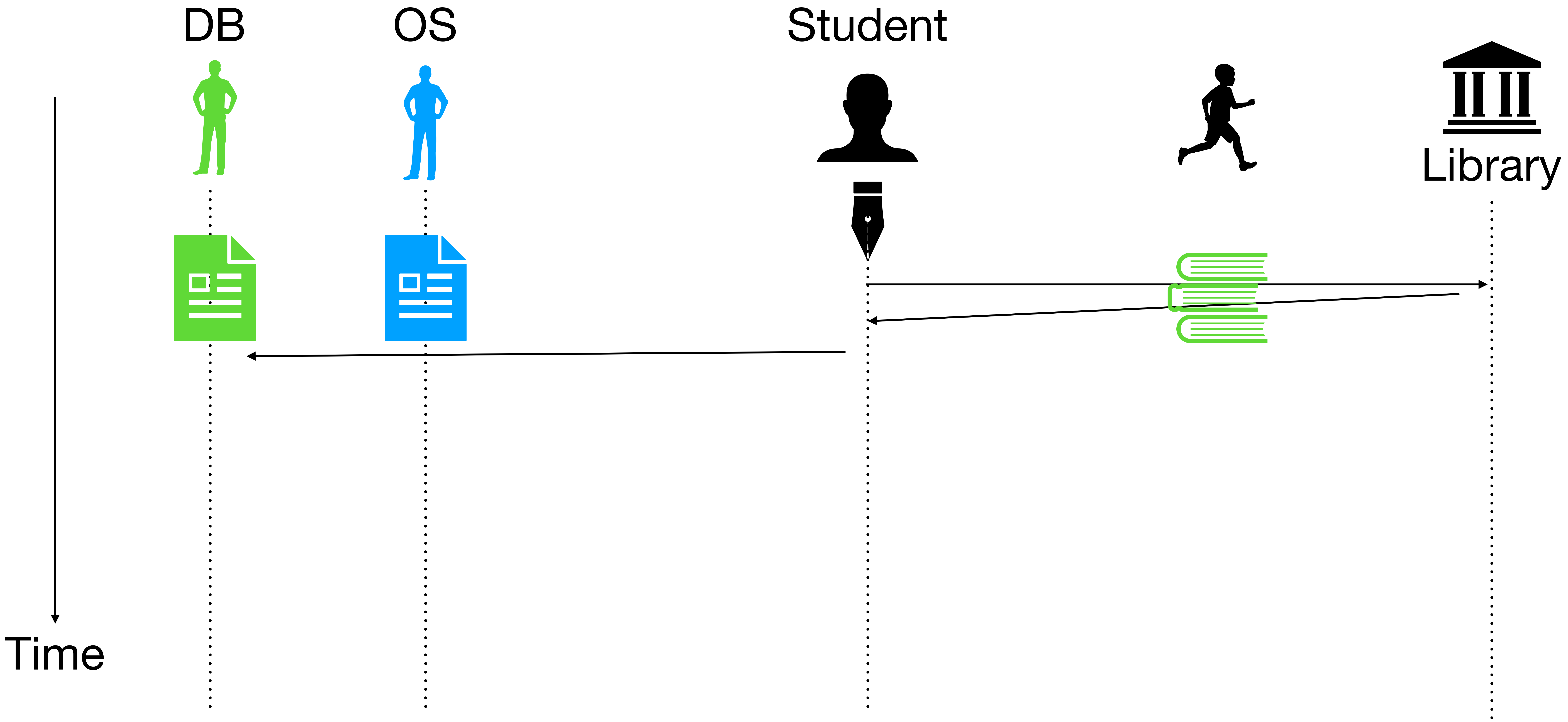
Thrashing: Library analogy



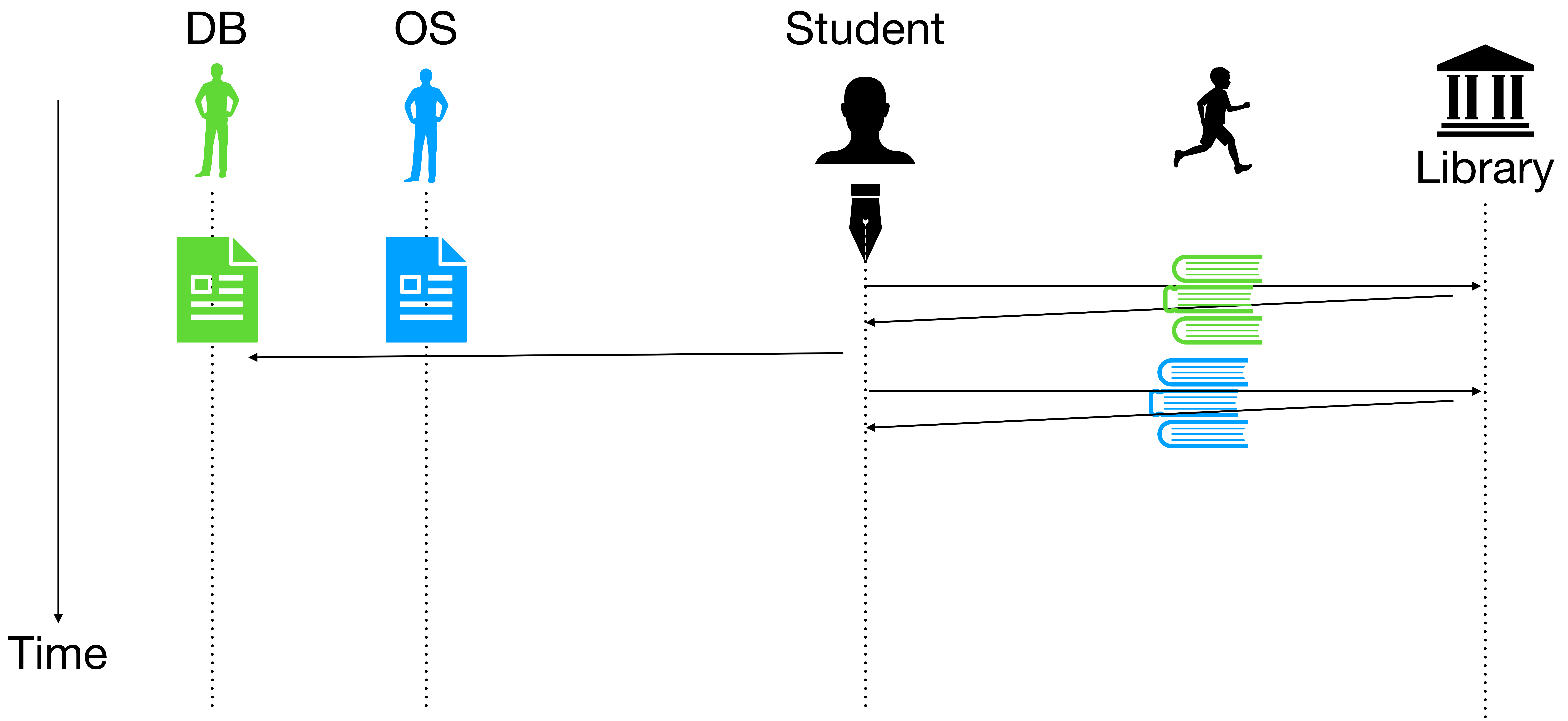
Thrashing: Library analogy



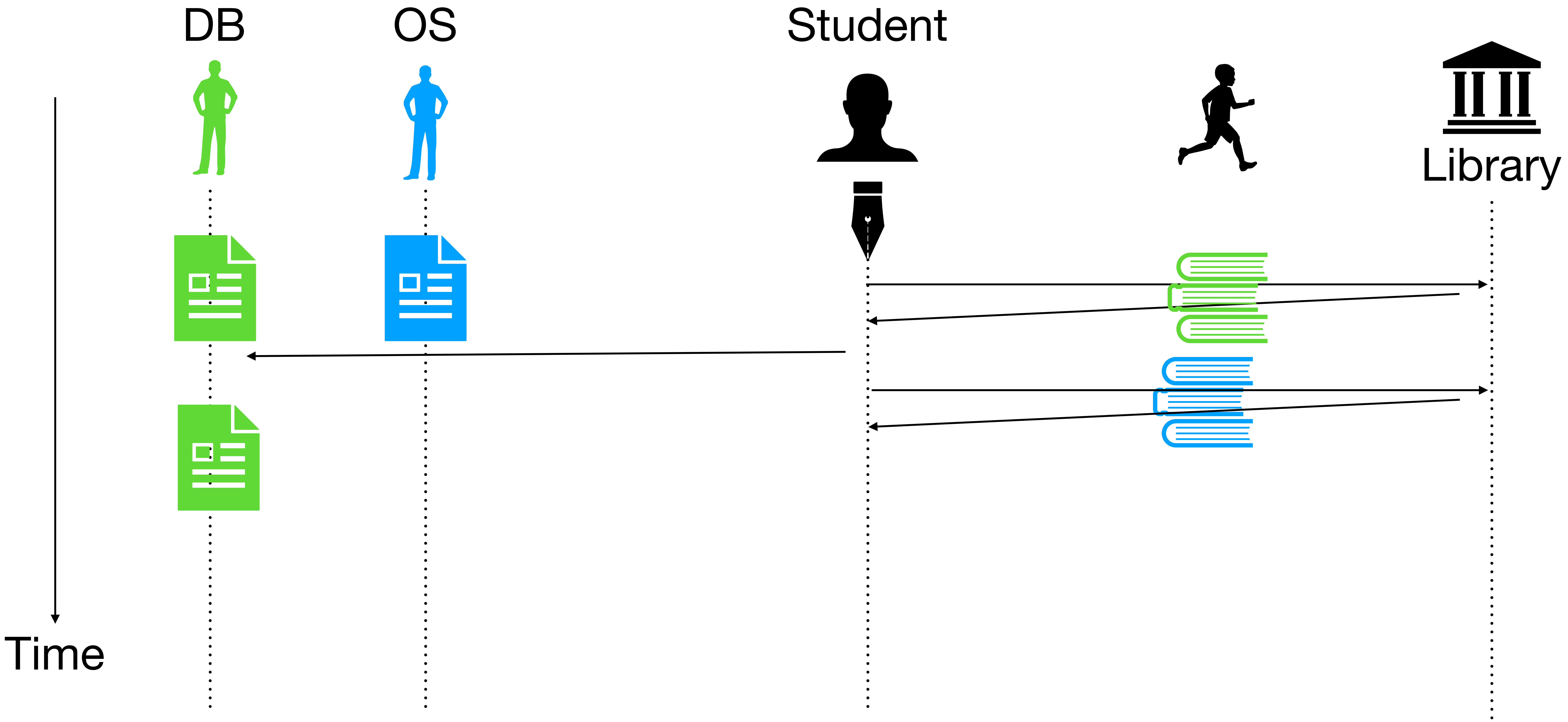
Thrashing: Library analogy



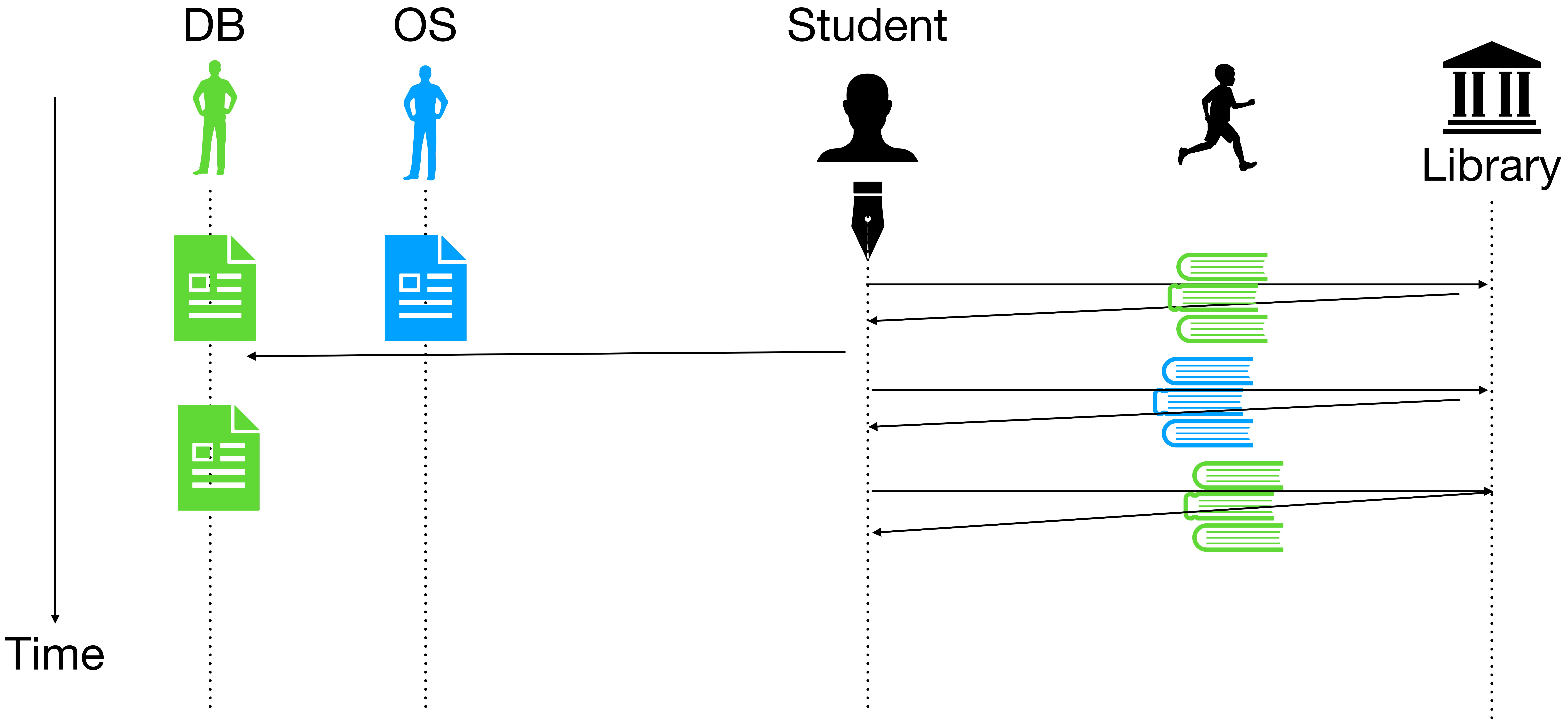
Thrashing: Library analogy



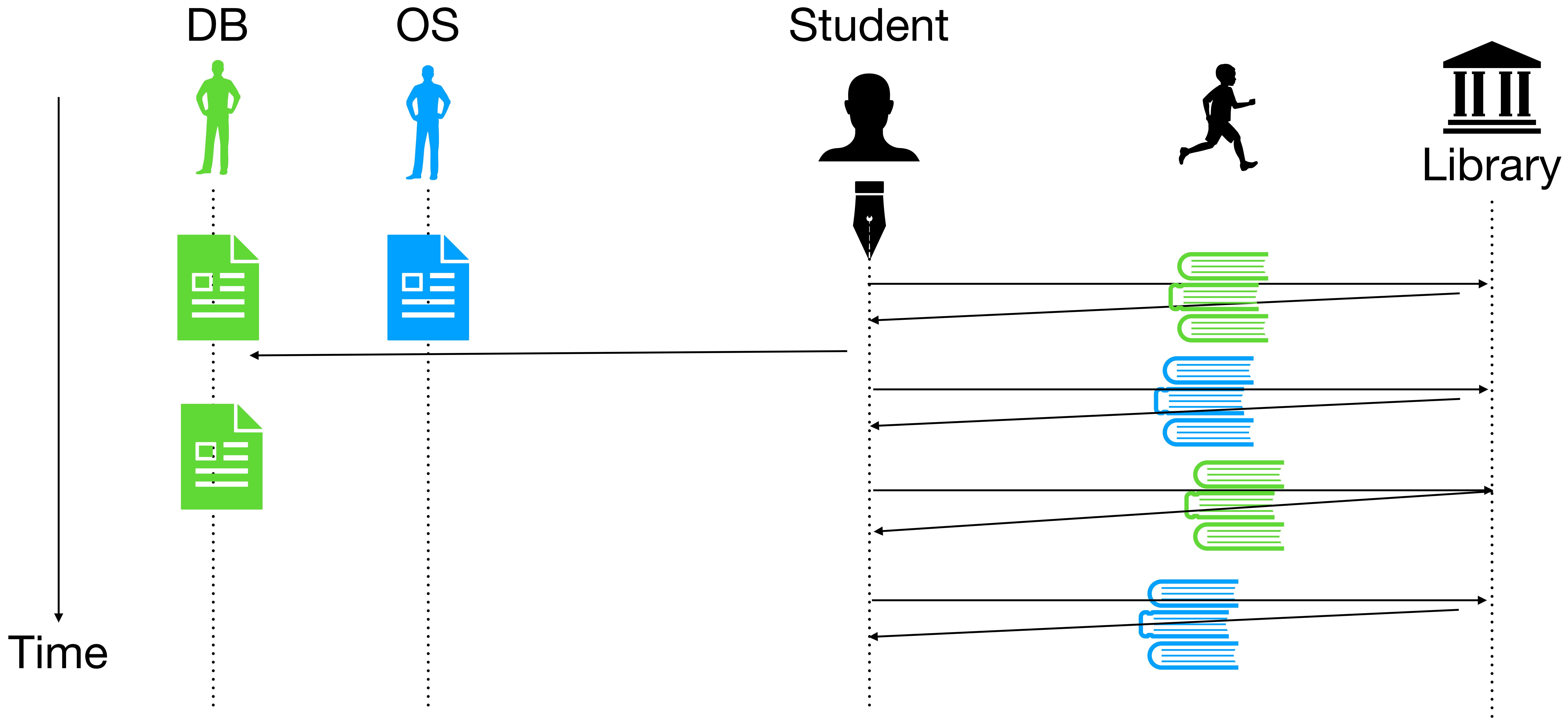
Thrashing: Library analogy



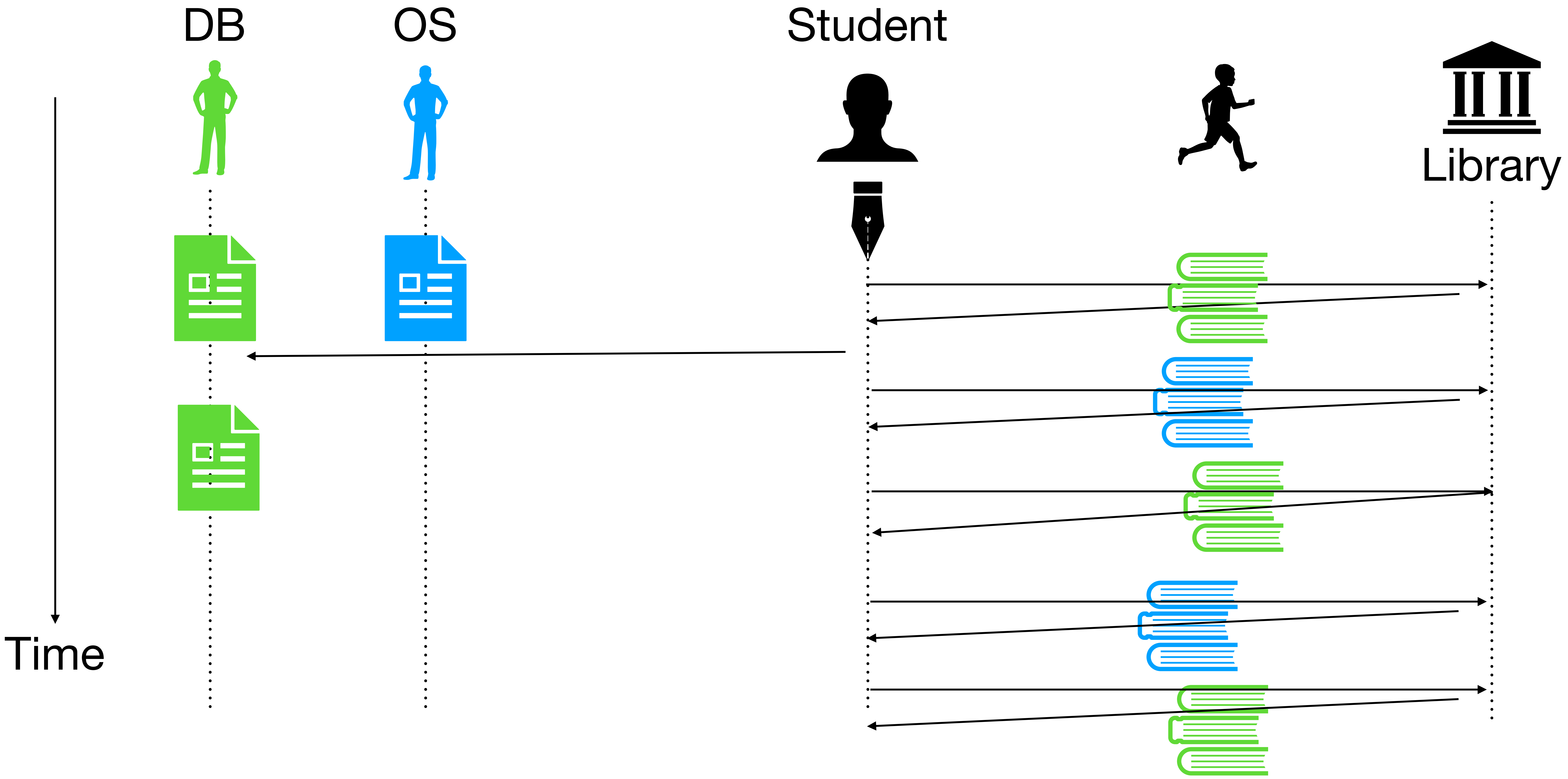
Thrashing: Library analogy



Thrashing: Library analogy



Thrashing: Library analogy



Thrashing: Library analogy (2)

- Problem: Library only allows one book to be checked out. Student is constantly running to/from the library. Not able to do work on any assignment.

Thrashing: Library analogy (2)

- Problem: Library only allows one book to be checked out. Student is constantly running to/from the library. Not able to do work on any assignment.
- Solution:

Thrashing: Library analogy (2)

- Problem: Library only allows one book to be checked out. Student is constantly running to/from the library. Not able to do work on any assignment.
- Solution:
  - Reduce “working set”

Thrashing: Library analogy (2)

- Problem: Library only allows one book to be checked out. Student is constantly running to/from the library. Not able to do work on any assignment.
- Solution:
  - Reduce “working set”
    - I will work on OS assignment completely before worrying about DB assignment

Thrashing: Library analogy (2)

- Problem: Library only allows one book to be checked out. Student is constantly running to/from the library. Not able to do work on any assignment.
- Solution:
  - Reduce “working set”
    - I will work on OS assignment completely before worrying about DB assignment
    - I will not work on DB assignment

Thrashing: Library analogy (2)

- Problem: Library only allows one book to be checked out. Student is constantly running to/from the library. Not able to do work on any assignment.
- Solution:
  - Reduce “working set”
    - I will work on OS assignment completely before worrying about DB assignment
    - I will not work on DB assignment
  - Buy a book to avoid going to library

Thrashing

- Total working set of running processes is larger than physical memory. Constantly swapping in and out pages to/from disk. Not able to work.

Thrashing

- Total working set of running processes is larger than physical memory. Constantly swapping in and out pages to/from disk. Not able to work.

- Solution:

Thrashing

- Total working set of running processes is larger than physical memory. Constantly swapping in and out pages to/from disk. Not able to work.
- Solution:
  - Reduce working set

Thrashing

- Total working set of running processes is larger than physical memory. Constantly swapping in and out pages to/from disk. Not able to work.
- Solution:
  - Reduce working set
    - Admission control: run some processes for some time and then some other

Thrashing

- Total working set of running processes is larger than physical memory. Constantly swapping in and out pages to/from disk. Not able to work.
- Solution:
  - Reduce working set
    - Admission control: run some processes for some time and then some other
    - Out-of-memory killer: Linux kills most memory intensive process

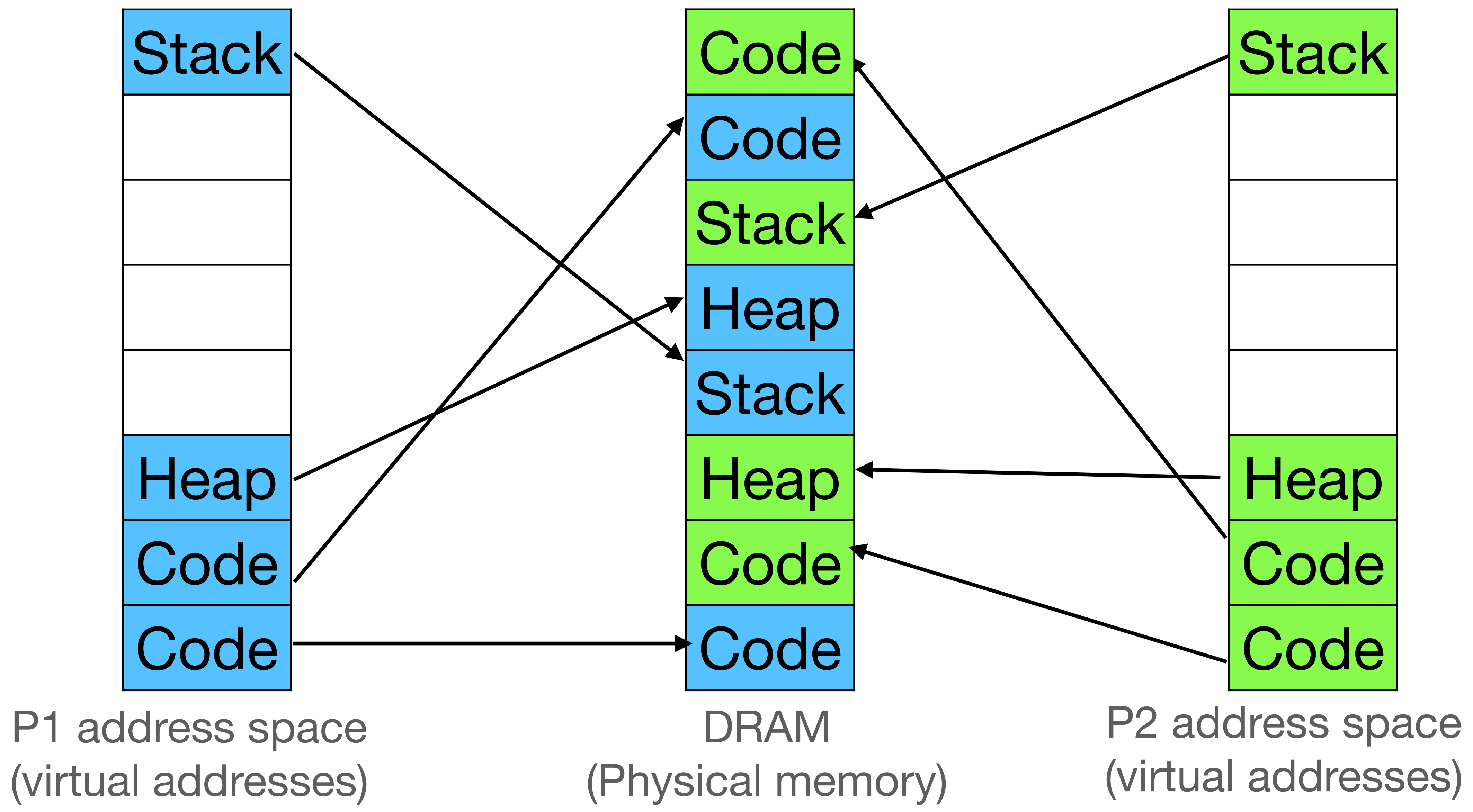
Thrashing

- Total working set of running processes is larger than physical memory. Constantly swapping in and out pages to/from disk. Not able to work.
- Solution:
  - Reduce working set
    - Admission control: run some processes for some time and then some other
    - Out-of-memory killer: Linux kills most memory intensive process
  - Buy more memory to avoid copying to/from disk

Reducing memory pressure

Kernel same page merging (KSM)

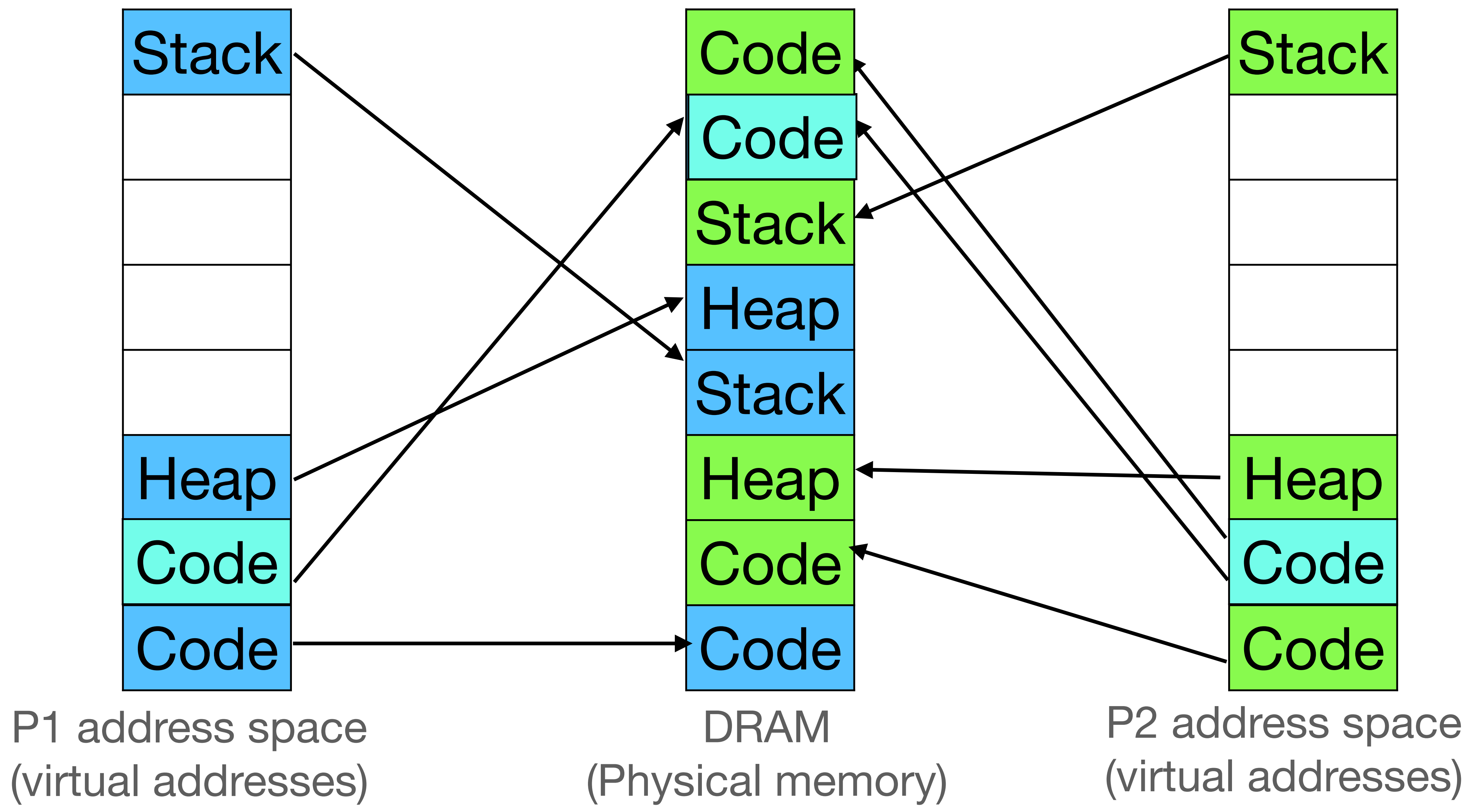
- OS periodically scans a few pages and computes their hash



Reducing memory pressure

Kernel same page merging (KSM)

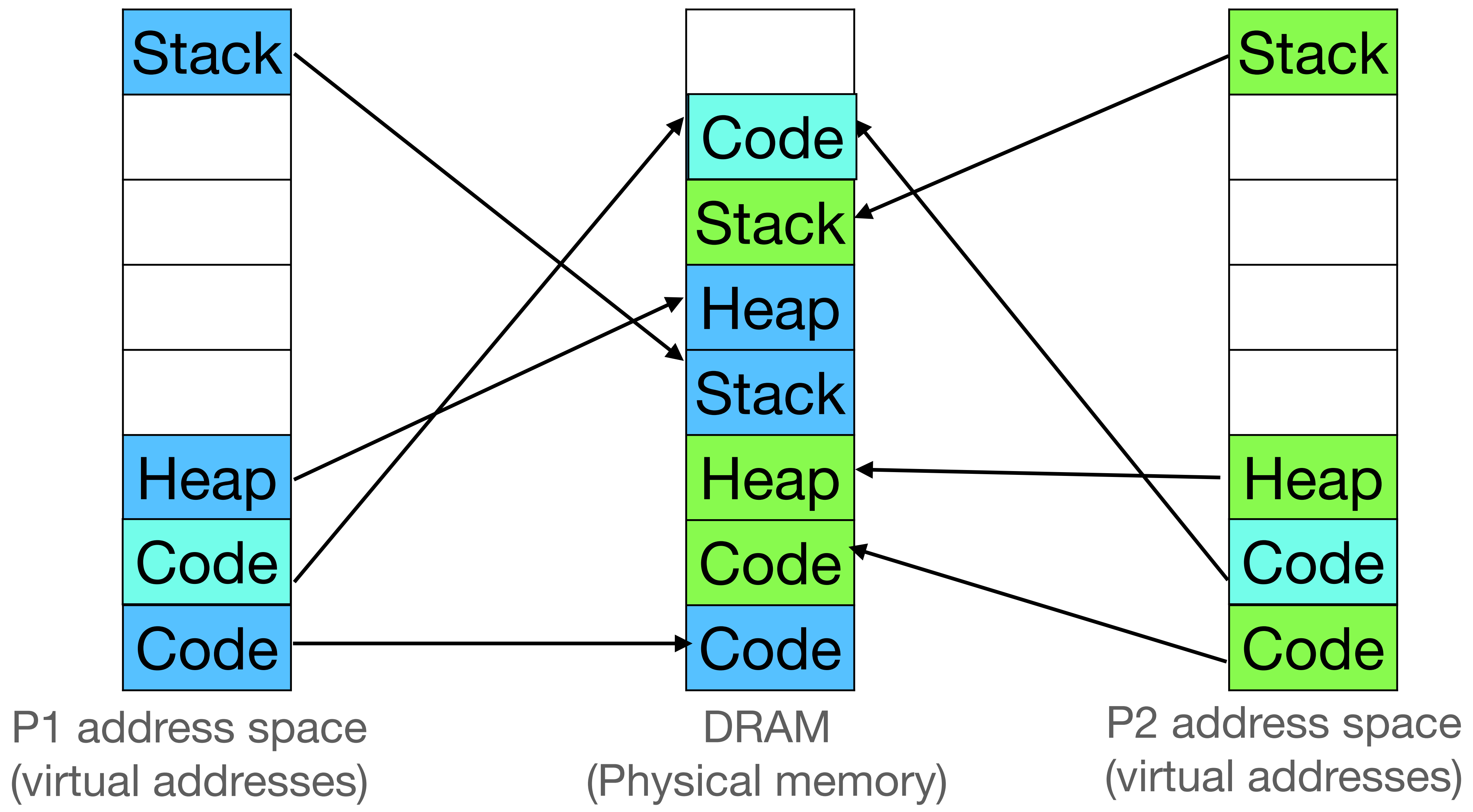
- OS periodically scans a few pages and computes their hash



Reducing memory pressure

Kernel same page merging (KSM)

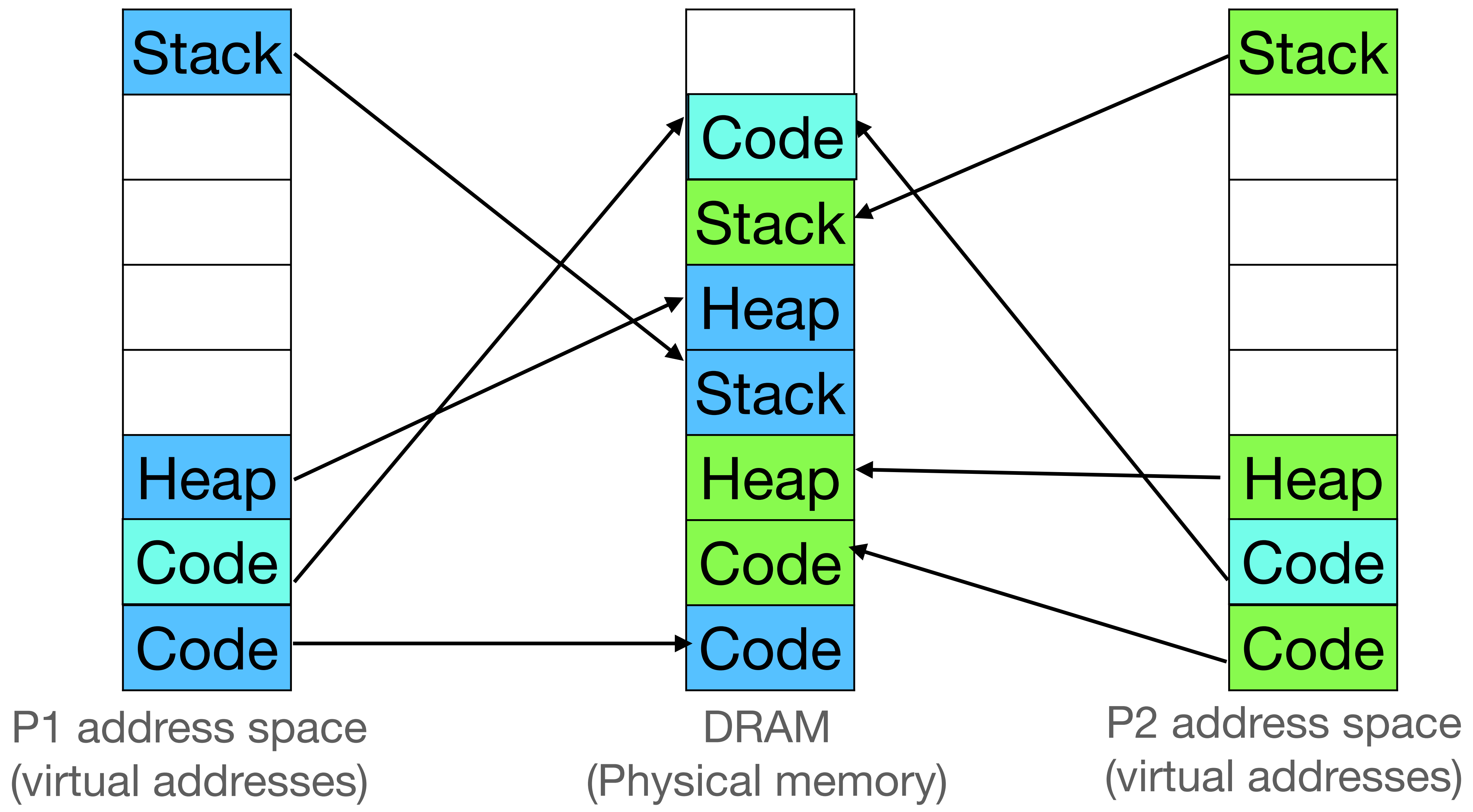
- OS periodically scans a few pages and computes their hash



Reducing memory pressure

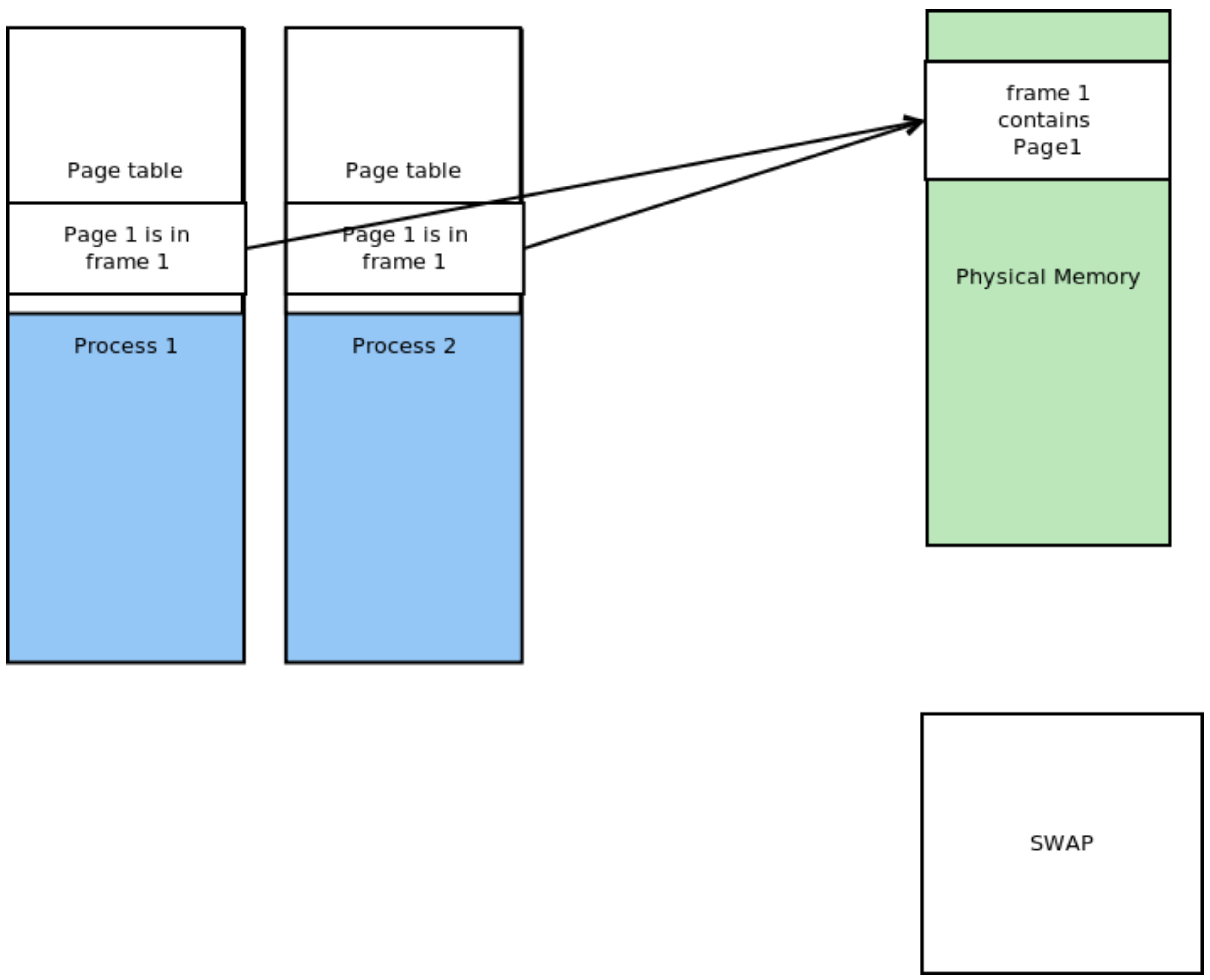
Kernel same page merging (KSM)

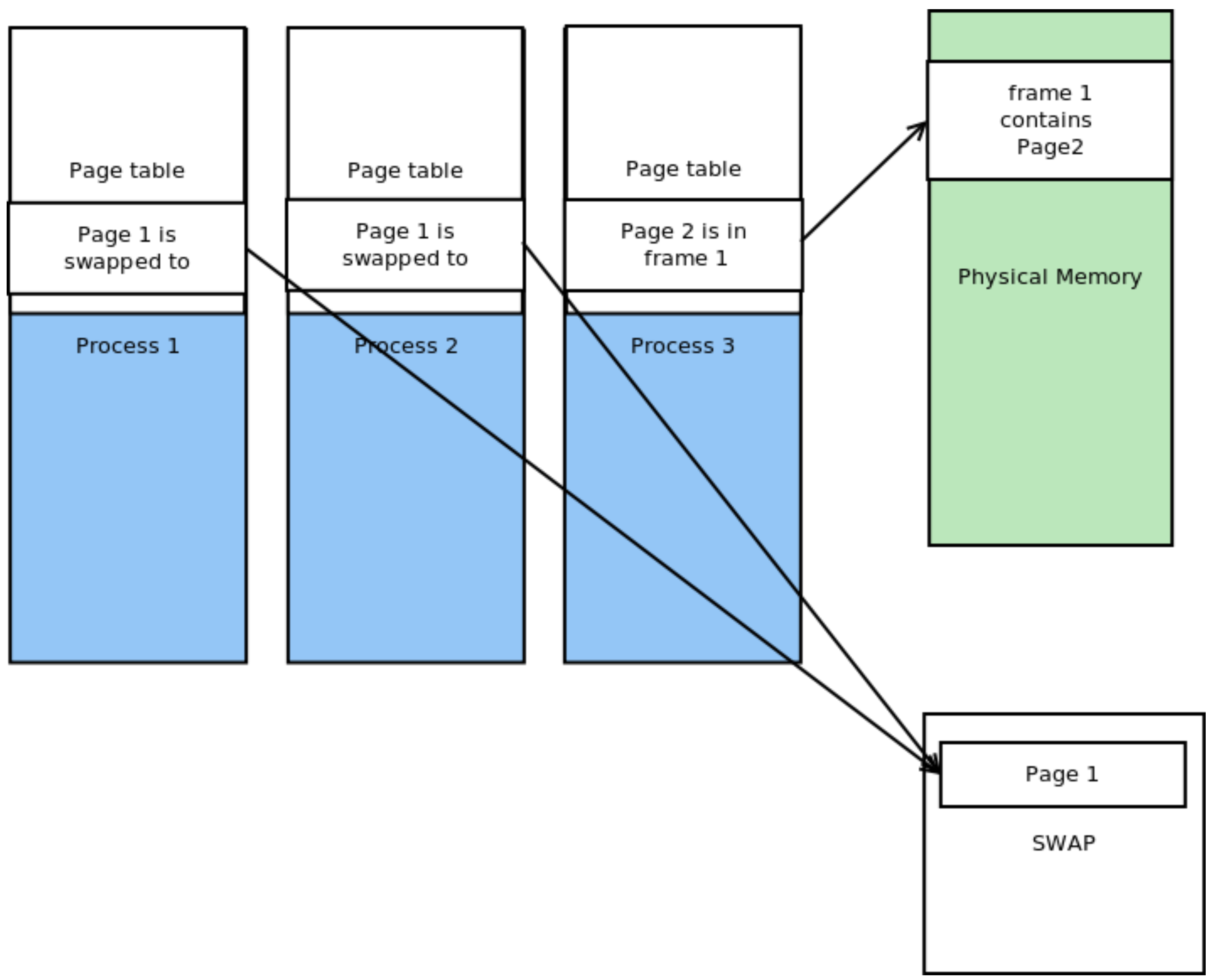
- OS periodically scans a few pages and computes their hash
- If two page hashes are same, deduplicate
  - Change PTE of one process to point to the common page
  - Add duplicate page to free list



Shared pages and demand paging

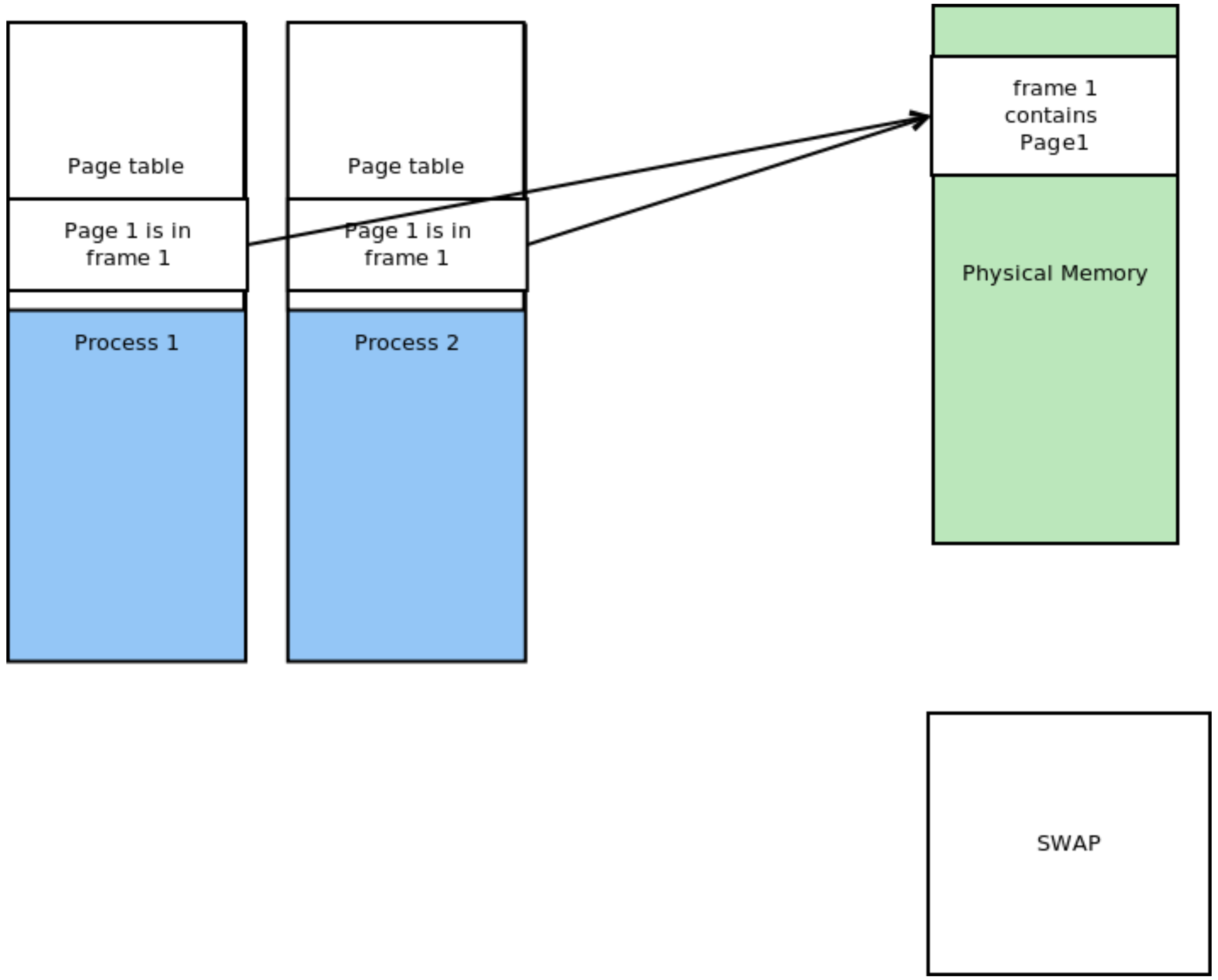
- Need to update multiple PTEs when pages are swapped out/in

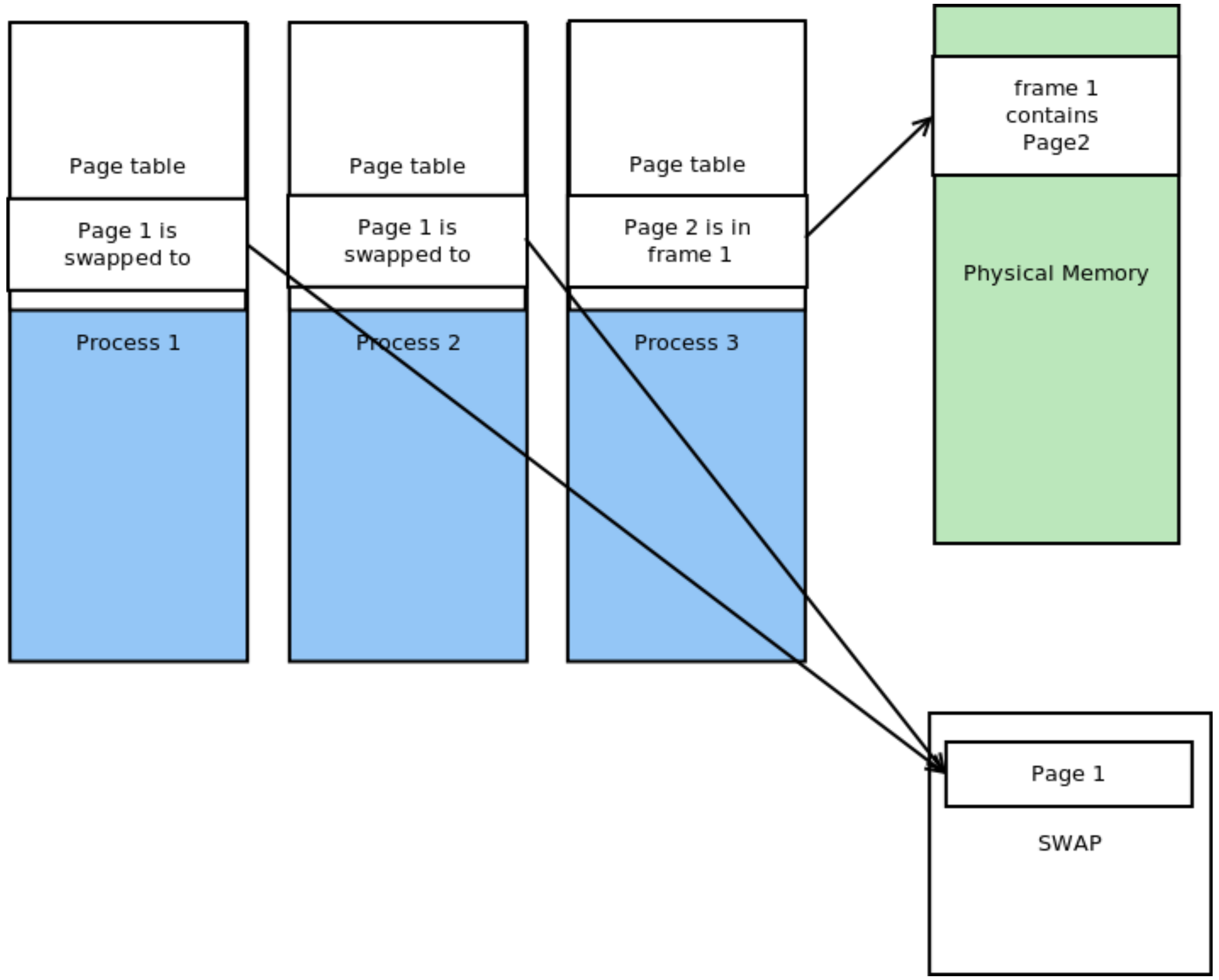




Reverse maps

- When swapping in/out a shared page, all PTEs must be updated.
- rmap: PPN -> list[*PTE]





Paging in action in xv6

xv6 book Ch 2

Process address space in xv6

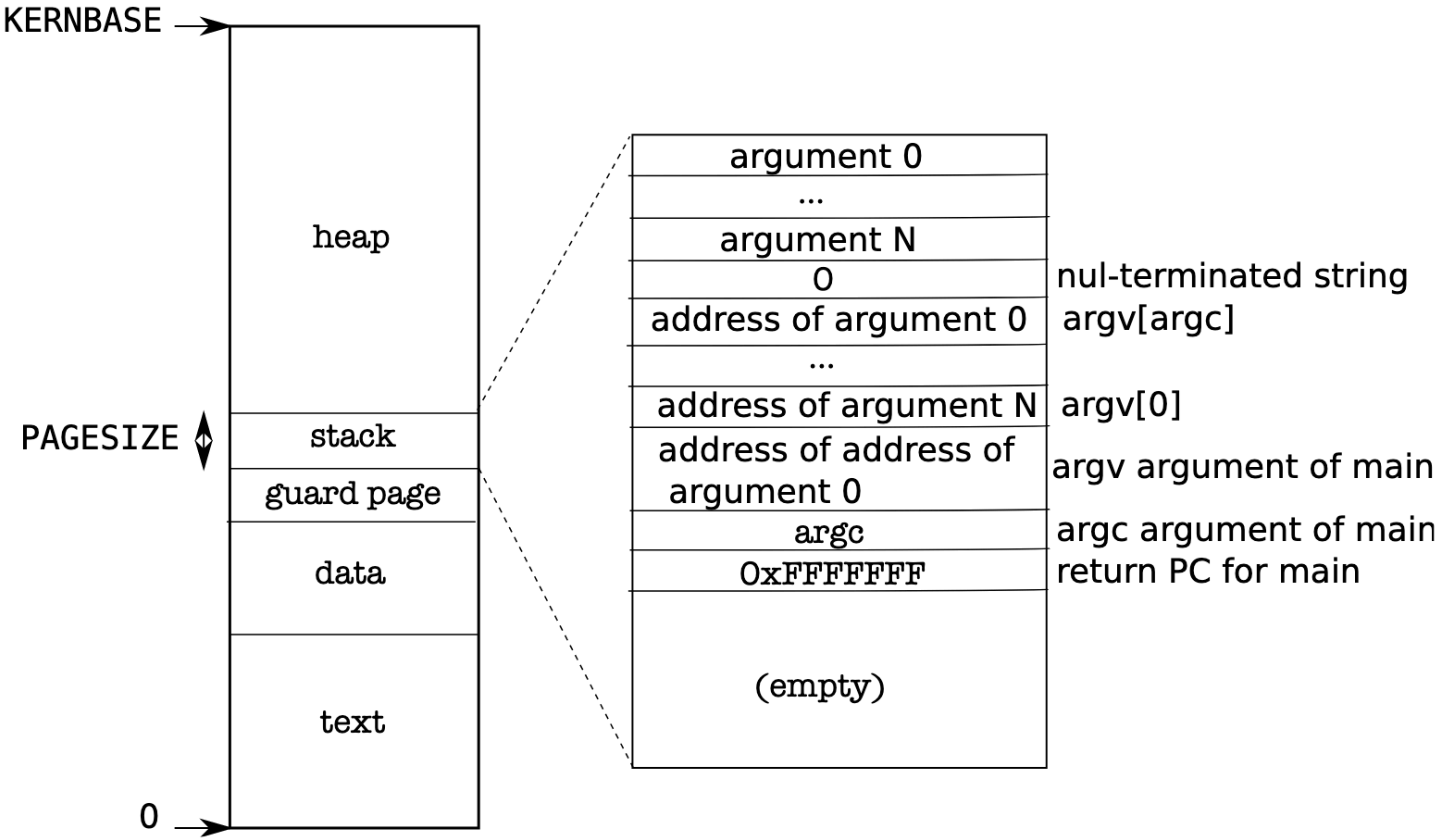


Figure 2-3. Memory layout of a user process with its initial stack.

Process address space in xv6

- Simplifies maintaining p->sz

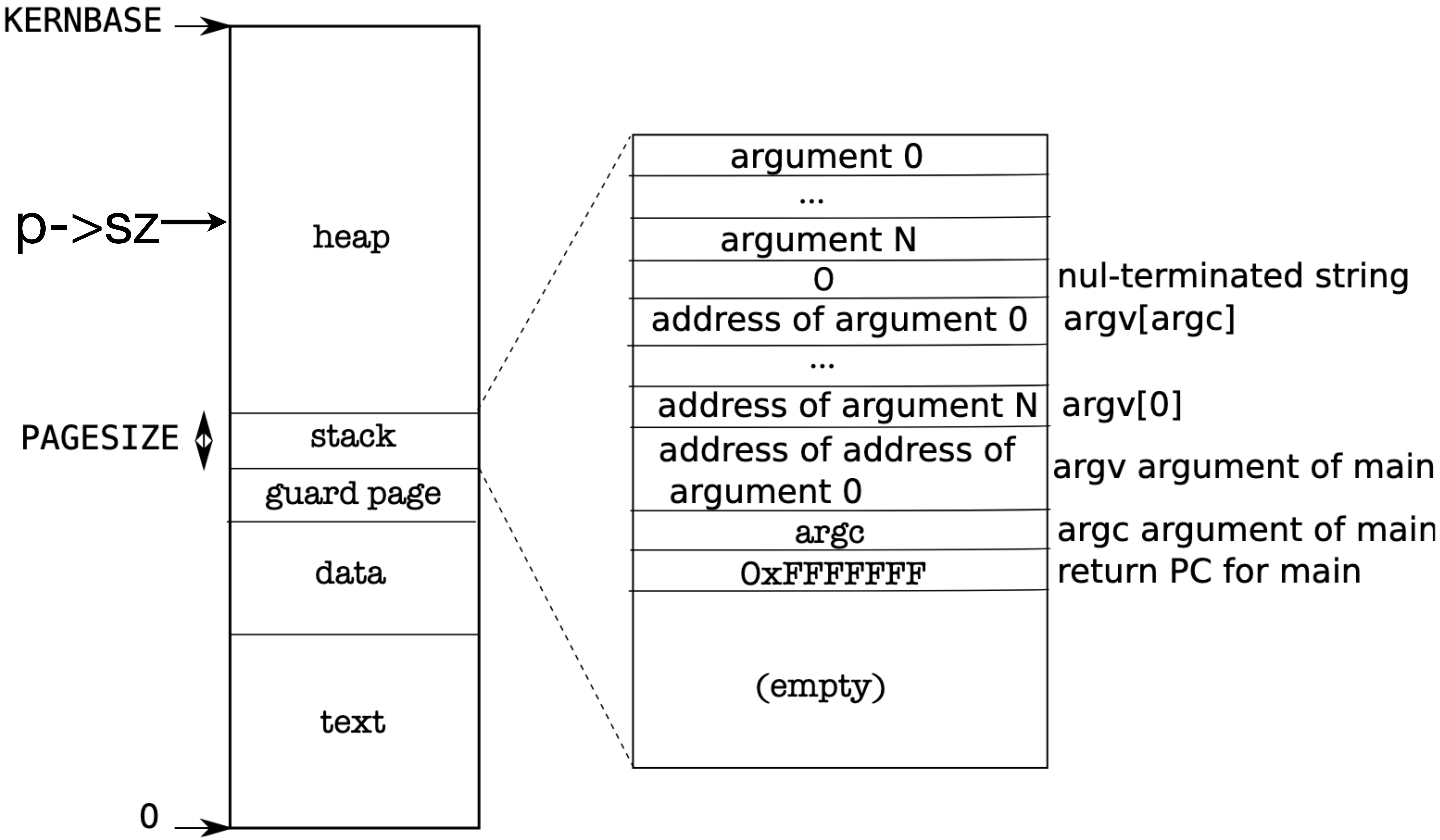


Figure 2-3. Memory layout of a user process with its initial stack.

Process address space in xv6

- Simplifies maintaining p->sz
- Heap grows upwards. malloc can ask for more memory via sbrk

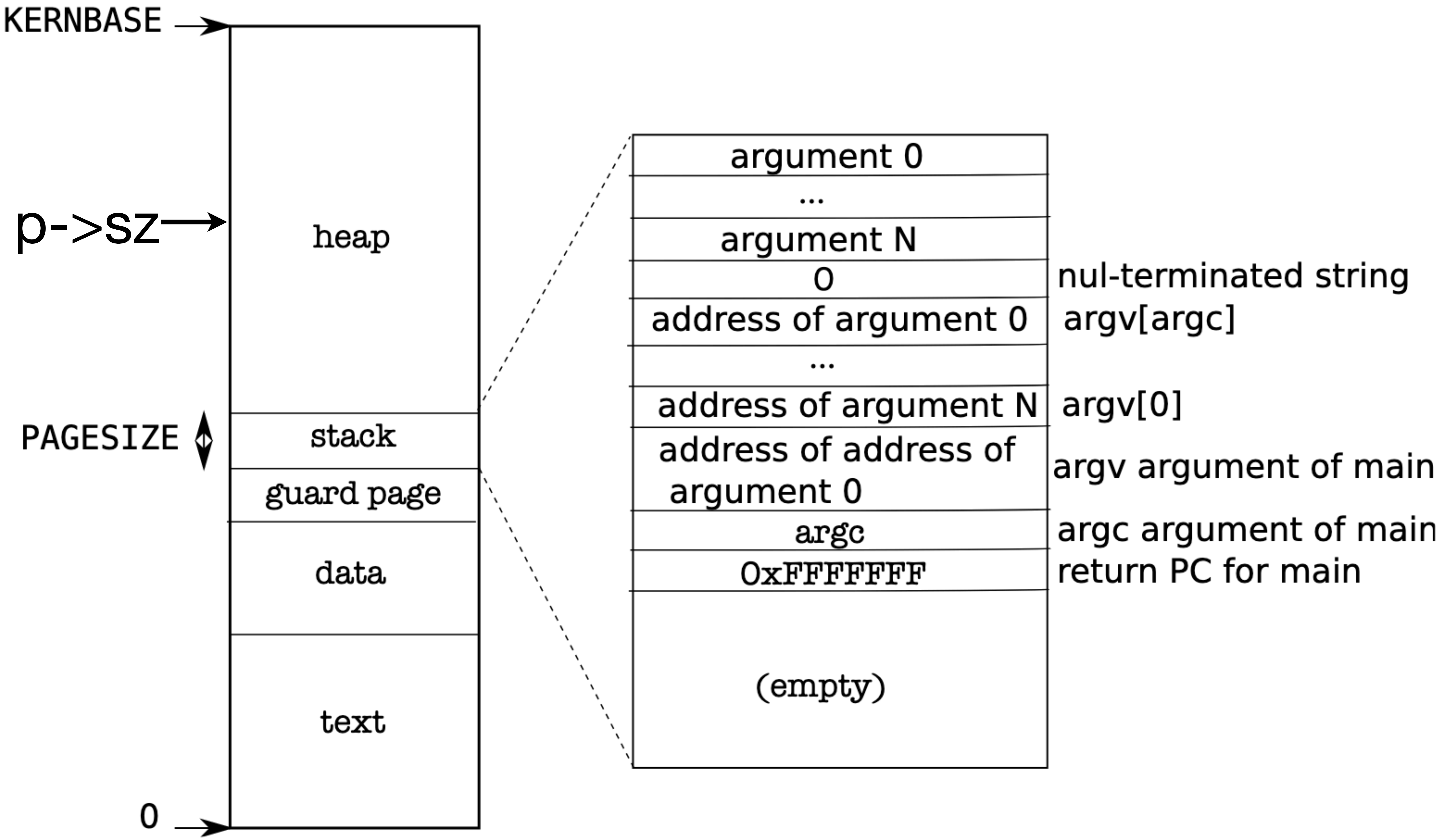


Figure 2-3. Memory layout of a user process with its initial stack.

Process address space in xv6

- Simplifies maintaining p->sz
- Heap grows upwards. malloc can ask for more memory via sbrk
- Stack is bounded to 4KB

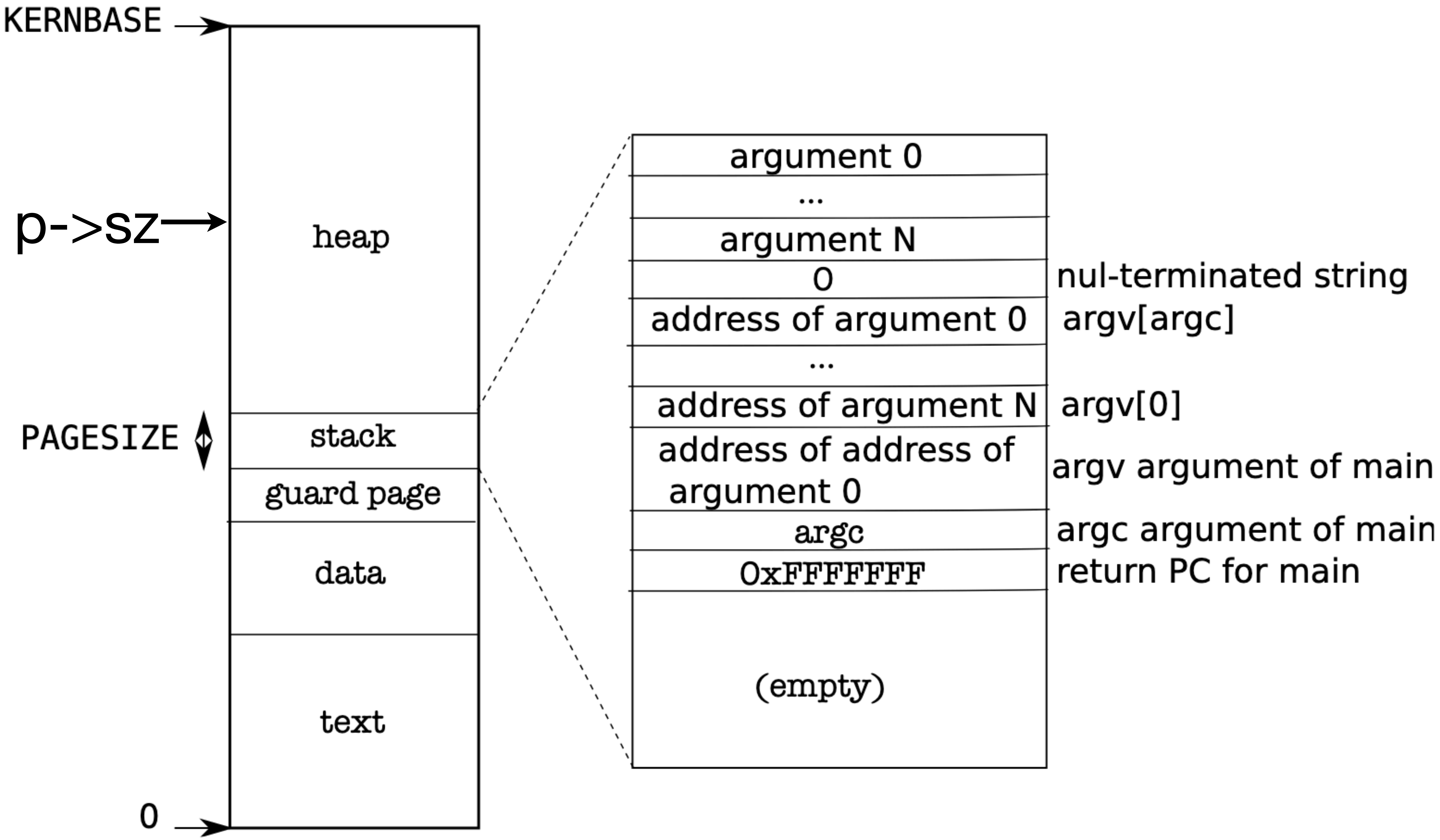
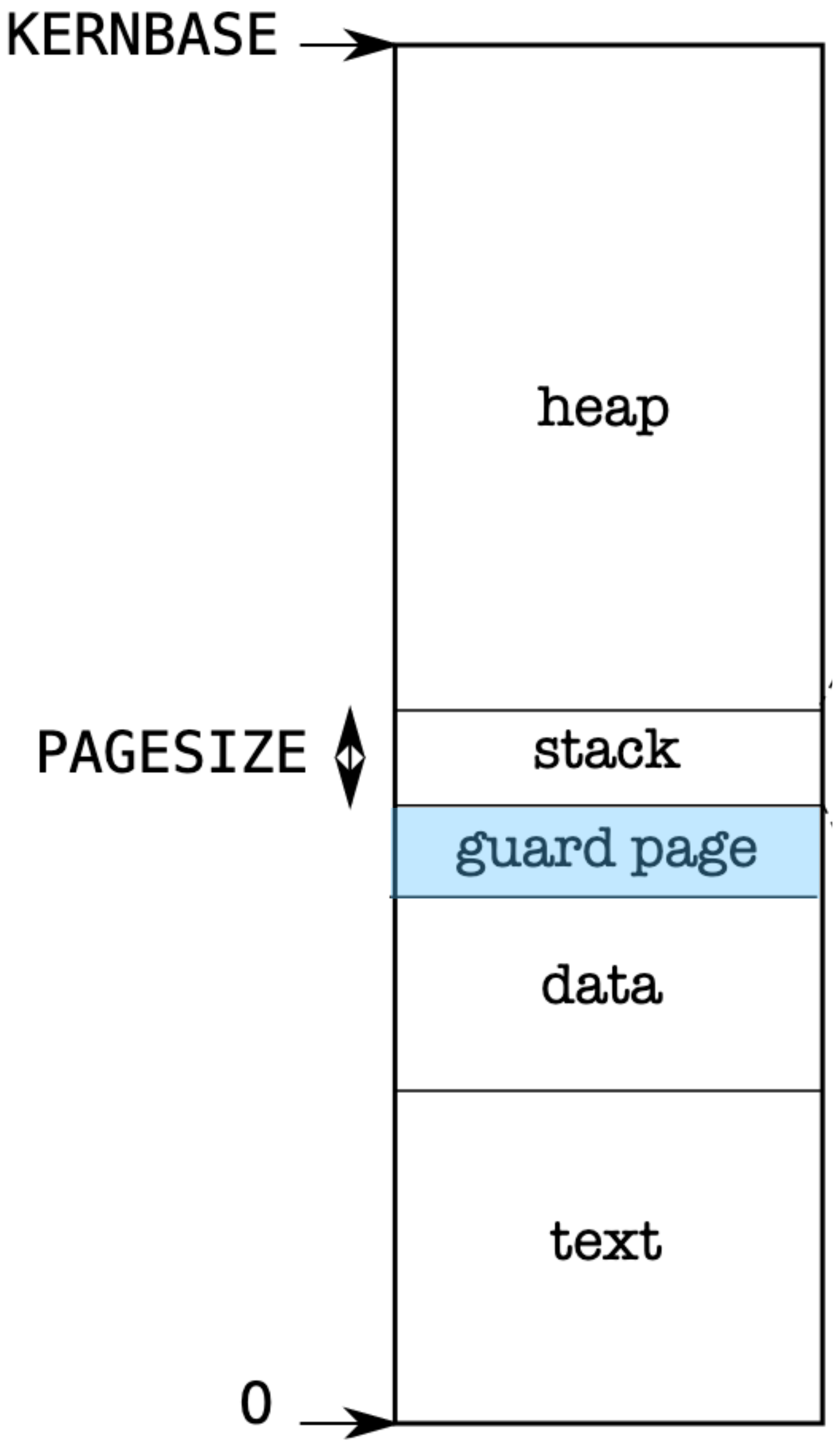


Figure 2-3. Memory layout of a user process with its initial stack.

Guard page

- Present bit is not set for the guard page
- Stack overflow traps into the OS
- OS will kill the process



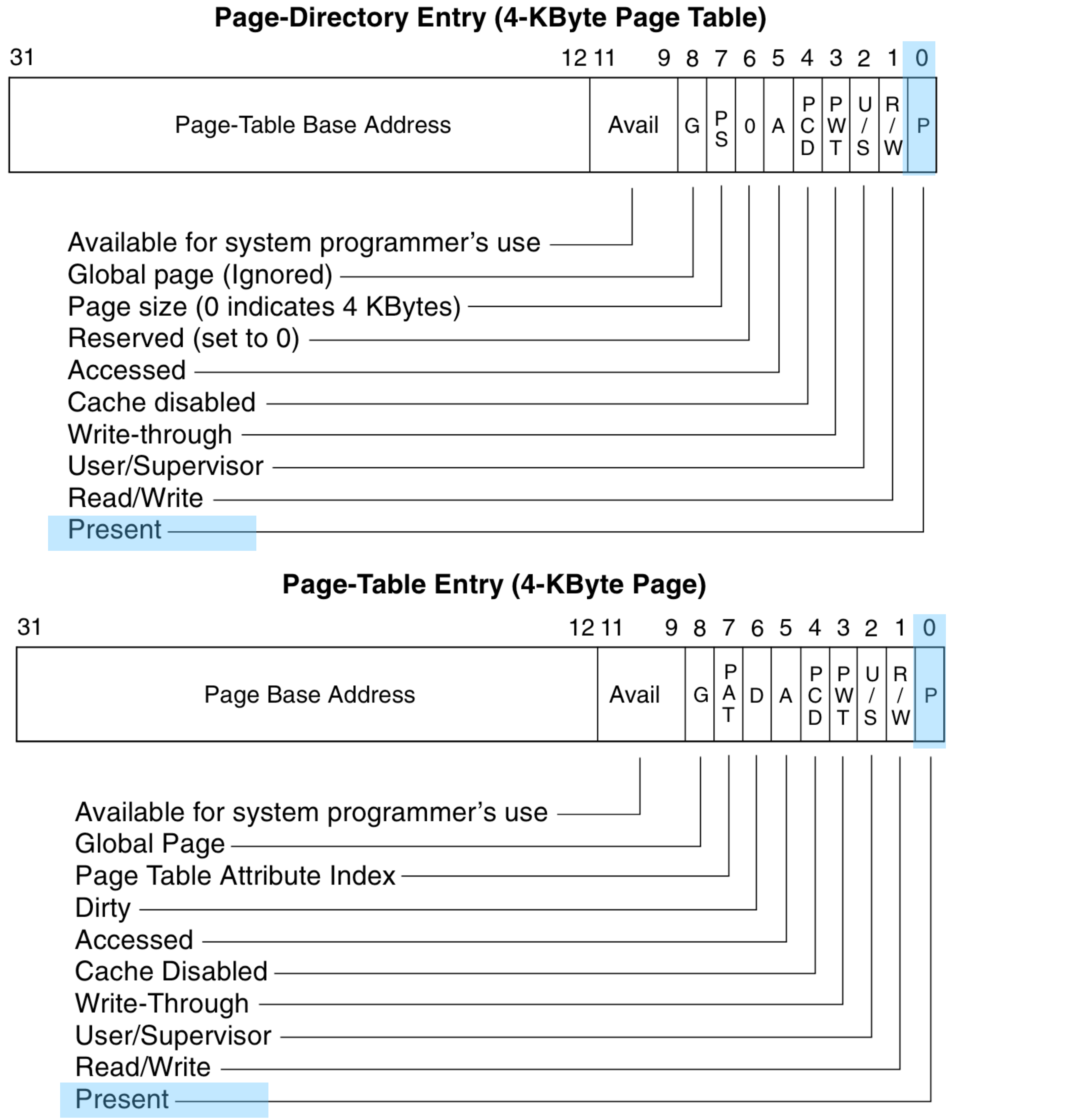
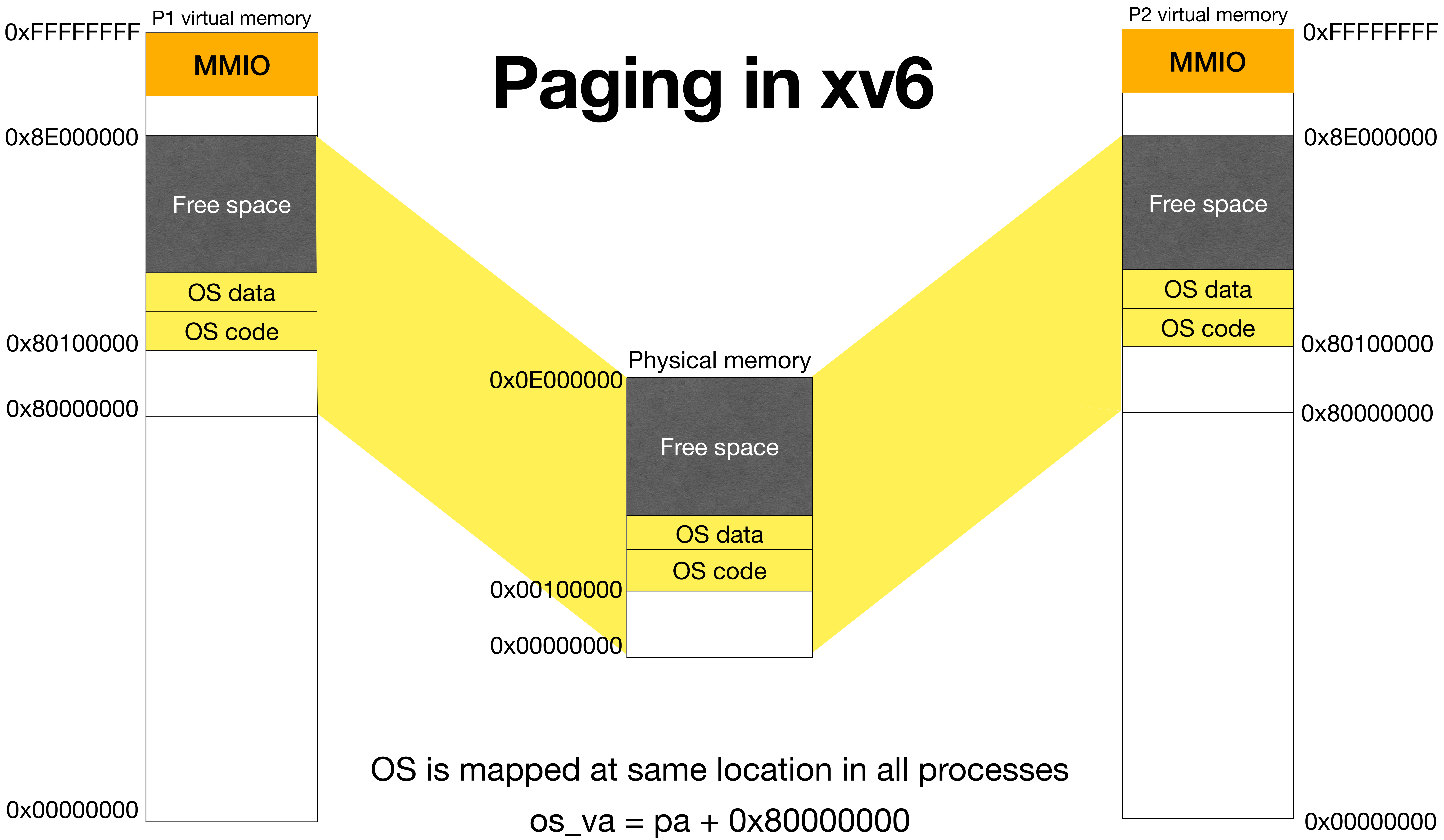


Figure 3-14. Format of Page-Directory and Page-Table Entries for 4-KByte Pages and 32-Bit Physical Addresses

Paging in xv6



OS is mapped at same location in all processes

os_va = pa + 0x80000000

Mapping OS into process address space

- User/supervisor bit is not set for OS pages

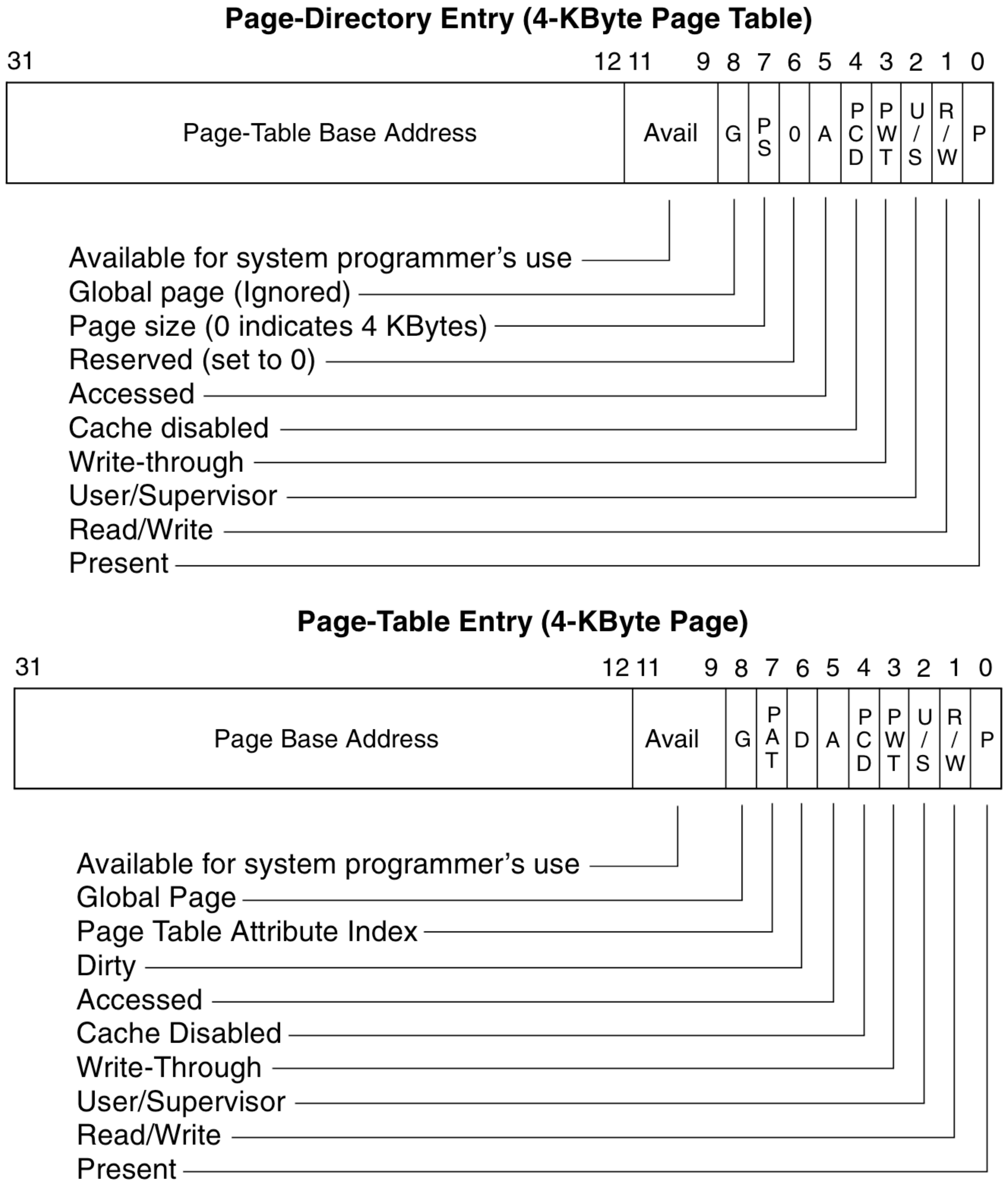


Figure 3-14. Format of Page-Directory and Page-Table Entries for 4-KByte Pages and 32-Bit Physical Addresses

Mapping OS into process address space

- User/supervisor bit is not set for OS pages

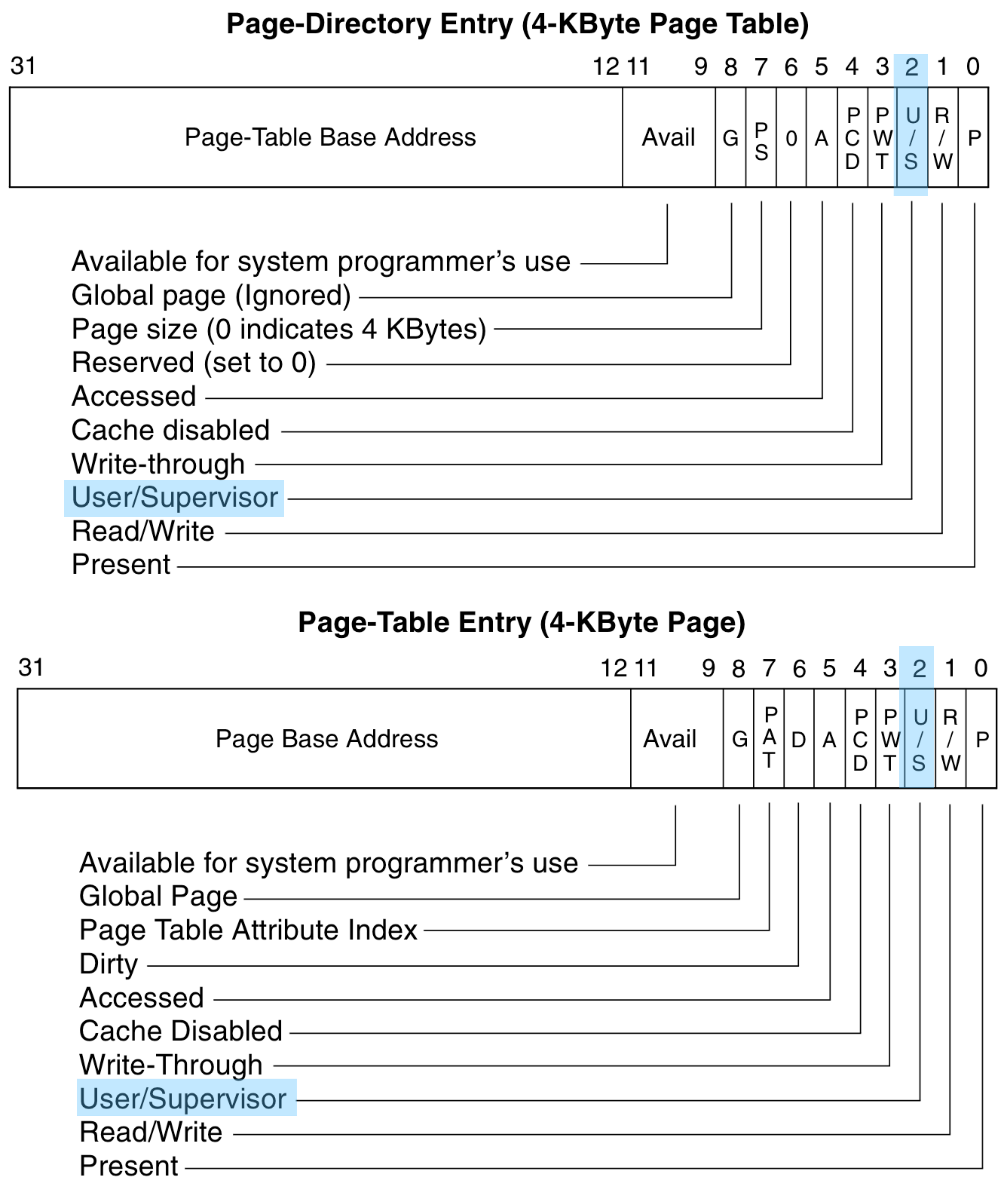
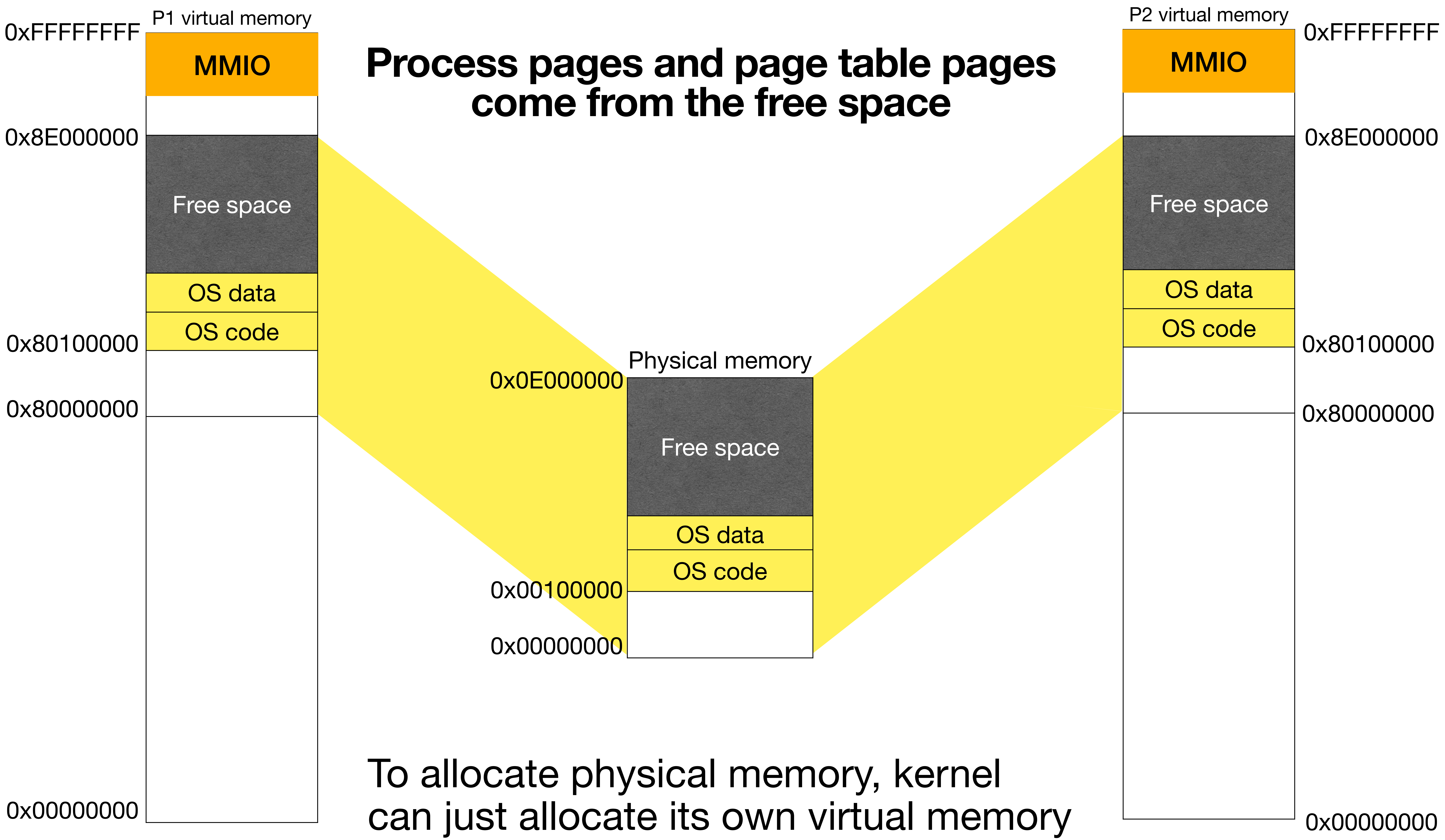


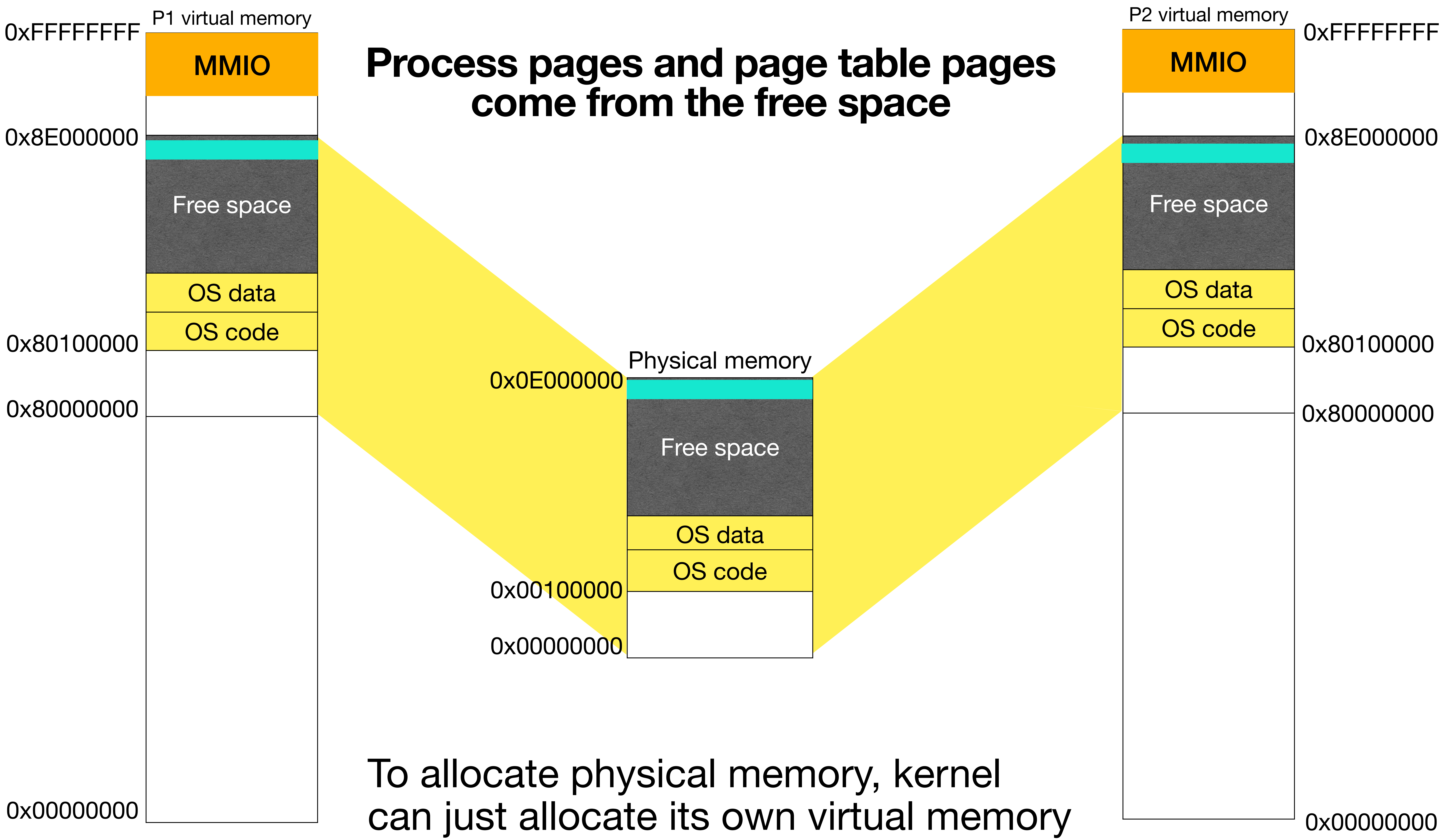
Figure 3-14. Format of Page-Directory and Page-Table Entries for 4-KByte Pages and 32-Bit Physical Addresses

Process pages and page table pages come from the free space



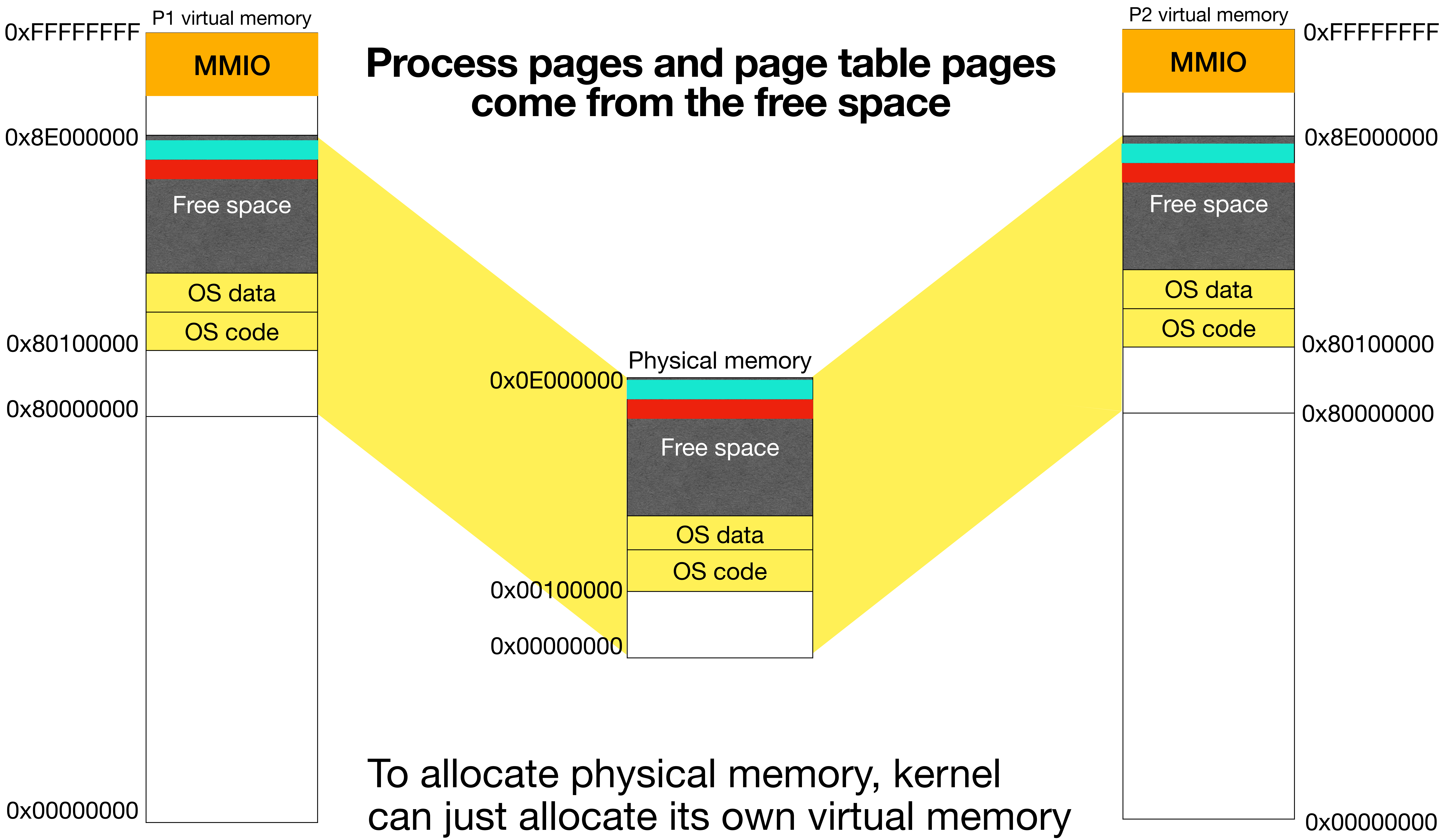
To allocate physical memory, kernel can just allocate its own virtual memory

Process pages and page table pages come from the free space



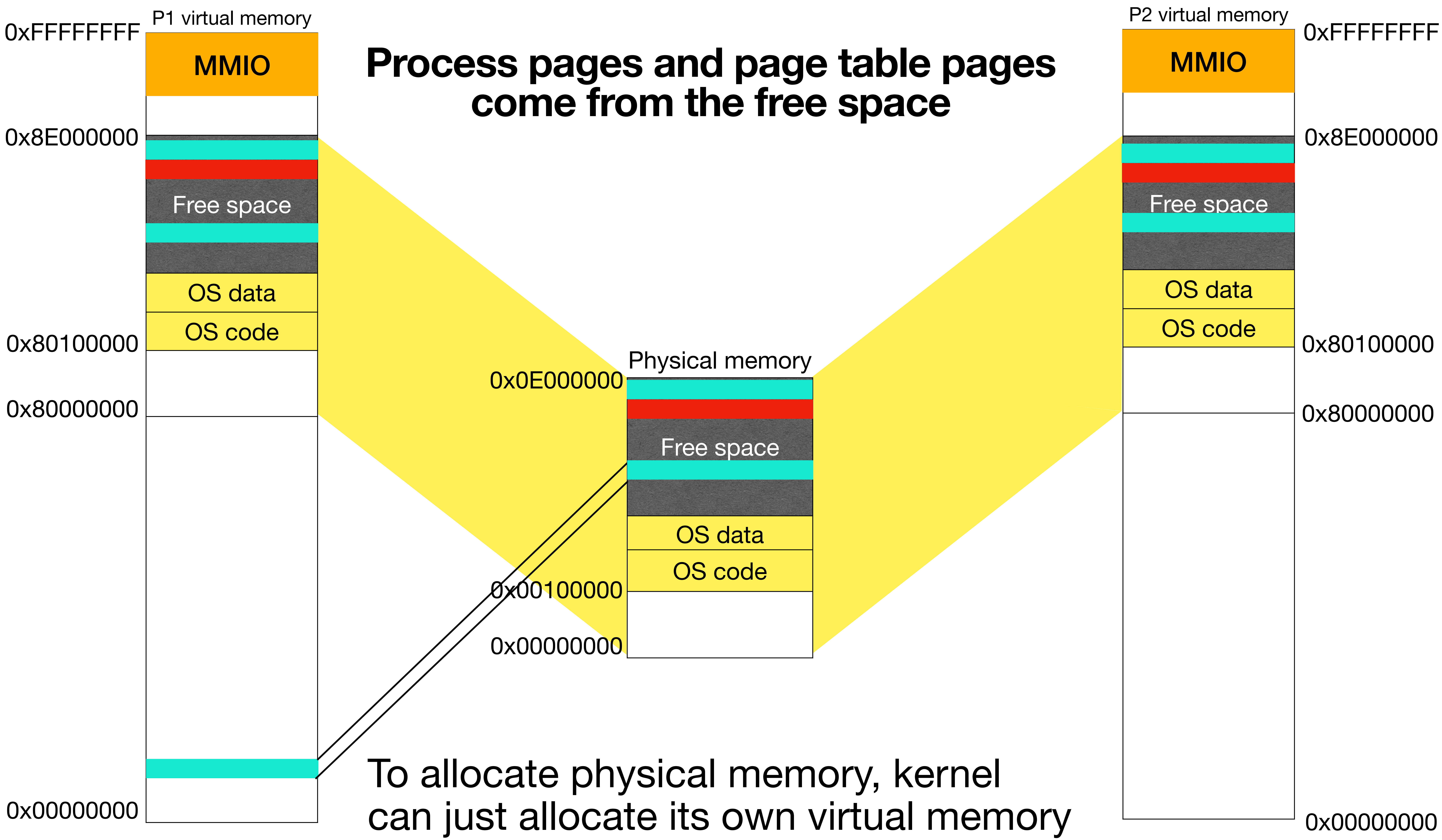
To allocate physical memory, kernel can just allocate its own virtual memory

Process pages and page table pages come from the free space



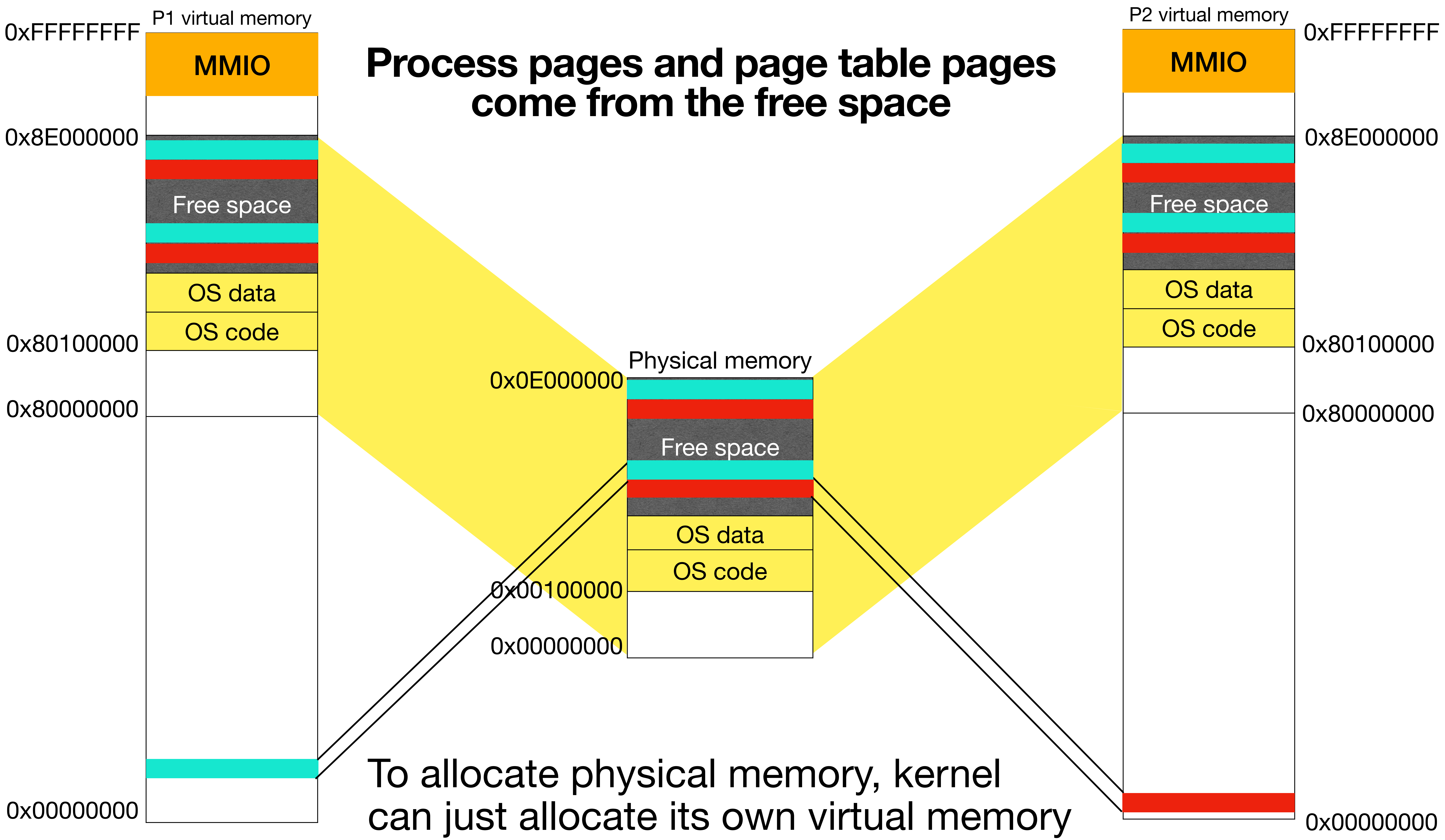
To allocate physical memory, kernel can just allocate its own virtual memory

Process pages and page table pages come from the free space



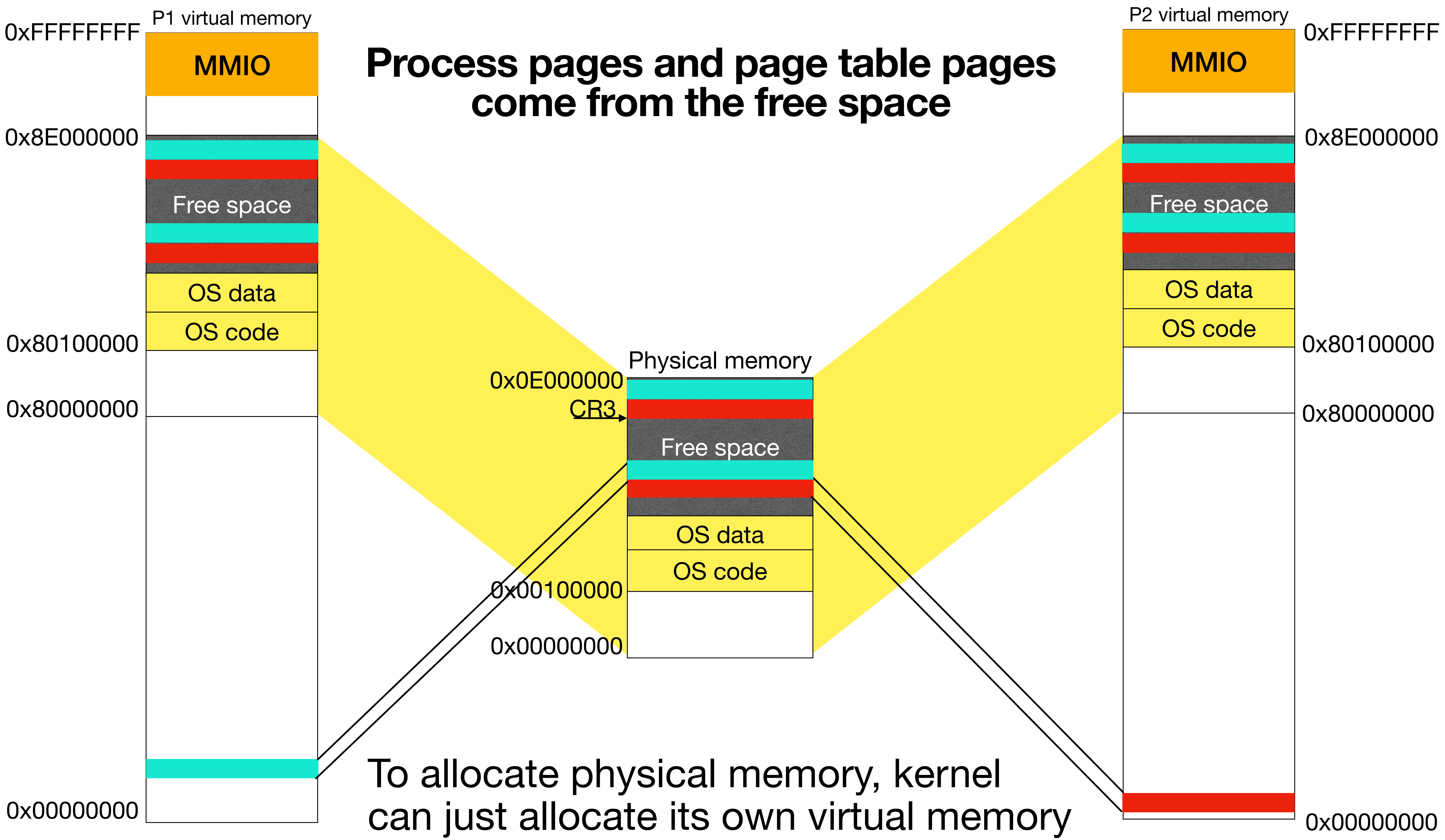
To allocate physical memory, kernel can just allocate its own virtual memory

Process pages and page table pages come from the free space



To allocate physical memory, kernel can just allocate its own virtual memory

Process pages and page table pages come from the free space

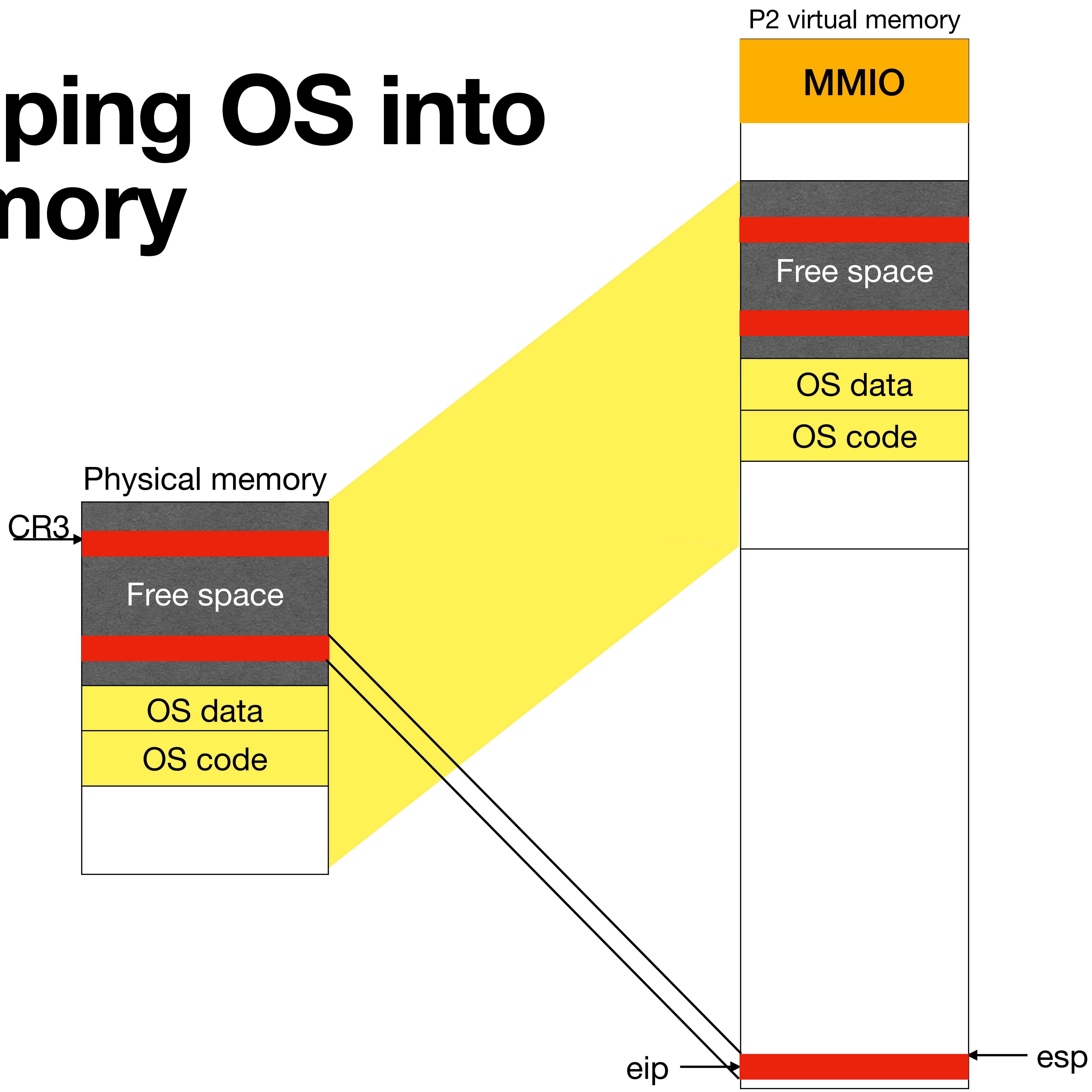


To allocate physical memory, kernel can just allocate its own virtual memory

Advantages of mapping OS into process virtual memory

Trap handling

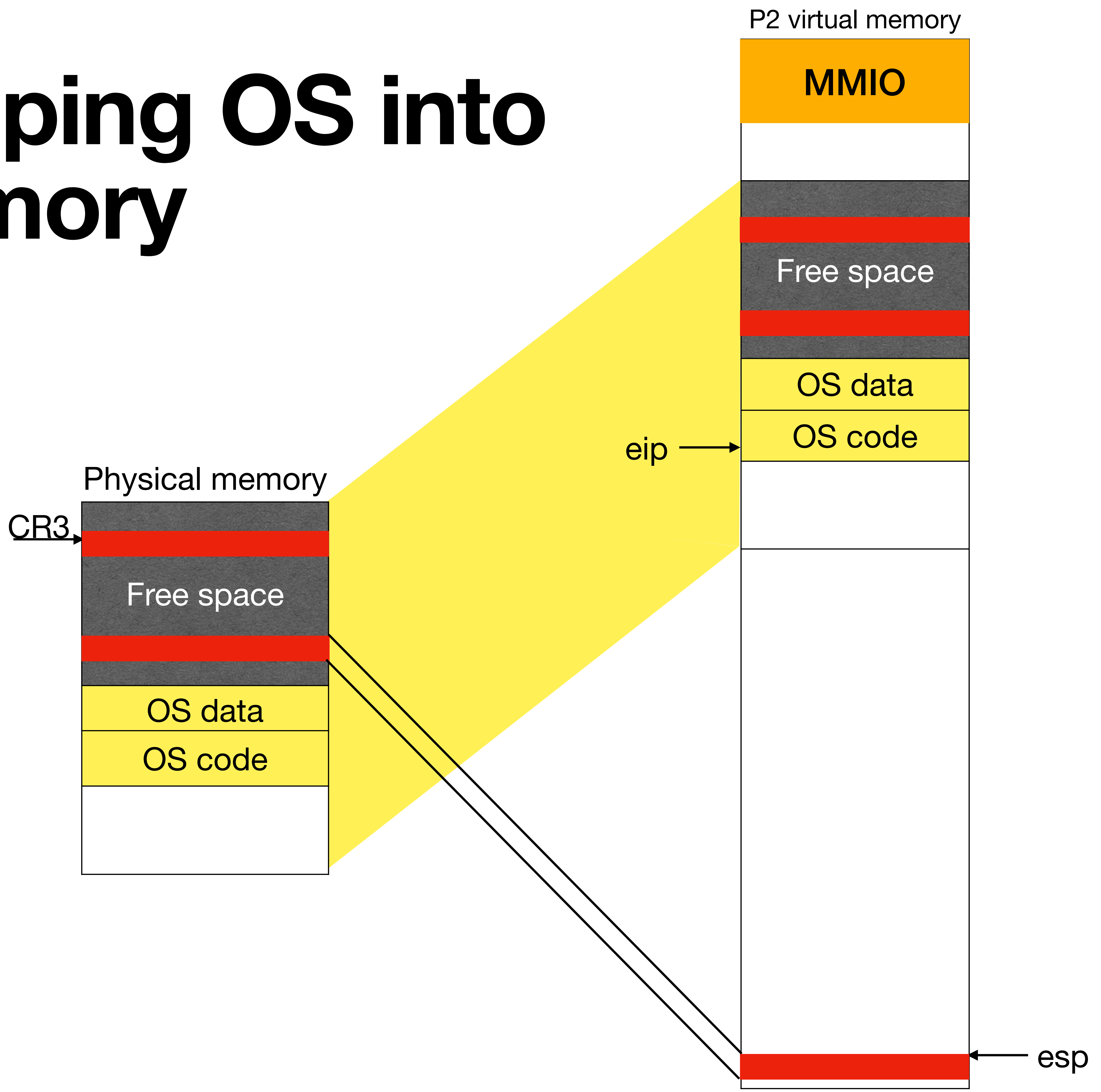
- Kernel stack and IDT are in the address space of the process. => Hardware need not switch page tables for handling interrupts



Advantages of mapping OS into process virtual memory

Trap handling

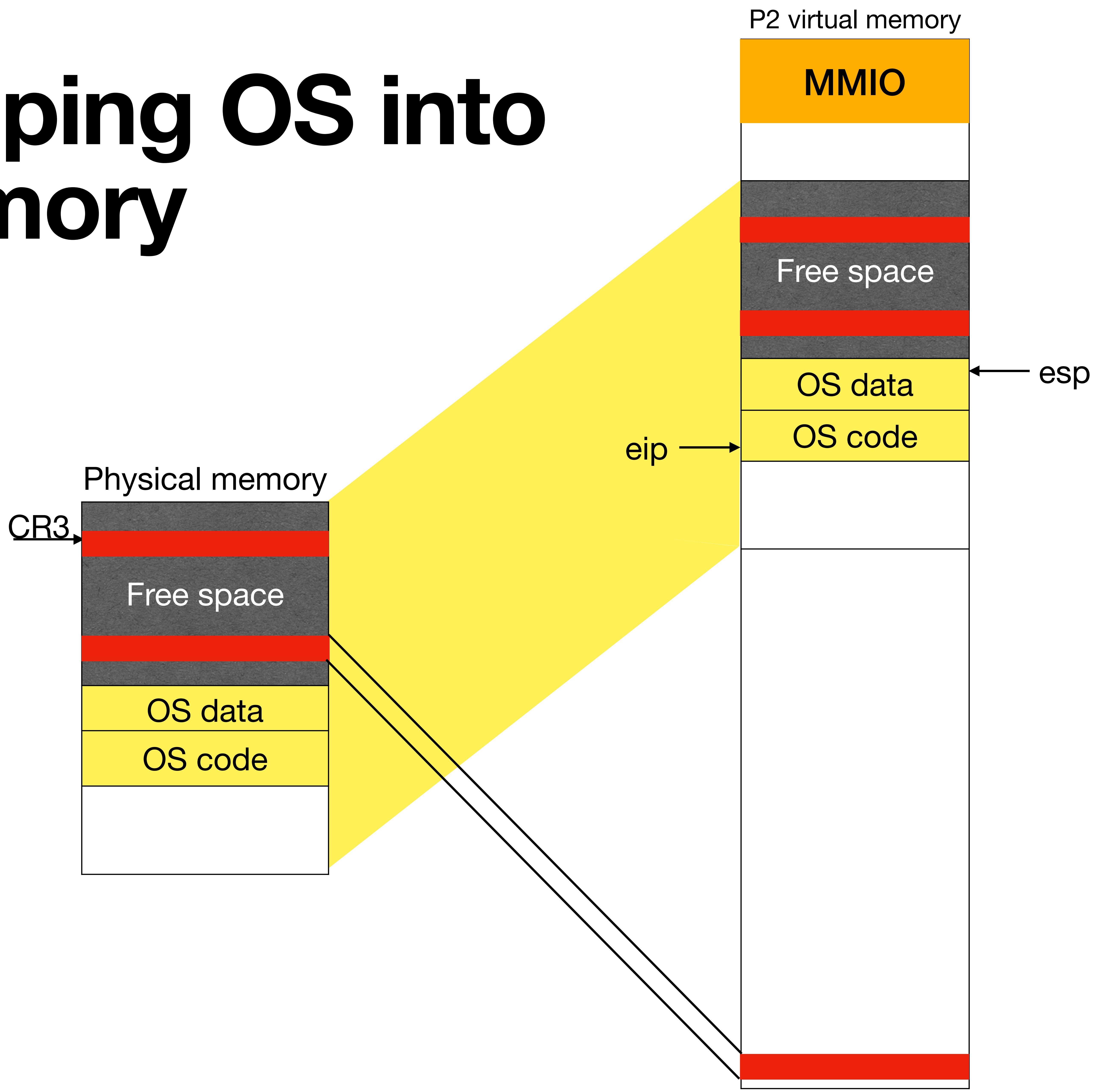
- Kernel stack and IDT are in the address space of the process. => Hardware need not switch page tables for handling interrupts



Advantages of mapping OS into process virtual memory

Trap handling

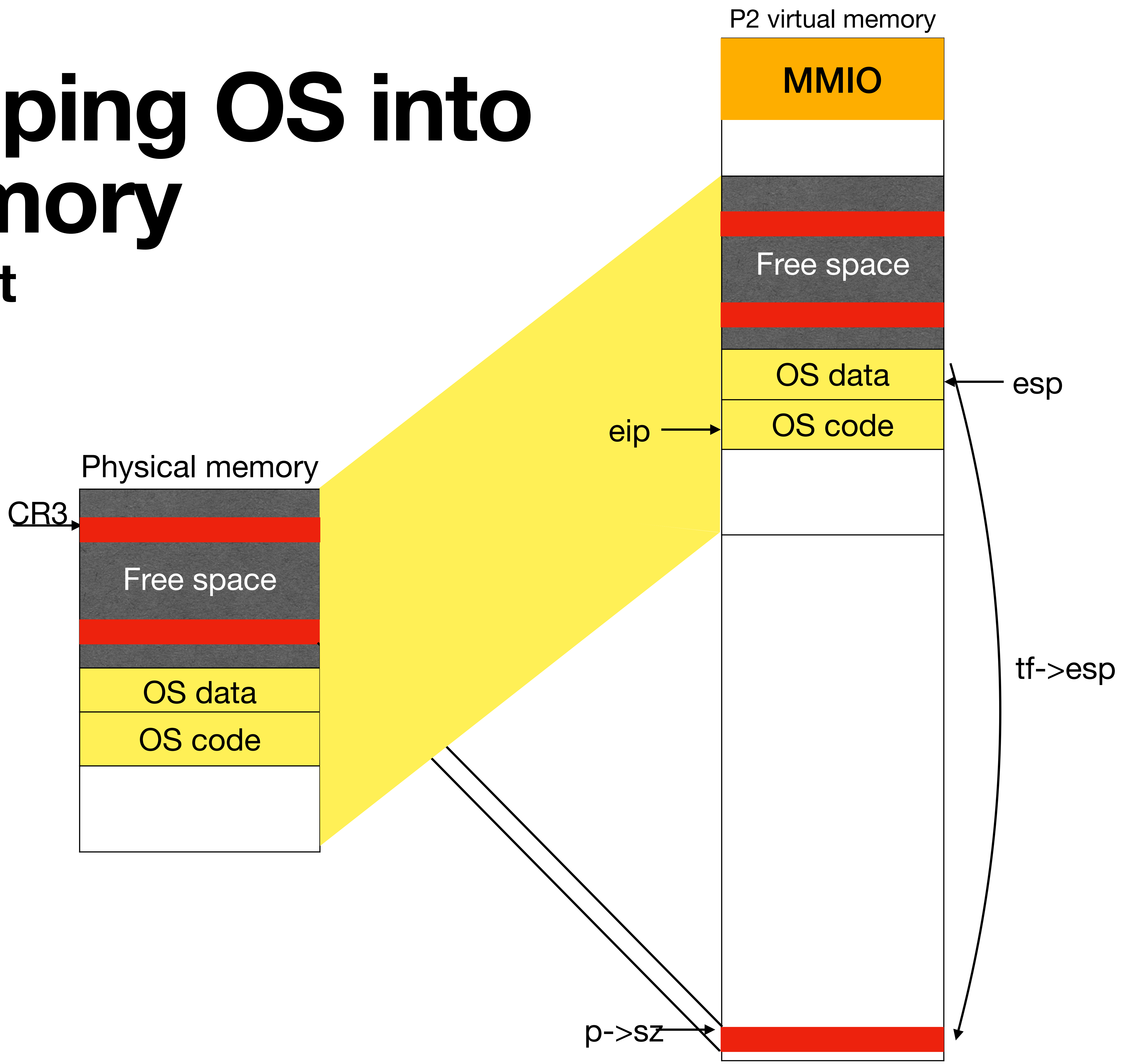
- Kernel stack and IDT are in the address space of the process. => Hardware need not switch page tables for handling interrupts



Advantages of mapping OS into process virtual memory

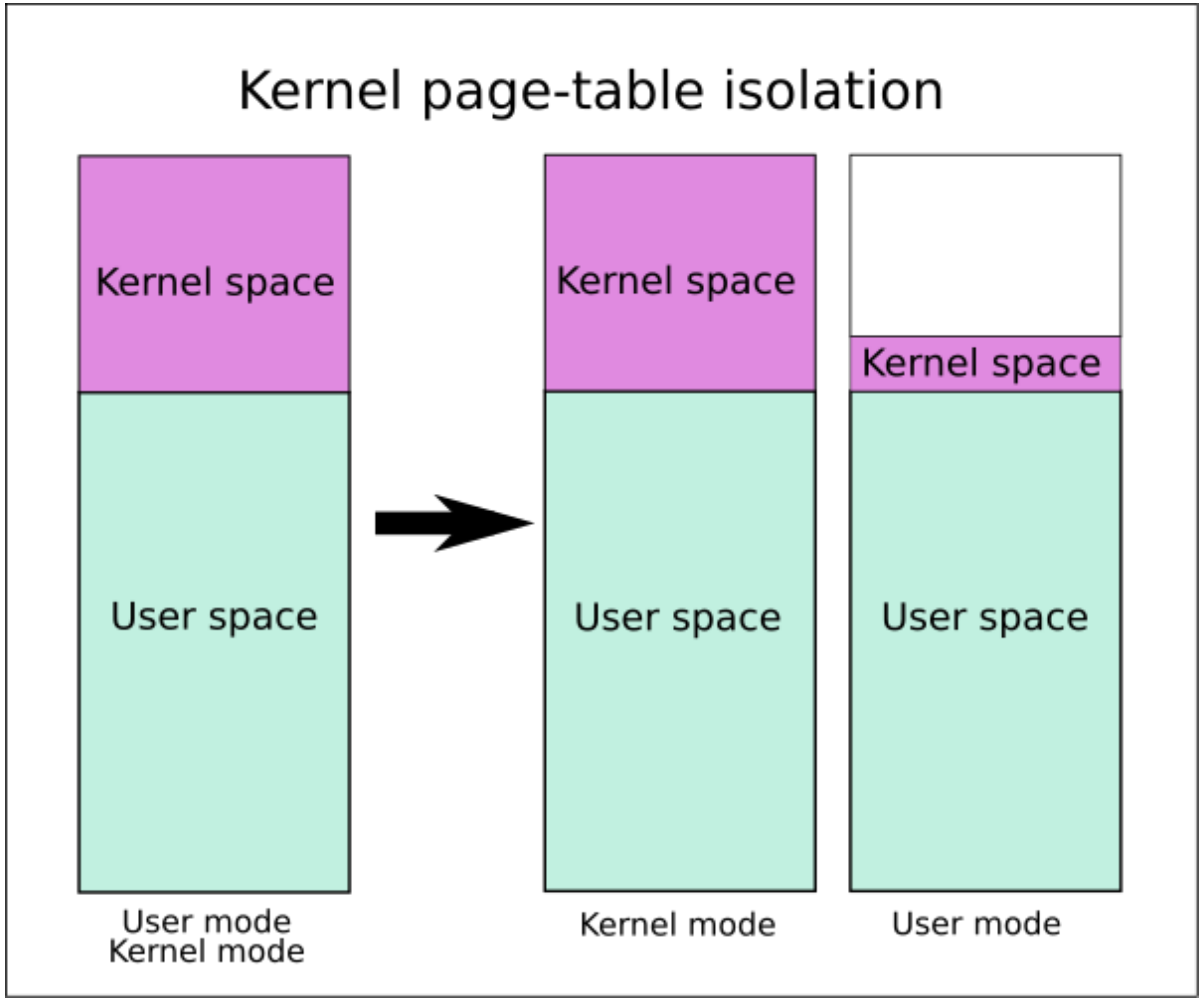
Read sys call parameters without switching page table

```
int fetchint(uint addr, int *ip) {
  if(addr >= p->sz || addr+4 > p->sz)
    return -1;
  *ip = *(int*)(addr + p->offset);
}

int argint(int n, int *ip) {
  return fetchint((myproc()->tf->esp)
                        + 4 + 4*n, ip);
}
```



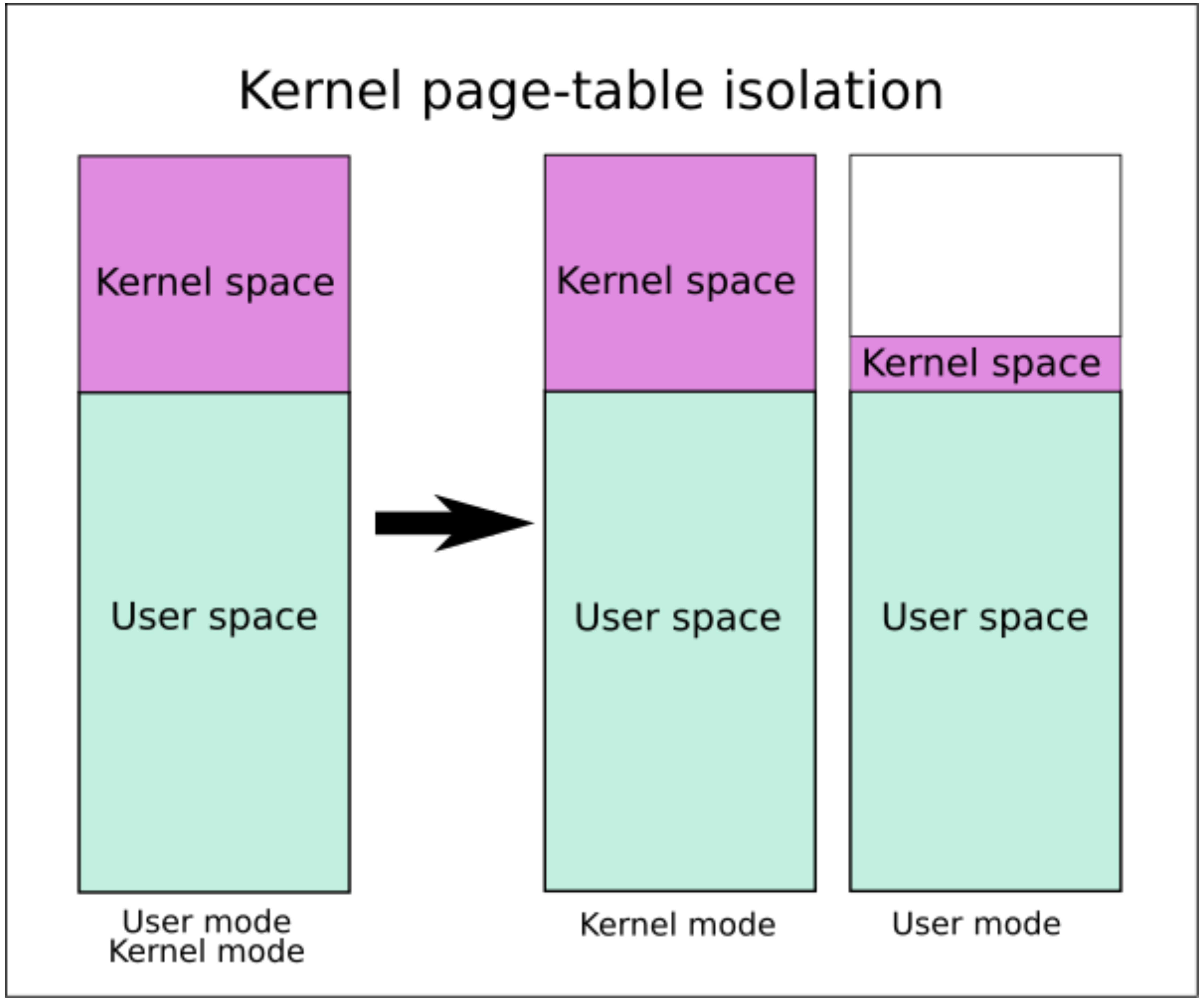
Kernel page table isolation (KPTI)

- Need to switch page tables to handle interrupts/syscalls



Kernel page table isolation (KPTI)

- Need to switch page tables to handle interrupts/syscalls
  - Syscall and interrupt heavy workloads like Postgres see 7-17% overheads (16-23% without tagged TLB)



Context switching

```
void seginit(void) {
  c->gdt[SEG_UCODE] = SEG(.., 0, 0xffffffff, DPL_USER);
  c->gdt[SEG_UDATA] = SEG(.., 0, 0xffffffff, DPL_USER);
}
```

- User segments map to entire memory since protection is done via paging

Context switching

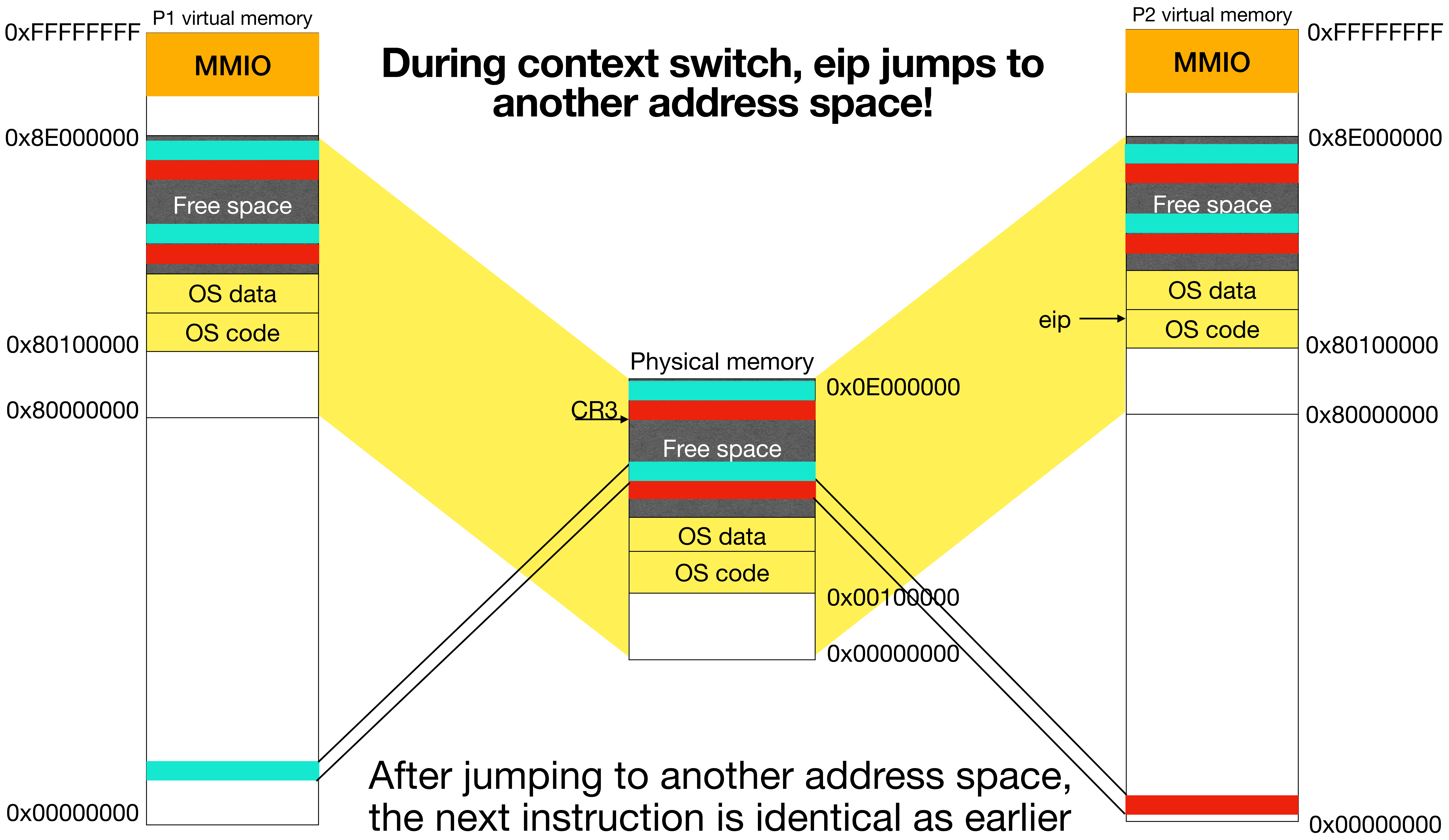
```
void seginit(void) {
  c->gdt[SEG_UCODE] = SEG(.., 0, 0xffffffff, DPL_USER);
  c->gdt[SEG_UDATA] = SEG(.., 0, 0xffffffff, DPL_USER);
}
```

- User segments map to entire memory since protection is done via paging
- switchuvm changes page tables instead of GDT entries

```
void switchuvm(struct proc *p) {
  mycpu()->gdt[SEG_TSS] = SEG16(STS_T32A, &mycpu()->ts,
                                sizeof(mycpu()->ts)-1, 0);
  mycpu()->ts.ss0 = SEG_KDATA << 3;
  mycpu()->ts.esp0 = (uint)p->kstack + KSTACKSIZE;
  ltr(SEG_TSS << 3);
  lcr3(V2P(p->pgdir));  // switch to process address space
}
```

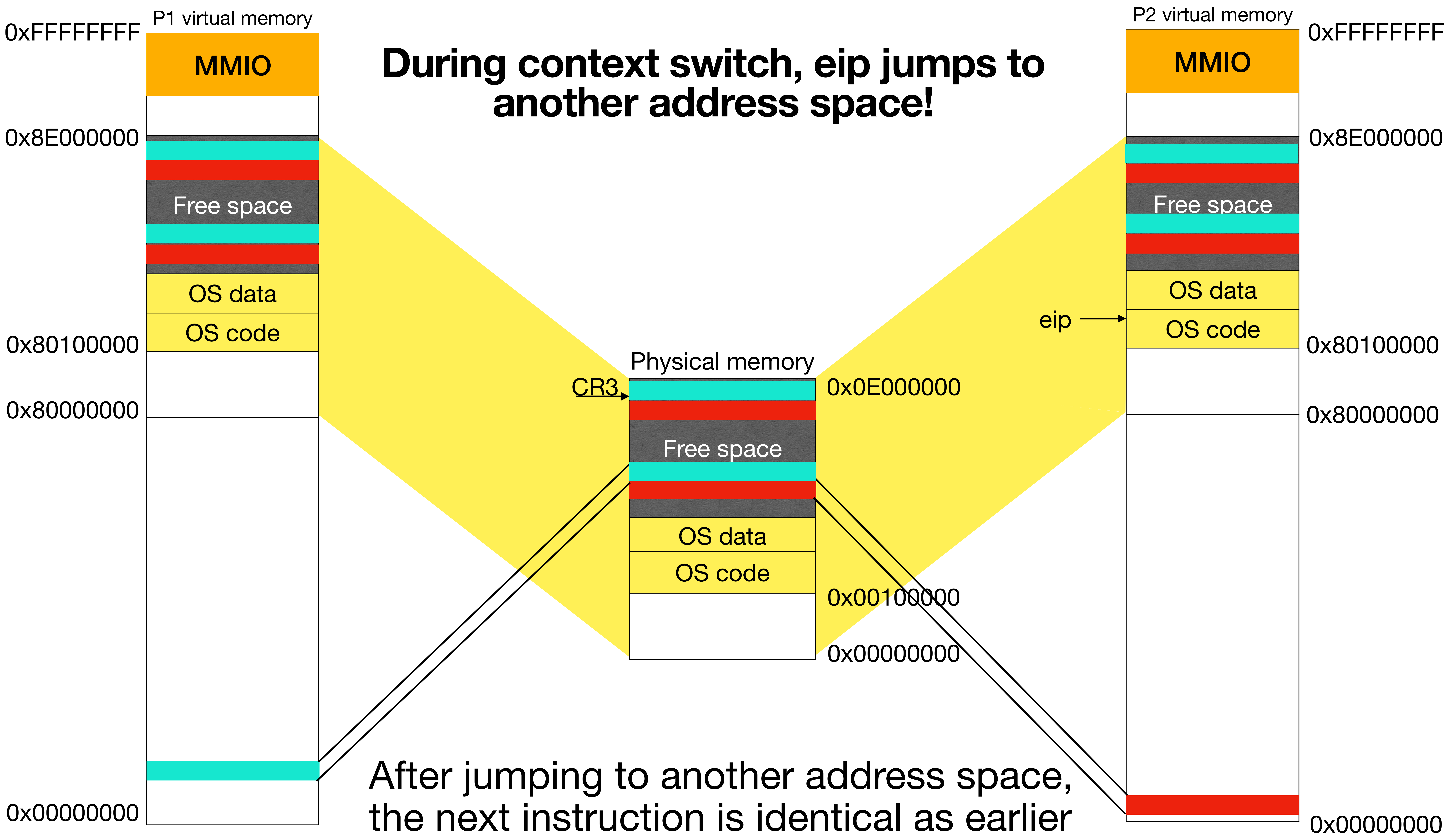
```
void scheduler(void) {
 // pick RUNNABLE process p
 switchuvm(p);
 swtch(p->context);
}
```

During context switch, eip jumps to another address space!



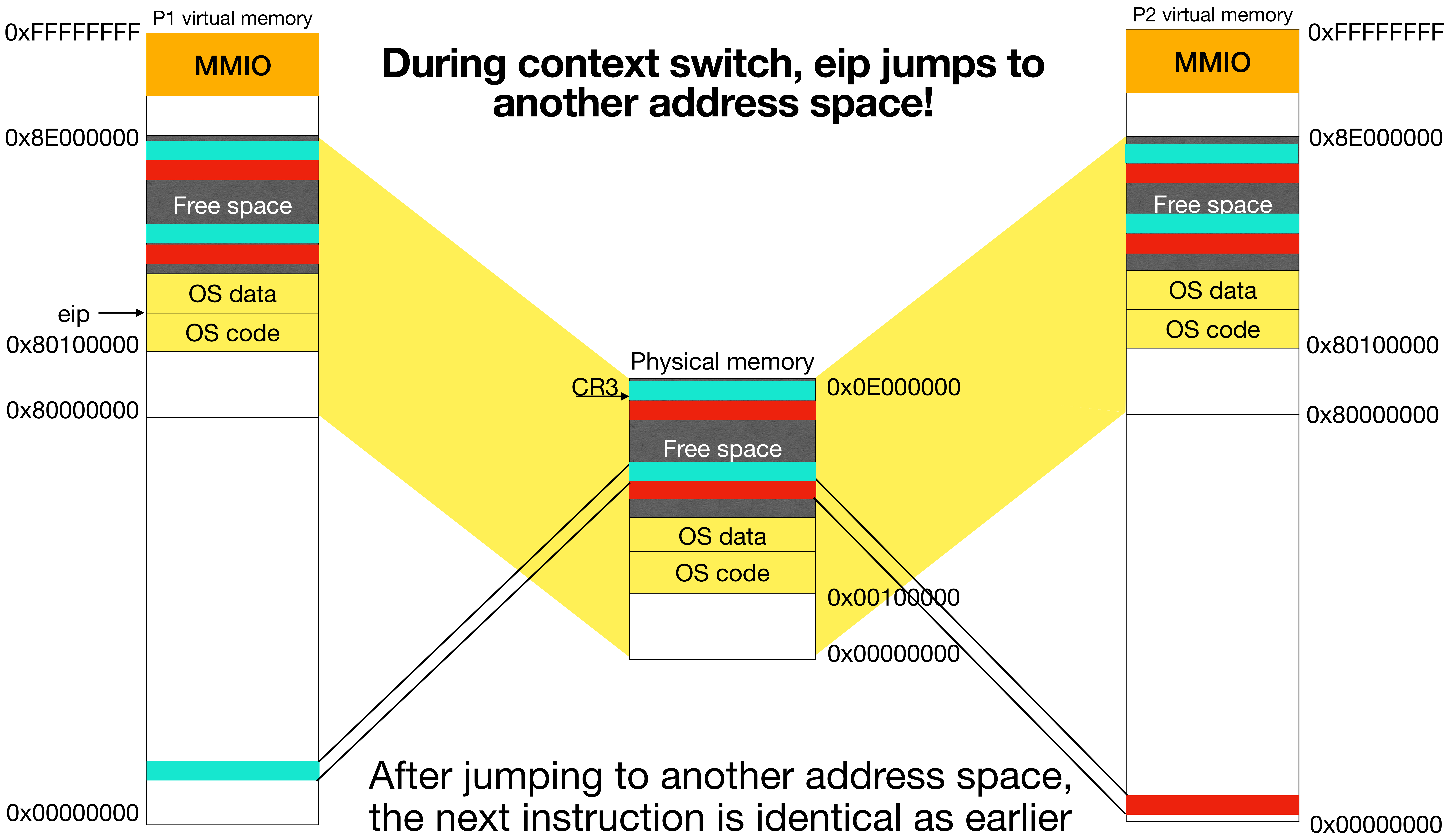
After jumping to another address space, the next instruction is identical as earlier

During context switch, eip jumps to another address space!



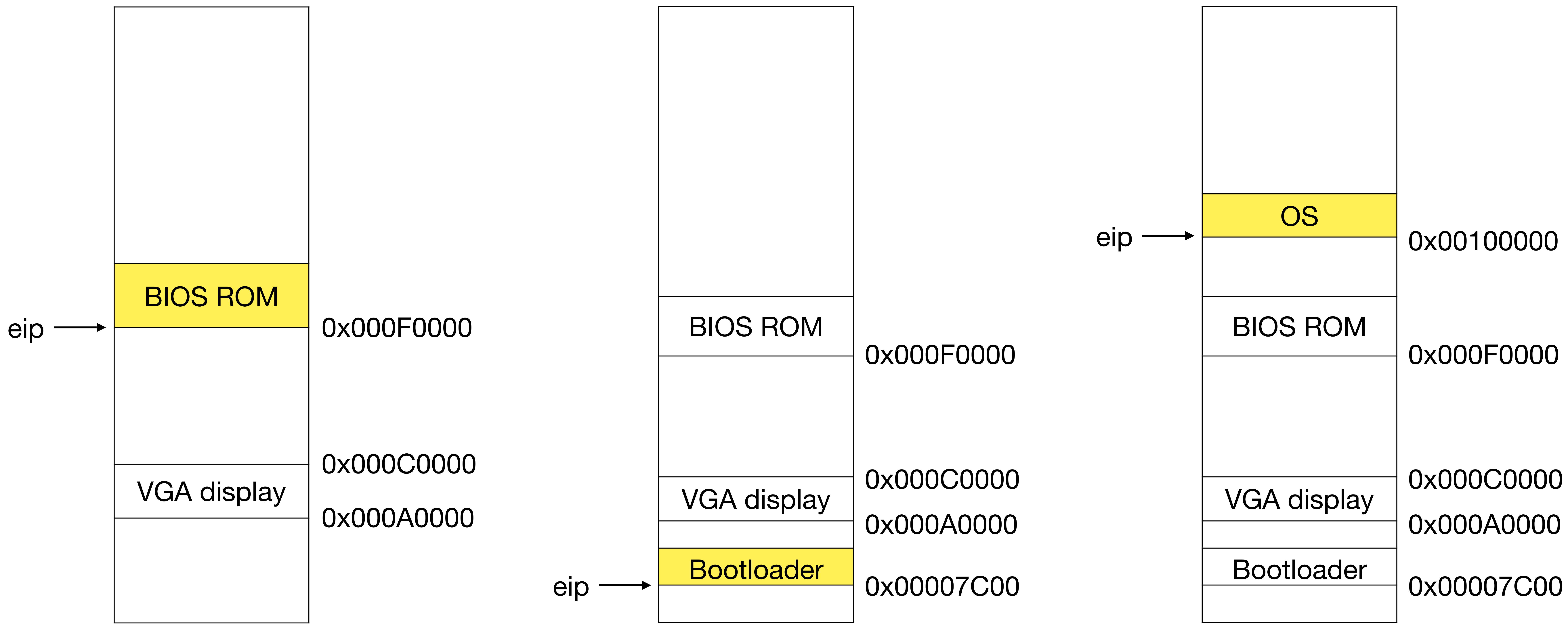
After jumping to another address space, the next instruction is identical as earlier

During context switch, eip jumps to another address space!



After jumping to another address space,
the next instruction is identical as earlier

Boot up sequence: BIOS to bootloader to OS



Kernel has different physical and virtual addresses

- kernel.ld declares virtual address 0x80100000, physical address 0x100000
- kernel.ld marks _start as entry point. _start is V2P_WO(entry) i.e, (0x8010000c - 0x80000000)

```
$ readelf -l kernel

Elf file type is EXEC (Executable file)
Entry point 0x10000c
There are 3 program headers, starting at offset 52

Program Headers:
  Type           Offset   VirtAddr   PhysAddr   FileSiz MemSiz  Flg Align
  LOAD           0x001000 0x80100000 0x00100000 0x07aab 0x07aab R E 0x1000
  LOAD           0x009000 0x80108000 0x00108000 0x02516 0x0d4a8 RW  0x1000
  GNU_STACK      0x000000 0x00000000 0x00000000 0x00000 0x00000 RWE 0x10

Section to Segment mapping:
  Segment Sections...
   00     .text .rodata
   01     .data .bss
   02
```

kernel.ld

```
ENTRY(_start)
SECTIONS {
  . = 0x80100000;
  .text : AT(0x100000)
```

entry.S

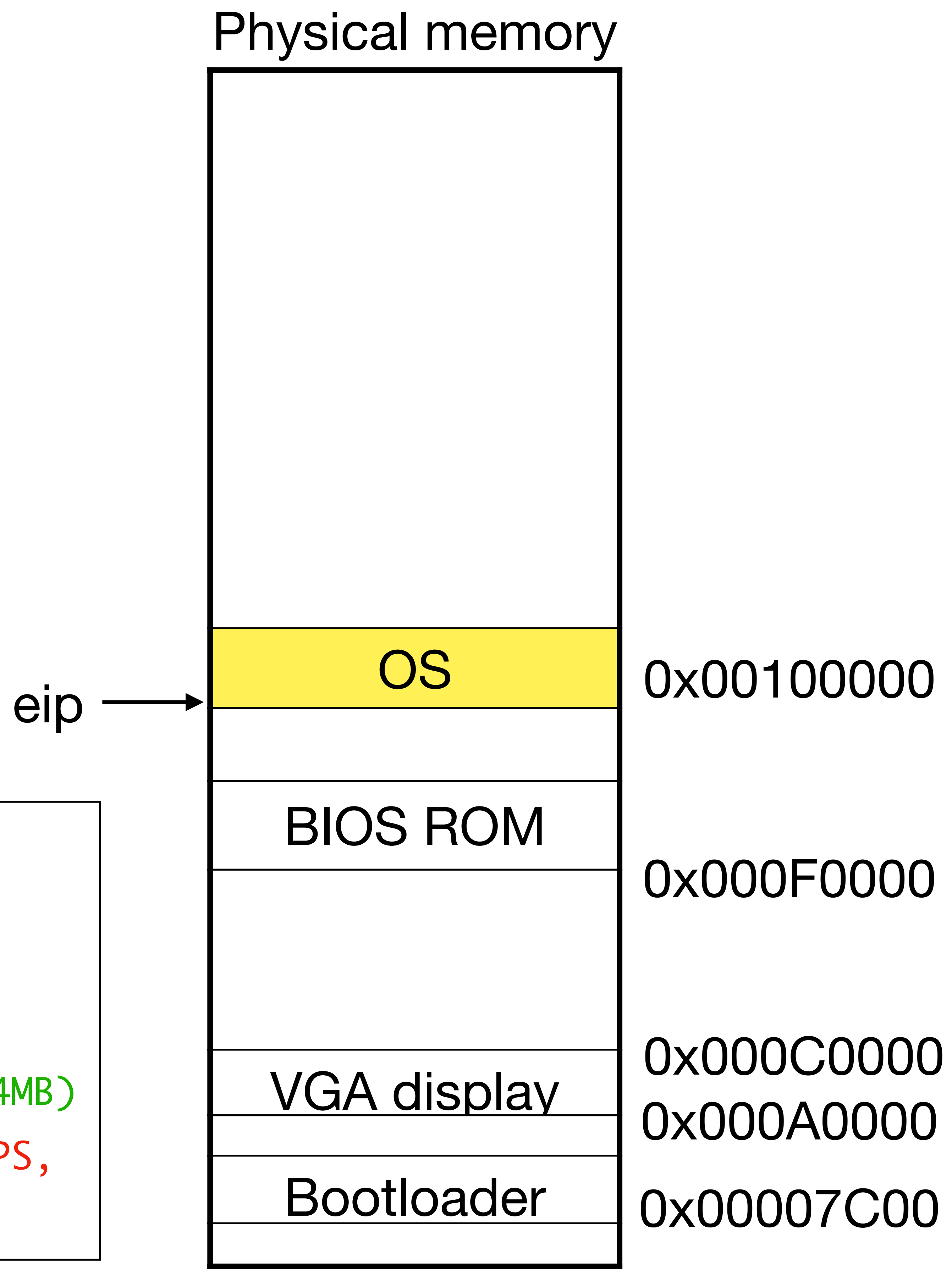
```
.globl _start
_start = V2P_WO(entry)
```

entry.S sets up an initial page table

eip →

```
entry:
  # Set page directory
  movl    $(V2P_WO(entrypgdir)), %eax
  movl    %eax, %cr3
  # Turn on paging.
  movl    %cr0, %eax
  orl     $(CR0_PG|CR0_WP), %eax
  movl    %eax, %cr0

  movl $(stack + KSTACKSIZE), %esp
  mov $main, %eax
  jmp *%eax

int main (void) {
  kinit1(end, P2V(4*1024*1024));
```

```
__attribute__((__aligned__(PGSIZE)))
pde_t entrypgdir[NPDENTRIES] = {
  // Map VA's [0, 4MB) to PA’s [0, 4MB)
  [0] = (0) | PTE_P | PTE_W | PTE_PS,
  // Map VA's [KERNBASE, KERNBASE+4MB) to PA's [0, 4MB)
  [KERNBASE>>PDXSHIFT] = (0) | PTE_P | PTE_W | PTE_PS,
};
```

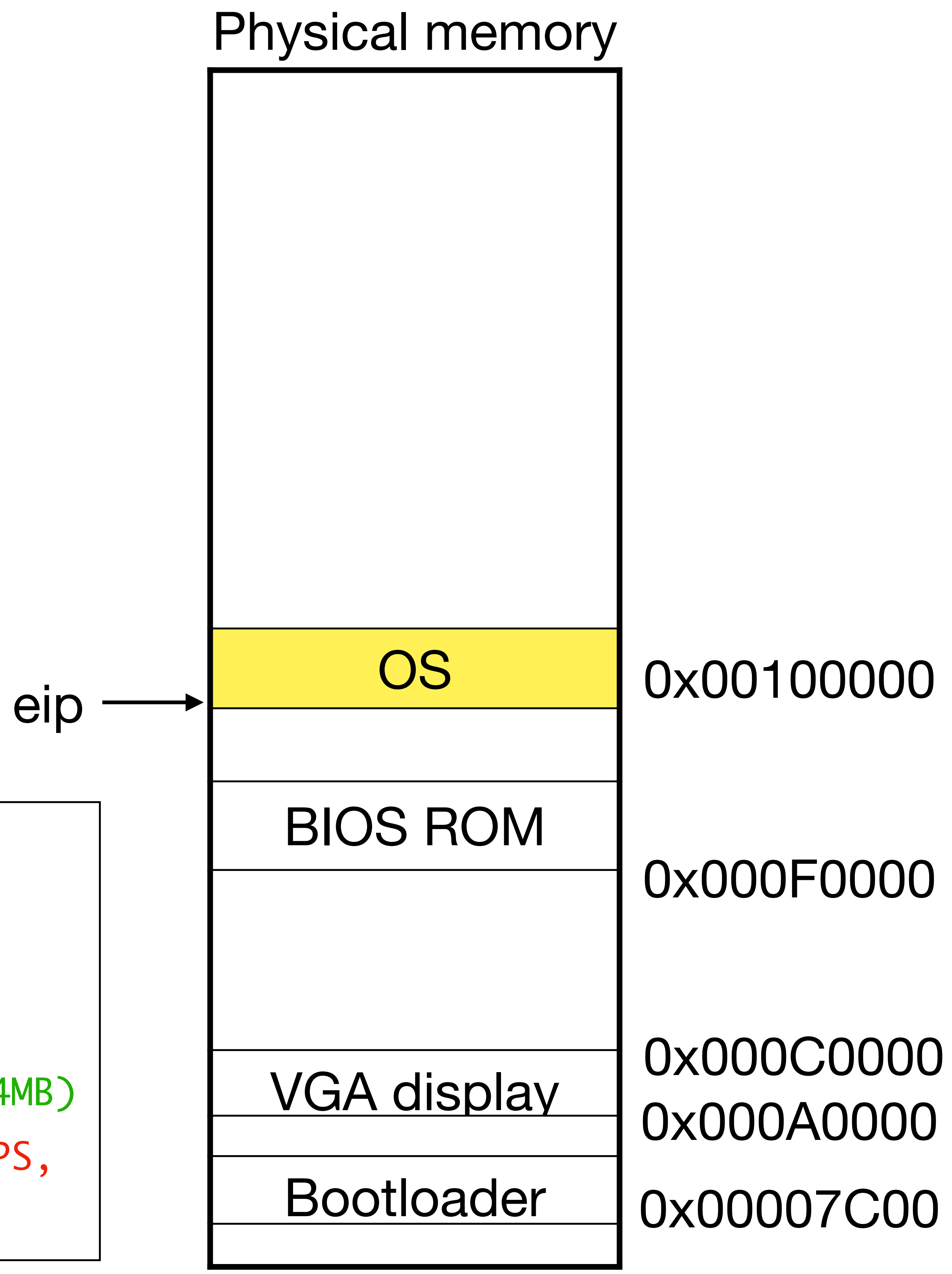


entry.S sets up an initial page table

eip →

```
entry:
  # Set page directory
  movl    $(V2P_WO(entrypgdir)), %eax
  movl    %eax, %cr3
  # Turn on paging.
  movl    %cr0, %eax
  orl     $(CR0_PG|CR0_WP), %eax
  movl    %eax, %cr0

  movl $(stack + KSTACKSIZE), %esp
  mov $main, %eax
  jmp *%eax

int main (void) {
  kinit1(end, P2V(4*1024*1024));
```

```
__attribute__((__aligned__(PGSIZE)))
pde_t entrypgdir[NPDENTRIES] = {
  // Map VA's [0, 4MB) to PA's [0, 4MB)
  [0] = (0) | PTE_P | PTE_W | PTE_PS,
  // Map VA's [KERNBASE, KERNBASE+4MB) to PA's [0, 4MB)
  [KERNBASE>>PDXSHIFT] = (0) | PTE_P | PTE_W | PTE_PS,
};
```



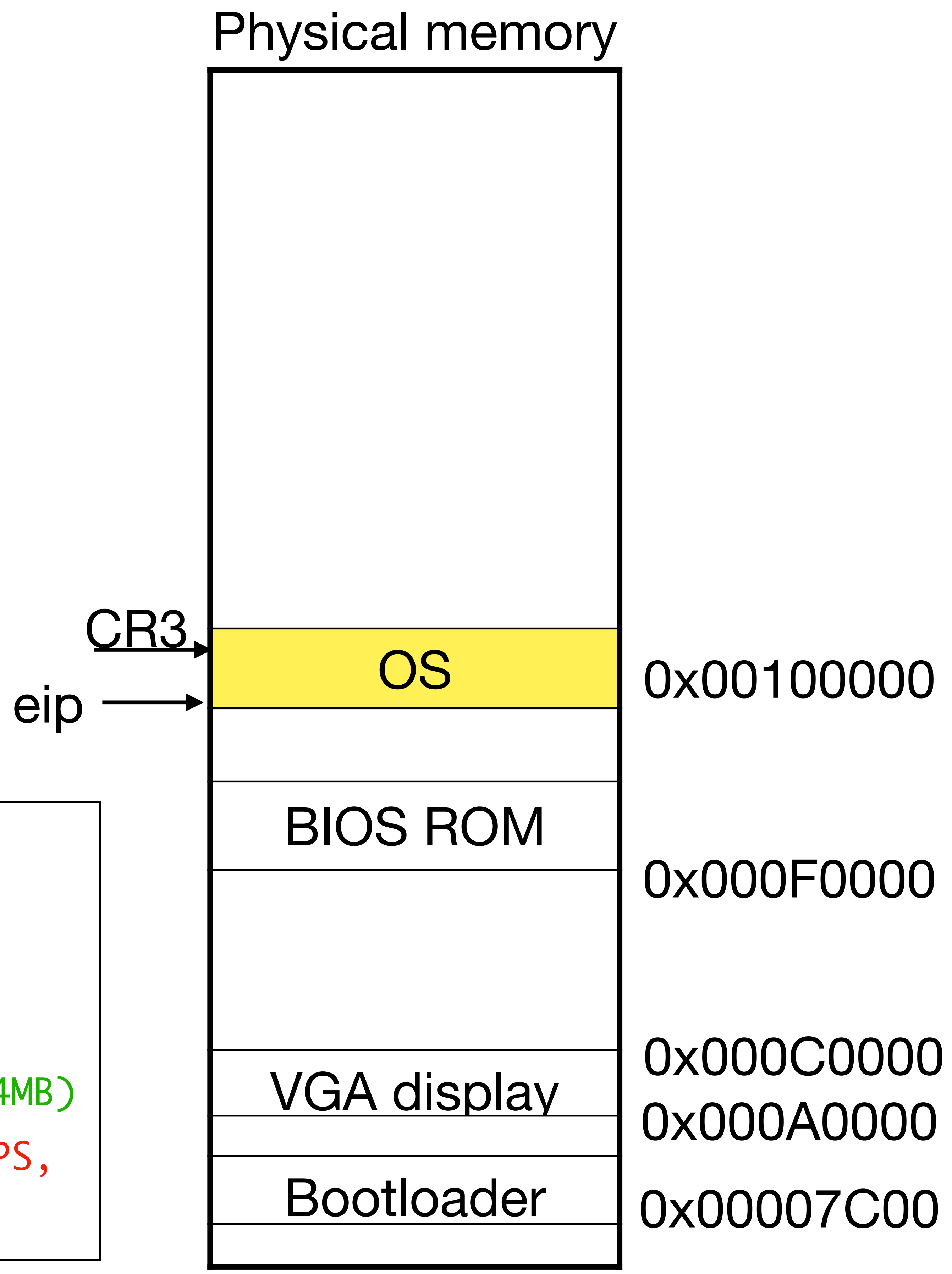
entry.S sets up an initial page table

eip

```
entry:
  # Set page directory
  movl    $(V2P_WO(entrypgdir)), %eax
  movl    %eax, %cr3
  # Turn on paging.
  movl    %cr0, %eax
  orl     $(CR0_PG|CR0_WP), %eax
  movl    %eax, %cr0

  movl $(stack + KSTACKSIZE), %esp
  mov $main, %eax
  jmp *%eax

int main (void) {
  kinit1(end, P2V(4*1024*1024));
```

CR3

```
__attribute__((__aligned__(PGSIZE)))
pde_t entrypgdir[NPDENTRIES] = {
  // Map VA's [0, 4MB) to PA's [0, 4MB)
  [0] = (0) | PTE_P | PTE_W | PTE_PS,
  // Map VA's [KERNBASE, KERNBASE+4MB) to PA's [0, 4MB)
  [KERNBASE>>PDXSHIFT] = (0) | PTE_P | PTE_W | PTE_PS,
};
```



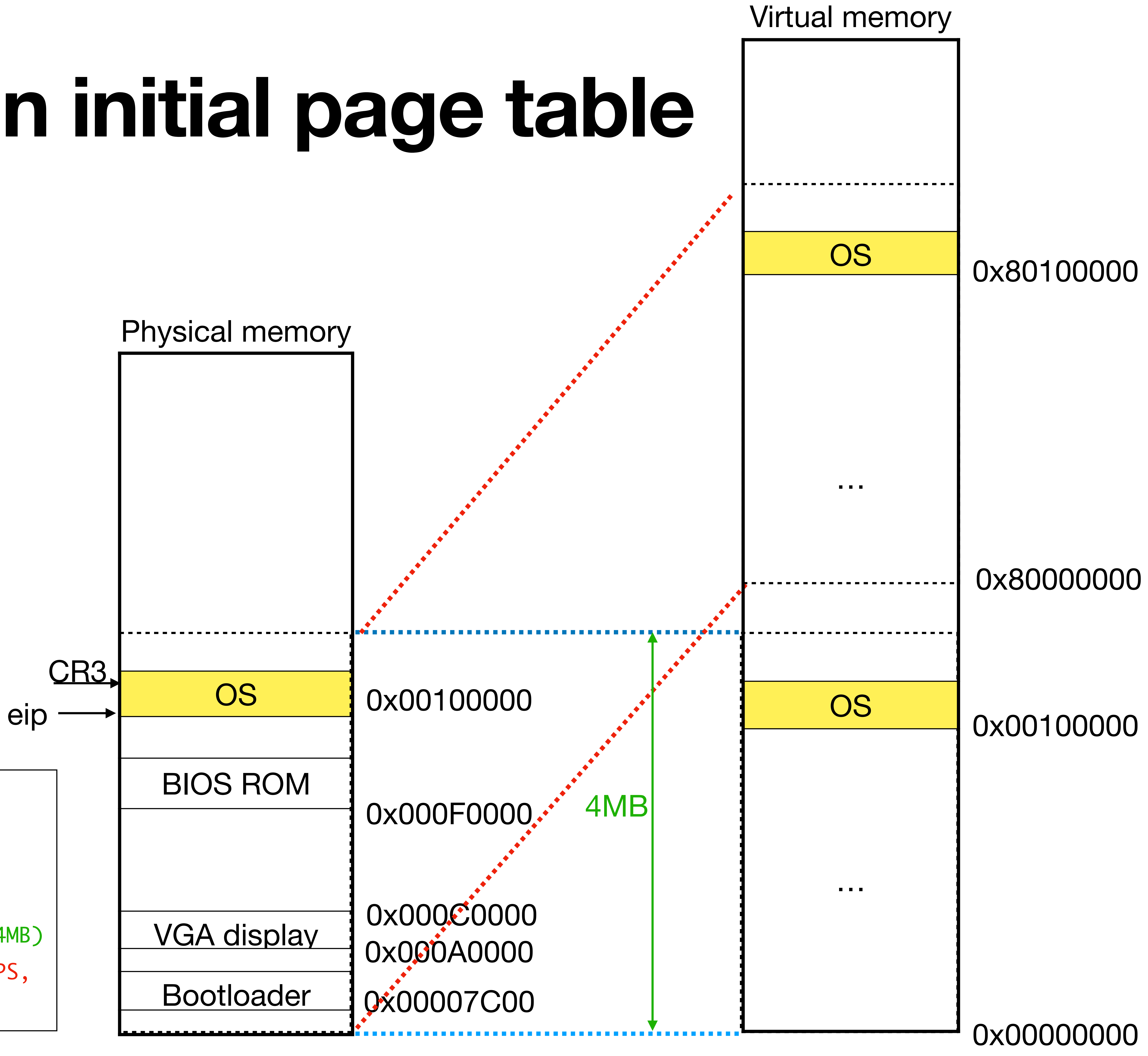
entry.S sets up an initial page table

```
entry:
  # Set page directory
  movl    $(V2P_WO(entrypgdir)), %eax
  movl    %eax, %cr3
  # Turn on paging.
  movl    %cr0, %eax
  orl     $(CR0_PG|CR0_WP), %eax
  movl    %eax, %cr0

  movl $(stack + KSTACKSIZE), %esp
  mov $main, %eax
  jmp *%eax

int main (void) {
  kinit1(end, P2V(4*1024*1024));
```

eip

CR3

```
__attribute__((__aligned__(PGSIZE)))
pde_t entrypgdir[NPDENTRIES] = {
  // Map VA's [0, 4MB) to PA's [0, 4MB)
  [0] = (0) | PTE_P | PTE_W | PTE_PS,
  // Map VA's [KERNBASE, KERNBASE+4MB) to PA's [0, 4MB)
  [KERNBASE>>PDXSHIFT] = (0) | PTE_P | PTE_W | PTE_PS,
};
```



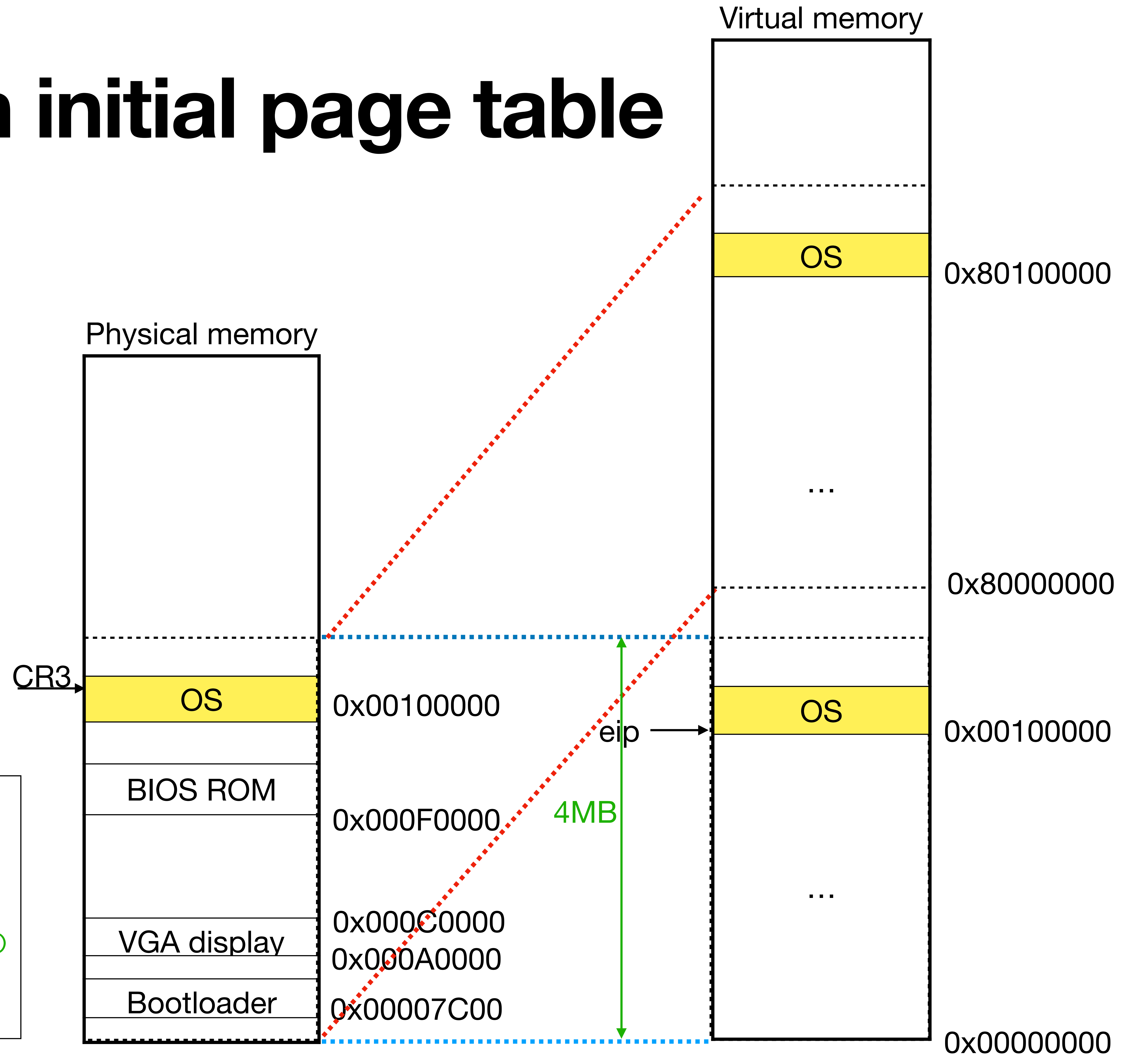
entry.S sets up an initial page table

eip →

```
entry:
  # Set page directory
  movl    $(V2P_WO(entrypgdir)), %eax
  movl    %eax, %cr3
  # Turn on paging.
  movl    %cr0, %eax
  orl     $(CR0_PG|CR0_WP), %eax
  movl    %eax, %cr0

  movl $(stack + KSTACKSIZE), %esp
  mov $main, %eax
  jmp *%eax

int main (void) {
  kinit1(end, P2V(4*1024*1024));
```

CR3 →

```
__attribute__((__aligned__(PGSIZE)))
pde_t entrypgdir[NPDENTRIES] = {
  // Map VA's [0, 4MB) to PA’s [0, 4MB)
  [0] = (0) | PTE_P | PTE_W | PTE_PS,
  // Map VA's [KERNBASE, KERNBASE+4MB) to PA's [0, 4MB)
  [KERNBASE>>PDXSHIFT] = (0) | PTE_P | PTE_W | PTE_PS,
};
```



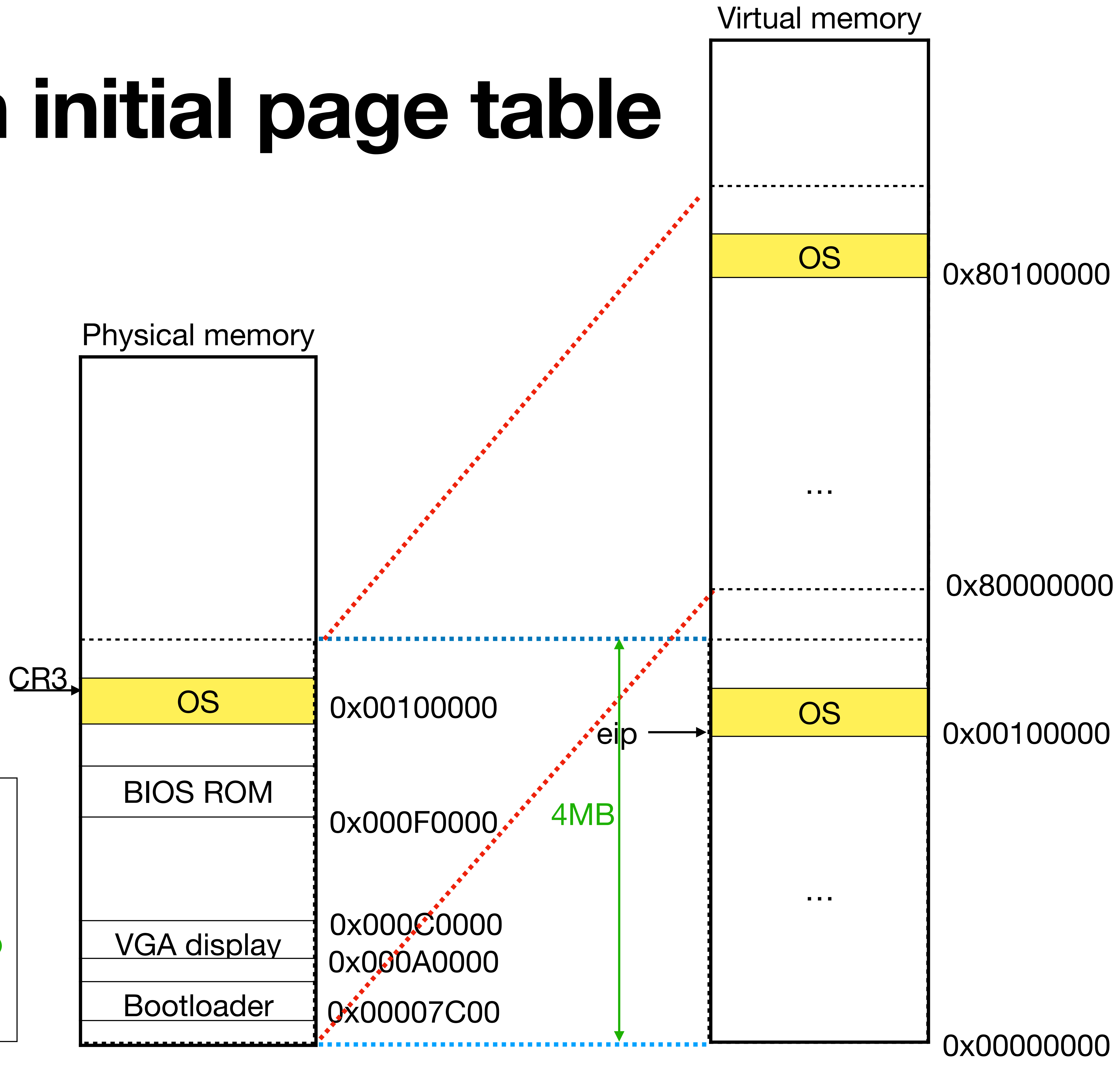
entry.S sets up an initial page table

```
entry:
  # Set page directory
  movl    $(V2P_WO(entrypgdir)), %eax
  movl    %eax, %cr3
  # Turn on paging.
  movl    %cr0, %eax
  orl     $(CR0_PG|CR0_WP), %eax
  movl    %eax, %cr0

  movl $(stack + KSTACKSIZE), %esp
  mov $main, %eax
  jmp *%eax

int main (void) {
  kinit1(end, P2V(4*1024*1024));
```

eip

CR3

```
__attribute__((__aligned__(PGSIZE)))
pde_t entrypgdir[NPDENTRIES] = {
  // Map VA's [0, 4MB) to PA's [0, 4MB)
  [0] = (0) | PTE_P | PTE_W | PTE_PS,
  // Map VA's [KERNBASE, KERNBASE+4MB) to PA's [0, 4MB)
  [KERNBASE>>PDXSHIFT] = (0) | PTE_P | PTE_W | PTE_PS,
};
```



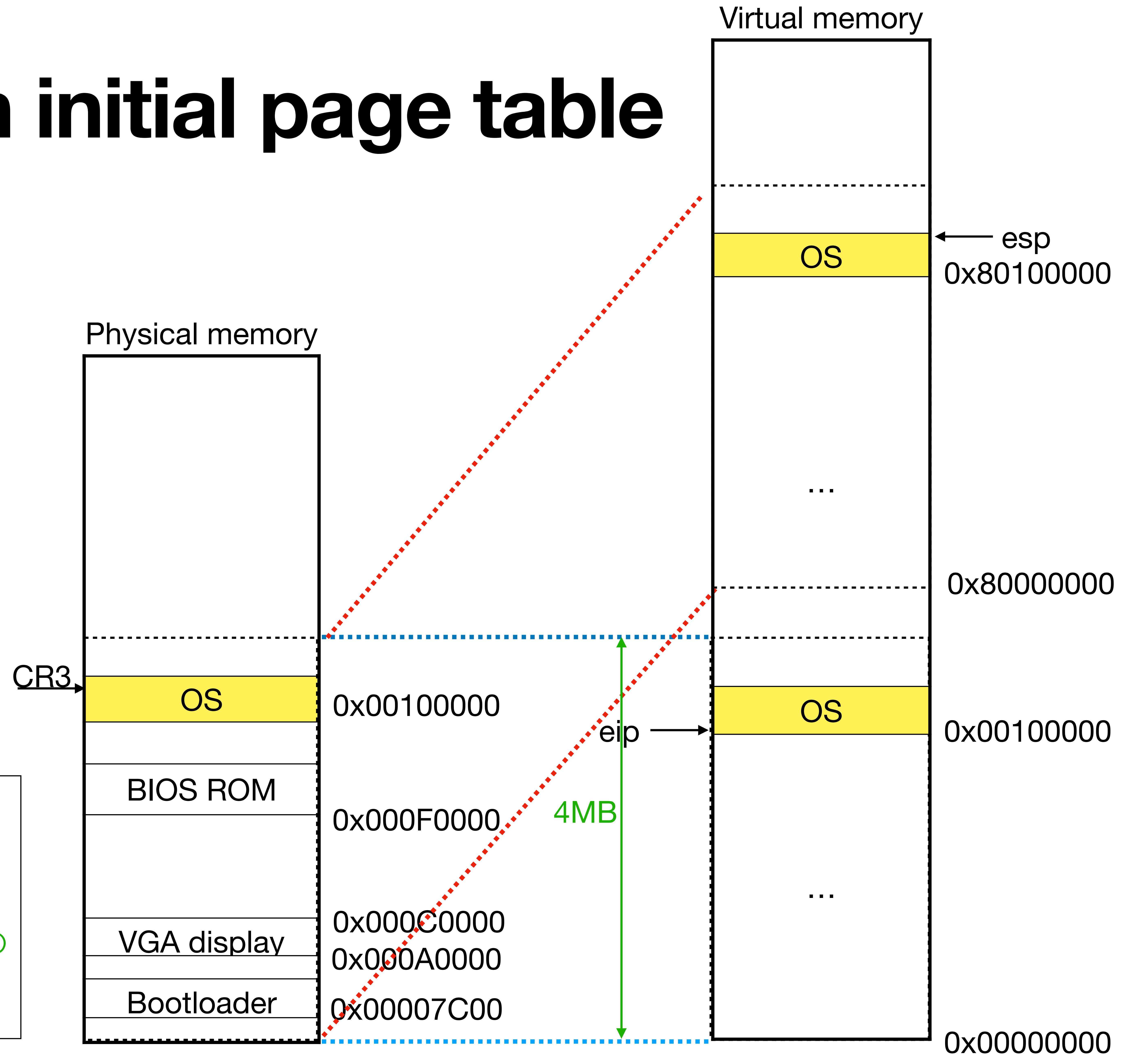
entry.S sets up an initial page table

eip →

```
entry:
  # Set page directory
  movl    $(V2P_WO(entrypgdir)), %eax
  movl    %eax, %cr3
  # Turn on paging.
  movl    %cr0, %eax
  orl     $(CR0_PG|CR0_WP), %eax
  movl    %eax, %cr0

  movl $(stack + KSTACKSIZE), %esp
  mov $main, %eax
  jmp *%eax

int main (void) {
  kinit1(end, P2V(4*1024*1024));
```

CR3 →

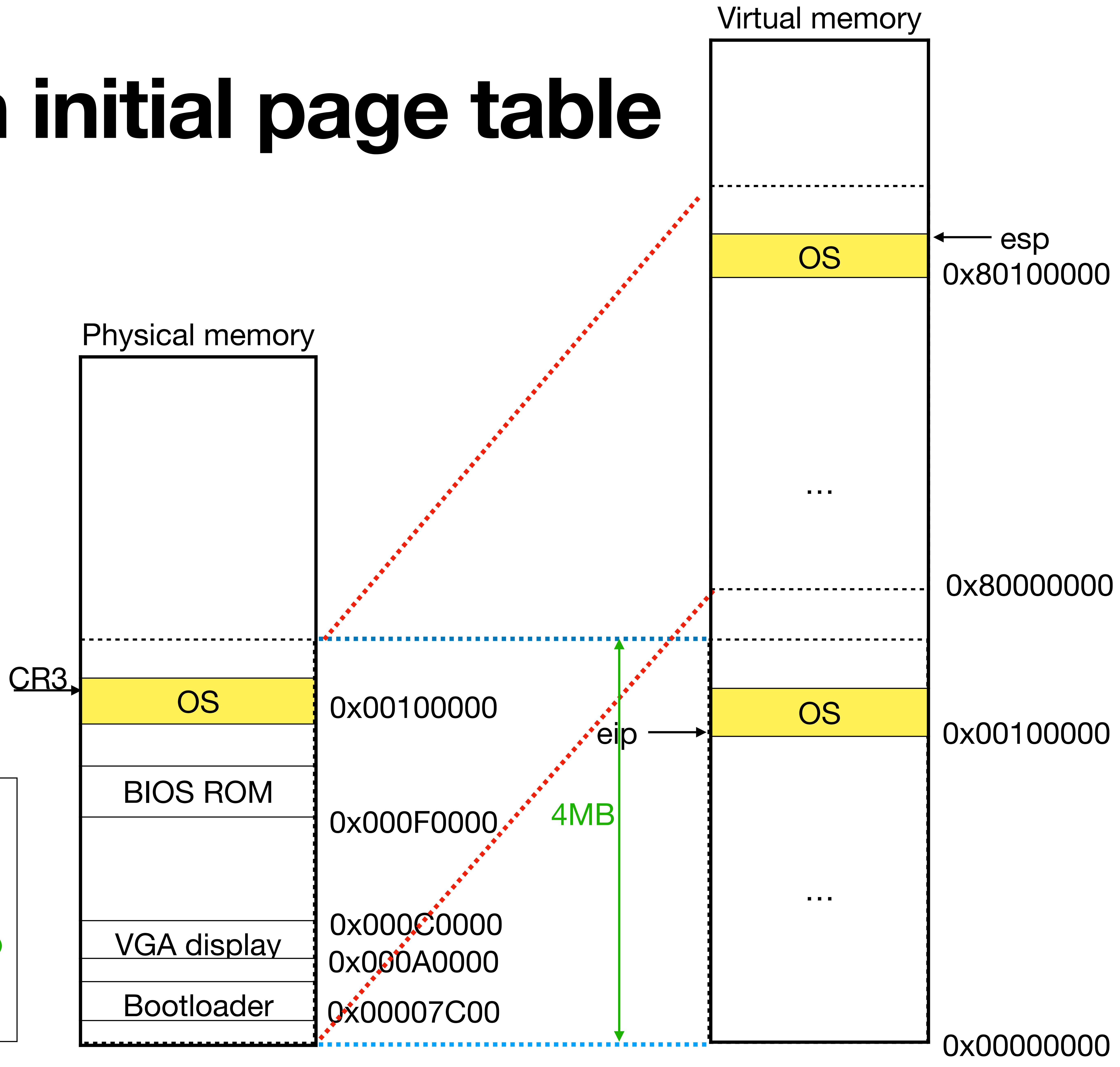
```
__attribute__((__aligned__(PGSIZE)))
pde_t entrypgdir[NPDENTRIES] = {
  // Map VA's [0, 4MB) to PA's [0, 4MB)
  [0] = (0) | PTE_P | PTE_W | PTE_PS,
  // Map VA's [KERNBASE, KERNBASE+4MB) to PA's [0, 4MB)
  [KERNBASE>>PDXSHIFT] = (0) | PTE_P | PTE_W | PTE_PS,
};
```



entry.S sets up an initial page table

eip →
```
entry:
  # Set page directory
  movl    $(V2P_WO(entrypgdir)), %eax
  movl    %eax, %cr3
  # Turn on paging.
  movl    %cr0, %eax
  orl     $(CR0_PG|CR0_WP), %eax
  movl    %eax, %cr0

  movl $(stack + KSTACKSIZE), %esp
  mov $main, %eax
  jmp *%eax

int main (void) {
  kinit1(end, P2V(4*1024*1024));
```

CR3 →
```
__attribute__((__aligned__(PGSIZE)))
pde_t entrypgdir[NPDENTRIES] = {
  // Map VA's [0, 4MB) to PA's [0, 4MB)
  [0] = (0) | PTE_P | PTE_W | PTE_PS,
  // Map VA's [KERNBASE, KERNBASE+4MB) to PA's [0, 4MB)
  [KERNBASE>>PDXSHIFT] = (0) | PTE_P | PTE_W | PTE_PS,
};
```



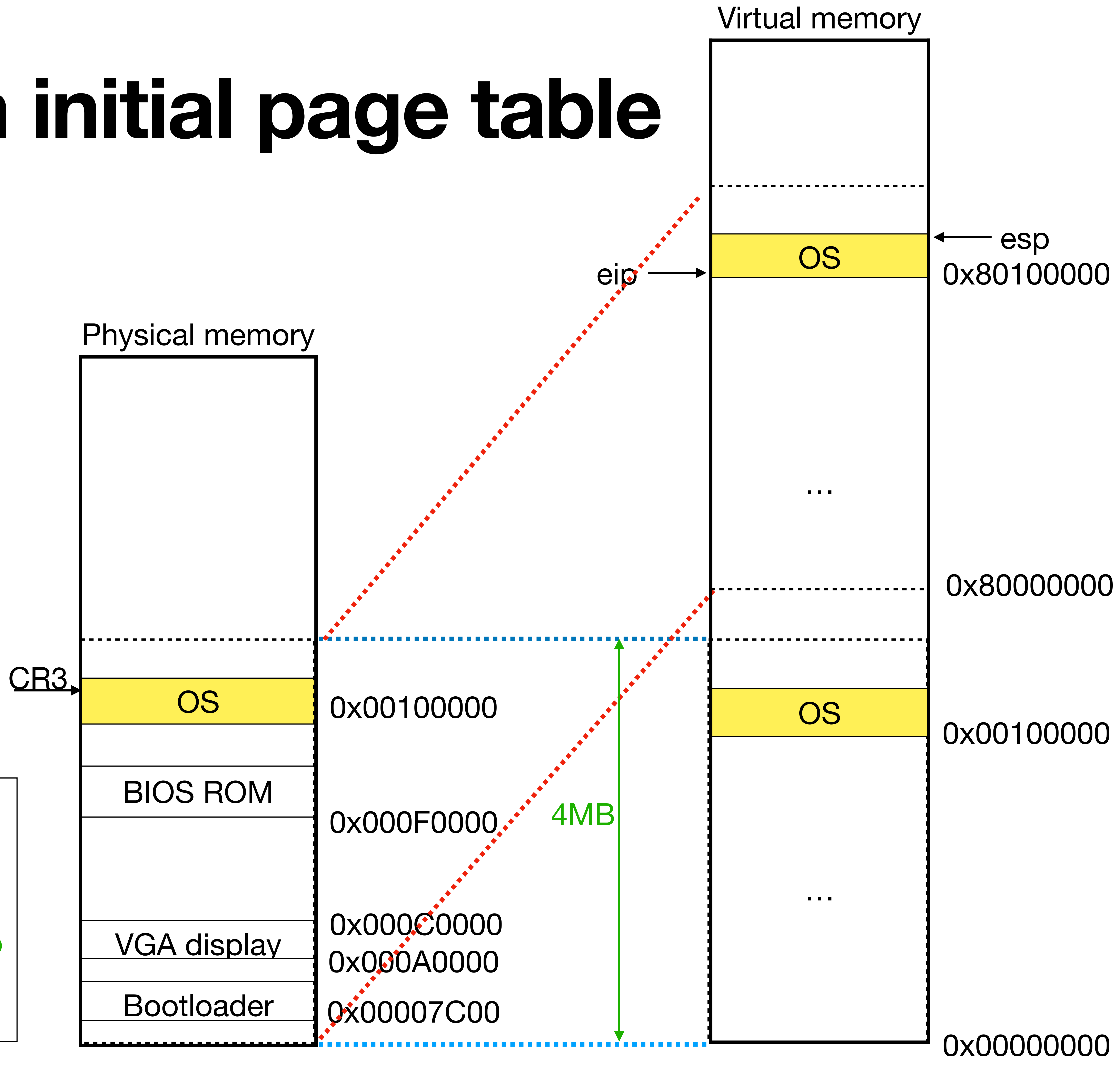
entry.S sets up an initial page table

```
entry:
  # Set page directory
  movl    $(V2P_WO(entrypgdir)), %eax
  movl    %eax, %cr3
  # Turn on paging.
  movl    %cr0, %eax
  orl     $(CR0_PG|CR0_WP), %eax
  movl    %eax, %cr0

  movl $(stack + KSTACKSIZE), %esp
  mov $main, %eax
  jmp *%eax

int main (void) {
  kinit1(end, P2V(4*1024*1024));
```

eip

CR3

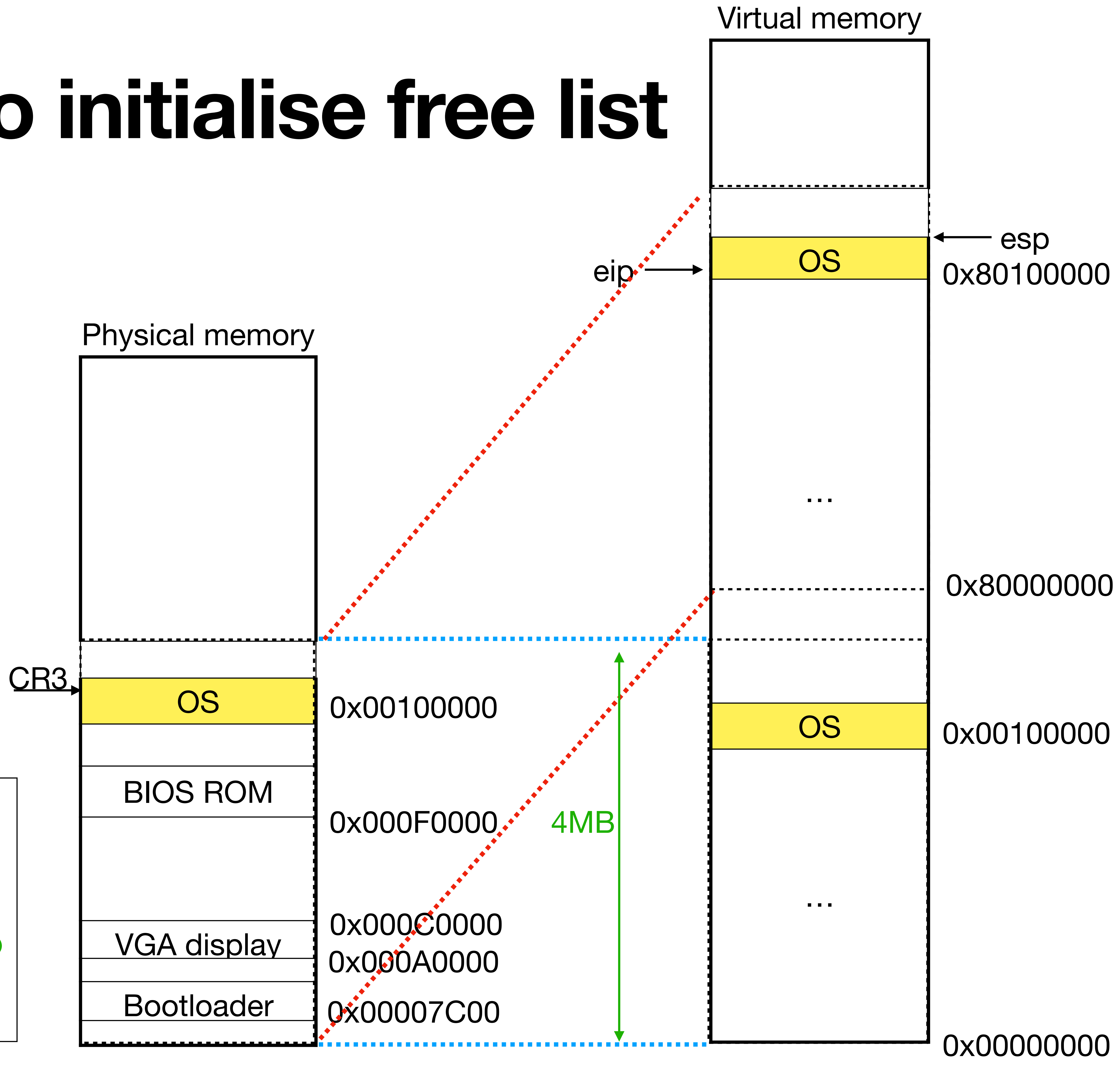
```
__attribute__((__aligned__(PGSIZE)))
pde_t entrypgdir[NPDENTRIES] = {
  // Map VA's [0, 4MB) to PA's [0, 4MB)
  [0] = (0) | PTE_P | PTE_W | PTE_PS,
  // Map VA's [KERNBASE, KERNBASE+4MB) to PA's [0, 4MB)
  [KERNBASE>>PDXSHIFT] = (0) | PTE_P | PTE_W | PTE_PS,
};
```



main calls kinit1 to initialise free list

Only 4MB is mapped in the initial page table

```
__attribute__((__aligned__(PGSIZE)))
pde_t entrypgdir[NPDENTRIES] = {
  // Map VA's [0, 4MB) to PA’s [0, 4MB)
  [0] = (0) | PTE_P | PTE_W | PTE_PS,
  // Map VA's [KERNBASE, KERNBASE+4MB) to PA's [0, 4MB)
  [KERNBASE>>PDXSHIFT] = (0) | PTE_P | PTE_W | PTE_PS,
};
```

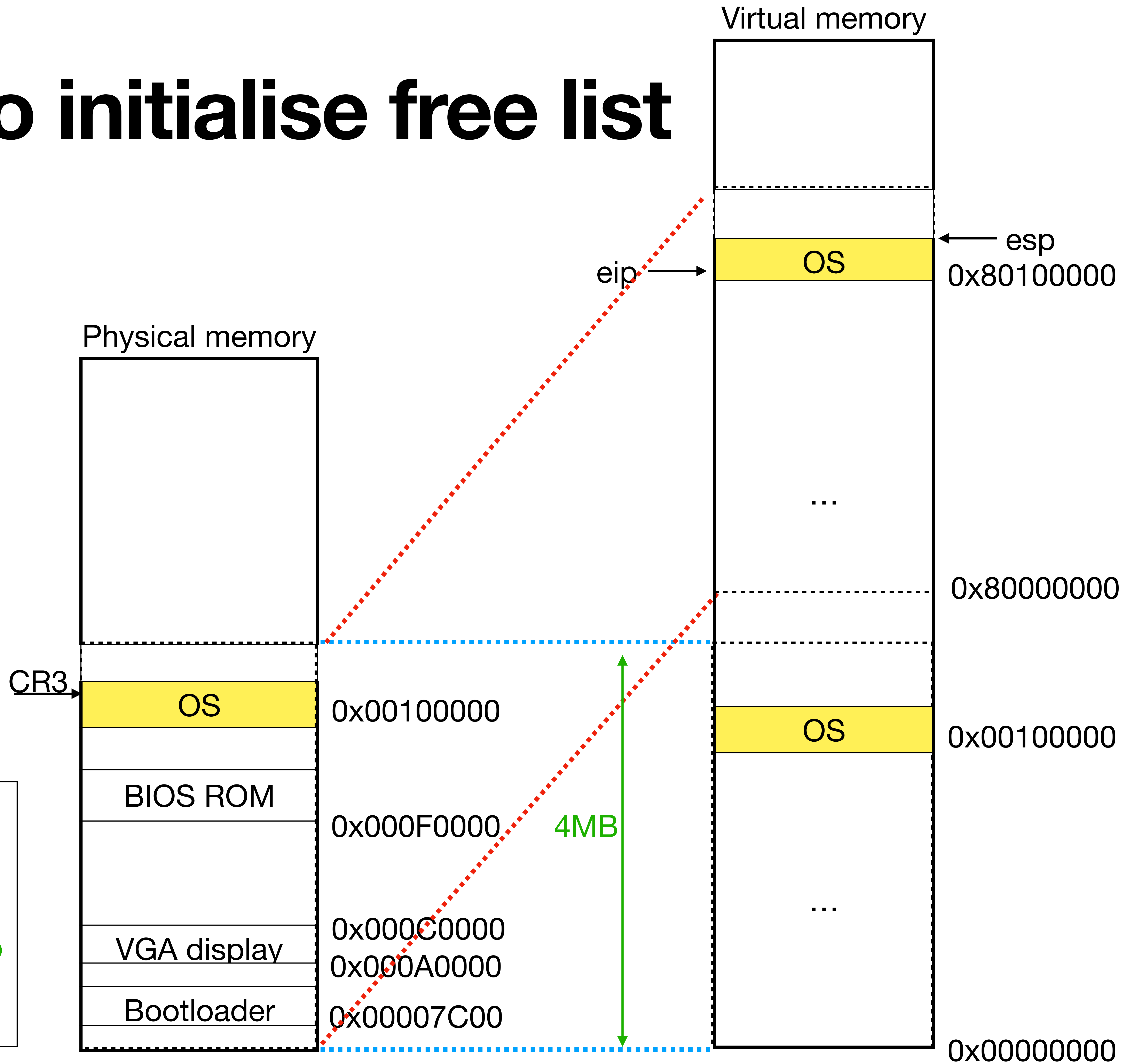


main calls kinit1 to initialise free list

Only 4MB is mapped in the initial page table

```
int main (void) {
  kinit1(end, P2V(4*1024*1024));
  …
}
```

```
__attribute__((__aligned__(PGSIZE)))
pde_t entrypgdir[NPDENTRIES] = {
  // Map VA's [0, 4MB) to PA’s [0, 4MB)
  [0] = (0) | PTE_P | PTE_W | PTE_PS,
  // Map VA's [KERNBASE, KERNBASE+4MB) to PA's [0, 4MB)
  [KERNBASE>>PDXSHIFT] = (0) | PTE_P | PTE_W | PTE_PS,
};
```

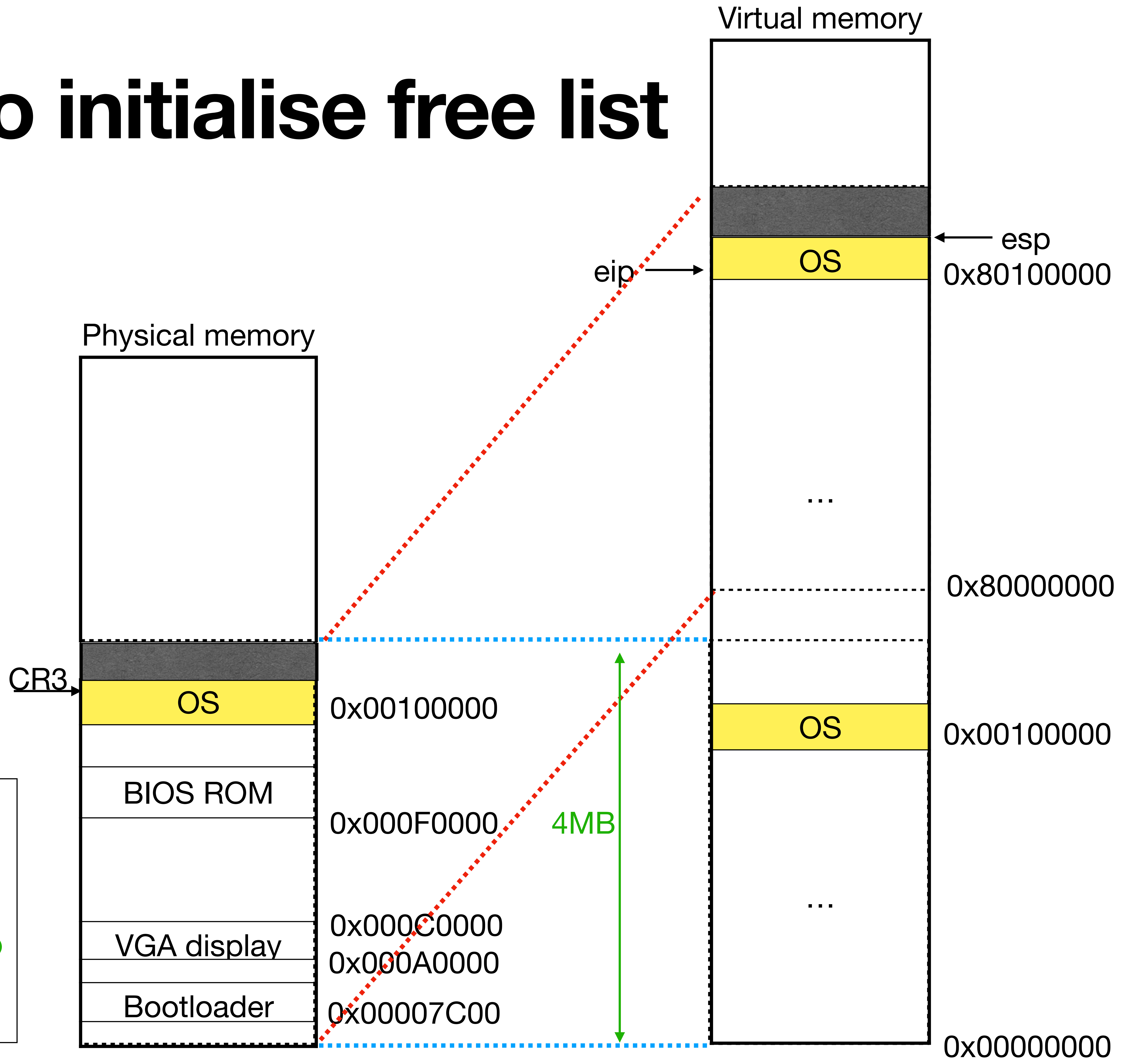


main calls kinit1 to initialise free list

Only 4MB is mapped in the initial page table

```
int main (void) {
  kinit1(end, P2V(4*1024*1024));
  …
}
```

```
__attribute__((__aligned__(PGSIZE)))
pde_t entrypgdir[NPDENTRIES] = {
  // Map VA's [0, 4MB) to PA’s [0, 4MB)
  [0] = (0) | PTE_P | PTE_W | PTE_PS,
  // Map VA's [KERNBASE, KERNBASE+4MB) to PA's [0, 4MB)
  [KERNBASE>>PDXSHIFT] = (0) | PTE_P | PTE_W | PTE_PS,
};
```



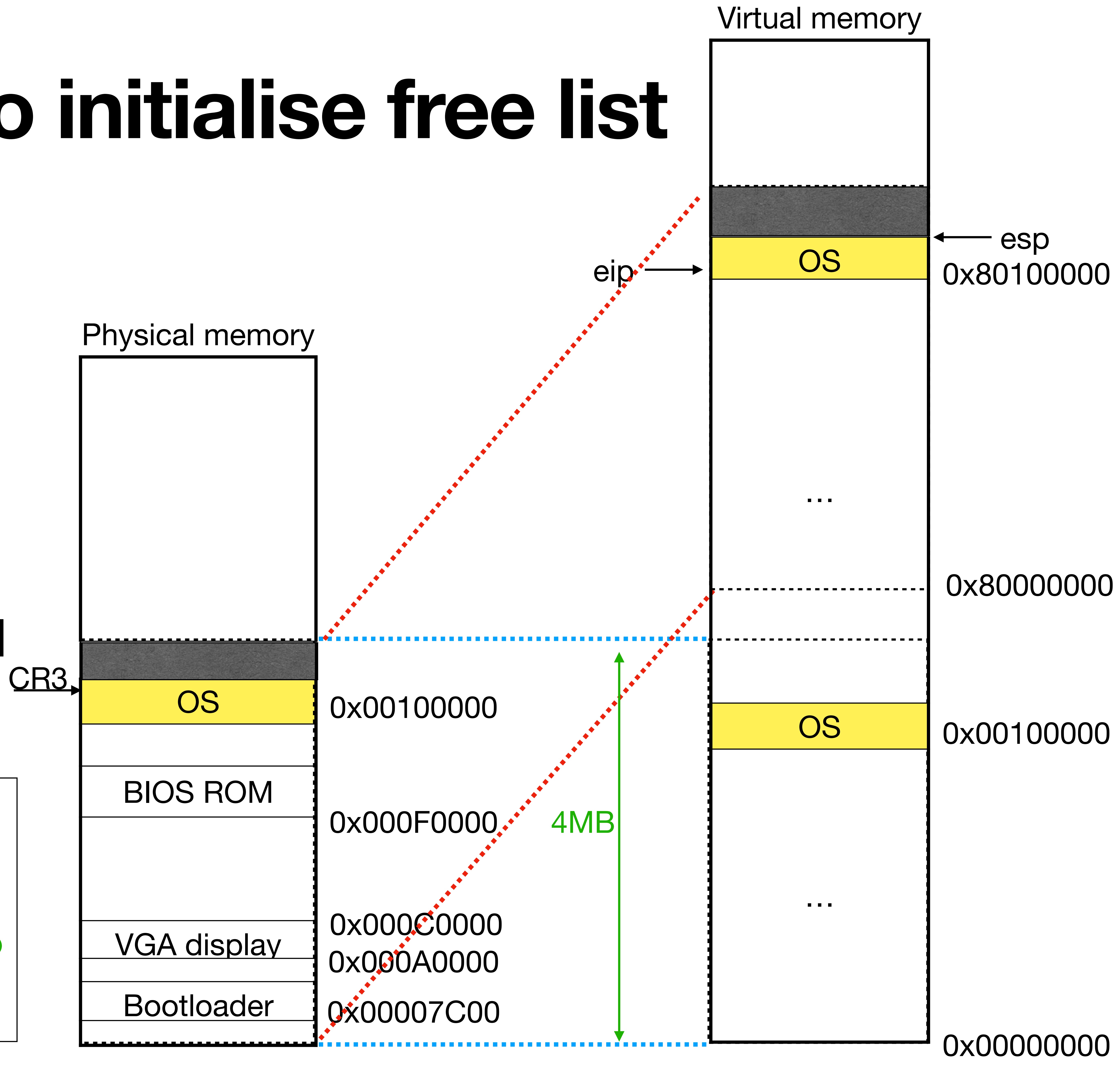
main calls kinit1 to initialise free list

Only 4MB is mapped in the initial page table

```
int main (void) {
  kinit1(end, P2V(4*1024*1024));
  …
}
```

Now pages can be allocated from the free list!

```
__attribute__((__aligned__(PGSIZE)))
pde_t entrypgdir[NPDENTRIES] = {
  // Map VA's [0, 4MB) to PA’s [0, 4MB)
  [0] = (0) | PTE_P | PTE_W | PTE_PS,
  // Map VA's [KERNBASE, KERNBASE+4MB) to PA's [0, 4MB)
  [KERNBASE>>PDXSHIFT] = (0) | PTE_P | PTE_W | PTE_PS,
};
```



OS sets up a new page table

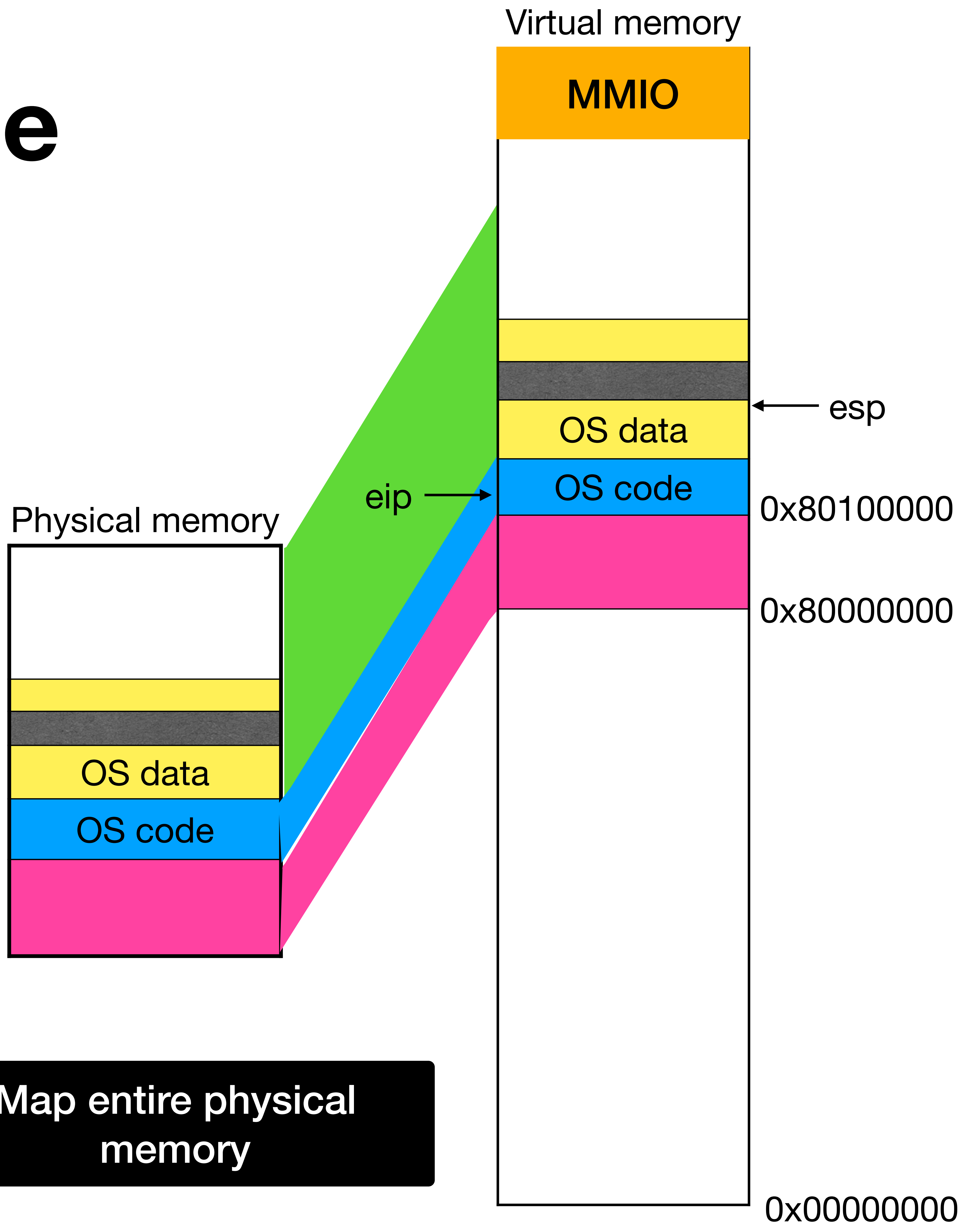
```
int main(void) {                    main.c
  kinit1(end, P2V(4*1024*1024));
  kvmalloc(); // kernel page table
  ..
}
```

Remove identity mapping in low addresses

```
static struct kmap {
  void *virt;
  uint phys_start;
  uint phys_end;
  int perm;
} kmap[] = {
{ (void*)KERNBASE, 0,             EXTMEM,    PTE_W},
{ (void*)KERNLINK, V2P(KERNLINK), V2P(data), 0},
{ (void*)data,     V2P(data),     PHYSTOP,   PTE_W},
{ (void*)DEVSPACE, DEVSPACE,      0,         PTE_W},
};
```

Mark code as read-only

Map entire physical memory



OS sets up a new page table

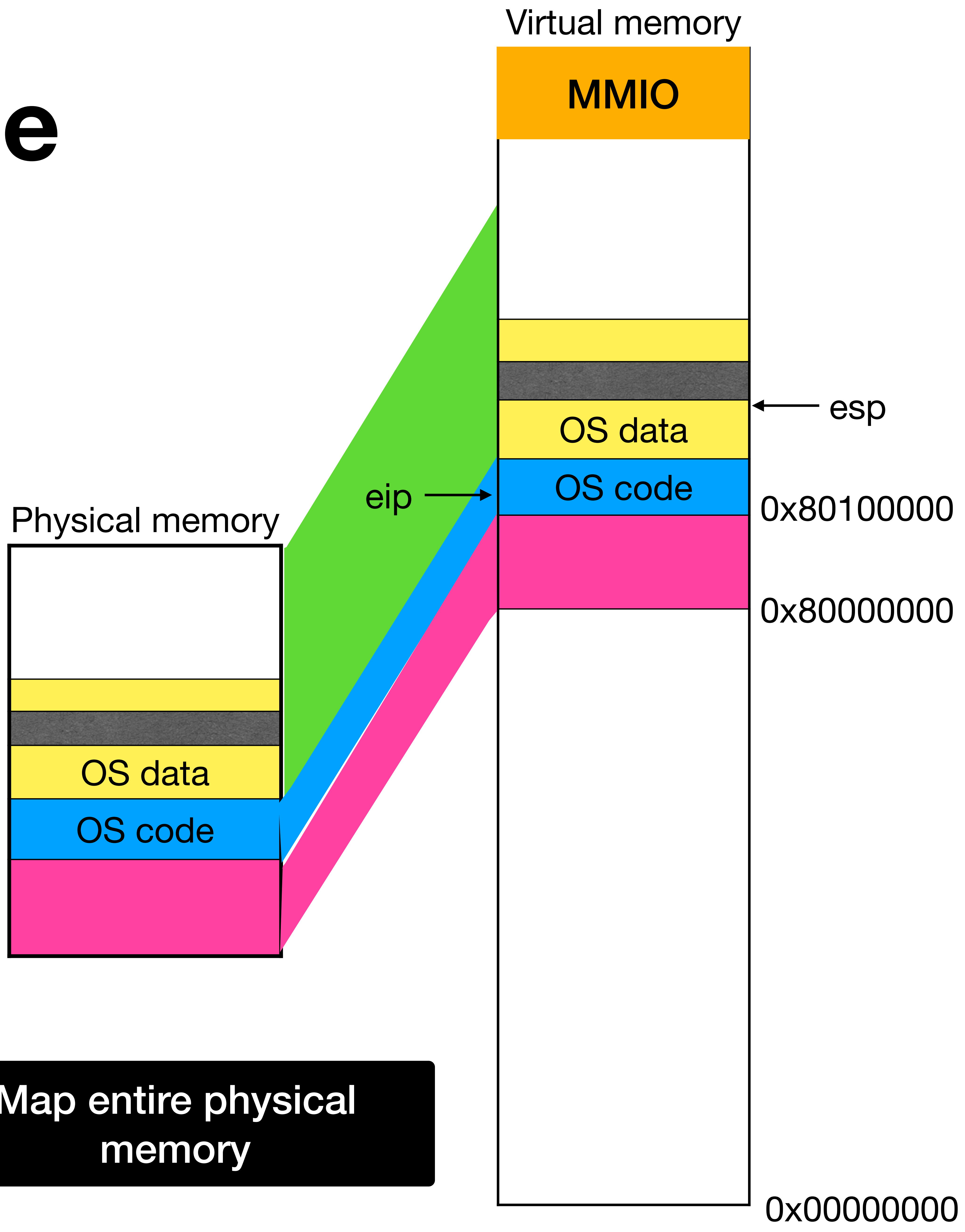
```
int main(void) {                main.c
  kinit1(end, P2V(4*1024*1024));
  kvmalloc(); // kernel page table
  ..
}
```

Remove identity mapping in low addresses

```
static struct kmap {
  void *virt;
  uint phys_start;
  uint phys_end;
  int perm;
} kmap[] = {
{ (void*)KERNBASE, 0,              EXTMEM,     PTE_W},
{ (void*)KERNLINK, V2P(KERNLINK), V2P(data), 0},
{ (void*)data,     V2P(data),     PHYSTOP,    PTE_W},
{ (void*)DEVSPACE, DEVSPACE,      0,          PTE_W},
};
```

Mark code as read-only

Map entire physical memory



OS sets up a new page table

```
kmap[] = {
{ (void*)KERNBASE, 0,              EXTMEM,     PTE_W},
{ (void*)KERNLINK, V2P(KERNLINK), V2P(data), 0},
{ (void*)data,     V2P(data),      PHYSTOP,    PTE_W},
{ (void*)DEVSPACE, DEVSPACE,       0,          PTE_W},
};
```

main.c

```
int main(void) {
  kinit1(end, P2V(4*1024*1024));
  kvmalloc(); // kernel page table
  ..
}
```

vm.c

```
// Allocate one page table for the machine for
// the kernel address space for scheduler processes.
void kvmalloc(void) {
  kpgdir = setupkvm();
  switchkvm();
}

pde_t* setupkvm(void) {
  pde_t *pgdir;
  struct kmap *k;
  pgdir = (pde_t*)kalloc();
  memset(pgdir, 0, PGSIZE);
  for(k = kmap; k < &kmap[NELEM(kmap)]; k++)
    mappages(pgdir, k->...)
}

// Switch h/w page table register to the kernel-only
// page table, for when no process is running.
void switchkvm(void) {
  lcr3(V2P(kpgdir));   // switch to the kernel page table
}
```

OS sets up a new page table

```
kmap[] = {
{ (void*)KERNBASE, 0,             EXTMEM,    PTE_W},
{ (void*)KERNLINK, V2P(KERNLINK), V2P(data), 0},
{ (void*)data,     V2P(data),     PHYSTOP,   PTE_W},
{ (void*)DEVSPACE, DEVSPACE,      0,         PTE_W},
};
```

```
int main(void) {                         main.c
  kinit1(end, P2V(4*1024*1024));
  kvmalloc(); // kernel page table
  ..
}
```

Allocate a new page for page

```
// Allocate one page table for the machine for      vm.c
// the kernel address space for scheduler processes.
void kvmalloc(void) {
  kpgdir = setupkvm();
  switchkvm();
}

pde_t* setupkvm(void) {
  pde_t *pgdir;
  struct kmap *k;
  pgdir = (pde_t*)kalloc();
  memset(pgdir, 0, PGSIZE);
  for(k = kmap; k < &kmap[NELEM(kmap)]; k++)
    mappages(pgdir, k->...)
}

// Switch h/w page table register to the kernel-only
// page table, for when no process is running.
void switchkvm(void) {
  lcr3(V2P(kpgdir));   // switch to the kernel page table
}
```

OS sets up a new page table

```
kmap[] = {
{ (void*)KERNBASE, 0,             EXTMEM,     PTE_W},
{ (void*)KERNLINK, V2P(KERNLINK), V2P(data),  0},
{ (void*)data,     V2P(data),     PHYSTOP,    PTE_W},
{ (void*)DEVSPACE, DEVSPACE,      0,          PTE_W},
};
```

```
int main(void) {                        main.c
  kinit1(end, P2V(4*1024*1024));
  kvmalloc(); // kernel page table
  ..
}
```

```
// Allocate one page table for the machine for     vm.c
// the kernel address space for scheduler processes.
void kvmalloc(void) {
  kpgdir = setupkvm();
  switchkvm();
}

pde_t* setupkvm(void) {
  pde_t *pgdir;
  struct kmap *k;
  pgdir = (pde_t*)kalloc();
  memset(pgdir, 0, PGSIZE);
  for(k = kmap; k < &kmap[NELEM(kmap)]; k++)
    mappages(pgdir, k->...)
}

// Switch h/w page table register to the kernel-only
// page table, for when no process is running.
void switchkvm(void) {
  lcr3(V2P(kpgdir));   // switch to the kernel page table
}
```

Allocate a new page for page

Add page table entries to map 4 areas

OS sets up a new page table

```
kmap[] = {
{ (void*)KERNBASE, 0,              EXTMEM,     PTE_W},
{ (void*)KERNLINK, V2P(KERNLINK), V2P(data), 0},
{ (void*)data,     V2P(data),      PHYSTOP,    PTE_W},
{ (void*)DEVSPACE, DEVSPACE,       0,          PTE_W},
};
```

```
int main(void) {                                    main.c
  kinit1(end, P2V(4*1024*1024));
  kvmalloc(); // kernel page table
  ..
}
```

```
// Allocate one page table for the machine for      vm.c
// the kernel address space for scheduler processes.
void kvmalloc(void) {
  kpgdir = setupkvm();
  switchkvm();
}

pde_t* setupkvm(void) {
  pde_t *pgdir;
  struct kmap *k;
  pgdir = (pde_t*)kalloc();
  memset(pgdir, 0, PGSIZE);
  for(k = kmap; k < &kmap[NELEM(kmap)]; k++)
    mappages(pgdir, k->...)
}

// Switch h/w page table register to the kernel-only
// page table, for when no process is running.
void switchkvm(void) {
  lcr3(V2P(kpgdir));   // switch to the kernel page table
}
```

Allocate a new page for page

Add page table entries to map 4 areas

Change CR3 to new page table

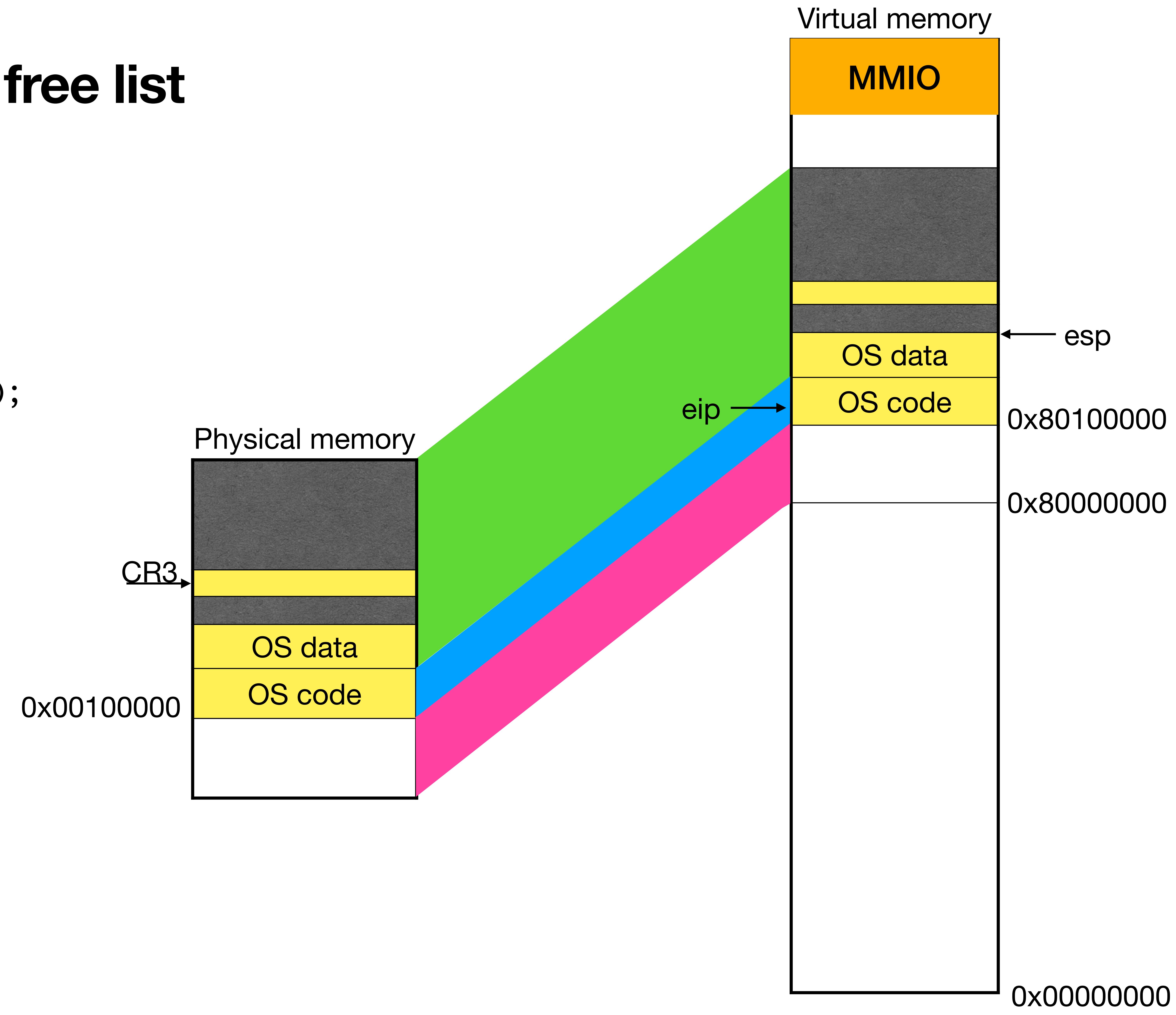
Mapping pages

- mappages takes page directory, virtual address, size, physical address, and permissions
  - It calls walkpgdir with alloc=1 (to allocate page table pages if they do not exist) to find the page table entry
  - It puts physical address in the PTE, marks it as present, and puts other permissions on PTE
- walkpgdir:
  - mimics hardware's page table walk. It takes first 10 bits to index into page directory to find page table page. It takes next 10 bits to index into page table page. It returns page table entry.
  - If page table page does not exist, it allocates a new page and adds it to page directory

main calls kinit2 to expand free list

```
int main (void) {
  kinit1(end, P2V(4*1024*1024));
  kvmalloc();
  …
  kinit2(P2V(4*1024*1024), P2V(PHYSTOP);
}
```

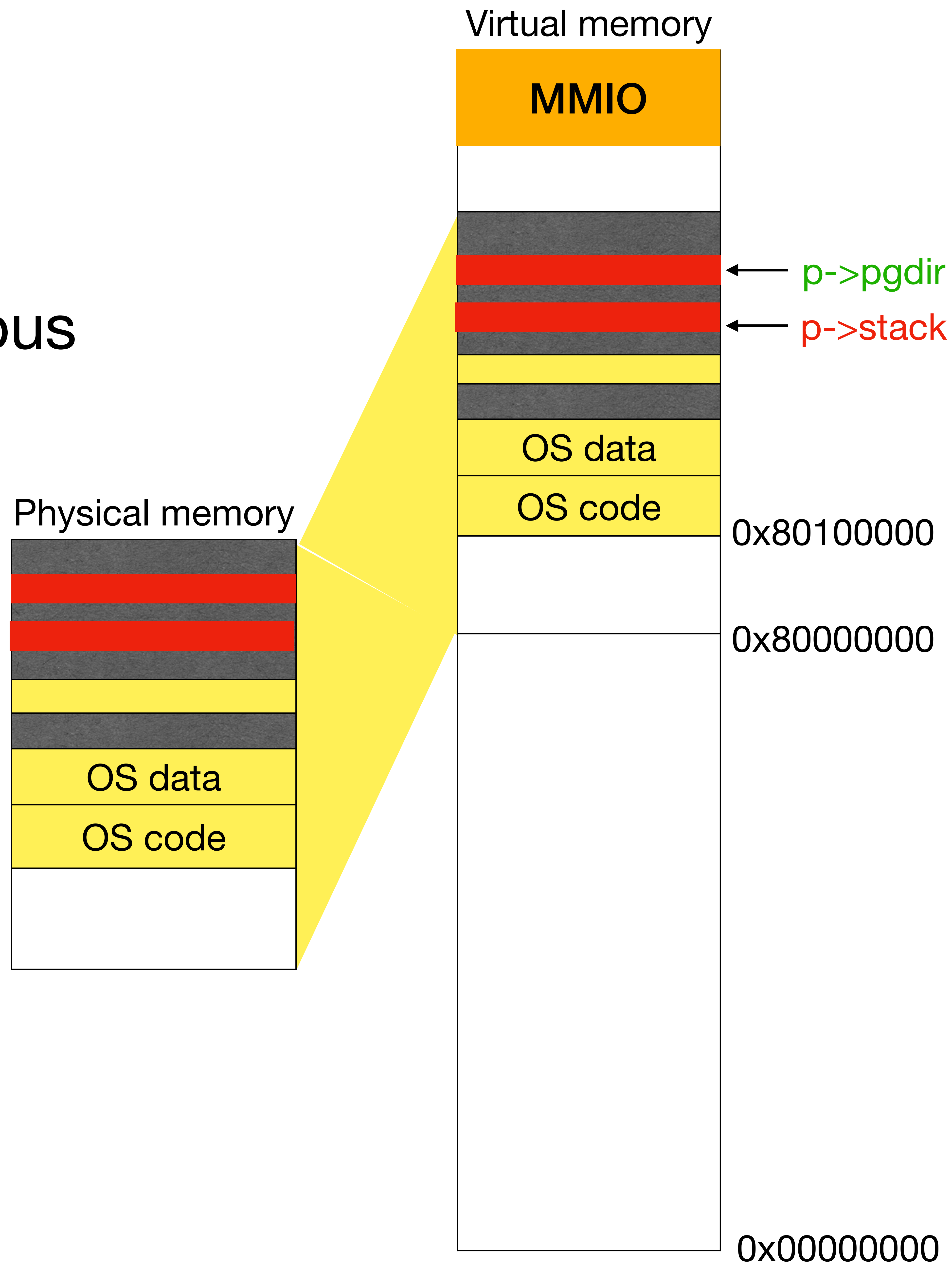
Now that full physical memory is mapped, rest of the physical memory is made available to allocator



Setting up processes

- kstack, page table pages need not be contiguous

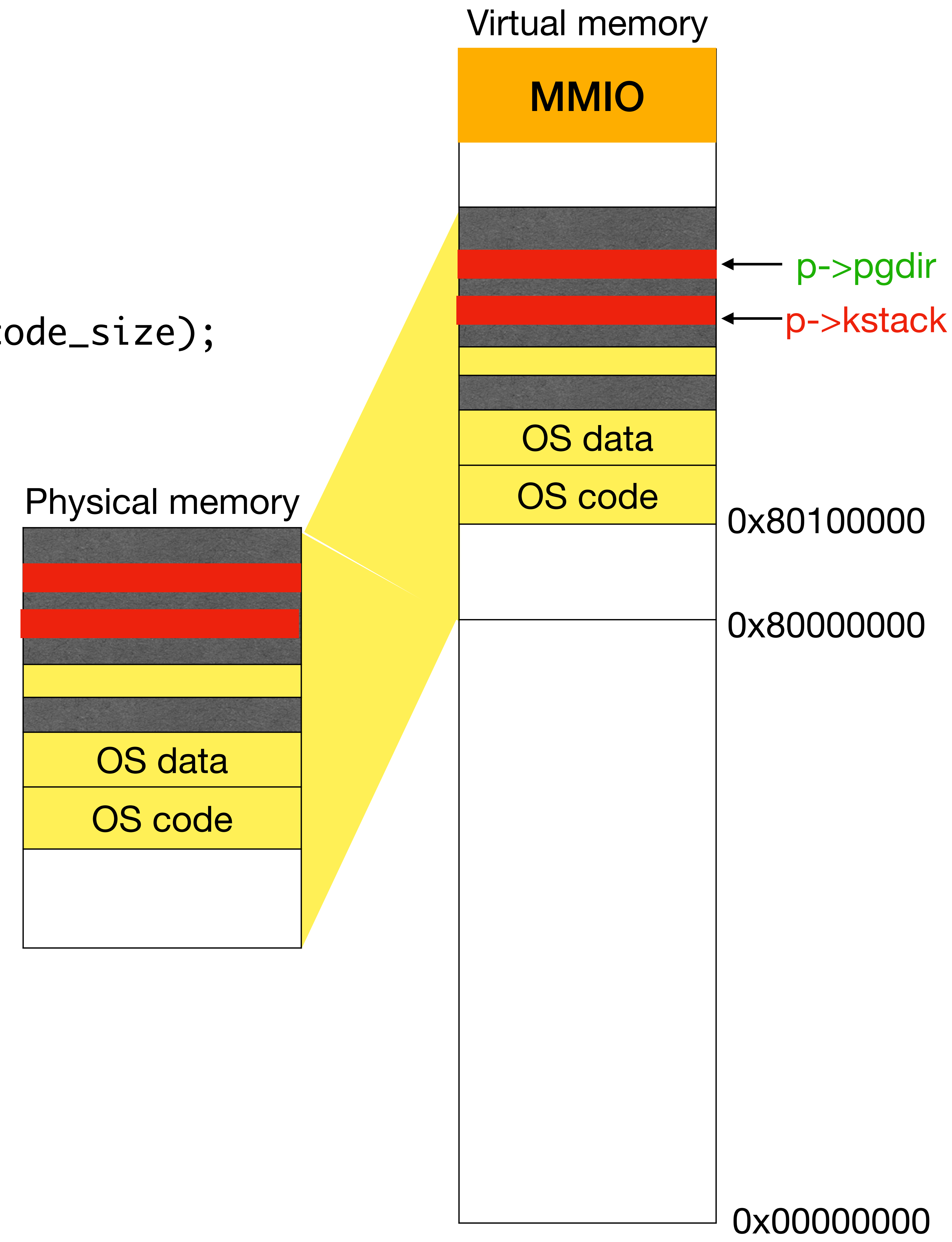
```
void userinit(){
  p = allocproc();
  p->pgdir = setupkvm();
  …
}
static struct proc* allocproc(void) {
  p->kstack = kalloc();
  sp = p->kstack + KSTACKSIZE;
  …
}
```



Key changes from paging

```
void userinit(){
  p->pgdir = setupkvm();
  inituvm(p->pgdir, _binary_initcode_start, (int)_binary_initcode_size);
  p->sz = PGSIZE;
 …
}
```

```
void inituvm(pde_t *pgdir, char *init, uint sz) {
  char *mem;
  mem = kalloc();
  memset(mem, 0, PGSIZE);
  mappages(pgdir, 0, PGSIZE, V2P(mem), PTE_W|PTE_U);
  memmove(mem, init, sz);
}
```

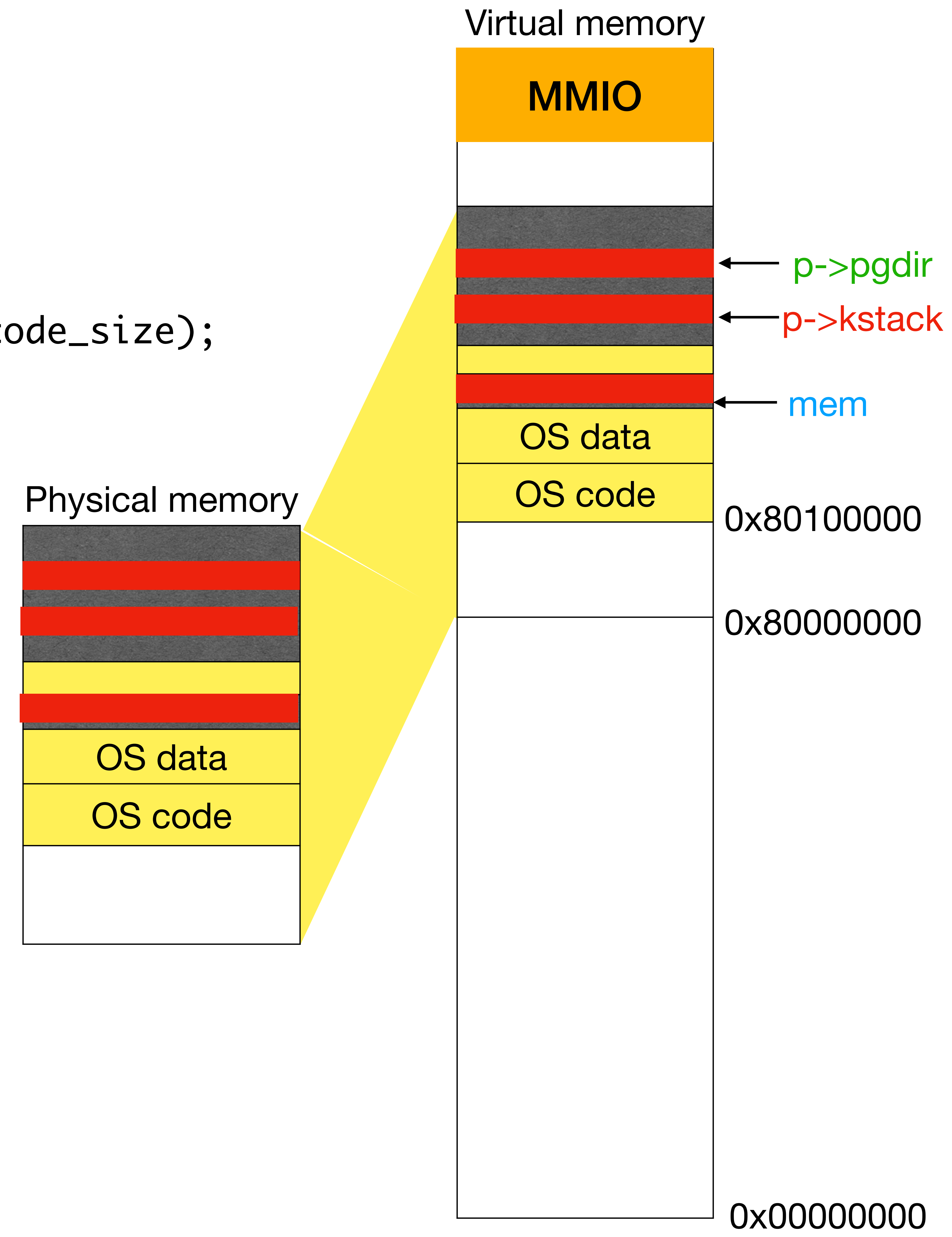


Key changes from paging

```
void userinit(){
  p->pgdir = setupkvm();
  inituvm(p->pgdir, _binary_initcode_start, (int)_binary_initcode_size);
  p->sz = PGSIZE;
 …
}
```

```
void inituvm(pde_t *pgdir, char *init, uint sz) {
  char *mem;
  mem = kalloc();
  memset(mem, 0, PGSIZE);
  mappages(pgdir, 0, PGSIZE, V2P(mem), PTE_W|PTE_U);
  memmove(mem, init, sz);
}
```

Allocate a page, clear it



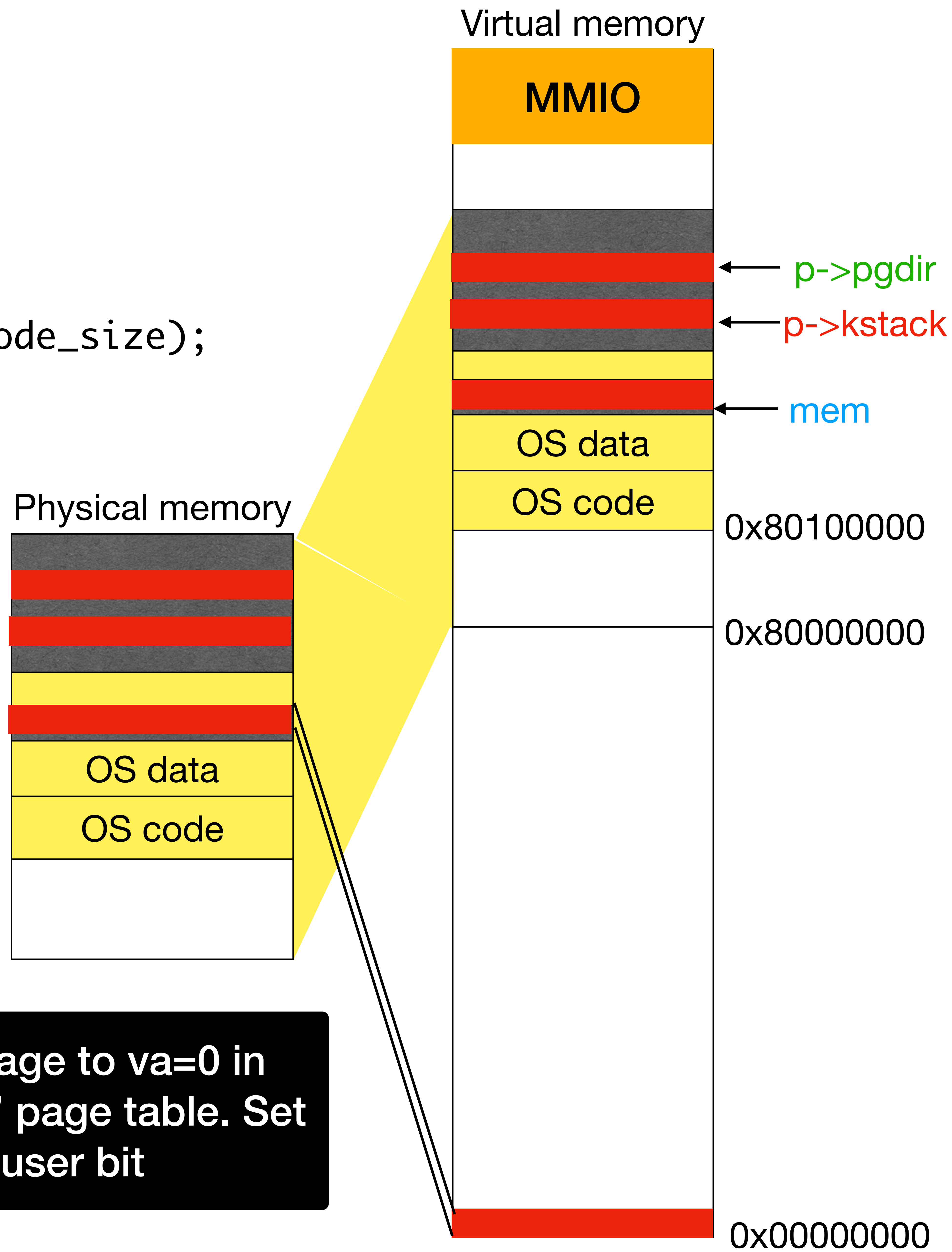
Key changes from paging

```
void userinit(){
  p->pgdir = setupkvm();
  inituvm(p->pgdir, _binary_initcode_start, (int)_binary_initcode_size);
  p->sz = PGSIZE;
 …
}
```

```
void inituvm(pde_t *pgdir, char *init, uint sz) {
  char *mem;
  mem = kalloc();
  memset(mem, 0, PGSIZE);
  mappages(pgdir, 0, PGSIZE, V2P(mem), PTE_W|PTE_U);
  memmove(mem, init, sz);
}
```

Allocate a page, clear it

Add page to va=0 in process' page table. Set user bit



Key changes from paging

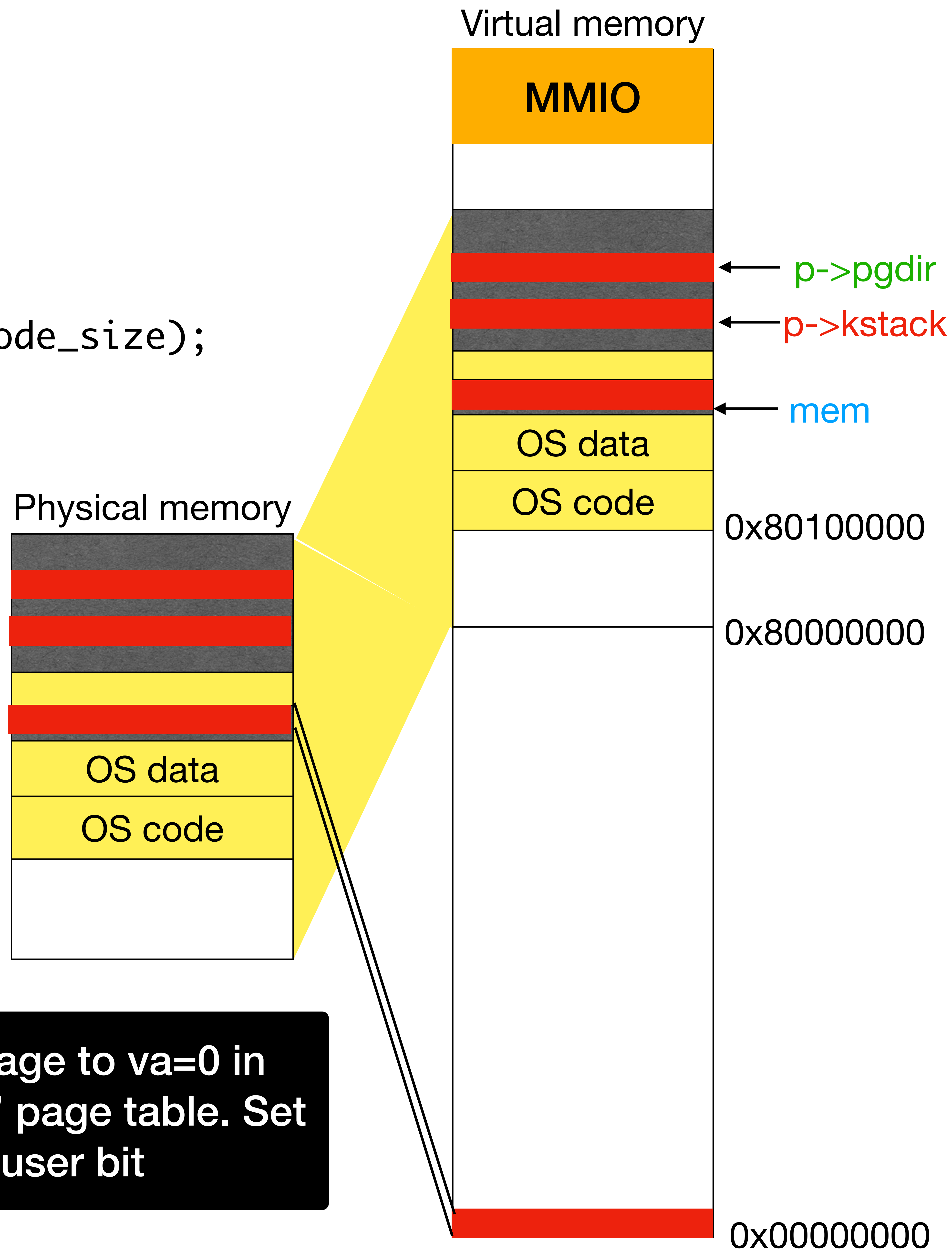
```
void userinit(){
  p->pgdir = setupkvm();
  inituvm(p->pgdir, _binary_initcode_start, (int)_binary_initcode_size);
  p->sz = PGSIZE;
 …
}
```

```
void inituvm(pde_t *pgdir, char *init, uint sz) {
  char *mem;
  mem = kalloc();
  memset(mem, 0, PGSIZE);
  mappages(pgdir, 0, PGSIZE, V2P(mem), PTE_W|PTE_U);
  memmove(mem, init, sz);
}
```

Allocate a page, clear it

Add page to va=0 in process' page table. Set user bit

Copy init binary



sbrk system call

```
int sys_sbrk(void) {
  int n;
  if(argint(0, &n) < 0)
    return -1;
  growproc(n)
  ..
}
```

```
int growproc(int n) {
  uint sz;
  struct proc *curproc = myproc();
  sz = curproc->sz;
  if(n > 0){
    sz = allocuvm(curproc->pgdir, sz, sz + n)
  } else if(n < 0){
    sz = deallocuvm(curproc->pgdir, sz, sz + n)
  }
  curproc->sz = sz;
  switchuvm(curproc);
```

Reload CR3 to evict all TLB entries (some of them maybe dealloc'ed)

Reject if address space will grow into the OS area

```
int allocuvm(pde_t *pgdir, uint oldsz, uint newsz) {
  if(newsz >= KERNBASE)
    return 0;
  uint a = PGROUNDUP(oldsz);
  for(; a < newsz; a += PGSIZE){
    char* mem = kalloc();
    memset(mem, 0, PGSIZE);
    mappages(pgdir, (char*)a, PGSIZE, V2P(mem), PTE_W|PTE_U)
```

Allocate a page, clear it, add to the user space

Summary

- Paging provides significant flexibility: we can spray process pages anywhere. Processes can grow/shrink easily since address space need not be contiguous
- It simplifies allocation and removes external fragmentation since pages are of fixed size
- Multi-level page tables help reduce size of the page tables. TLBs help reduce page table walks
- Address translation hardware is much more complex than segmentation
- Demand paging transparently moves pages between memory and disk
- OS maps itself at fixed area in virtual address space of all processes to ease trap handling, sys call parameter reading, and switching page tables